

本节内容

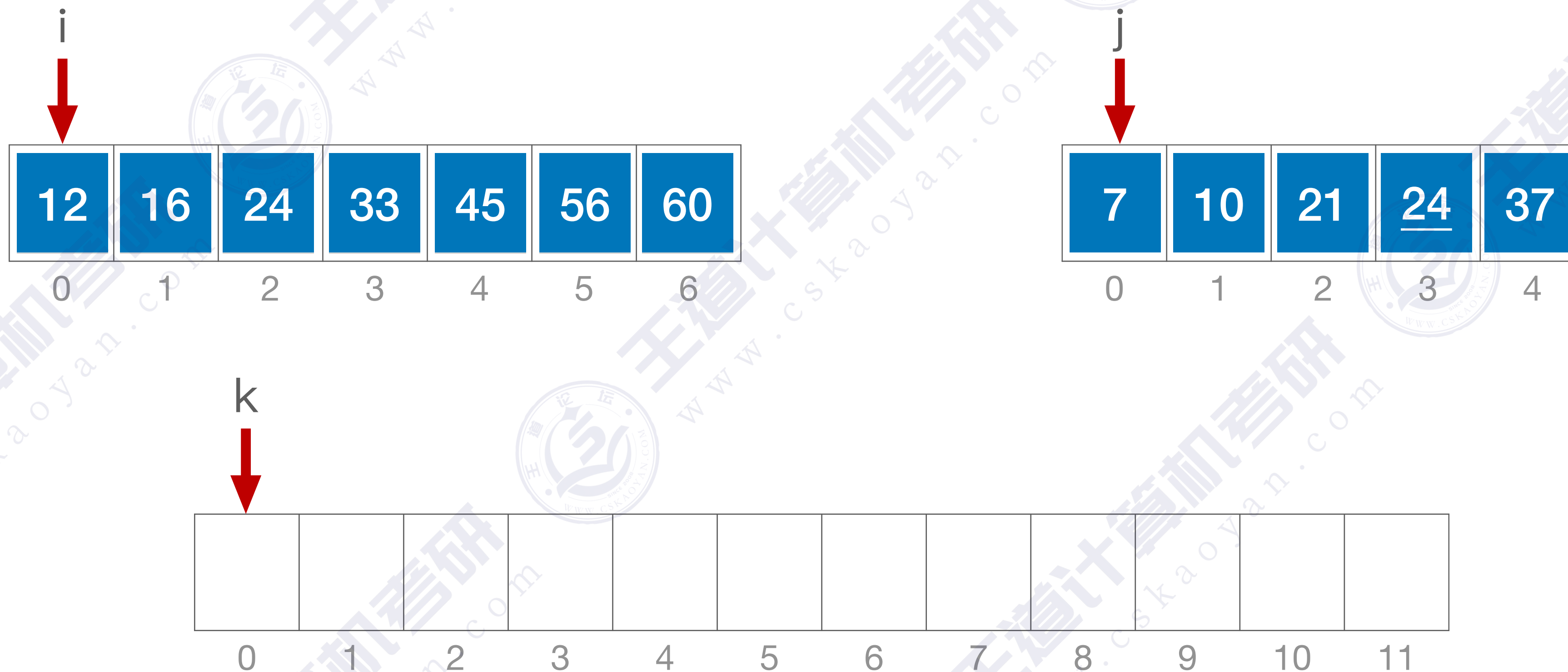
归并排序

(Merge Sort)

什么是 Merge（归并/合并）？



归并：把两个或多个已经有序的序列合并成一个

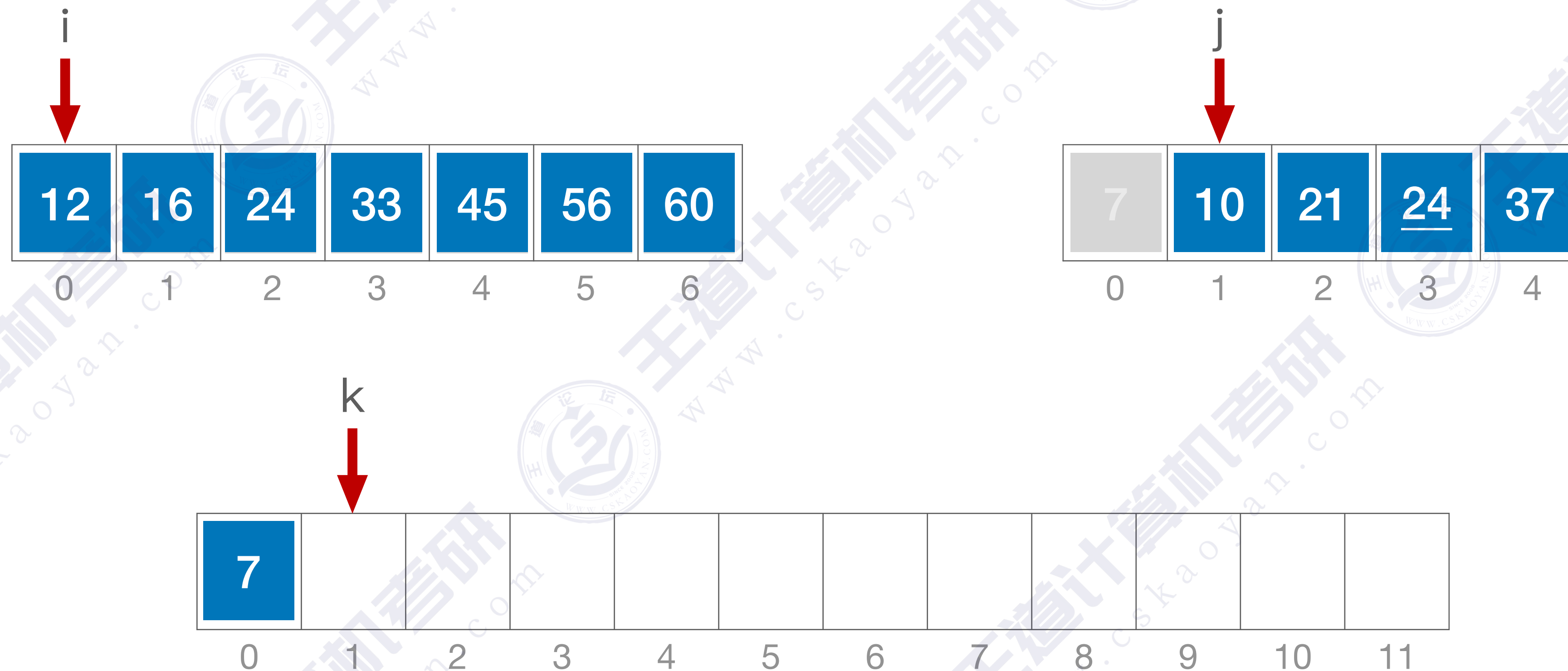


对比 i 、 j 所指元素，选择更小的一个放入 k 所指位置

什么是 Merge（归并/合并）？



归并：把两个或多个已经有序的序列合并成一个

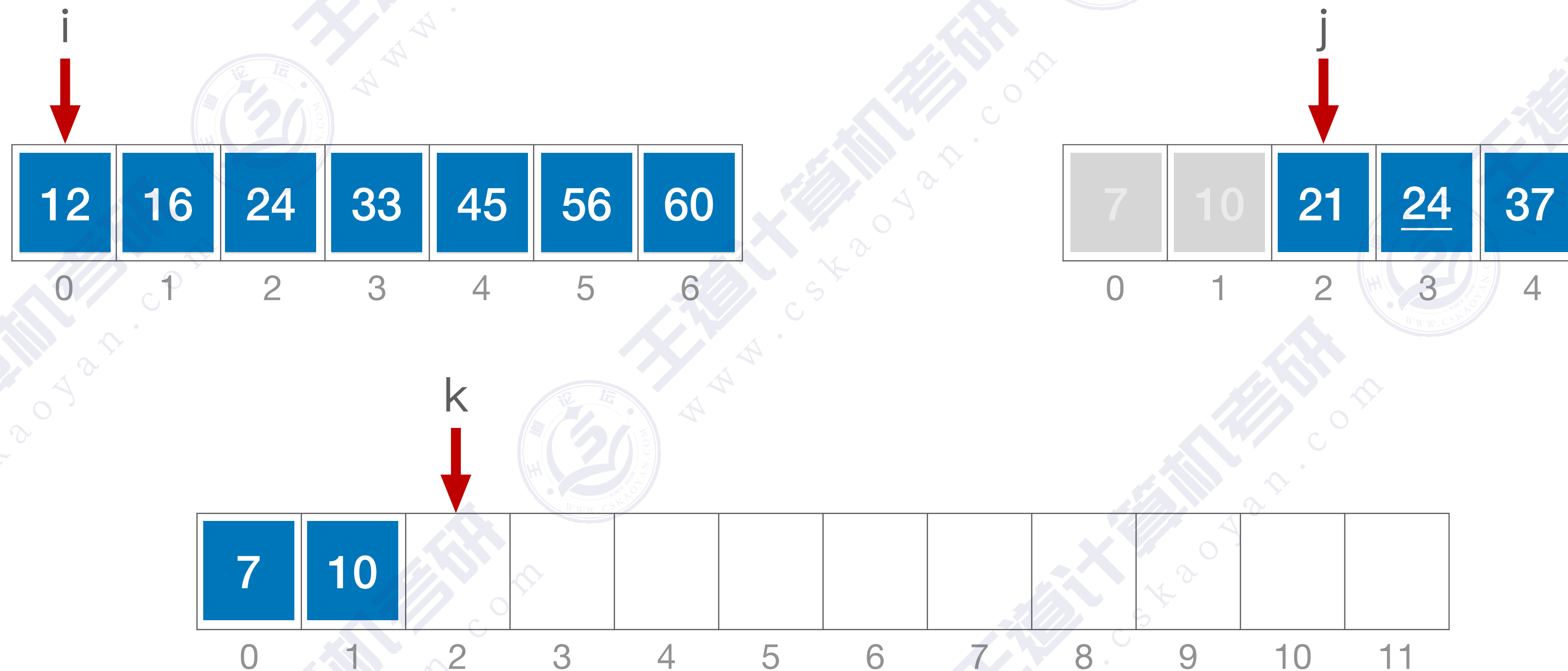


对比 i 、 j 所指元素，选择更小的一个放入 k 所指位置

什么是 Merge（归并/合并）？



归并：把两个或多个已经有序的序列合并成一个

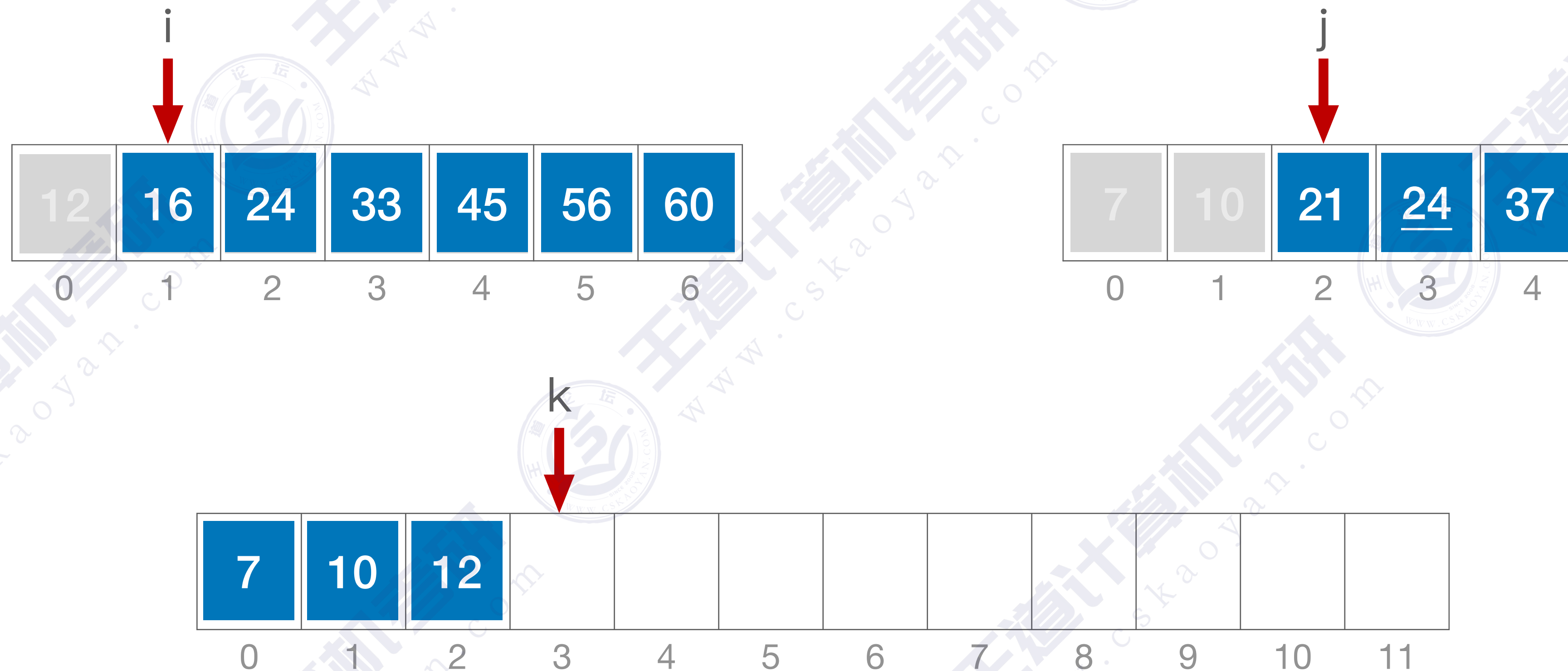


对比 i 、 j 所指元素，选择更小的一个放入 k 所指位置

什么是 Merge（归并/合并）？



归并：把两个或多个已经有序的序列合并成一个

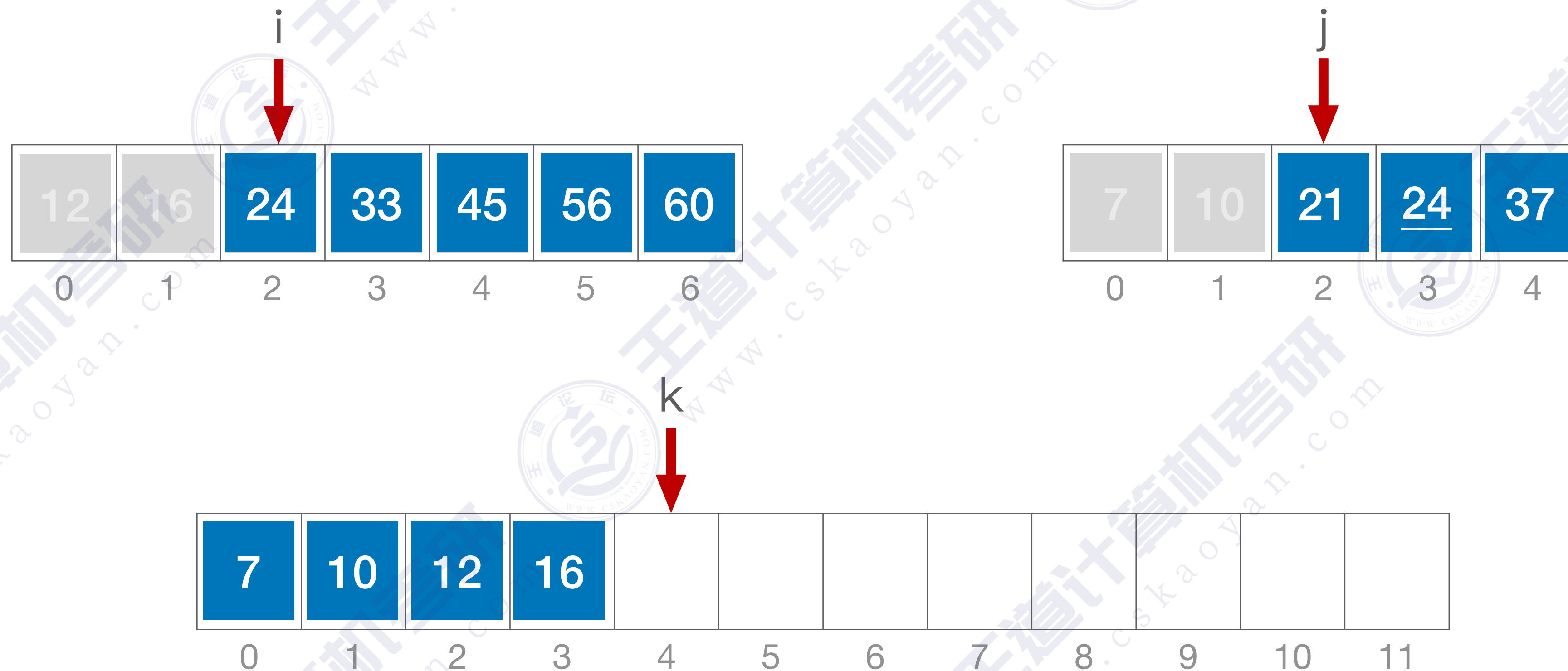


对比 i 、 j 所指元素，选择更小的一个放入 k 所指位置

什么是 Merge (归并/合并) ?



归并：把两个或多个已经有序的序列合并成一个

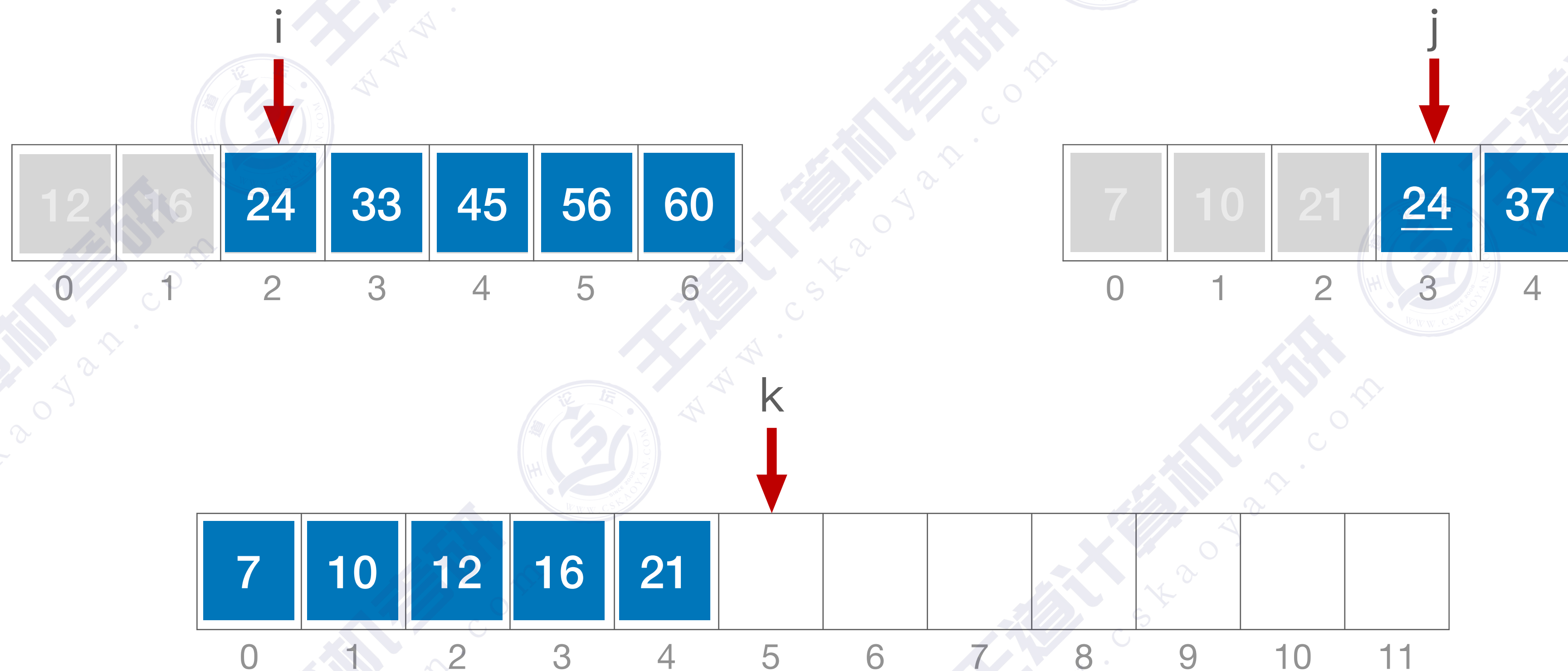


对比 i 、 j 所指元素，选择更小的一个放入 k 所指位置

什么是 Merge（归并/合并）？



归并：把两个或多个已经有序的序列合并成一个

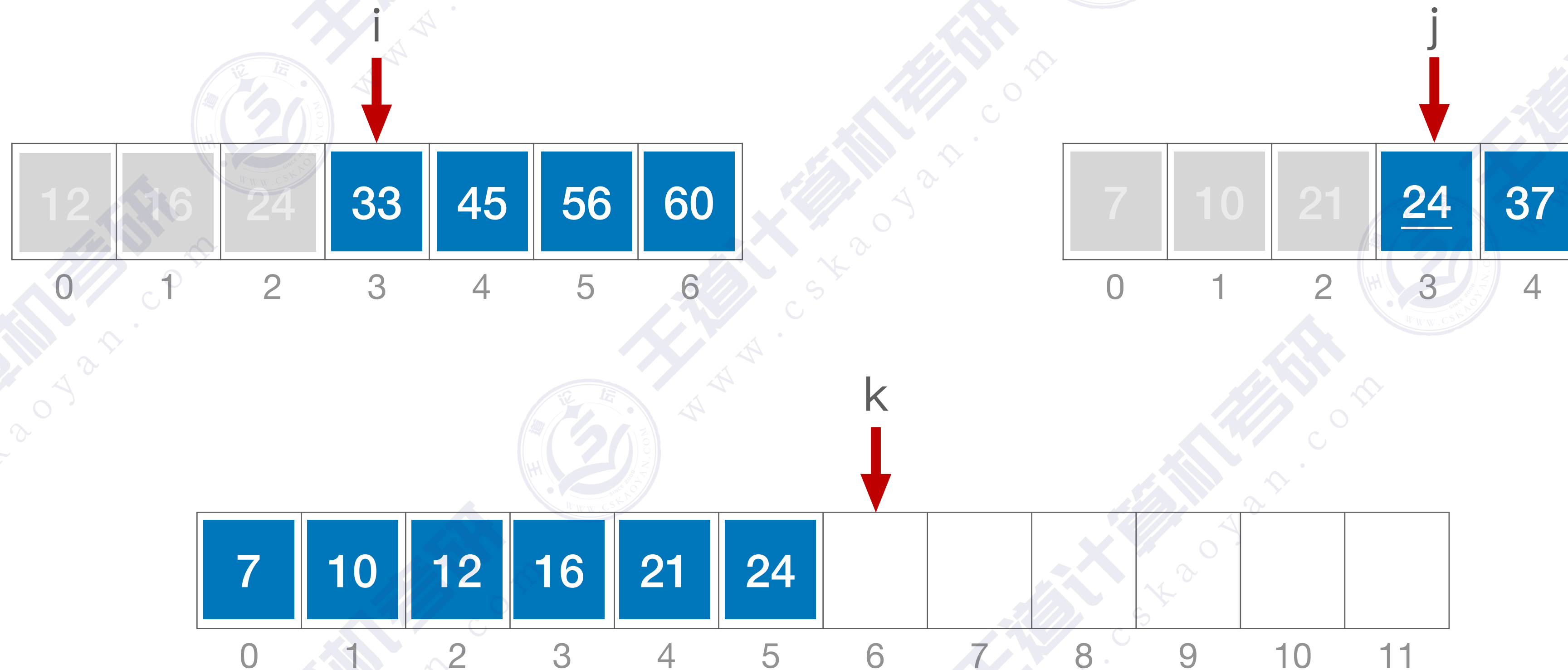


对比 i 、 j 所指元素，选择更小的一个放入 k 所指位置

什么是 Merge (归并/合并) ?



归并：把两个或多个已经有序的序列合并成一个

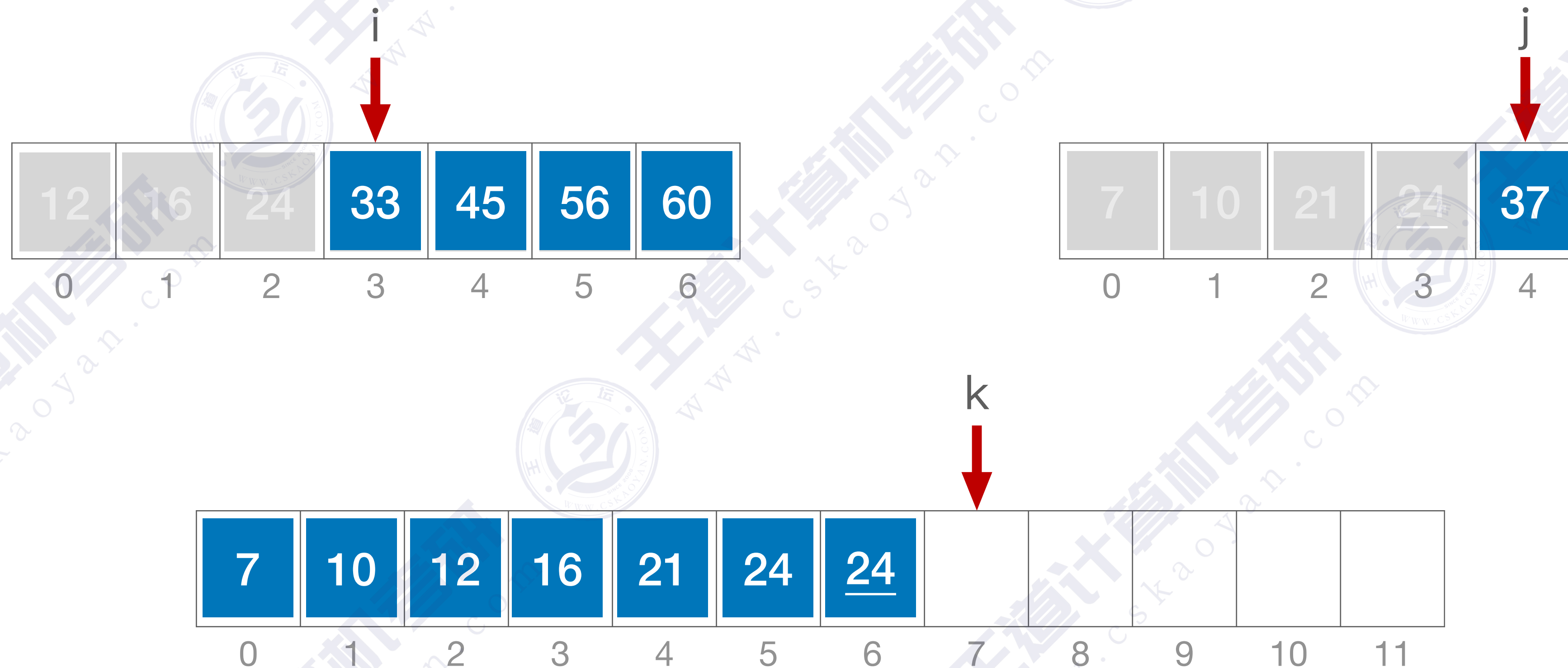


对比 i 、 j 所指元素，选择更小的一个放入 k 所指位置

什么是 Merge (归并/合并) ?



归并：把两个或多个已经有序的序列合并成一个

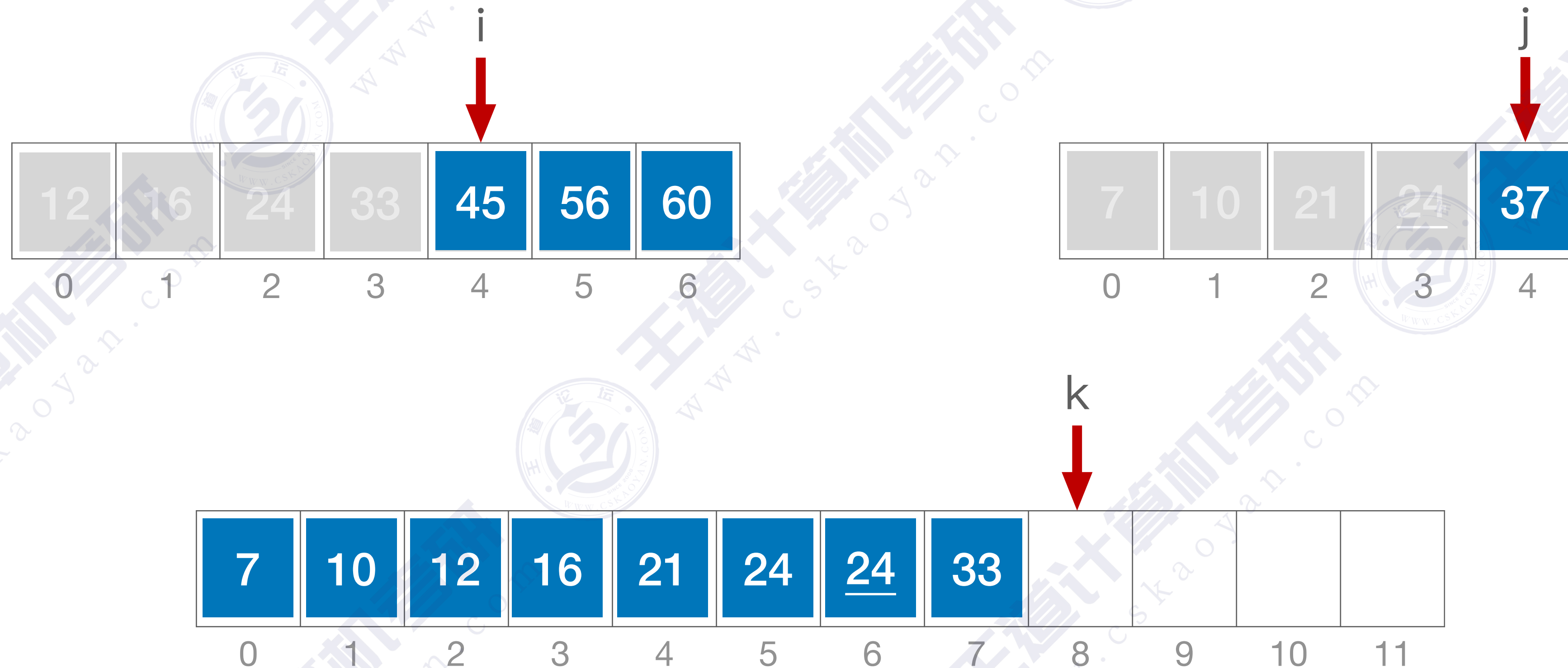


对比 i 、 j 所指元素，选择更小的一个放入 k 所指位置

什么是 Merge（归并/合并）？



归并：把两个或多个已经有序的序列合并成一个

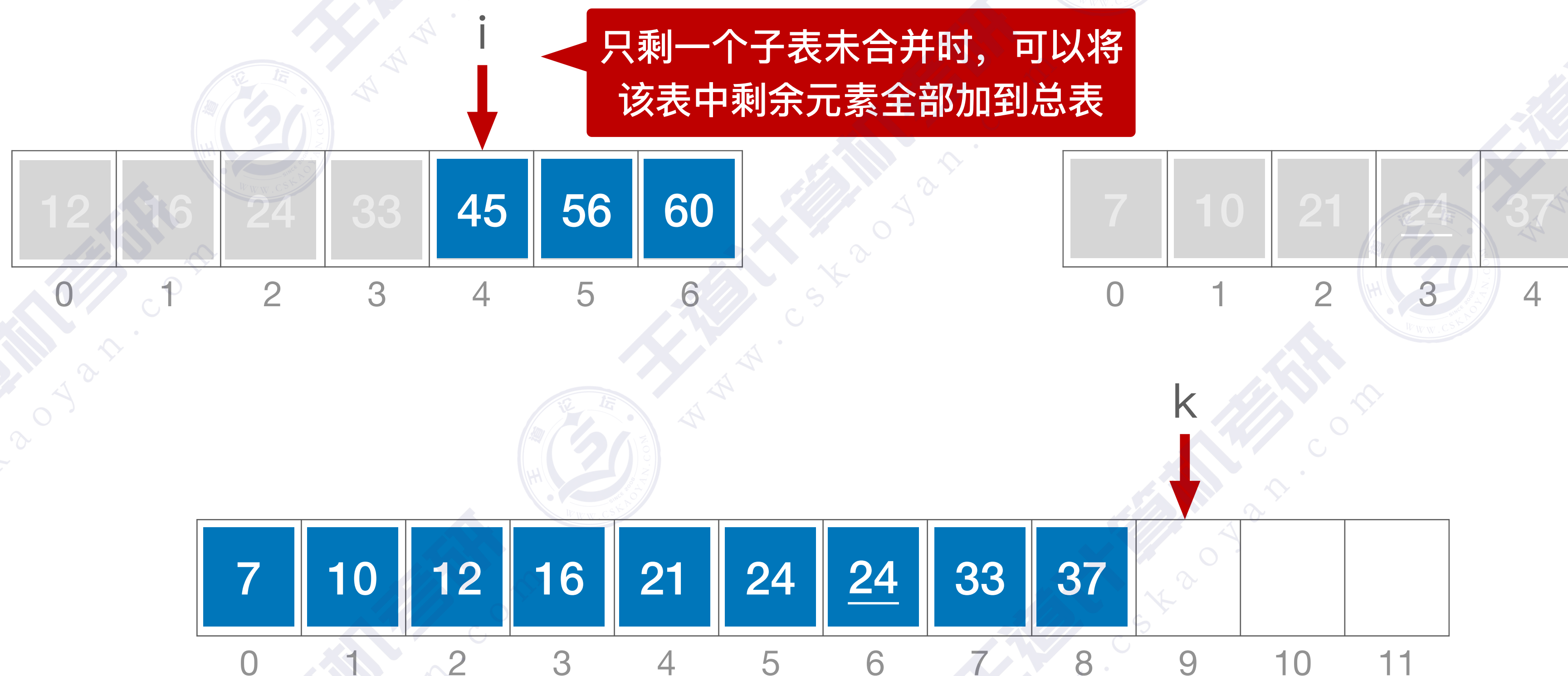


对比 *i*、*j* 所指元素，选择更小的一个放入 *k* 所指位置

什么是 Merge（归并/合并）？



归并：把两个或多个已经有序的序列合并成一个

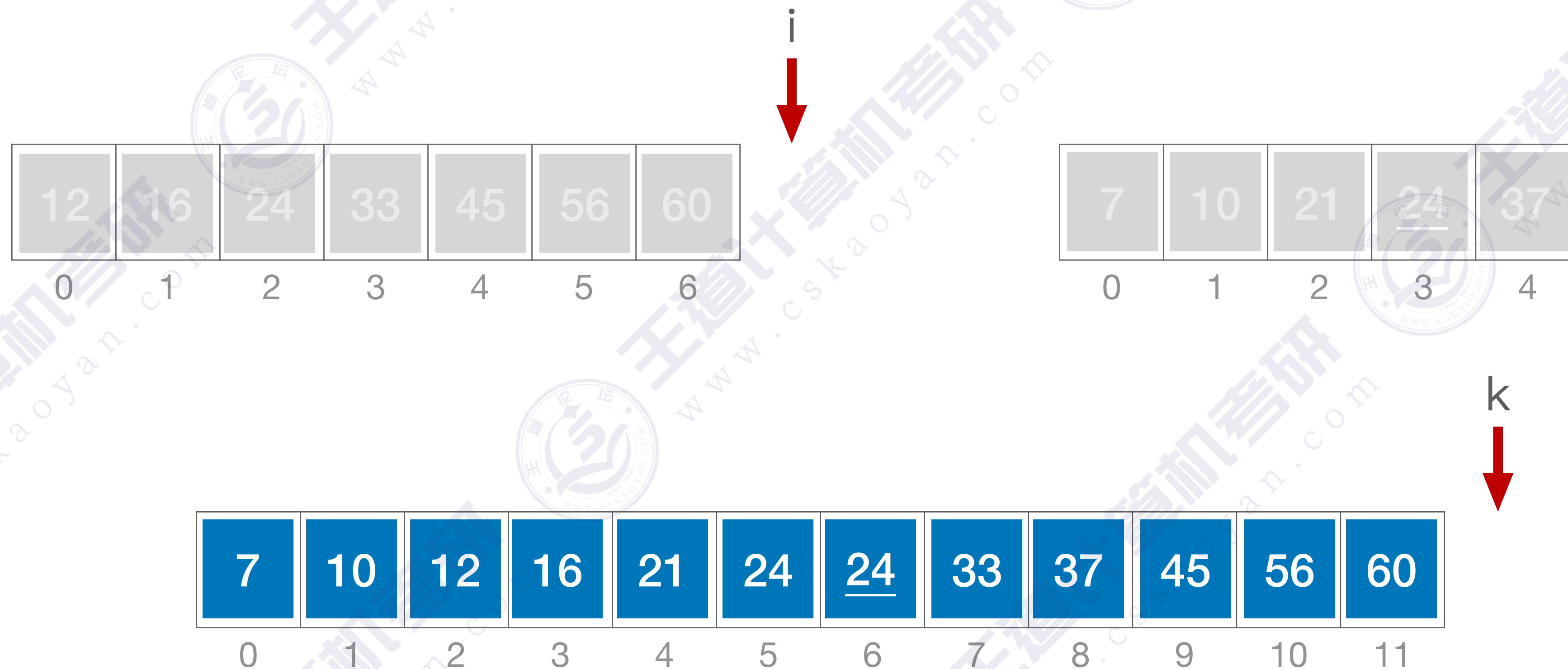


对比 i 、 j 所指元素，选择更小的一个放入 k 所指位置

什么是 Merge（归并/合并）？



归并：把两个或多个已经有序的序列合并成一个

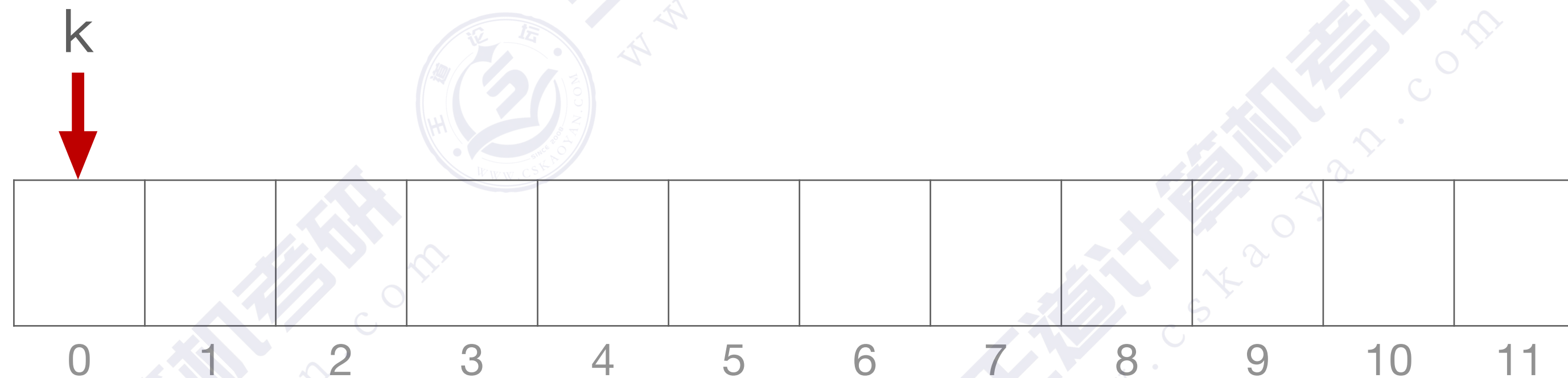
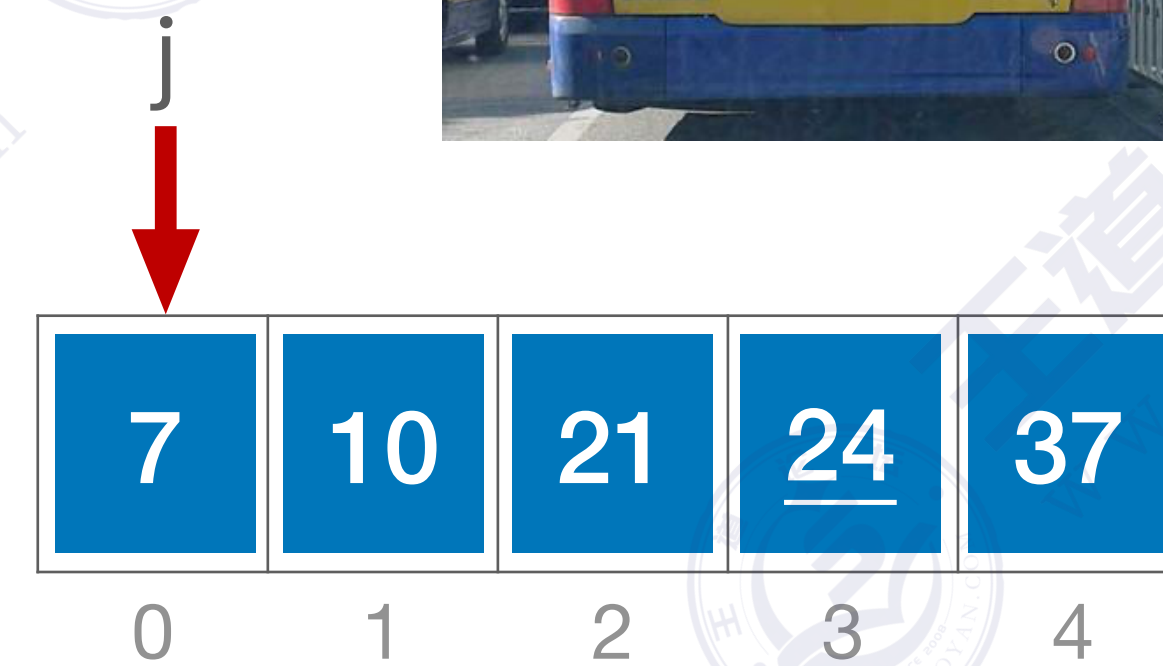
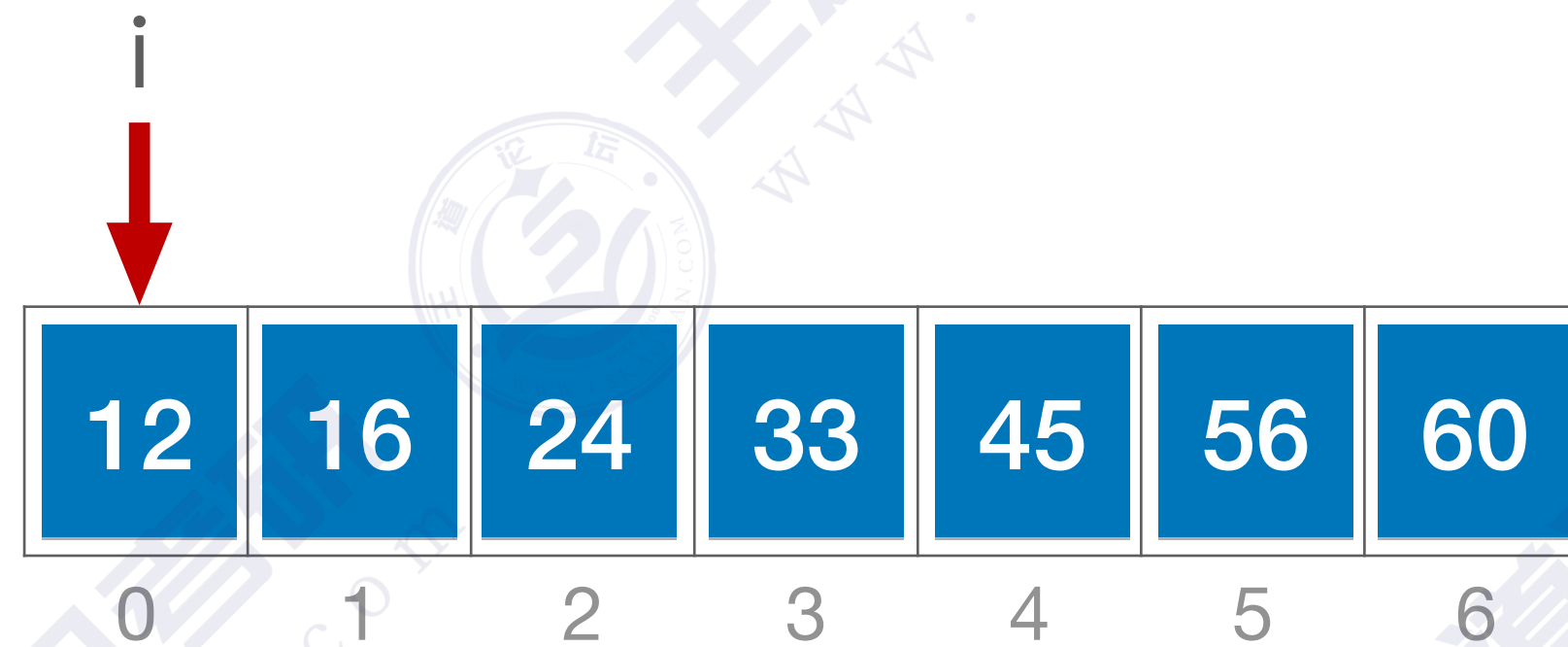


对比 i 、 j 所指元素，选择更小的一个放入 k 所指位置

“2路”——二合一

“2路”归并

归并：把两个或多个已经有序的序列合并成一个



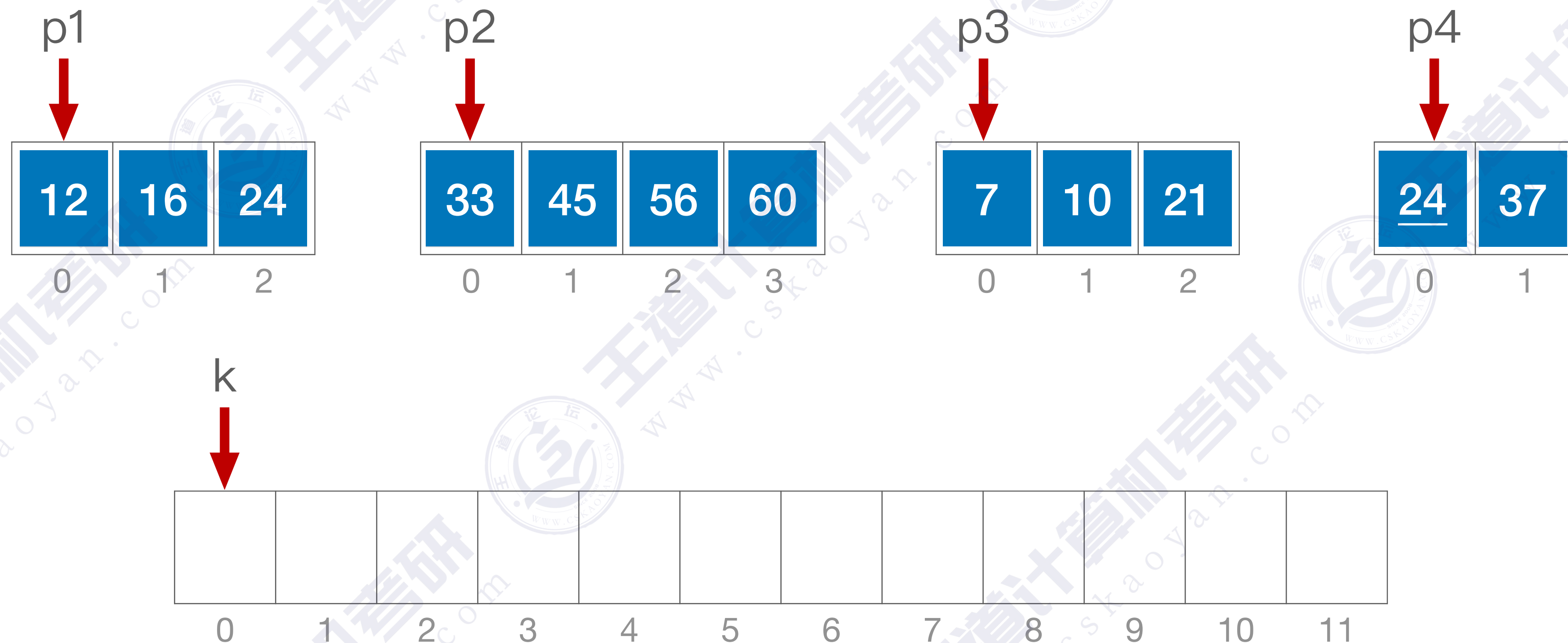
“2路”归并——每选出一个元素需对比关键字1次

对比 i、j 所指元素，选择更小的一个放入 k 所指位置



“4路”——四合一 → “4路”归并

归并：把两个或多个已经有序的序列合并成一个



“4路”归并——每选出一个元素需对比关键字3次

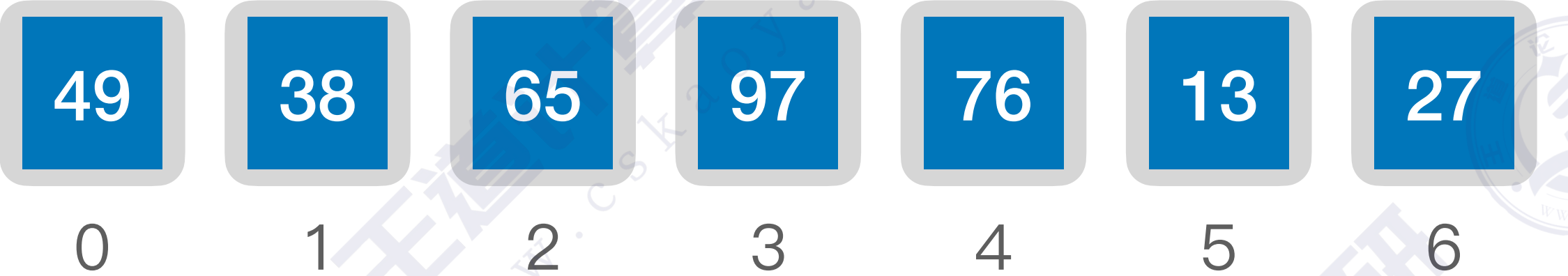
对比 p1、p2、p3、p4所指元素，选择更小的一个放入 k 所指位置

结论：m路归并，每选出一个元素需对比关键字 m-1 次

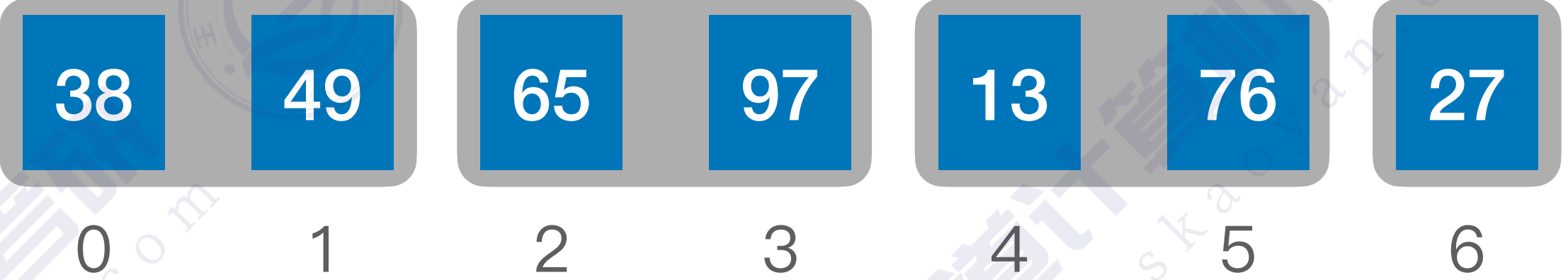
在内部排序中一般采用2路归并

归并排序（手算模拟）

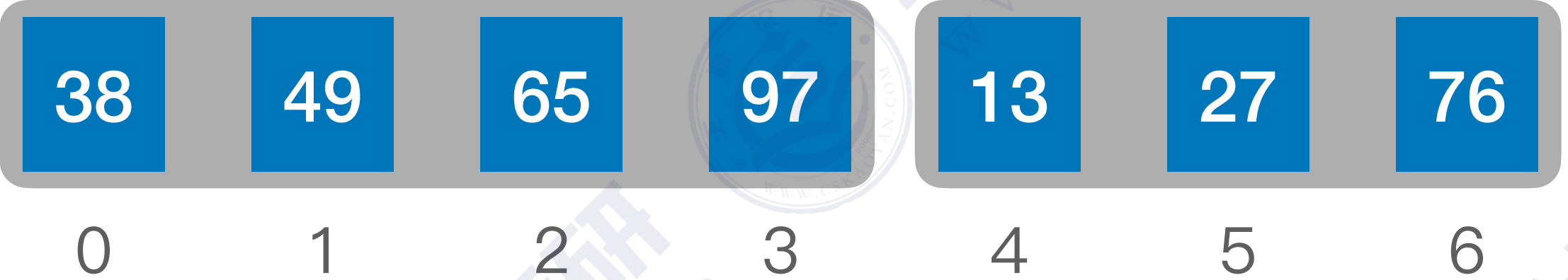
初始序列:



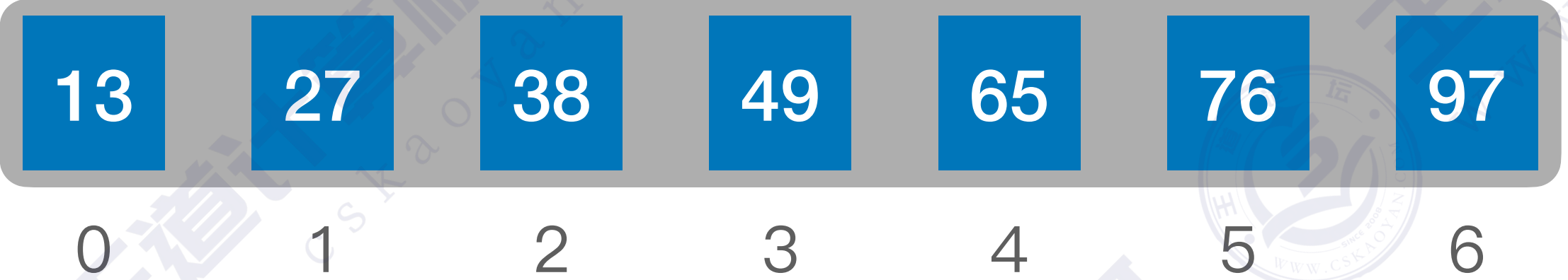
一趟归并后:



二趟归并后:

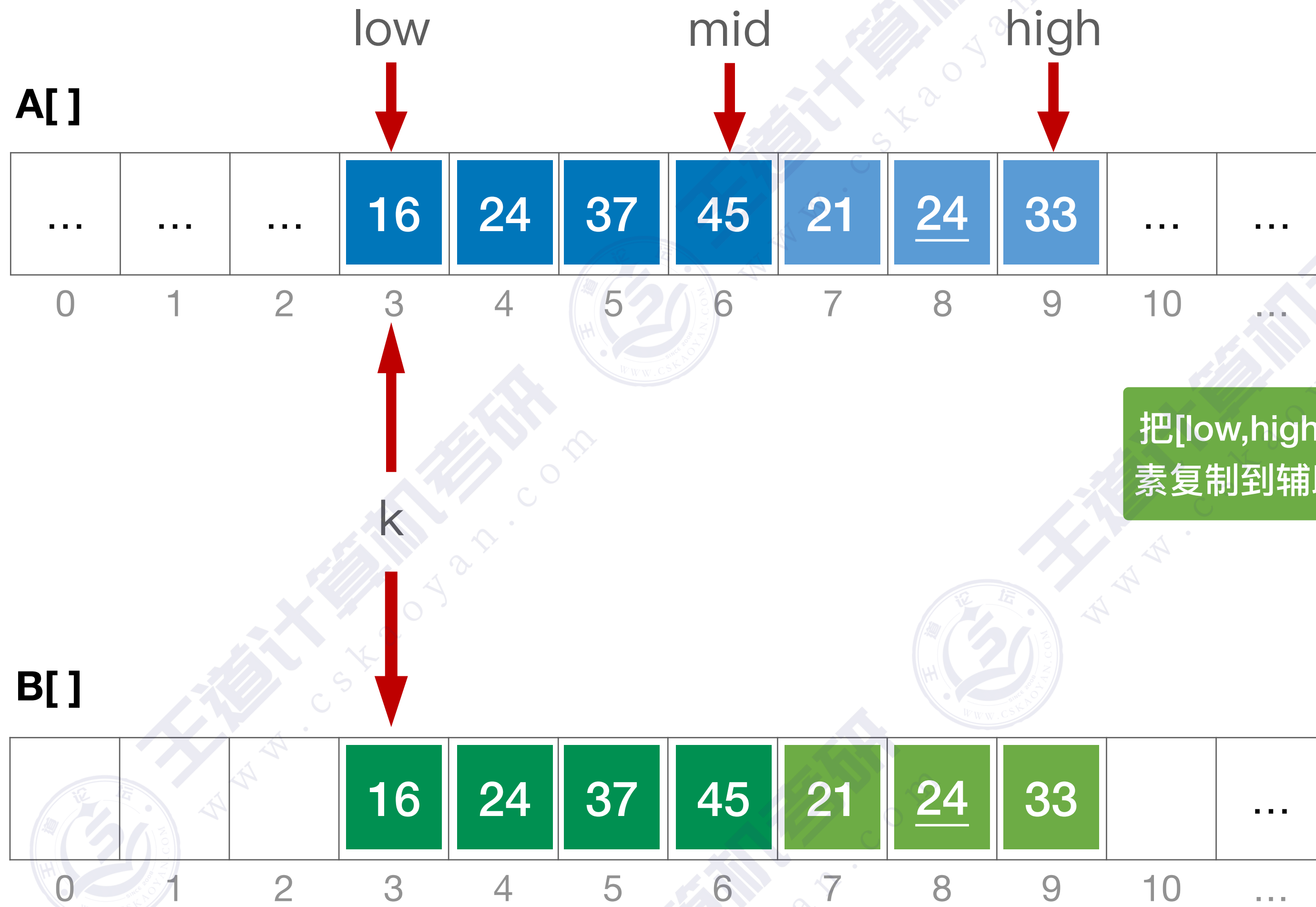


三趟归并后:



核心操作：把数组内的两个有序序列归并为一个

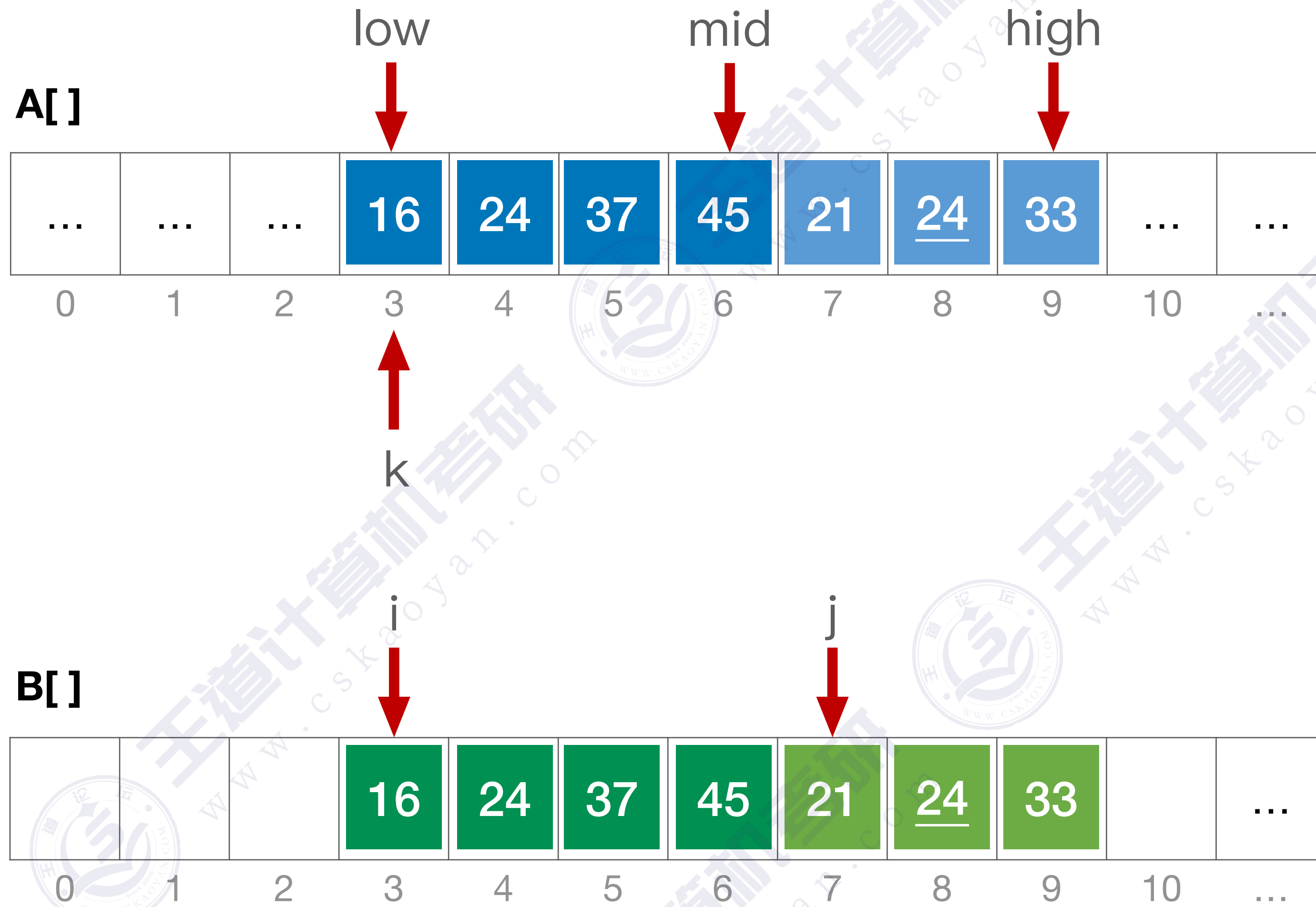
代码实现



```
int *B=(int *)malloc(n*sizeof(int)); //辅助数组B

//A[low...mid]和A[mid+1...high]各自有序，将两个部分归并
void Merge(int A[],int low,int mid,int high){
    int i,j,k;
    for(k=low;k<=high;k++)
        B[k]=A[k]; //将A中所有元素复制到B中
    for(i=low,j=mid+1,k=i;i<=mid&&j<=high;k++){
        if(B[i]<=B[j])
            A[k]=B[i++]; //将较小值复制到A中
        else
            A[k]=B[j++];
    }//for
    while(i<=mid)    A[k++]=B[i++];
    while(j<=high)  A[k++]=B[j++];
}
```

代码实现

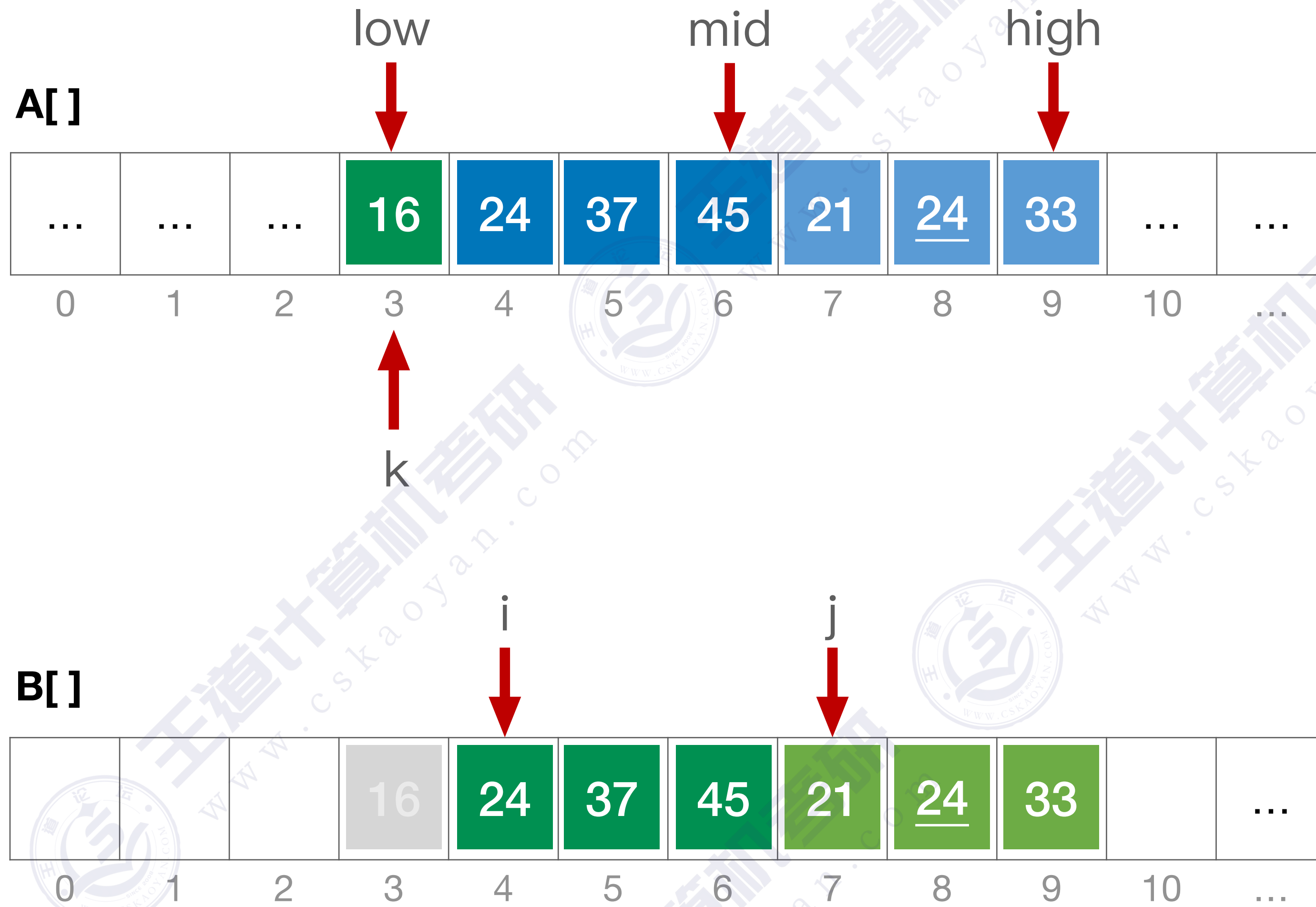


```
int *B=(int *)malloc(n*sizeof(int)); //辅助数组B

//A[low...mid]和A[mid+1...high]各自有序，将两个部分归并
void Merge(int A[],int low,int mid,int high){
    int i,j,k;
    for(k=low;k<=high;k++)
        B[k]=A[k]; //将A中所有元素复制到B中
    for(i=low,j=mid+1,k=i;i<=mid&&j<=high;k++){
        if(B[i]<=B[j])
            A[k]=B[i++]; //将较小值复制到A中
        else
            A[k]=B[j++];
    }//for
    while(i<=mid)    A[k++]=B[i++];
    while(j<=high)  A[k++]=B[j++];
}
```

归并

代码实现

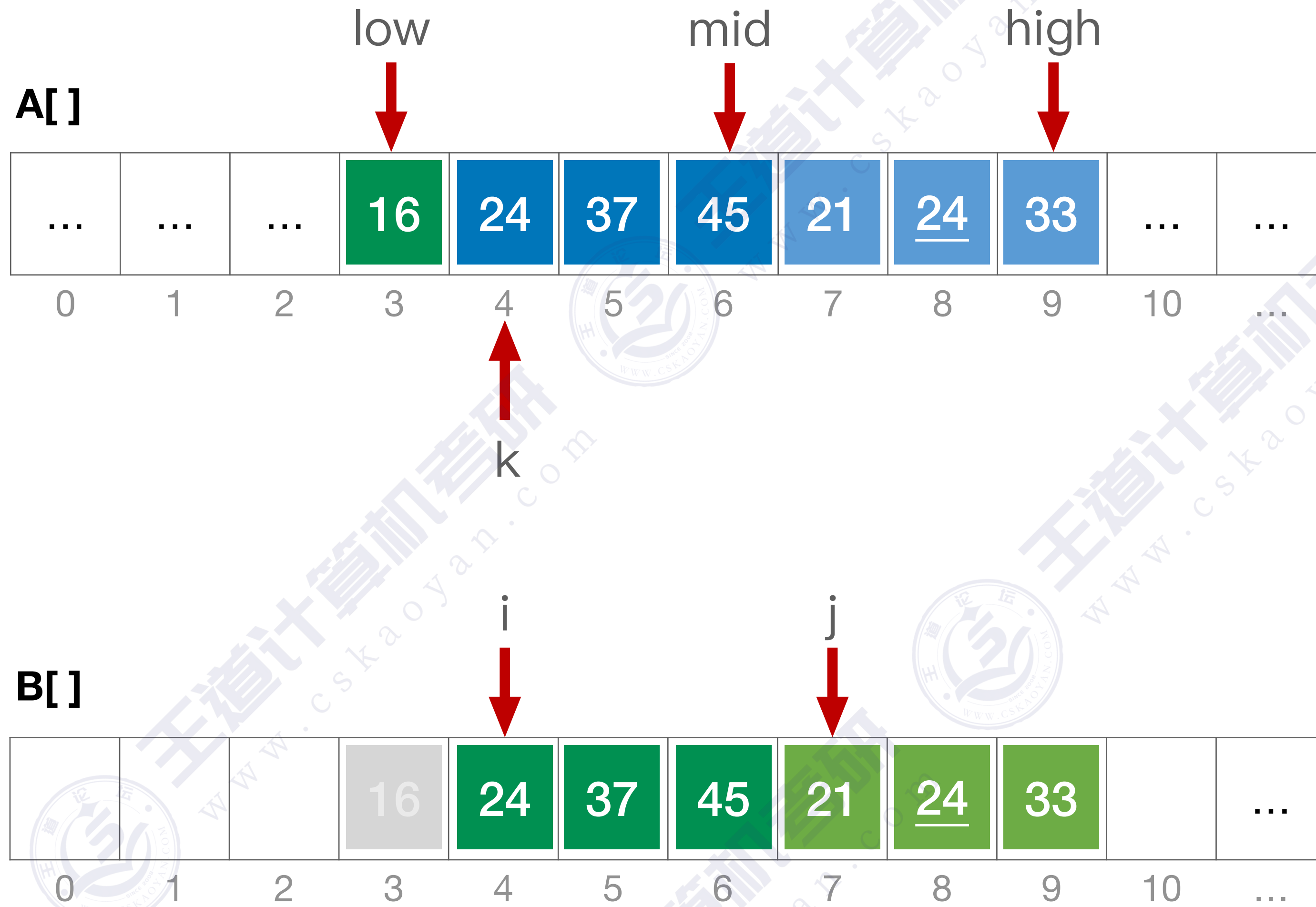


```
int *B=(int *)malloc(n*sizeof(int)); //辅助数组B

//A[low...mid]和A[mid+1...high]各自有序，将两个部分归并
void Merge(int A[],int low,int mid,int high){
    int i,j,k;
    for(k=low;k<=high;k++)
        B[k]=A[k]; //将A中所有元素复制到B中
    for(i=low,j=mid+1,k=i;i<=mid&&j<=high;k++){
        if(B[i]<=B[j])
            A[k]=B[i++]; //将较小值复制到A中
        else
            A[k]=B[j++];
    }//for
    while(i<=mid)    A[k++]=B[i++];
    while(j<=high)  A[k++]=B[j++];
}
```

归并

代码实现

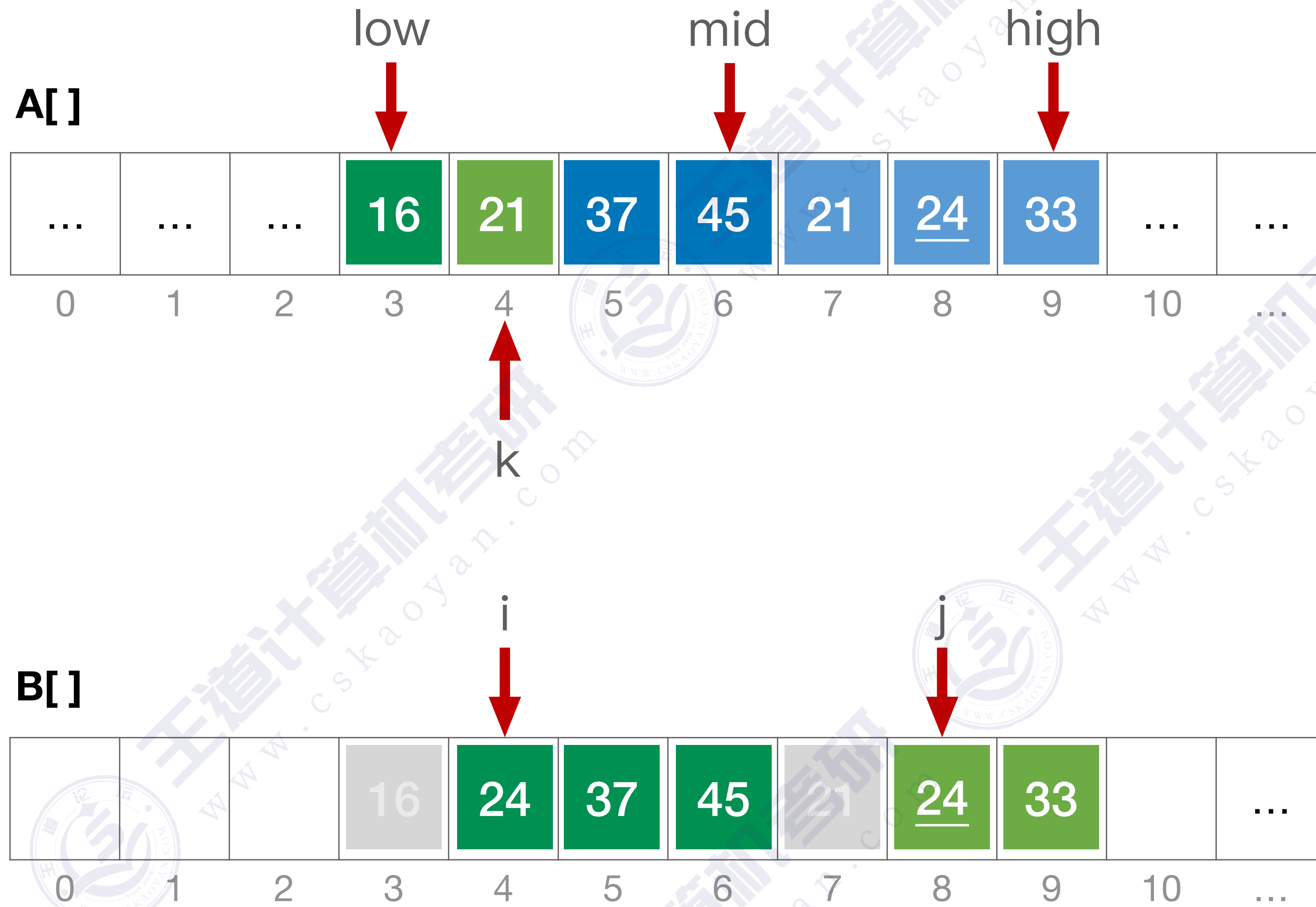


```
int *B=(int *)malloc(n*sizeof(int)); //辅助数组B

//A[low...mid]和A[mid+1...high]各自有序，将两个部分归并
void Merge(int A[],int low,int mid,int high){
    int i,j,k;
    for(k=low;k<=high;k++)
        B[k]=A[k]; //将A中所有元素复制到B中
    for(i=low,j=mid+1,k=i;i<=mid&&j<=high;k++){
        if(B[i]<=B[j])
            A[k]=B[i++]; //将较小值复制到A中
        else
            A[k]=B[j++];
    }//for
    while(i<=mid)    A[k++]=B[i++];
    while(j<=high)  A[k++]=B[j++];
}
```

归并

代码实现

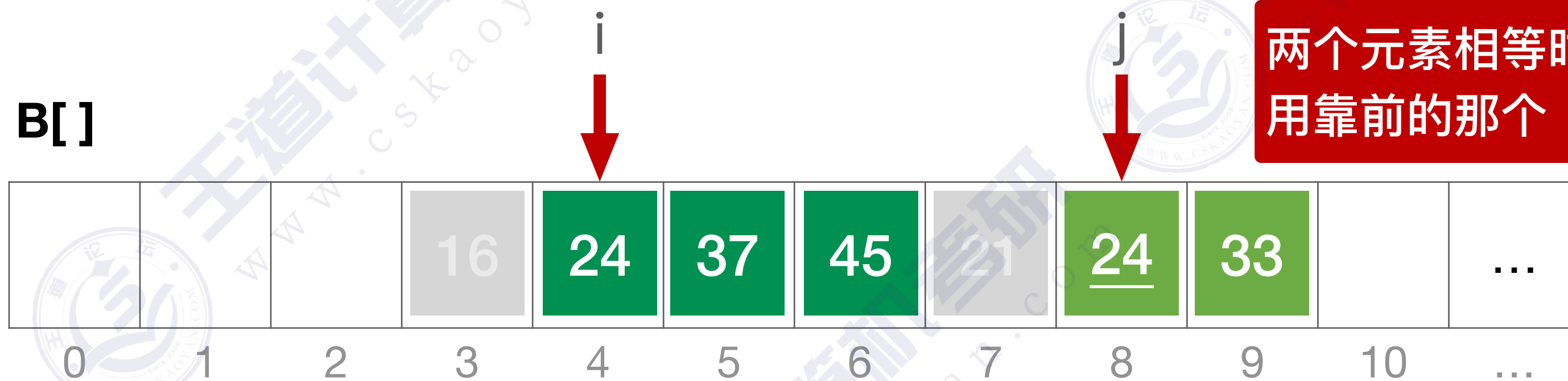
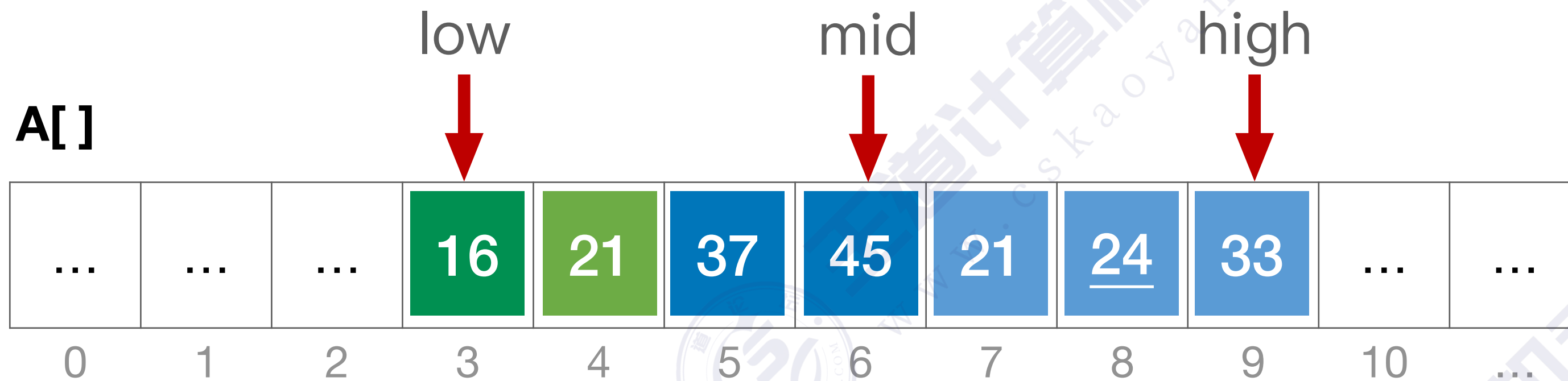


```
int *B=(int *)malloc(n*sizeof(int)); //辅助数组B

//A[low...mid]和A[mid+1...high]各自有序，将两个部分归并
void Merge(int A[],int low,int mid,int high){
    int i,j,k;
    for(k=low;k<=high;k++)
        B[k]=A[k]; //将A中所有元素复制到B中
    for(i=low,j=mid+1,k=i;i<=mid&&j<=high;k++){
        if(B[i]<=B[j])
            A[k]=B[i++]; //将较小值复制到A中
        else
            A[k]=B[j++];
    }//for
    while(i<=mid)    A[k++]=B[i++];
    while(j<=high)  A[k++]=B[j++];
}
```

归并

代码实现



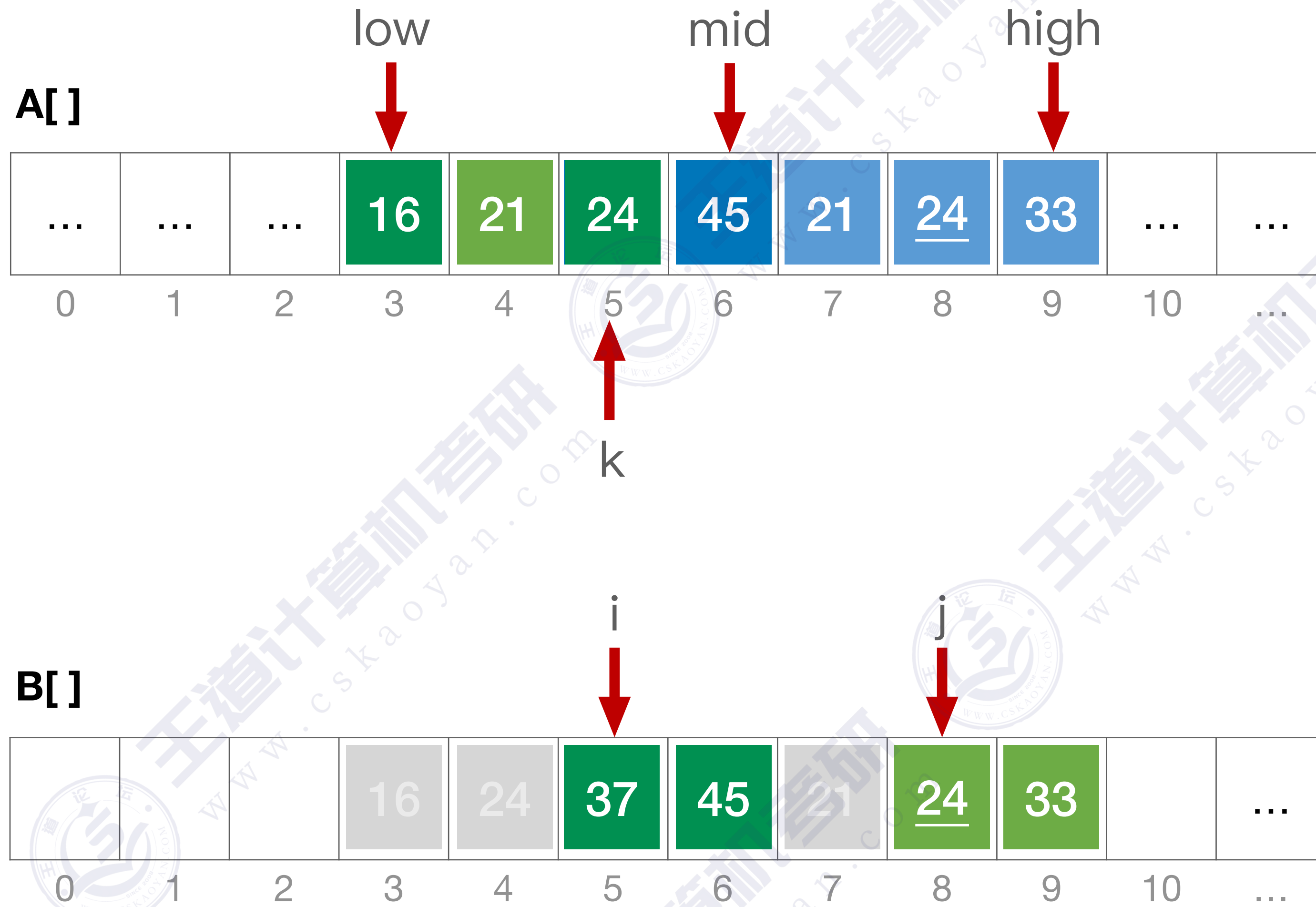
```
int *B=(int *)malloc(n*sizeof(int)); //辅助数组B

//A[low...mid]和A[mid+1...high]各自有序，将两个部分归并
void Merge(int A[],int low,int mid,int high){
    int i,j,k;
    for(k=low;k<=high;k++)
        B[k]=A[k]; //将A中所有元素复制到B中
    for(i=low,j=mid+1,k=i;i<=mid&&j<=high;k++){
        if(B[i]<=B[j])
            A[k]=B[i++]; //将较小值复制到A中
        else
            A[k]=B[j++];
    } //for
    while(i<=mid) A[k++]=B[i++];
    while(j<=high) A[k++]=B[j++];
}
```

归并

两个元素相等时，优先使用靠前的那个（稳定性）

代码实现

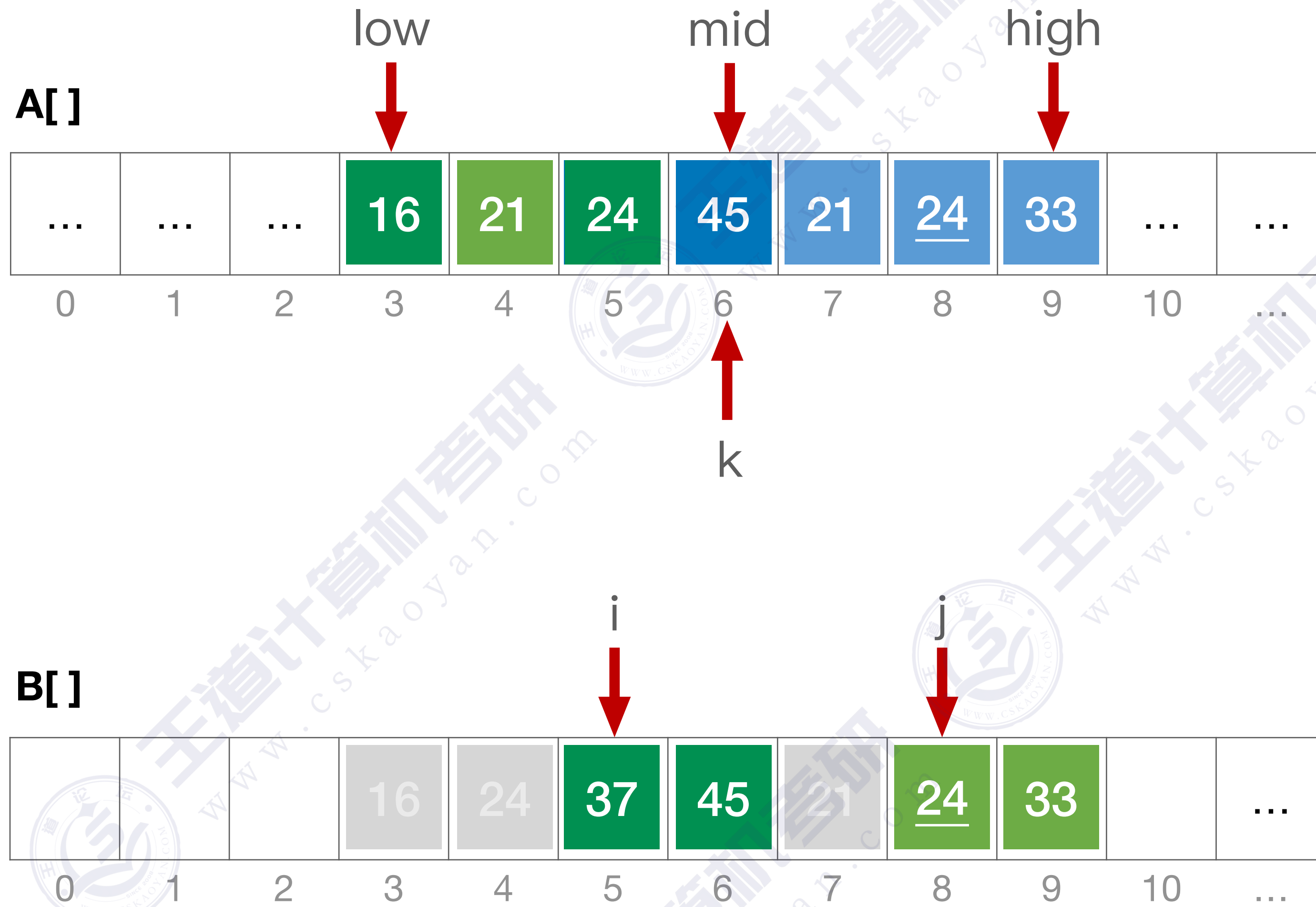


```
int *B=(int *)malloc(n*sizeof(int)); //辅助数组B

//A[low...mid]和A[mid+1...high]各自有序，将两个部分归并
void Merge(int A[],int low,int mid,int high){
    int i,j,k;
    for(k=low;k<=high;k++)
        B[k]=A[k]; //将A中所有元素复制到B中
    for(i=low,j=mid+1,k=i;i<=mid&&j<=high;k++){
        if(B[i]<=B[j])
            A[k]=B[i++]; //将较小值复制到A中
        else
            A[k]=B[j++];
    }//for
    while(i<=mid) A[k++]=B[i++];
    while(j<=high) A[k++]=B[j++];
}
```

归并

代码实现

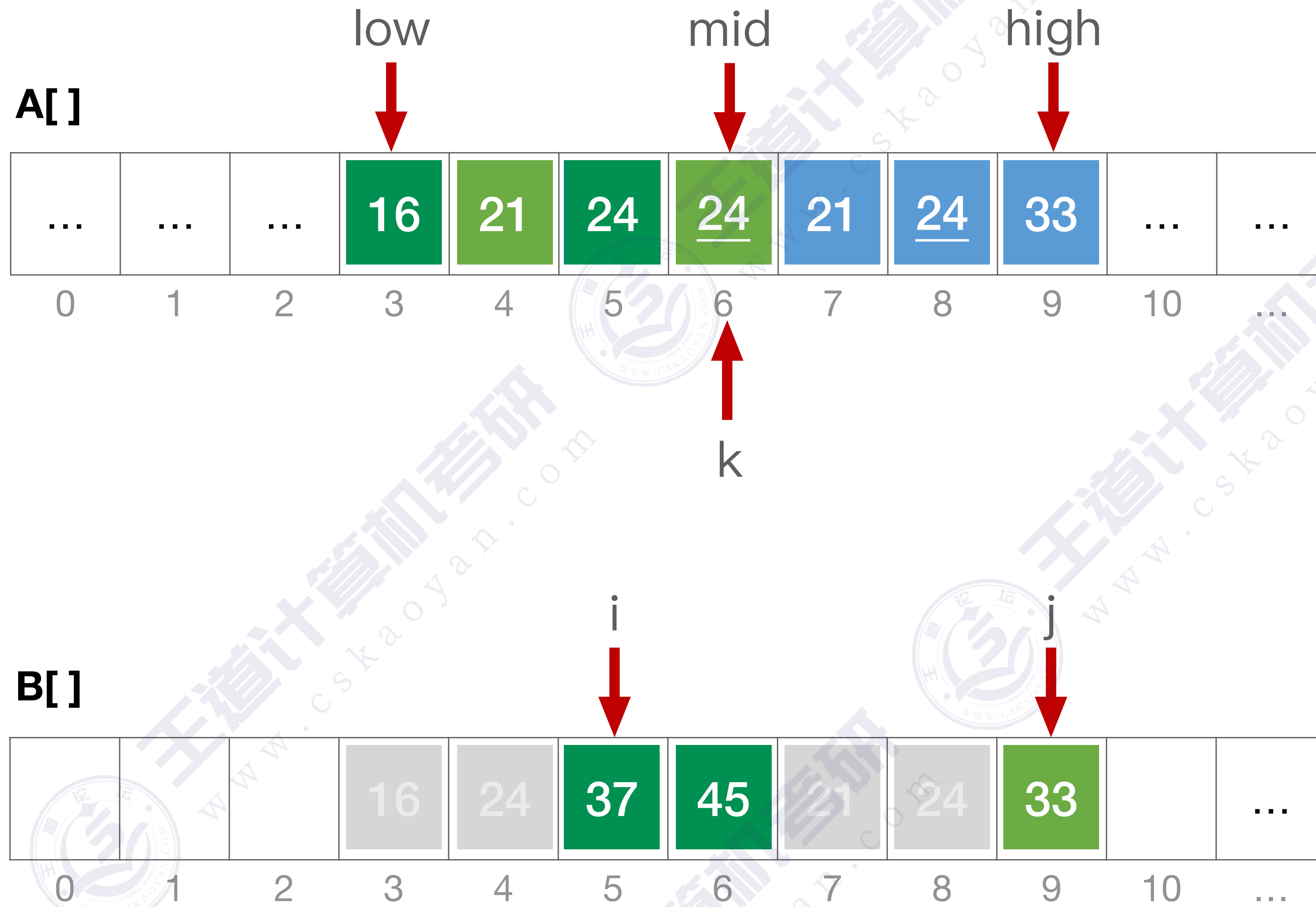


```
int *B=(int *)malloc(n*sizeof(int)); //辅助数组B

//A[low...mid]和A[mid+1...high]各自有序，将两个部分归并
void Merge(int A[],int low,int mid,int high){
    int i,j,k;
    for(k=low;k<=high;k++)
        B[k]=A[k]; //将A中所有元素复制到B中
    for(i=low,j=mid+1,k=i;i<=mid&&j<=high;k++){
        if(B[i]<=B[j])
            A[k]=B[i++]; //将较小值复制到A中
        else
            A[k]=B[j++];
    }//for
    while(i<=mid)    A[k++]=B[i++];
    while(j<=high)  A[k++]=B[j++];
}
```

归并

代码实现

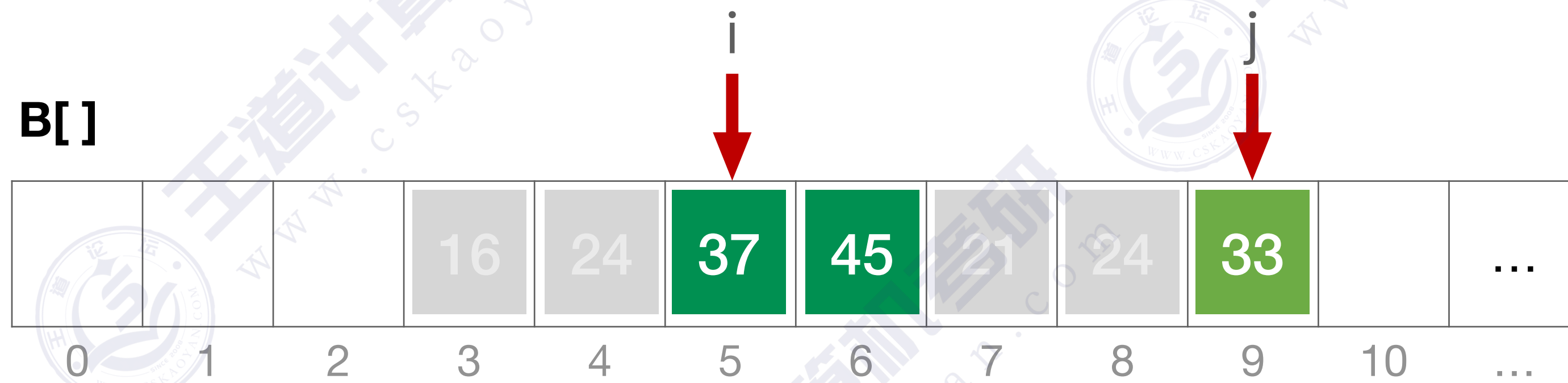
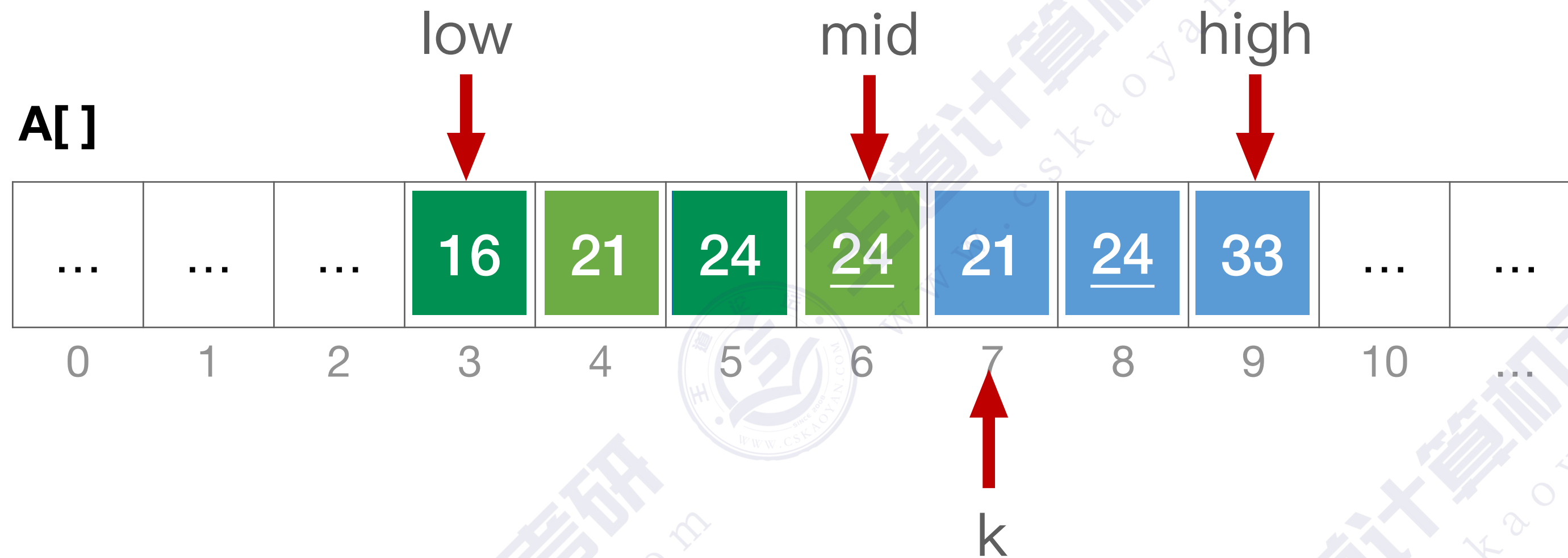


```
int *B=(int *)malloc(n*sizeof(int)); //辅助数组B

//A[low...mid]和A[mid+1...high]各自有序，将两个部分归并
void Merge(int A[],int low,int mid,int high){
    int i,j,k;
    for(k=low;k<=high;k++)
        B[k]=A[k]; //将A中所有元素复制到B中
    for(i=low,j=mid+1,k=i;i<=mid&&j<=high;k++){
        if(B[i]<=B[j])
            A[k]=B[i++]; //将较小值复制到A中
        else
            A[k]=B[j++];
    }//for
    while(i<=mid)    A[k++]=B[i++];
    while(j<=high)  A[k++]=B[j++];
}
```

归并

代码实现



```
int *B=(int *)malloc(n*sizeof(int)); //辅助数组B
```

```
//A[low...mid]和A[mid+1...high]各自有序，将两个部分归并
```

```
void Merge(int A[],int low,int mid,int high){
```

```
    int i,j,k;
```

```
    for(k=low;k<=high;k++){
```

```
        B[k]=A[k]; //将A中所有元素复制到B中
```

归并

```
    for(i=low,j=mid+1,k=i;i<=mid&&j<=high;k++){
```

```
        if(B[i]<=B[j])
```

```
            A[k]=B[i++]; //将较小值复制到A中
```

```
        else
```

```
            A[k]=B[j++];
```

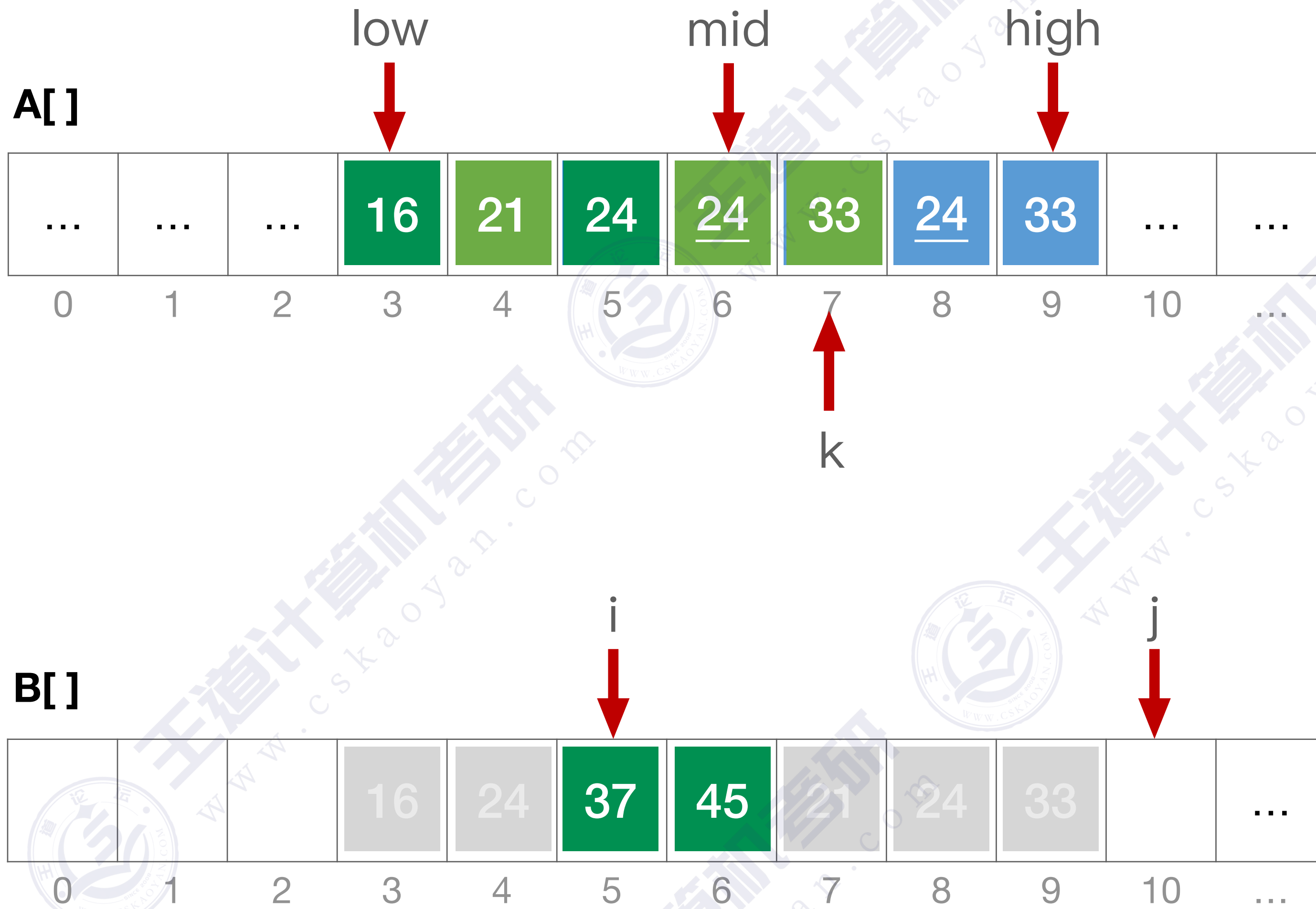
```
    }//for
```

```
    while(i<=mid) A[k++]=B[i++];
```

```
    while(j<=high) A[k++]=B[j++];
```

```
}
```


代码实现

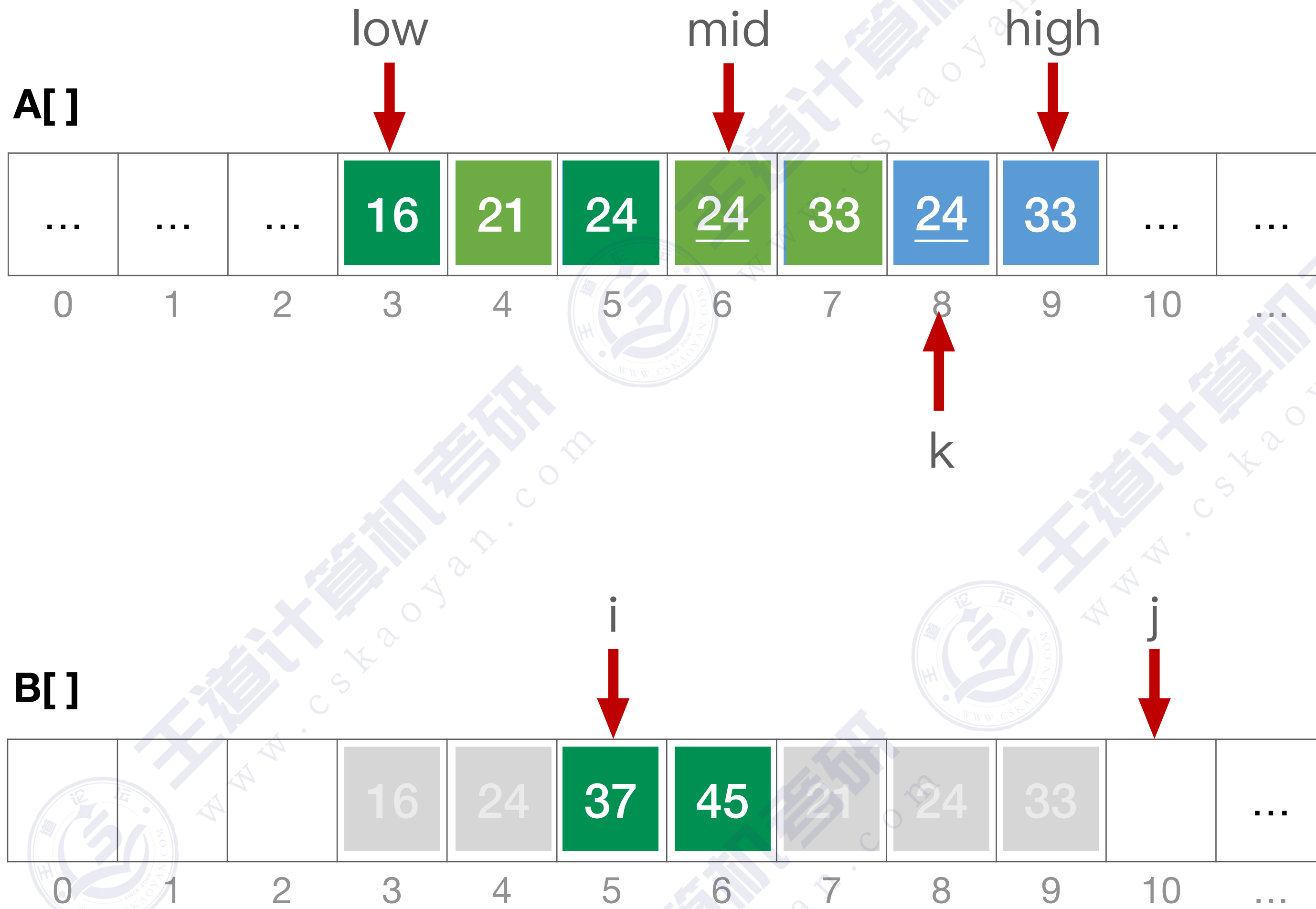


```
int *B=(int *)malloc(n*sizeof(int)); //辅助数组B

//A[low...mid]和A[mid+1...high]各自有序，将两个部分归并
void Merge(int A[],int low,int mid,int high){
    int i,j,k;
    for(k=low;k<=high;k++)
        B[k]=A[k]; //将A中所有元素复制到B中
    for(i=low,j=mid+1,k=i;i<=mid&&j<=high;k++){
        if(B[i]<=B[j])
            A[k]=B[i++]; //将较小值复制到A中
        else
            A[k]=B[j++];
    }//for
    while(i<=mid)    A[k++]=B[i++];
    while(j<=high)  A[k++]=B[j++];
}
```

归并

代码实现



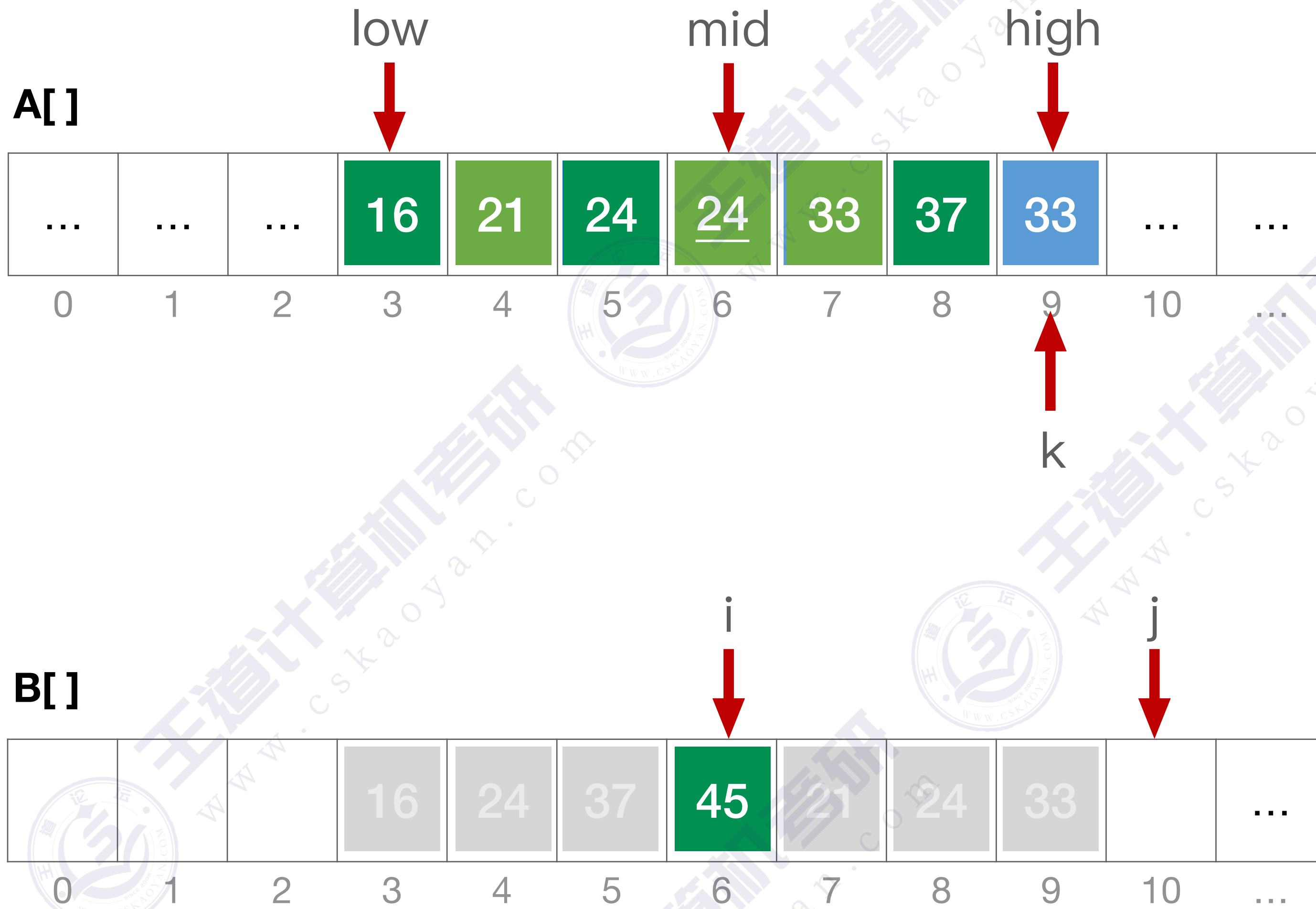
```
int *B=(int *)malloc(n*sizeof(int)); //辅助数组B

//A[low...mid]和A[mid+1...high]各自有序，将两个部分归并
void Merge(int A[],int low,int mid,int high){
    int i,j,k;
    for(k=low;k<=high;k++)
        B[k]=A[k]; //将A中所有元素复制到B中
    for(i=low,j=mid+1,k=i;i<=mid&&j<=high;k++){
        if(B[i]<=B[j])
            A[k]=B[i++]; //将较小值复制到A中
        else
            A[k]=B[j++];
    }//for
    while(i<=mid) A[k++]=B[i++];
    while(j<=high) A[k++]=B[j++];
}
```

归并

没有归并完的部分复制到尾部

代码实现



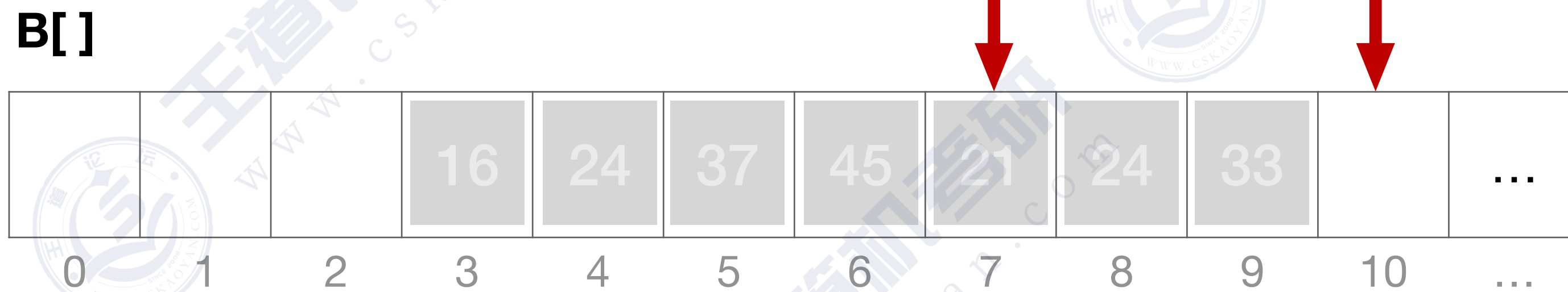
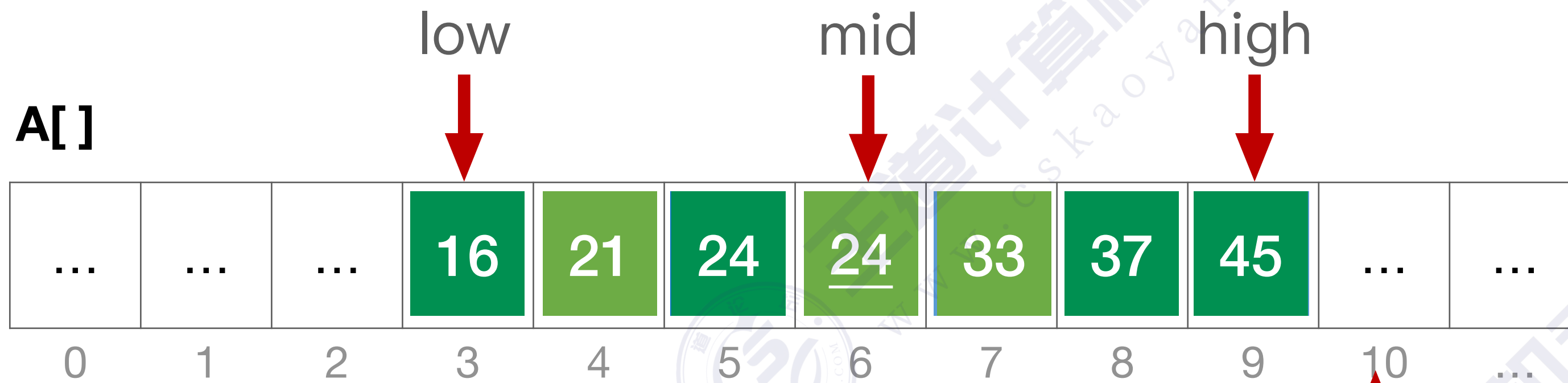
```
int *B=(int *)malloc(n*sizeof(int)); //辅助数组B

//A[low...mid]和A[mid+1...high]各自有序，将两个部分归并
void Merge(int A[],int low,int mid,int high){
    int i,j,k;
    for(k=low;k<=high;k++)
        B[k]=A[k]; //将A中所有元素复制到B中
    for(i=low,j=mid+1,k=i;i<=mid&&j<=high;k++){
        if(B[i]<=B[j])
            A[k]=B[i++]; //将较小值复制到A中
        else
            A[k]=B[j++];
    }//for
    while(i<=mid) A[k++]=B[i++];
    while(j<=high) A[k++]=B[j++];
}
```

归并

没有归并完的部分复制到尾部

代码实现



```
int *B=(int *)malloc(n*sizeof(int)); //辅助数组B
```

```
//A[low...mid]和A[mid+1...high]各自有序，将两个部分归并
```

```
void Merge(int A[],int low,int mid,int high){
```

```
    int i,j,k;
```

```
    for(k=low;k<=high;k++)
```

```
        B[k]=A[k]; //将A中所有元素复制到B中
```

```
    for(i=low,j=mid+1,k=i;i<=mid&&j<=high;k++){
```

```
        if(B[i]<=B[j])
```

```
            A[k]=B[i++]; //将较小值复制到A中
```

```
        else
```

```
            A[k]=B[j++];
```

```
    }//for
```

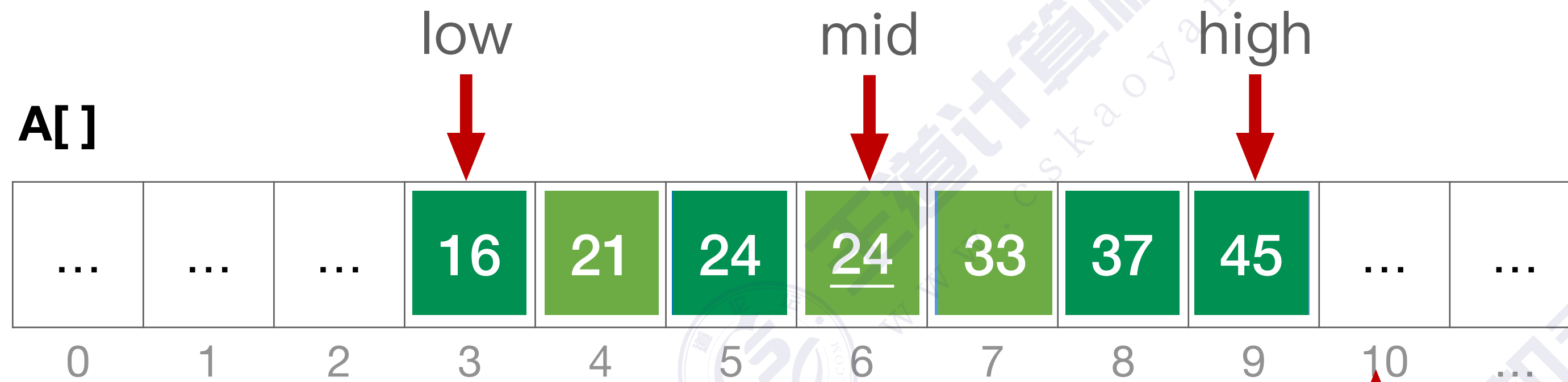
```
    while(i<=mid) A[k++]=B[i++];
```

```
    while(j<=high) A[k++]=B[j++];
```

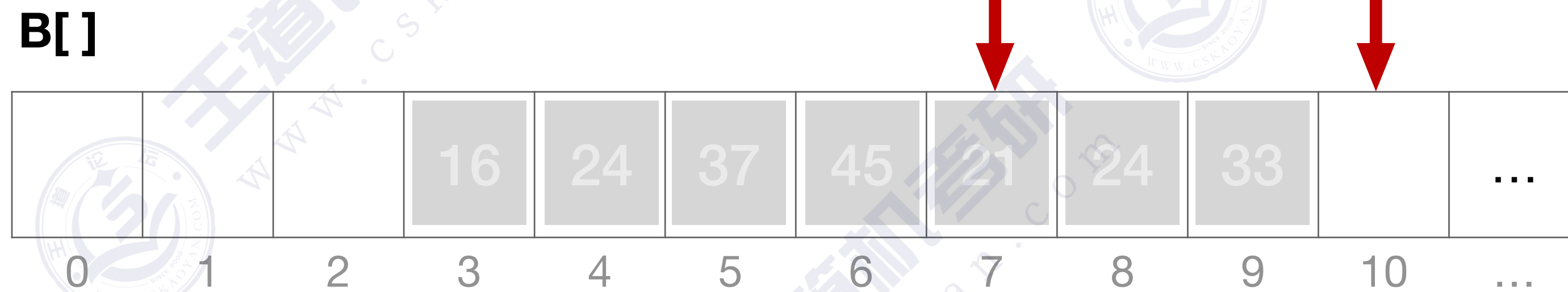
归并

没有归并完的部分复制到尾部

代码实现



经过 Merge 处理,
A[low~high]整体有序



```
int *B=(int *)malloc(n*sizeof(int)); //辅助数组B
```

```
//A[low...mid]和A[mid+1...high]各自有序, 将两个部分归并
```

```
void Merge(int A[],int low,int mid,int high){
```

```
int i,j,k;
```

```
for(k=low;k<=high;k++)
```

```
    B[k]=A[k]; //将A中所有元素复制到B中
```

```
for(i=low,j=mid+1,k=i;i<=mid&&j<=high;k++){
```

```
    if(B[i]<=B[j])
```

```
        A[k]=B[i++]; //将较小值复制到A中
```

```
    else
```

```
        A[k]=B[j++];
```

```
}//for
```

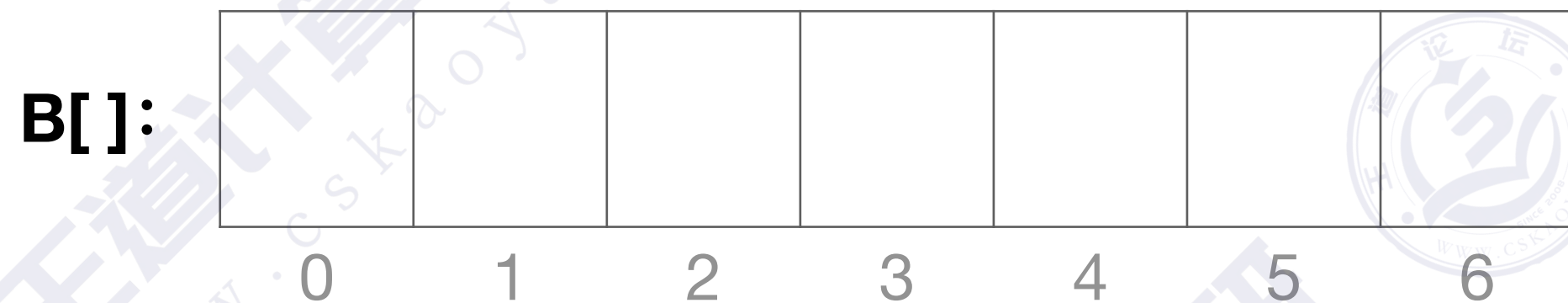
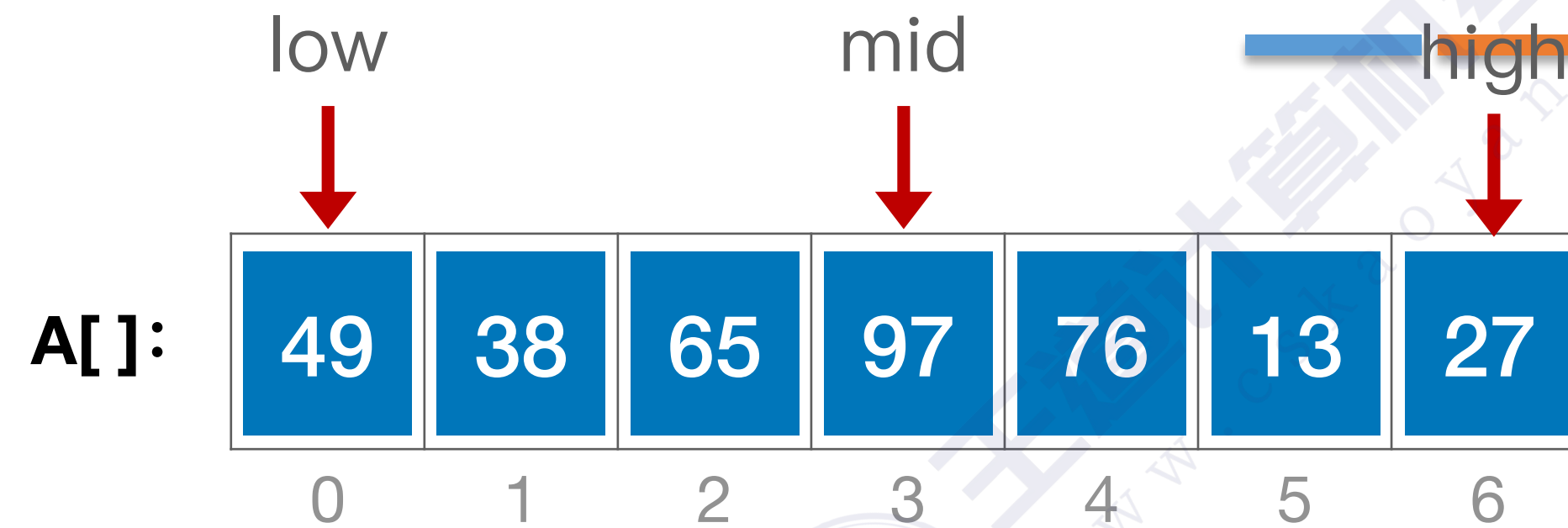
```
while(i<=mid) A[k++]=B[i++];
```

```
while(j<=high) A[k++]=B[j++];
```

```
}
```

归并

代码实现



```
int *B=(int *)malloc(n*sizeof(int)); //辅助数组B
```

```
//A[low...mid]和A[mid+1...high]各自有序，将两个部分归并
```

```
void Merge(int A[],int low,int mid,int high){
```

```
int i,j,k;
```

```
for(k=low;k<=high;k++)
```

```
    B[k]=A[k]; //将A中所有元素复制到B中
```

```
for(i=low,j=mid+1,k=i;i<=mid&&j<=high;k++){
```

```
    if(B[i]<=B[j])
```

```
        A[k]=B[i++]; //将较小值复制到A中
```

```
    else
```

```
        A[k]=B[j++];
```

```
}//for
```

```
while(i<=mid) A[k++]=B[i++];
```

```
while(j<=high) A[k++]=B[j++];
```

```
}
```

```
void MergeSort(int A[],int low,int high){
```

```
if(low<high){
```

```
    int mid=(low+high)/2; //从中间划分
```

```
    MergeSort(A,low,mid); //对左半部分归并排序
```

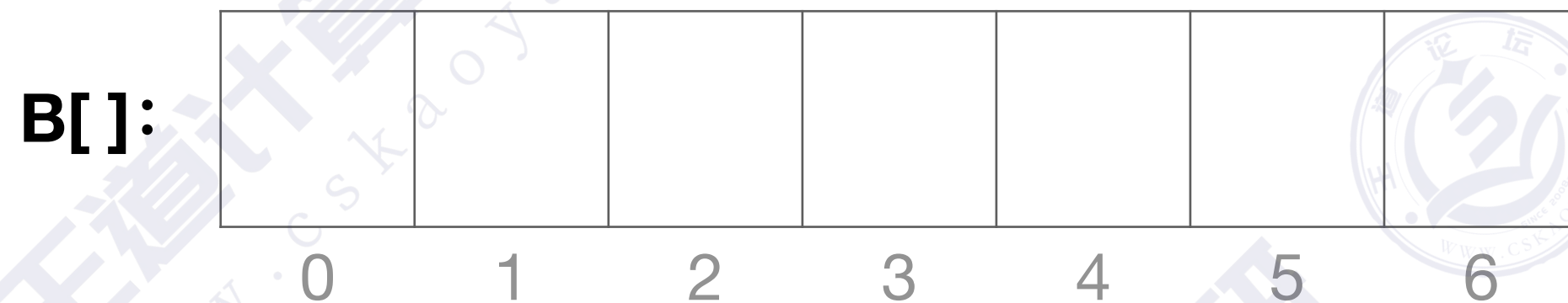
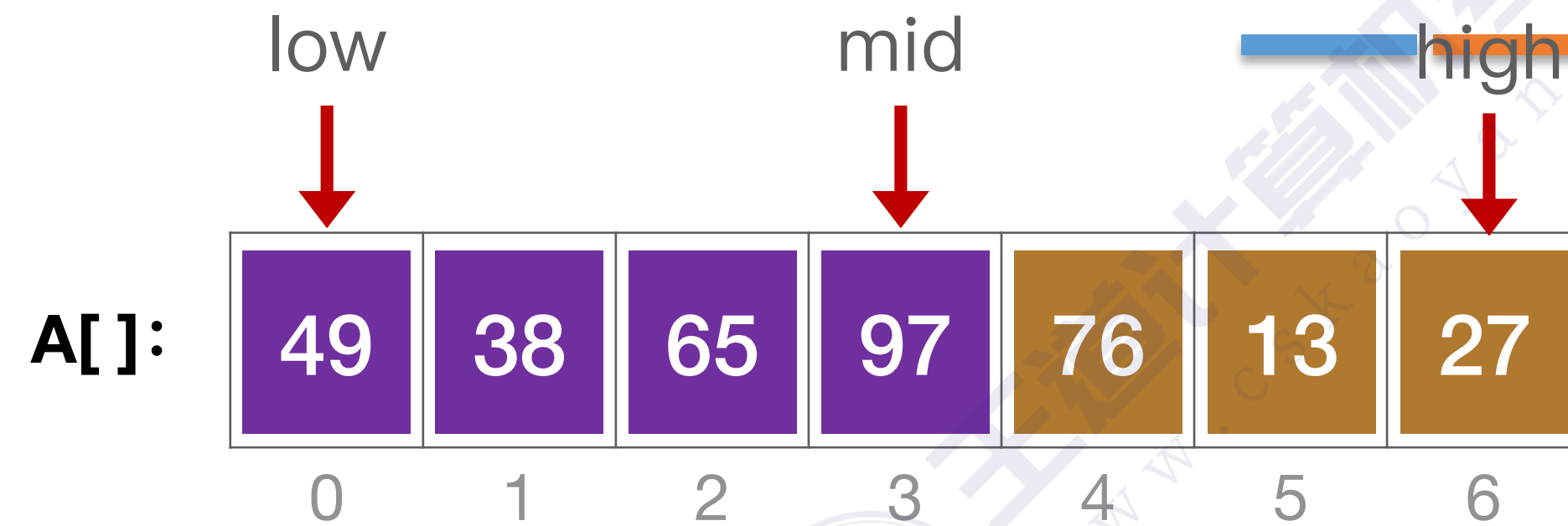
```
    MergeSort(A,mid+1,high); //对右半部分归并排序
```

```
    Merge(A,low,mid,high); //归并
```

```
}//if
```

```
}
```


代码实现



```
int *B=(int *)malloc(n*sizeof(int)); //辅助数组B
```

```
//A[low...mid]和A[mid+1...high]各自有序，将两个部分归并
```

```
void Merge(int A[],int low,int mid,int high){
```

```
int i,j,k;
```

```
for(k=low;k<=high;k++)
```

```
    B[k]=A[k]; //将A中所有元素复制到B中
```

```
for(i=low,j=mid+1,k=i;i<=mid&&j<=high;k++){
```

```
    if(B[i]<=B[j])
```

```
        A[k]=B[i++]; //将较小值复制到A中
```

```
    else
```

```
        A[k]=B[j++];
```

```
}//for
```

```
while(i<=mid) A[k++]=B[i++];
```

```
while(j<=high) A[k++]=B[j++];
```

```
}
```

```
void MergeSort(int A[],int low,int high){
```

```
if(low<high){
```

```
    int mid=(low+high)/2; //从中间划分
```

```
    MergeSort(A,low,mid); //对左半部分归并排序
```

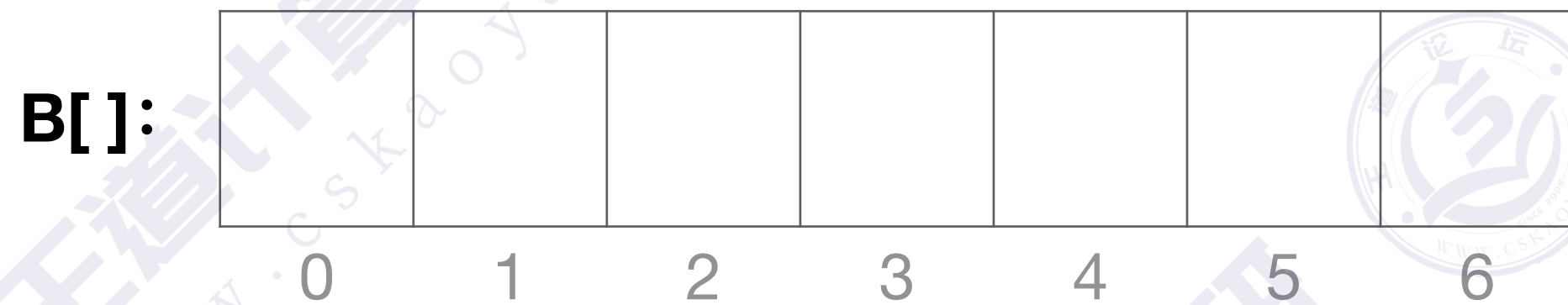
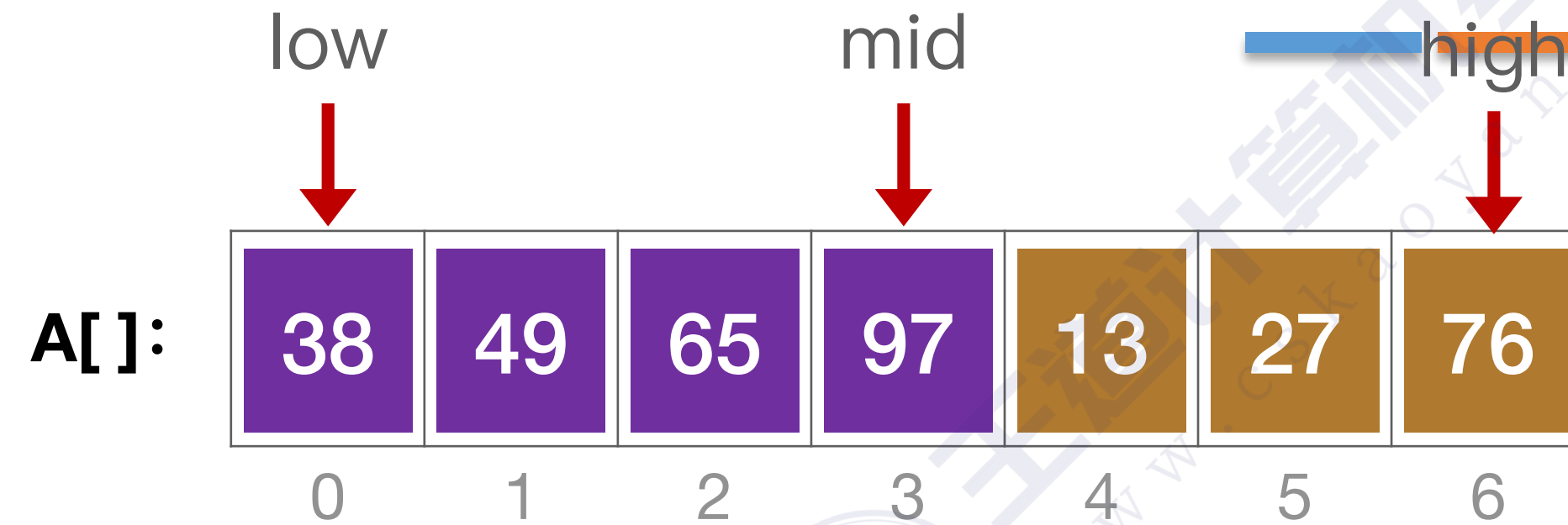
```
    MergeSort(A,mid+1,high); //对右半部分归并排序
```

```
    Merge(A,low,mid,high); //归并
```

```
}//if
```

```
}
```


代码实现



将左右两个子序列分别进行归并排序

```
int *B=(int *)malloc(n*sizeof(int)); //辅助数组B
```

```
//A[low...mid]和A[mid+1...high]各自有序，将两个部分归并
```

```
void Merge(int A[],int low,int mid,int high){
```

```
int i,j,k;
```

```
for(k=low;k<=high;k++)
```

```
B[k]=A[k]; //将A中所有元素复制到B中
```

```
for(i=low,j=mid+1,k=i;i<=mid&&j<=high;k++){
```

```
if(B[i]<=B[j])
```

```
A[k]=B[i++]; //将较小值复制到A中
```

```
else
```

```
A[k]=B[j++];
```

```
}//for
```

```
while(i<=mid) A[k++]=B[i++];
```

```
while(j<=high) A[k++]=B[j++];
```

```
}
```

```
void MergeSort(int A[],int low,int high){
```

```
if(low<high){
```

```
int mid=(low+high)/2; //从中间划分
```

```
MergeSort(A,low,mid); //对左半部分归并排序
```

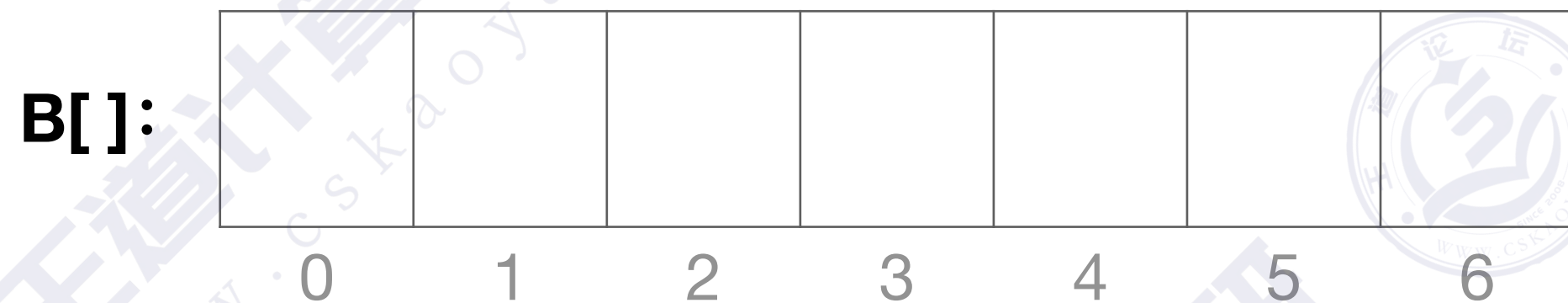
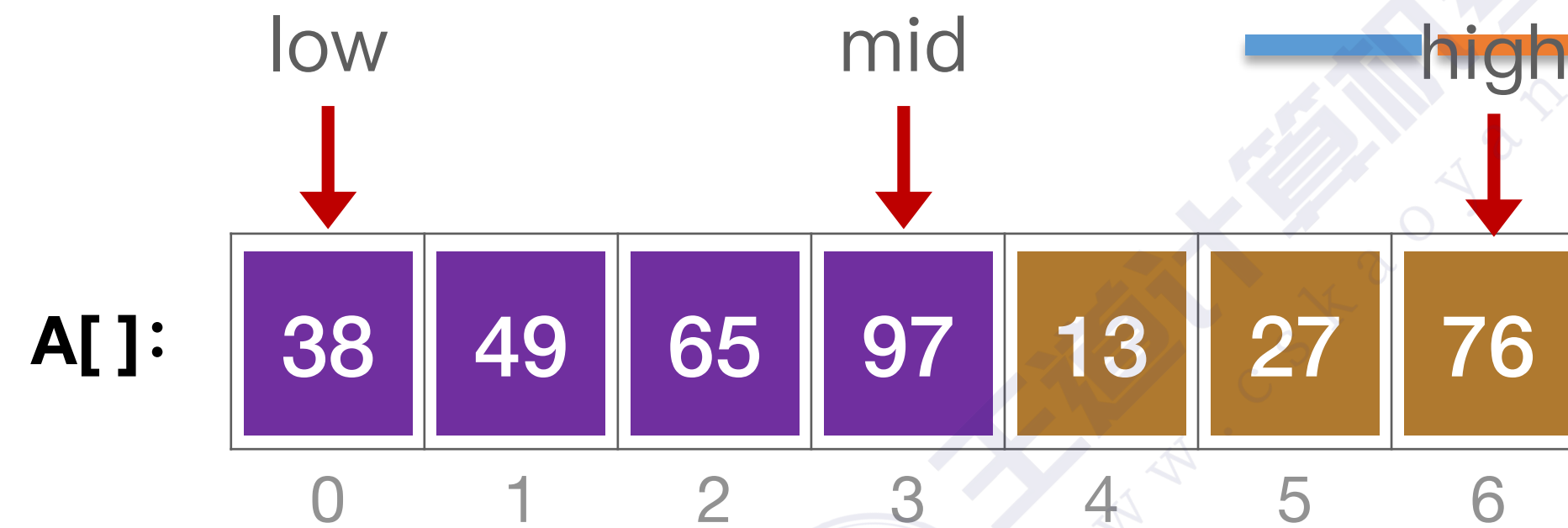
```
MergeSort(A,mid+1,high); //对右半部分归并排序
```

```
Merge(A,low,mid,high); //归并
```

```
}//if
```

```
}
```

代码实现



左右两个子序列分别有序之后再将二者归并

```
int *B=(int *)malloc(n*sizeof(int)); //辅助数组B
```

```
//A[low...mid]和A[mid+1...high]各自有序，将两个部分归并
```

```
void Merge(int A[],int low,int mid,int high){
```

```
int i,j,k;
```

```
for(k=low;k<=high;k++)
```

```
B[k]=A[k]; //将A中所有元素复制到B中
```

```
for(i=low,j=mid+1,k=i;i<=mid&&j<=high;k++){
```

```
if(B[i]<=B[j])
```

```
A[k]=B[i++]; //将较小值复制到A中
```

```
else
```

```
A[k]=B[j++];
```

```
}//for
```

```
while(i<=mid) A[k++]=B[i++];
```

```
while(j<=high) A[k++]=B[j++];
```

```
}
```

```
void MergeSort(int A[],int low,int high){
```

```
if(low<high){
```

```
int mid=(low+high)/2; //从中间划分
```

```
MergeSort(A,low,mid); //对左半部分归并排序
```

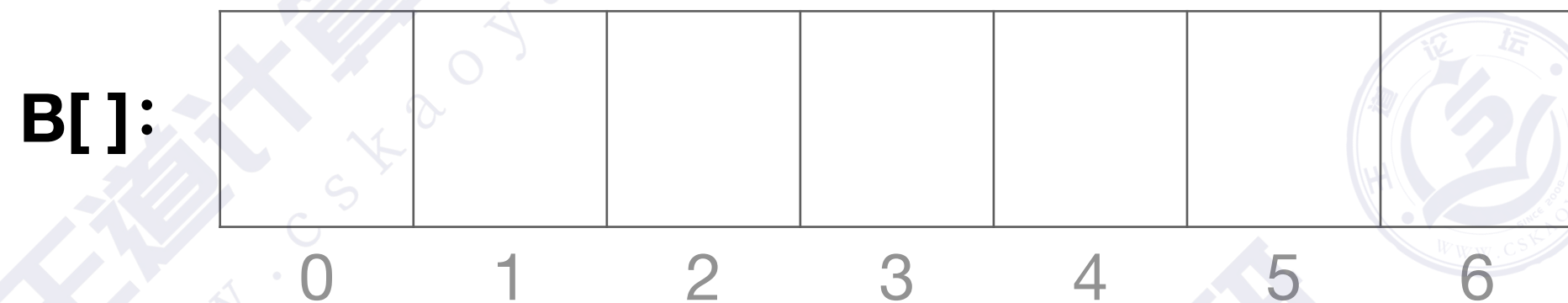
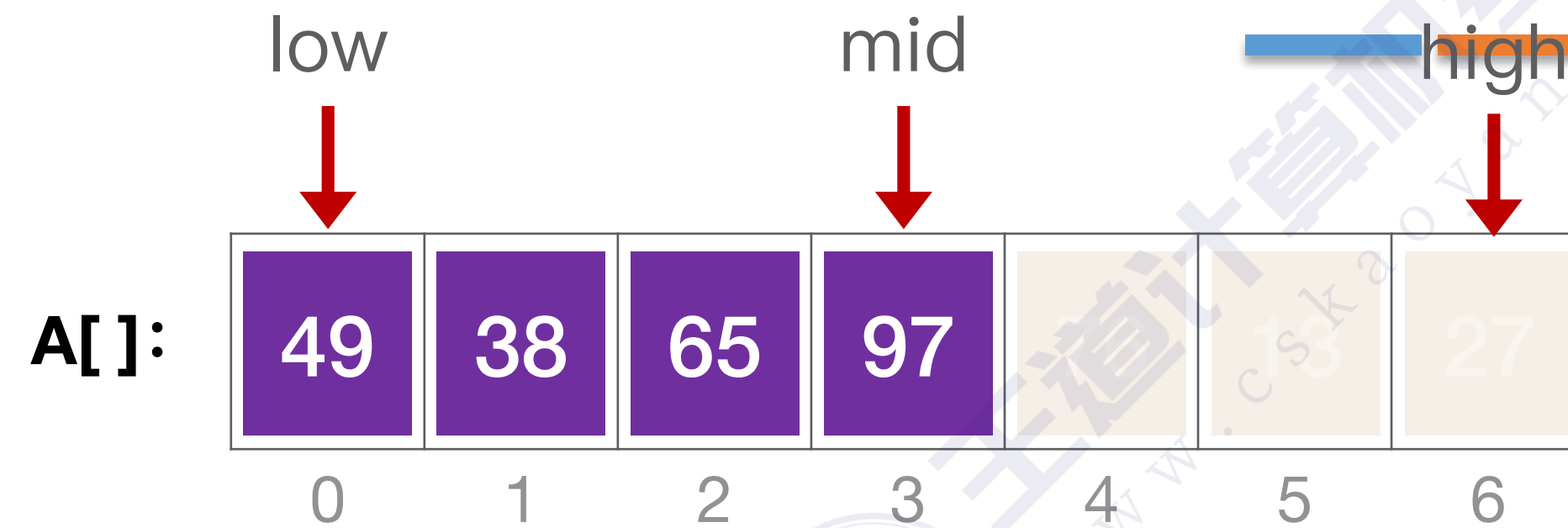
```
MergeSort(A,mid+1,high); //对右半部分归并排序
```

```
Merge(A,low,mid,high); //归并
```

```
}//if
```

```
}
```


代码实现



```
int *B=(int *)malloc(n*sizeof(int)); //辅助数组B
```

```
//A[low...mid]和A[mid+1...high]各自有序，将两个部分归并
```

```
void Merge(int A[],int low,int mid,int high){
```

```
    int i,j,k;
```

```
    for(k=low;k<=high;k++)
```

```
        B[k]=A[k]; //将A中所有元素复制到B中
```

```
    for(i=low,j=mid+1,k=i;i<=mid&&j<=high;k++){
```

```
        if(B[i]<=B[j])
```

```
            A[k]=B[i++]; //将较小值复制到A中
```

```
        else
```

```
            A[k]=B[j++];
```

```
    }//for
```

```
    while(i<=mid) A[k++]=B[i++];
```

```
    while(j<=high) A[k++]=B[j++];
```

```
}
```

```
void MergeSort(int A[],int low,int high){
```

```
    if(low<high){
```

```
        int mid=(low+high)/2; //从中间划分
```

```
        MergeSort(A,low,mid); //对左半部分归并排序
```

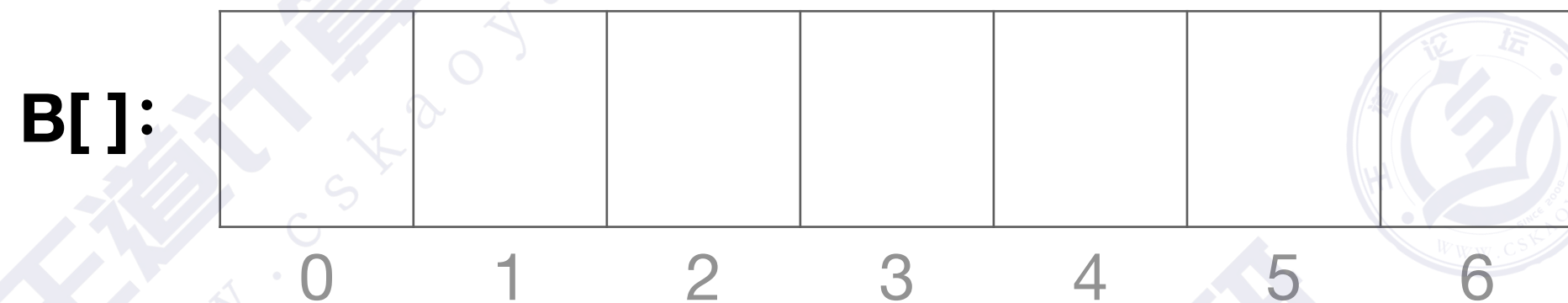
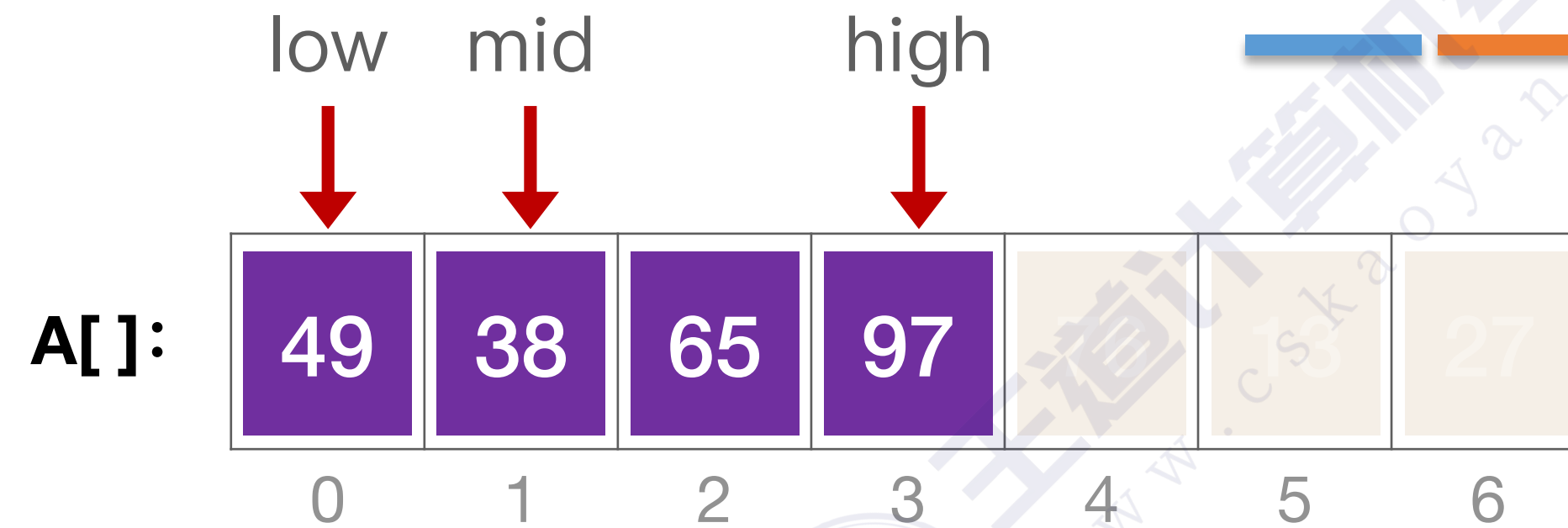
```
        MergeSort(A,mid+1,high); //对右半部分归并排序
```

```
        Merge(A,low,mid,high); //归并
```

```
    }//if
```

```
}
```


代码实现



```
int *B=(int *)malloc(n*sizeof(int)); //辅助数组B
```

```
//A[low...mid]和A[mid+1...high]各自有序，将两个部分归并
```

```
void Merge(int A[],int low,int mid,int high){
```

```
int i,j,k;
```

```
for(k=low;k<=high;k++)
```

```
B[k]=A[k]; //将A中所有元素复制到B中
```

```
for(i=low,j=mid+1,k=i;i<=mid&&j<=high;k++){
```

```
if(B[i]<=B[j])
```

```
A[k]=B[i++]; //将较小值复制到A中
```

```
else
```

```
A[k]=B[j++];
```

```
}//for
```

```
while(i<=mid) A[k++]=B[i++];
```

```
while(j<=high) A[k++]=B[j++];
```

```
}
```

```
void MergeSort(int A[],int low,int high){
```

```
if(low<high){
```

```
int mid=(low+high)/2; //从中间划分
```

```
MergeSort(A,low,mid); //对左半部分归并排序
```

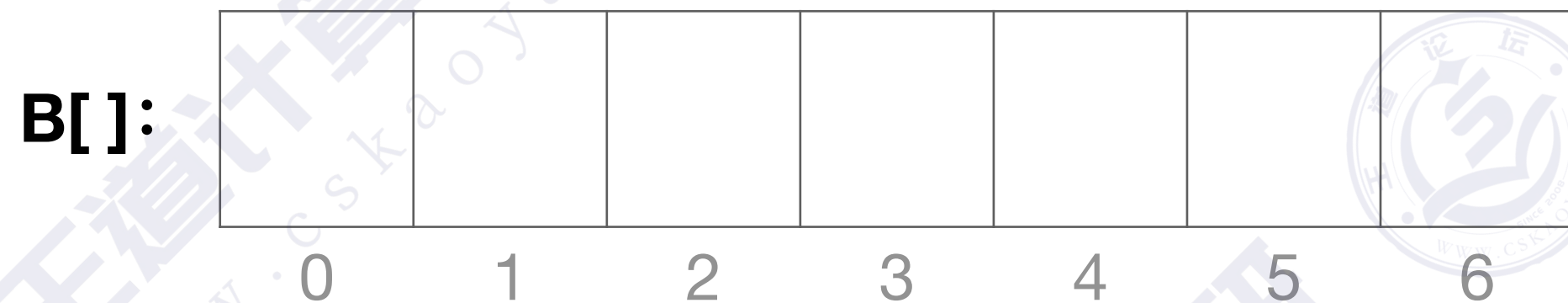
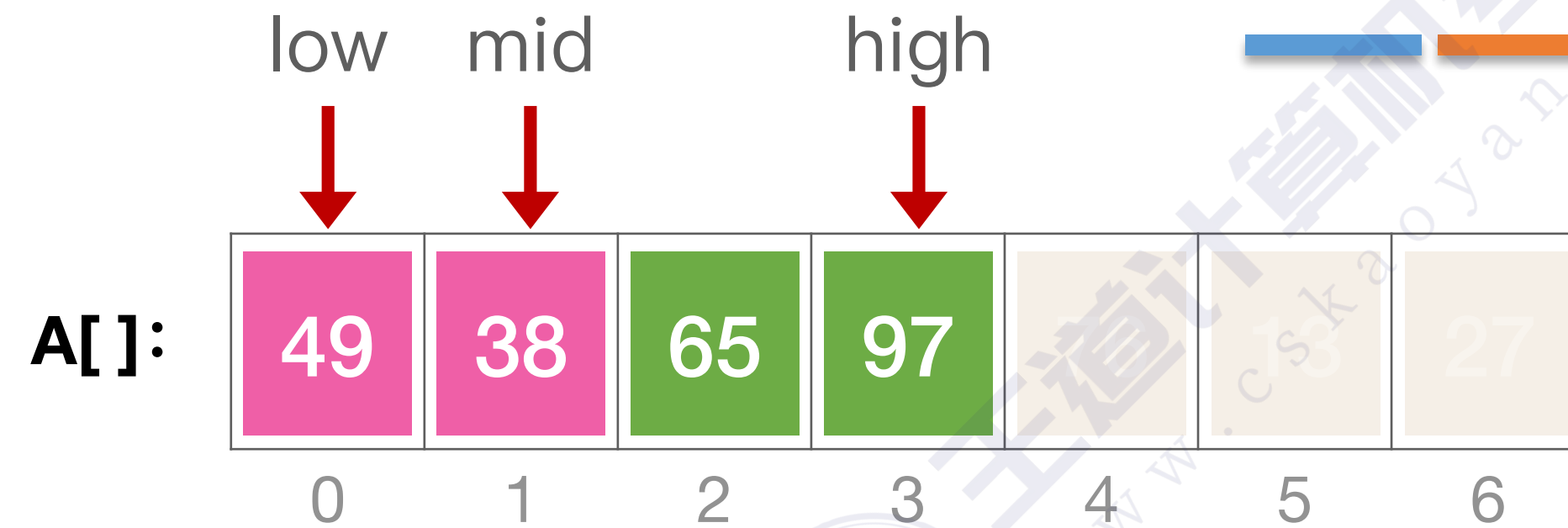
```
MergeSort(A,mid+1,high); //对右半部分归并排序
```

```
Merge(A,low,mid,high); //归并
```

```
}//if
```

```
}
```

代码实现



```
int *B=(int *)malloc(n*sizeof(int)); //辅助数组B
```

```
//A[low...mid]和A[mid+1...high]各自有序，将两个部分归并
```

```
void Merge(int A[],int low,int mid,int high){
```

```
int i,j,k;
```

```
for(k=low;k<=high;k++)
```

```
B[k]=A[k]; //将A中所有元素复制到B中
```

```
for(i=low,j=mid+1,k=i;i<=mid&&j<=high;k++){
```

```
if(B[i]<=B[j])
```

```
A[k]=B[i++]; //将较小值复制到A中
```

```
else
```

```
A[k]=B[j++];
```

```
}//for
```

```
while(i<=mid) A[k++]=B[i++];
```

```
while(j<=high) A[k++]=B[j++];
```

```
}
```

```
void MergeSort(int A[],int low,int high){
```

```
if(low<high){
```

```
int mid=(low+high)/2; //从中间划分
```

```
➡ MergeSort(A,low,mid); //对左半部分归并排序
```

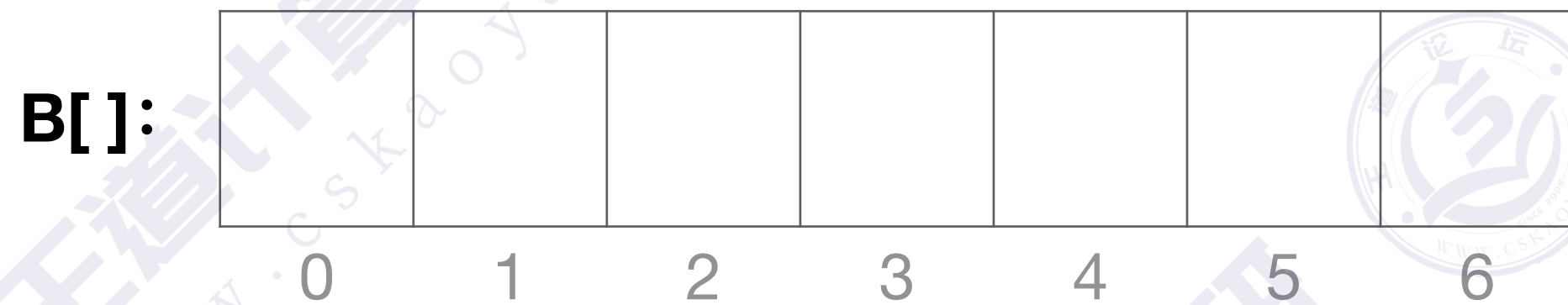
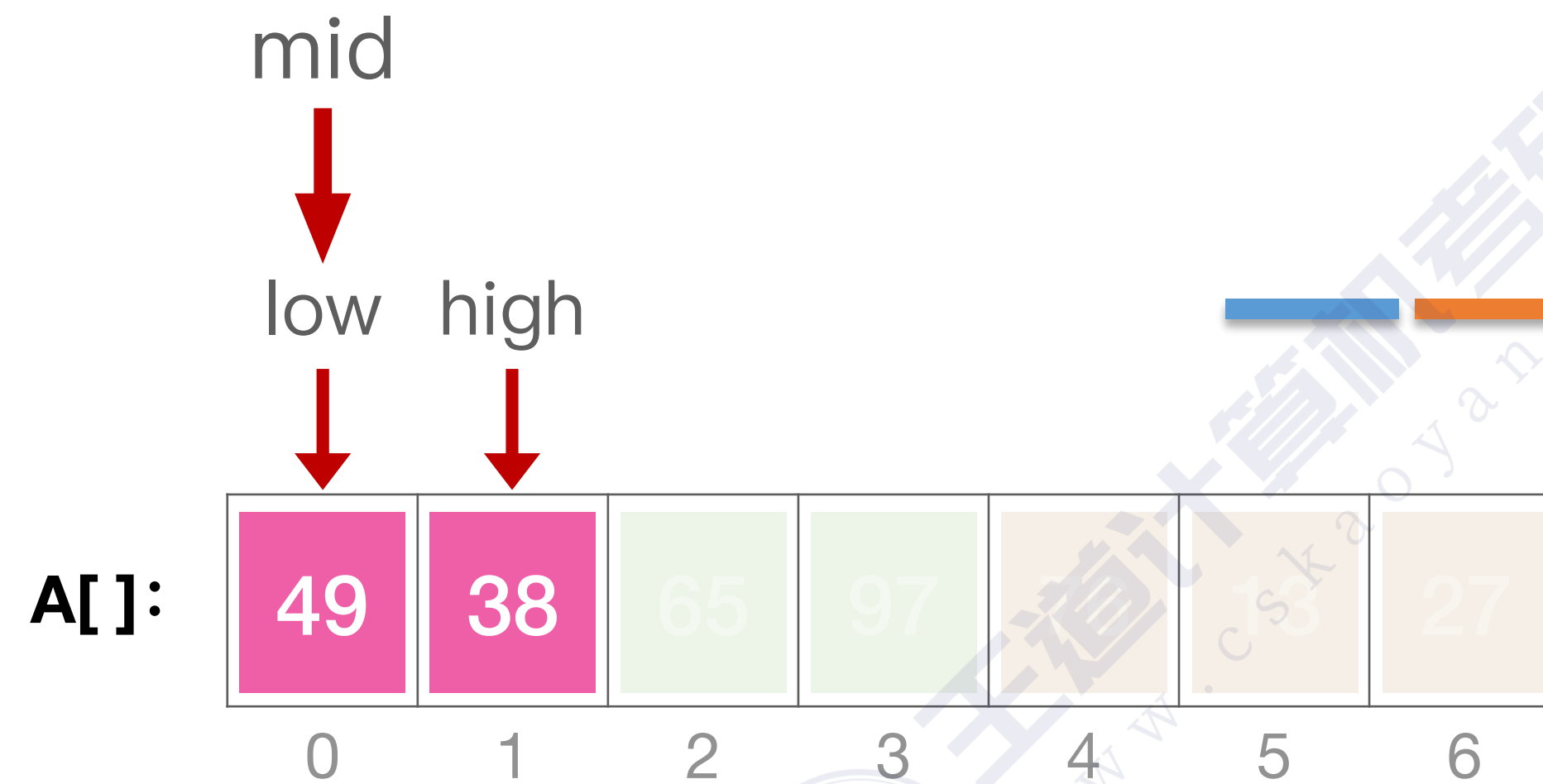
```
➡ MergeSort(A,mid+1,high); //对右半部分归并排序
```

```
Merge(A,low,mid,high); //归并
```

```
}//if
```

```
}
```


代码实现



```
int *B=(int *)malloc(n*sizeof(int)); //辅助数组B
```

```
//A[low...mid]和A[mid+1...high]各自有序，将两个部分归并
```

```
void Merge(int A[],int low,int mid,int high){
```

```
int i,j,k;
```

```
for(k=low;k<=high;k++)
```

```
    B[k]=A[k]; //将A中所有元素复制到B中
```

```
for(i=low,j=mid+1,k=i;i<=mid&&j<=high;k++){
```

```
    if(B[i]<=B[j])
```

```
        A[k]=B[i++]; //将较小值复制到A中
```

```
    else
```

```
        A[k]=B[j++];
```

```
}//for
```

```
while(i<=mid) A[k++]=B[i++];
```

```
while(j<=high) A[k++]=B[j++];
```

```
}
```

```
void MergeSort(int A[],int low,int high){
```

```
if(low<high){
```

```
    int mid=(low+high)/2; //从中间划分
```

```
    MergeSort(A,low,mid); //对左半部分归并排序
```

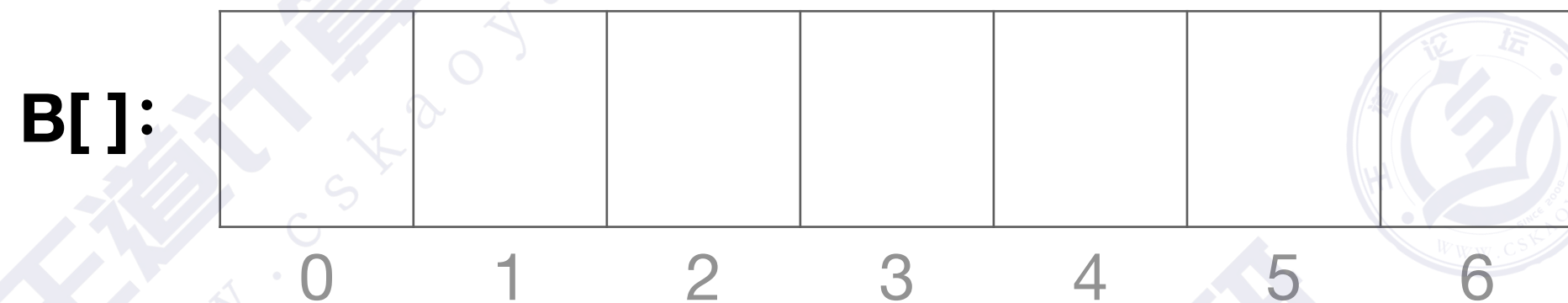
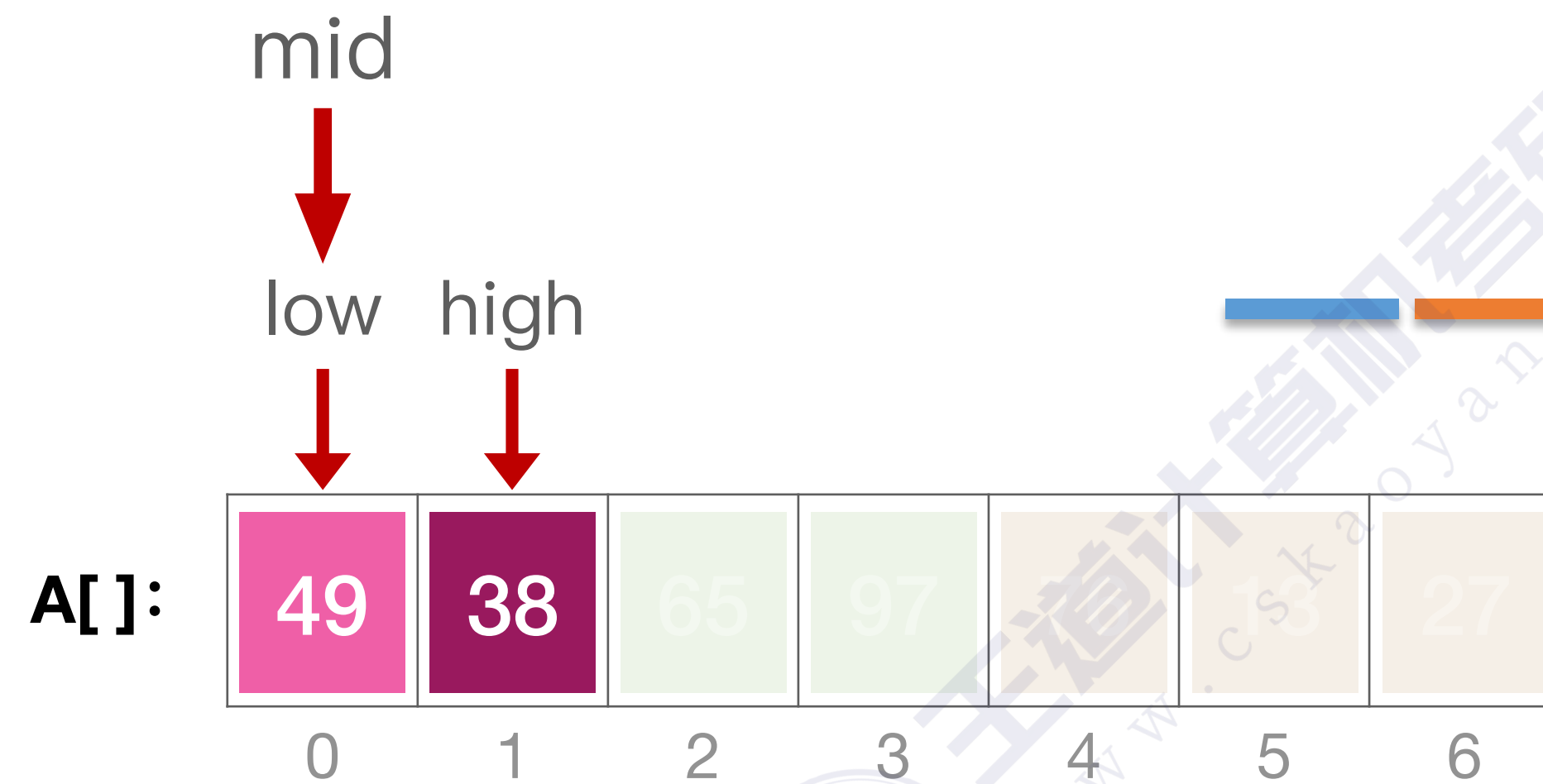
```
    MergeSort(A,mid+1,high); //对右半部分归并排序
```

```
    Merge(A,low,mid,high); //归并
```

```
}//if
```

```
}
```


代码实现



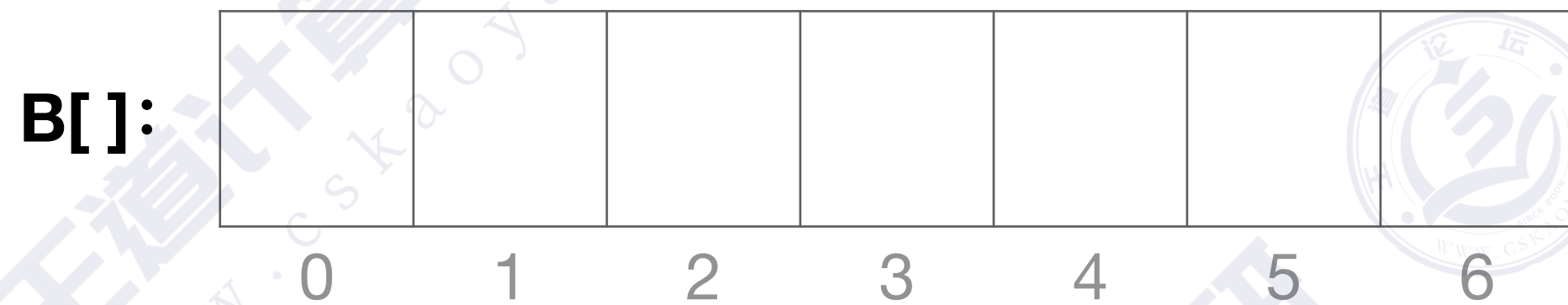
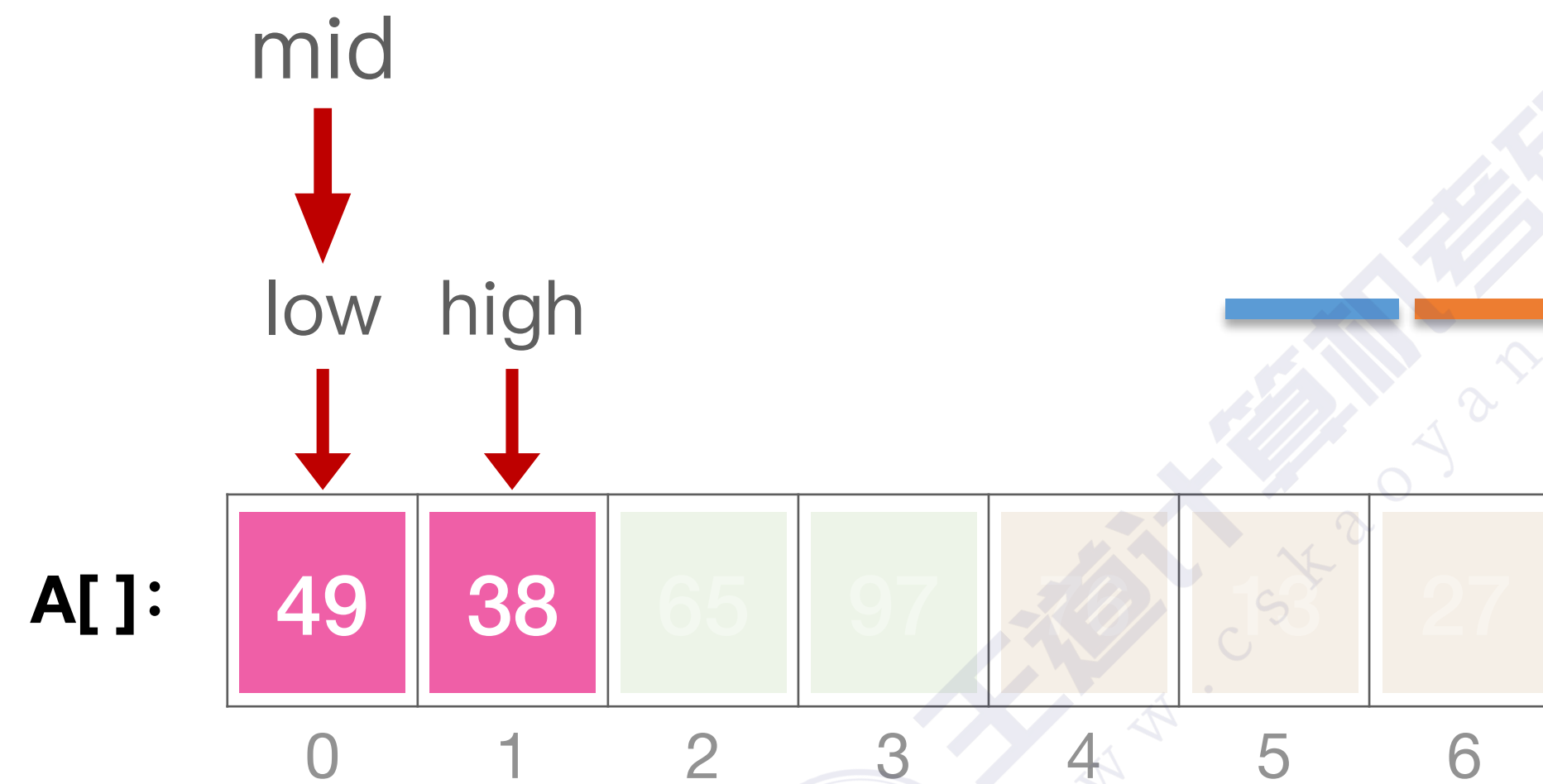
禁止套娃

将左右两个子序列分别进行归并排序（每个子序列只含有1个元素）

```
int *B=(int *)malloc(n*sizeof(int)); //辅助数组B
//A[low...mid]和A[mid+1...high]各自有序，将两个部分归并
void Merge(int A[],int low,int mid,int high){
    int i,j,k;
    for(k=low;k<=high;k++)
        B[k]=A[k]; //将A中所有元素复制到B中
    for(i=low,j=mid+1,k=i;i<=mid&&j<=high;k++){
        if(B[i]<=B[j])
            A[k]=B[i++]; //将较小值复制到A中
        else
            A[k]=B[j++];
    }//for
    while(i<=mid) A[k++]=B[i++];
    while(j<=high) A[k++]=B[j++];
}

void MergeSort(int A[],int low,int high){
    if(low<high){
        int mid=(low+high)/2; //从中间划分
        MergeSort(A,low,mid); //对左半部分归并排序
        MergeSort(A,mid+1,high); //对右半部分归并排序
        Merge(A,low,mid,high); //归并
    }//if
}
```

代码实现



```
int *B=(int *)malloc(n*sizeof(int)); //辅助数组B
```

```
//A[low...mid]和A[mid+1...high]各自有序，将两个部分归并
```

```
void Merge(int A[],int low,int mid,int high){
```

```
int i,j,k;
```

```
for(k=low;k<=high;k++)
```

```
    B[k]=A[k]; //将A中所有元素复制到B中
```

```
for(i=low,j=mid+1,k=i;i<=mid&&j<=high;k++){
```

```
    if(B[i]<=B[j])
```

```
        A[k]=B[i++]; //将较小值复制到A中
```

```
    else
```

```
        A[k]=B[j++];
```

```
}//for
```

```
while(i<=mid) A[k++]=B[i++];
```

```
while(j<=high) A[k++]=B[j++];
```

```
}
```

```
void MergeSort(int A[],int low,int high){
```

```
if(low<high){
```

```
    int mid=(low+high)/2; //从中间划分
```

```
    MergeSort(A,low,mid); //对左半部分归并排序
```

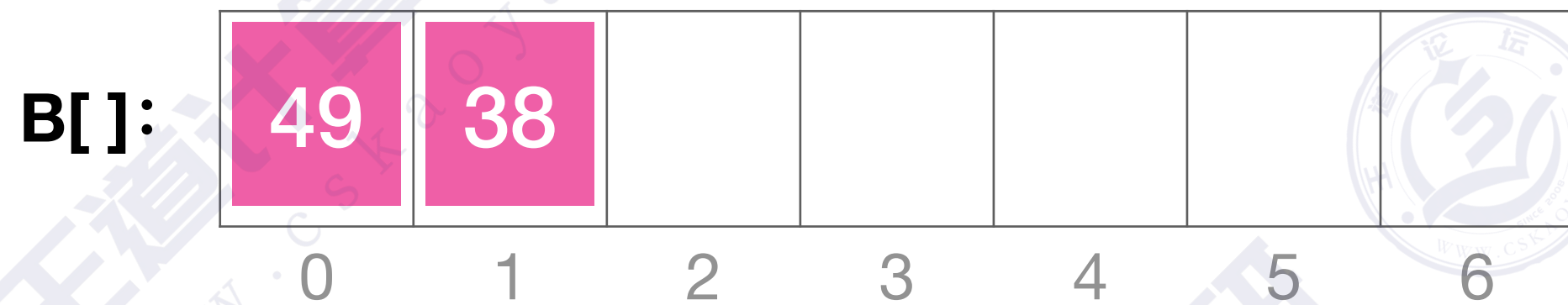
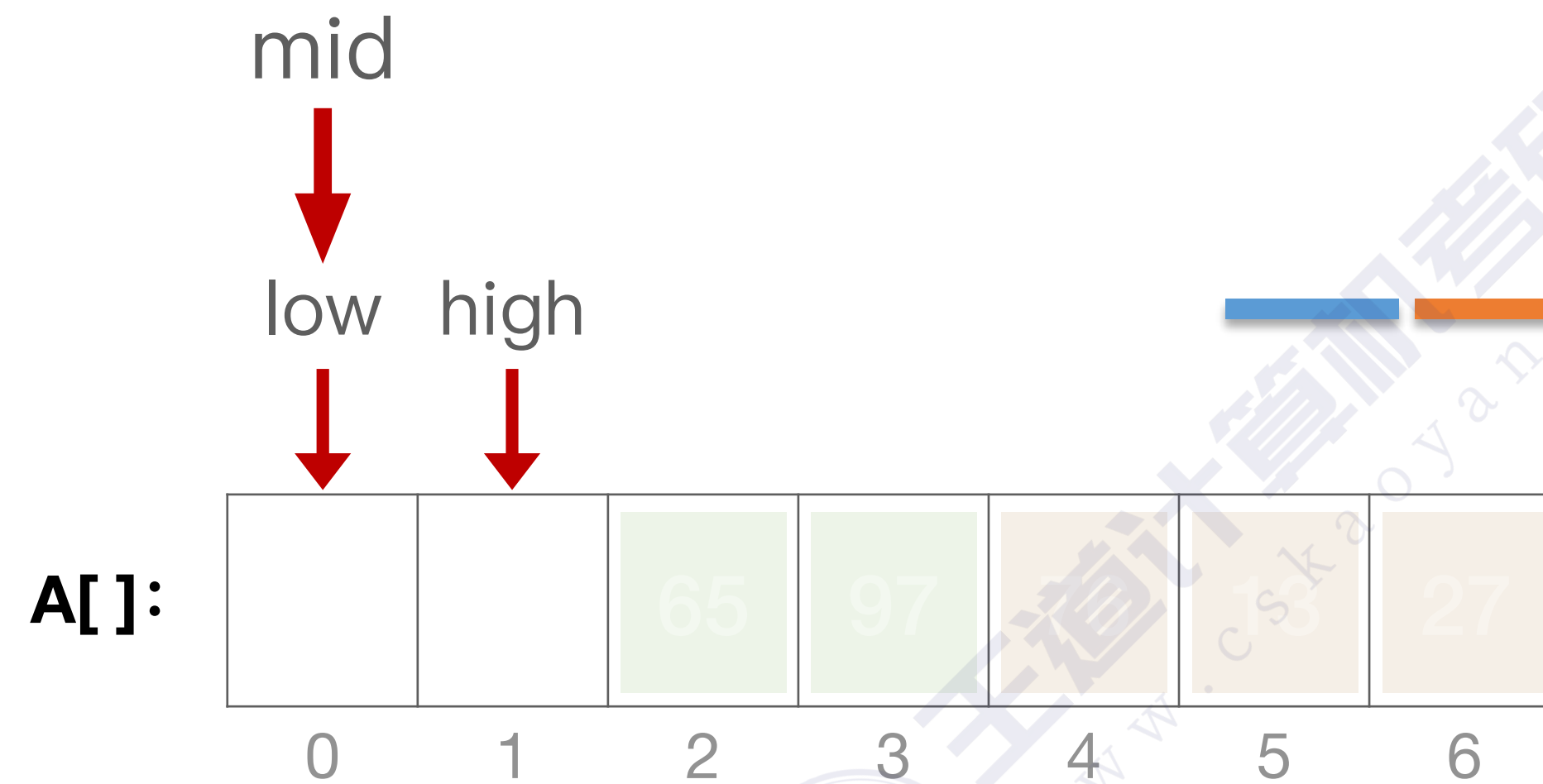
```
    MergeSort(A,mid+1,high); //对右半部分归并排序
```

```
    Merge(A,low,mid,high); //归并
```

```
}//if
```

```
}
```


代码实现



```
int *B=(int *)malloc(n*sizeof(int)); //辅助数组B
```

```
//A[low...mid]和A[mid+1...high]各自有序，将两个部分归并
```

```
void Merge(int A[],int low,int mid,int high){
```

```
int i,j,k;
```

```
for(k=low;k<=high;k++)
```

```
    B[k]=A[k]; //将A中所有元素复制到B中
```

```
for(i=low,j=mid+1,k=i;i<=mid&&j<=high;k++){
```

```
    if(B[i]<=B[j])
```

```
        A[k]=B[i++]; //将较小值复制到A中
```

```
    else
```

```
        A[k]=B[j++];
```

```
}//for
```

```
while(i<=mid) A[k++]=B[i++];
```

```
while(j<=high) A[k++]=B[j++];
```

```
}
```

```
void MergeSort(int A[],int low,int high){
```

```
if(low<high){
```

```
    int mid=(low+high)/2; //从中间划分
```

```
    MergeSort(A,low,mid); //对左半部分归并排序
```

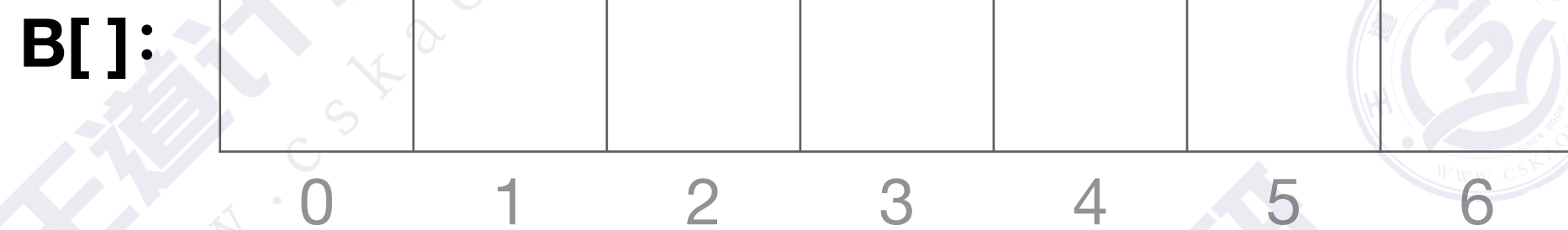
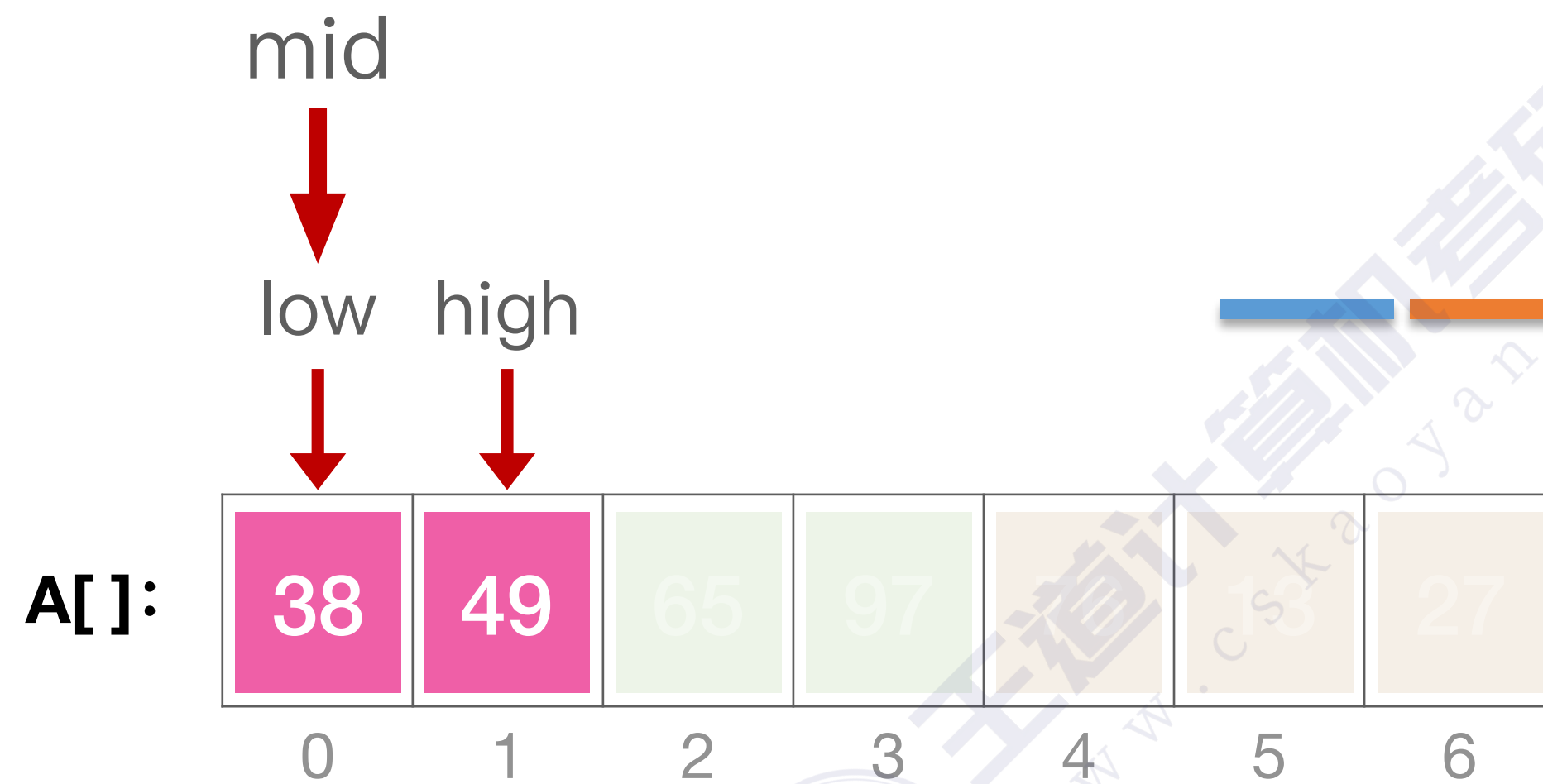
```
    MergeSort(A,mid+1,high); //对右半部分归并排序
```

```
    Merge(A,low,mid,high); //归并
```

```
}//if
```

```
}
```


代码实现



```
int *B=(int *)malloc(n*sizeof(int)); //辅助数组B
```

```
//A[low...mid]和A[mid+1...high]各自有序，将两个部分归并
```

```
void Merge(int A[],int low,int mid,int high){
```

```
int i,j,k;
```

```
for(k=low;k<=high;k++)
```

```
    B[k]=A[k]; //将A中所有元素复制到B中
```

```
for(i=low,j=mid+1,k=i;i<=mid&&j<=high;k++){
```

```
    if(B[i]<=B[j])
```

```
        A[k]=B[i++]; //将较小值复制到A中
```

```
    else
```

```
        A[k]=B[j++];
```

```
}//for
```

```
while(i<=mid) A[k++]=B[i++];
```

```
while(j<=high) A[k++]=B[j++];
```

```
}
```

```
void MergeSort(int A[],int low,int high){
```

```
if(low<high){
```

```
    int mid=(low+high)/2; //从中间划分
```

```
    MergeSort(A,low,mid); //对左半部分归并排序
```

```
    MergeSort(A,mid+1,high); //对右半部分归并排序
```

```
    Merge(A,low,mid,high); //归并
```

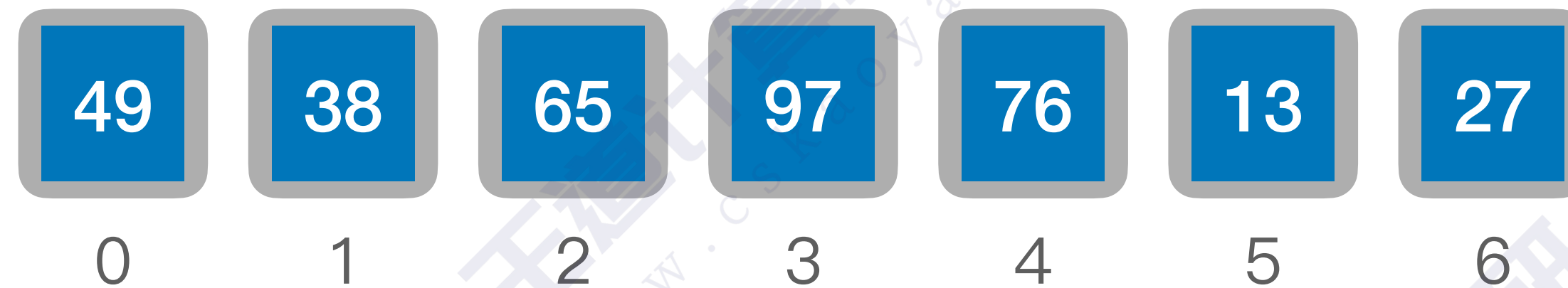
```
}//if
```

```
}
```

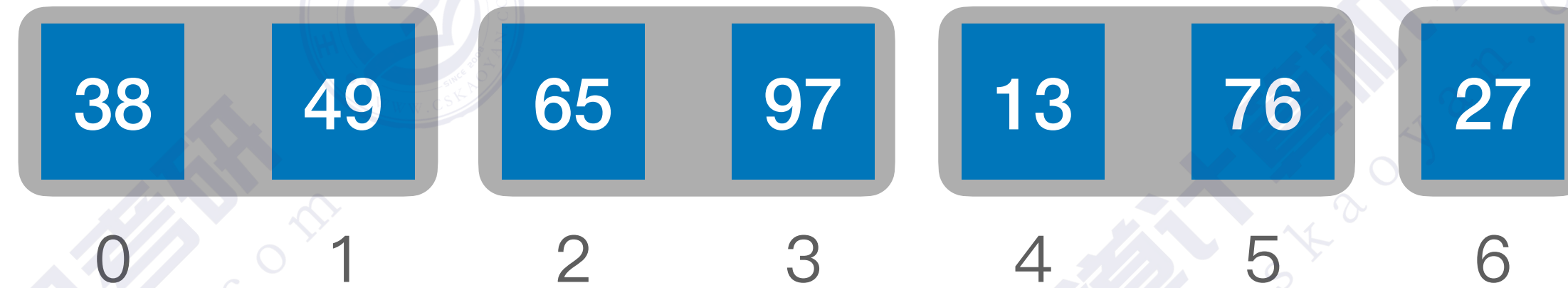
算法效率分析

2路归并的“归并树”——形态上就是一棵倒立的二叉树

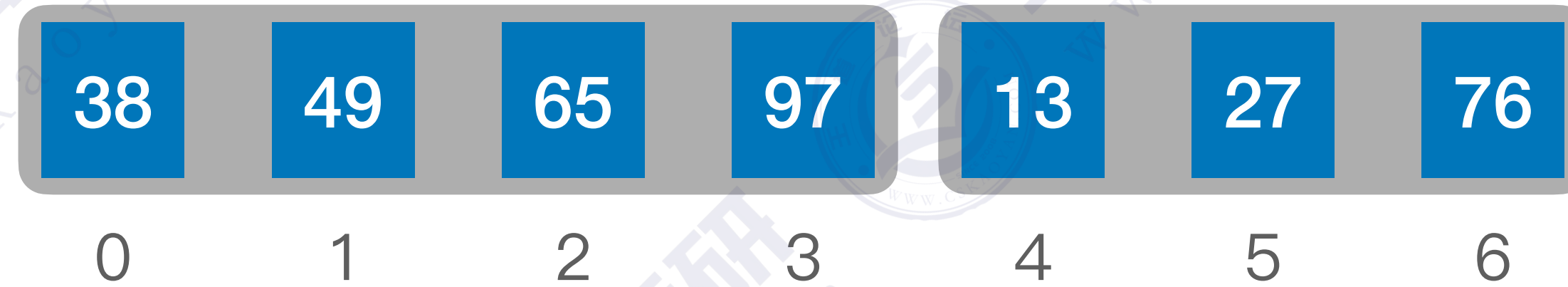
初始序列:



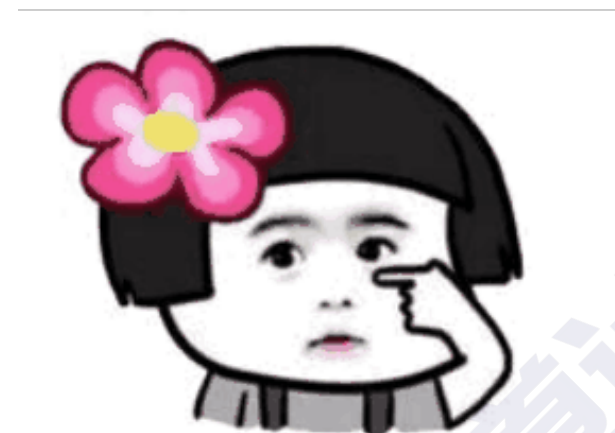
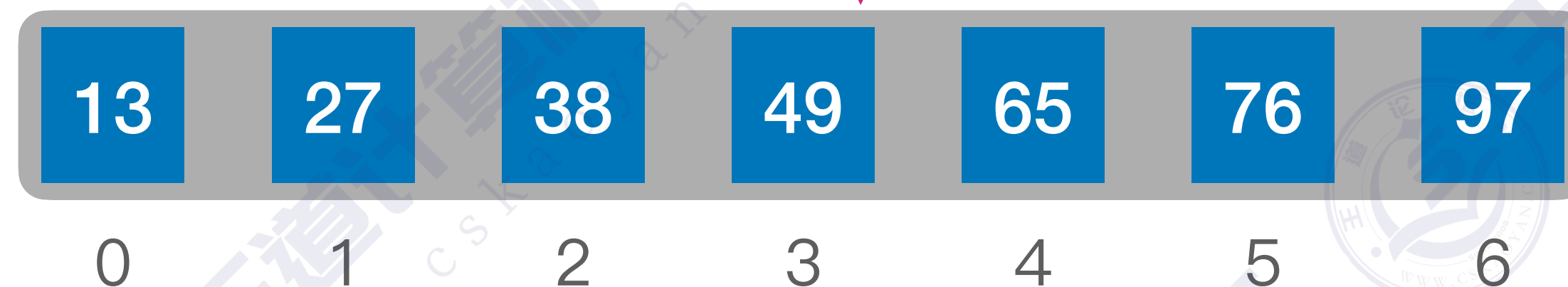
一趟归并后:



二趟归并后:



三趟归并后:



看左边

二叉树的第 h 层最多有 2^{h-1} 个结点
若树高为 h , 则应满足 $n \leq 2^{h-1}$
即 $h - 1 = \lceil \log_2 n \rceil$

结论: n 个元素进行2路归并排序, 归并趟数 $= \lceil \log_2 n \rceil$

每趟归并时间复杂度为 $O(n)$, 则算法时间复杂度为 $O(n \log_2 n)$

空间复杂度 $= O(n)$, 来自于辅助数组B

知识回顾与重要考点

归并排序

Merge (归并)

把两个或多个有序的子序列合并为一个

2 路归并——二合一

k 路归并——k 合一

归并排序算法

① 若 $low < high$, 则将序列分从中间 $mid = (low + high) / 2$ 分开

② 对左半部分 $[low, mid]$ 递归地进行归并排序

③ 对右半部分 $[mid + 1, high]$ 递归地进行归并排序

④ 将左右两个有序子序列 Merge 为一个

性能

空间复杂度 $O(n)$

时间复杂度 $O(n \log n)$

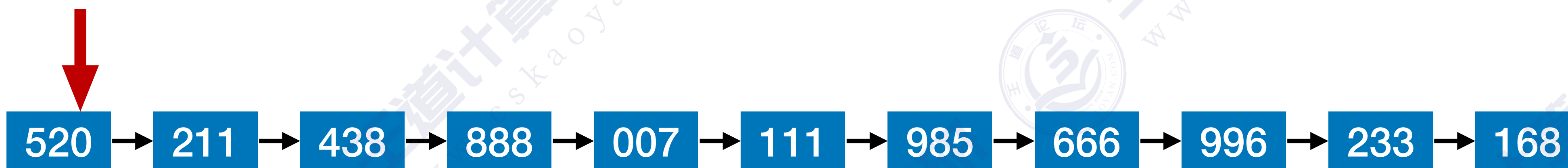
稳定性 稳定的

本节内容

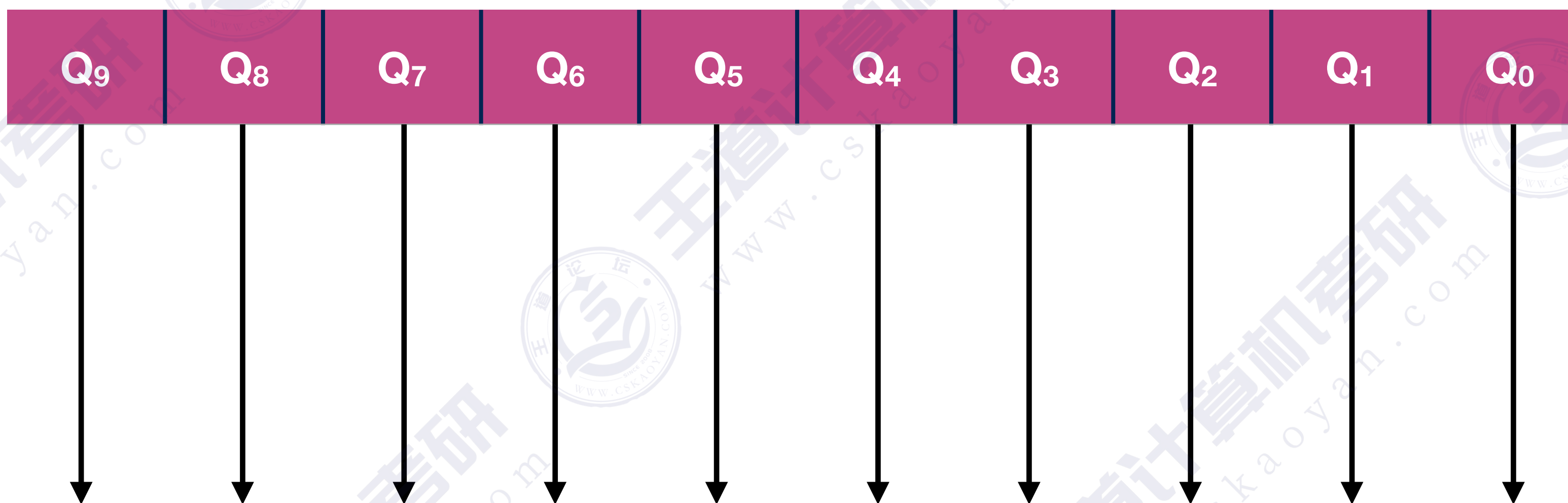
基数排序

(Radix Sort)

基数排序

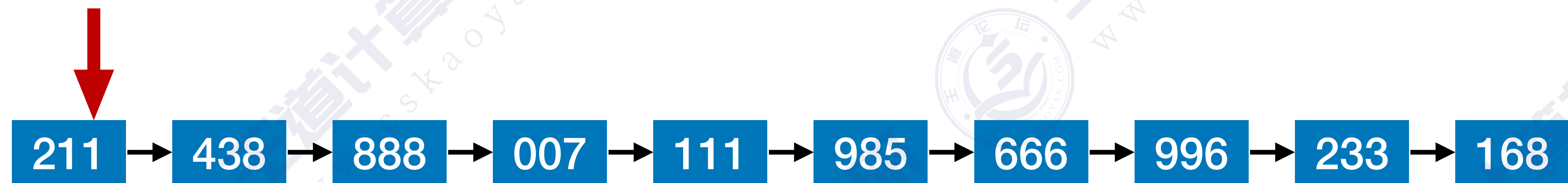


第一趟：以“个位”进行“分配”

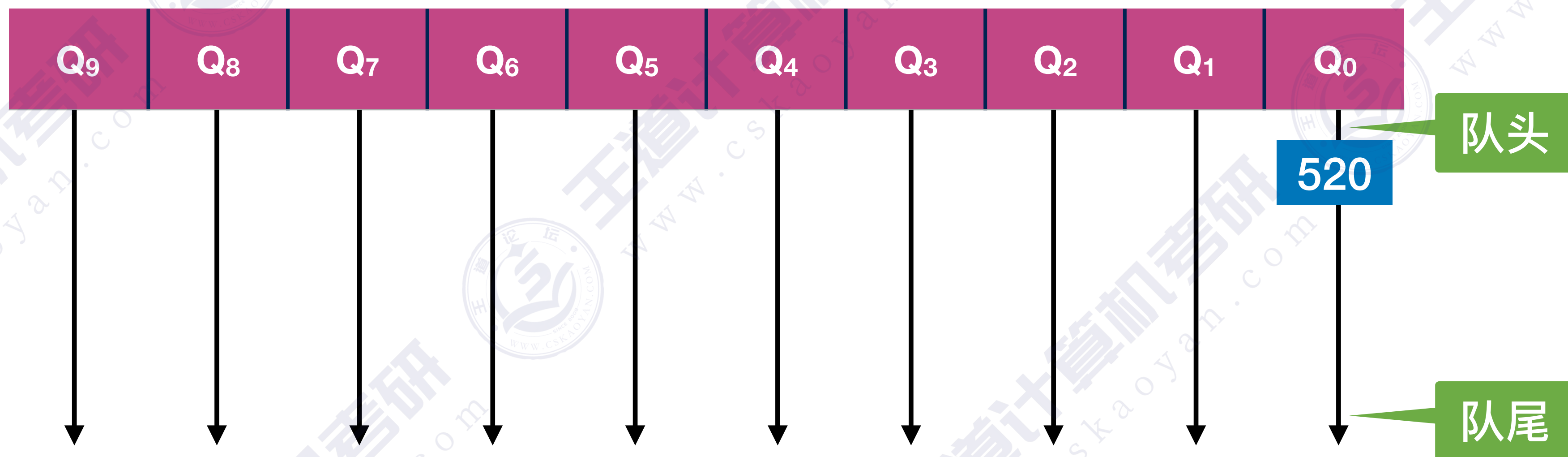


要求：得到按关键字“递减”的有序序列。

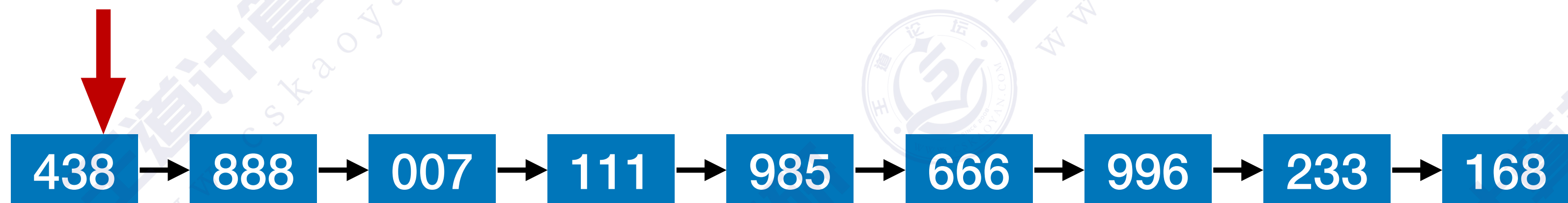
基数排序



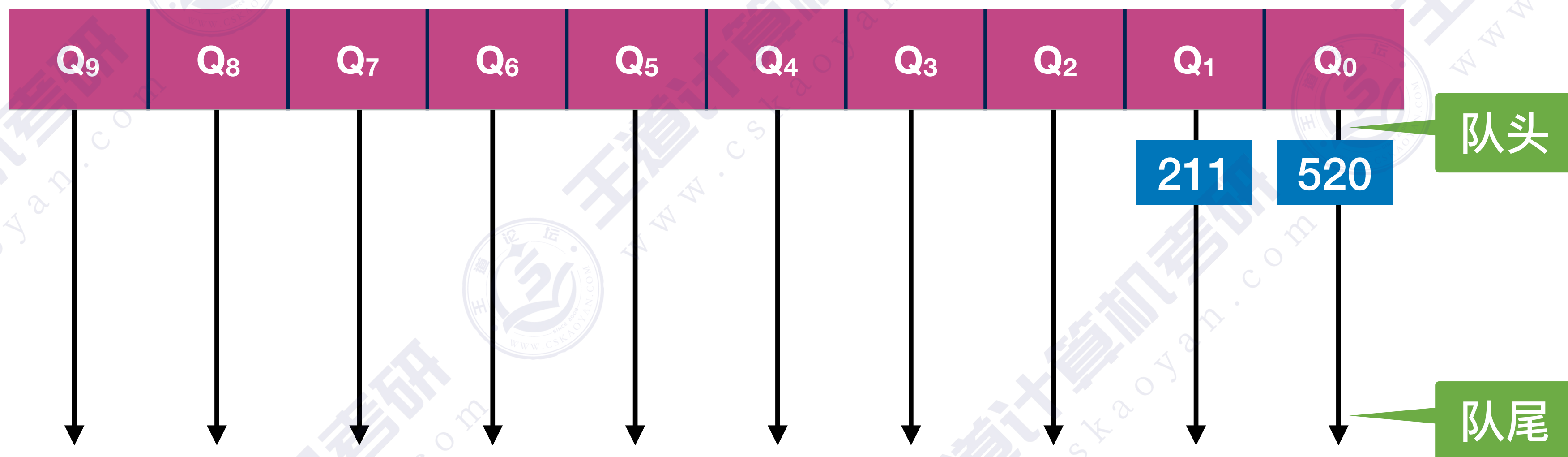
第一趟：以“个位”进行“分配”



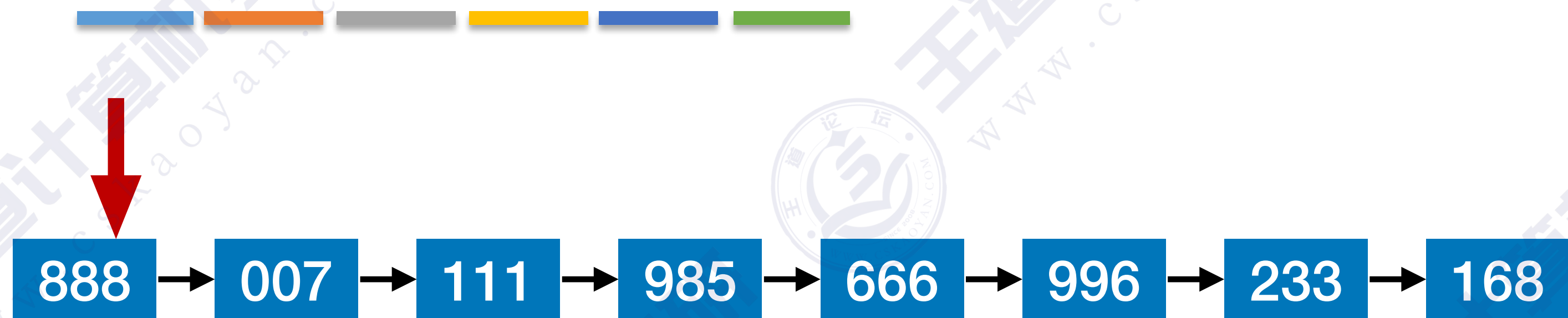
基数排序



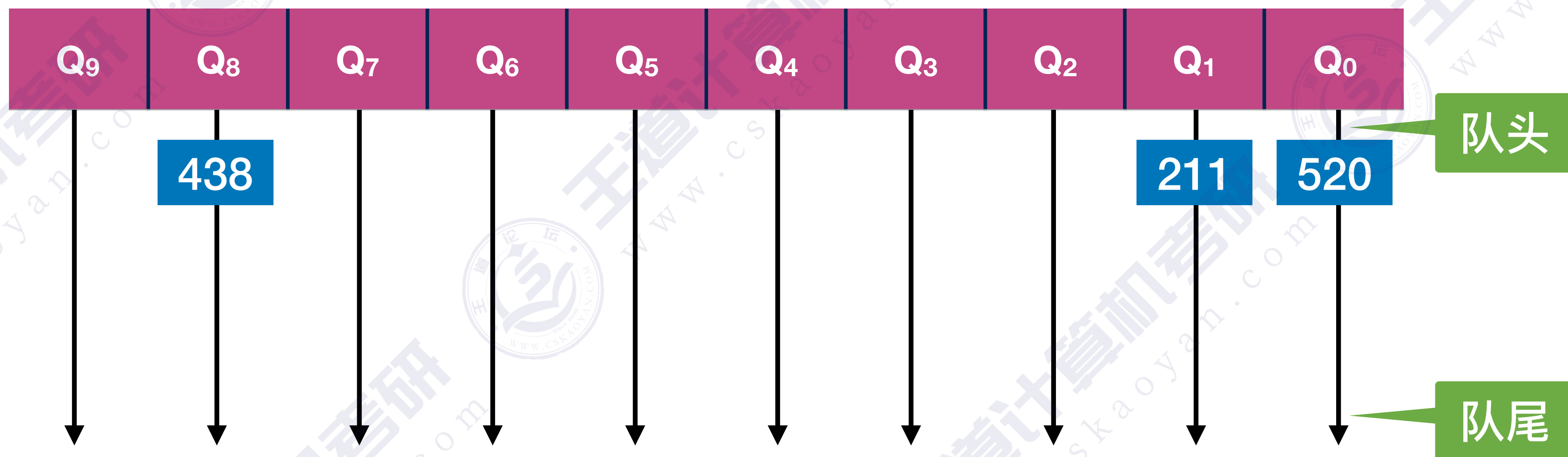
第一趟：以“个位”进行“分配”



基数排序



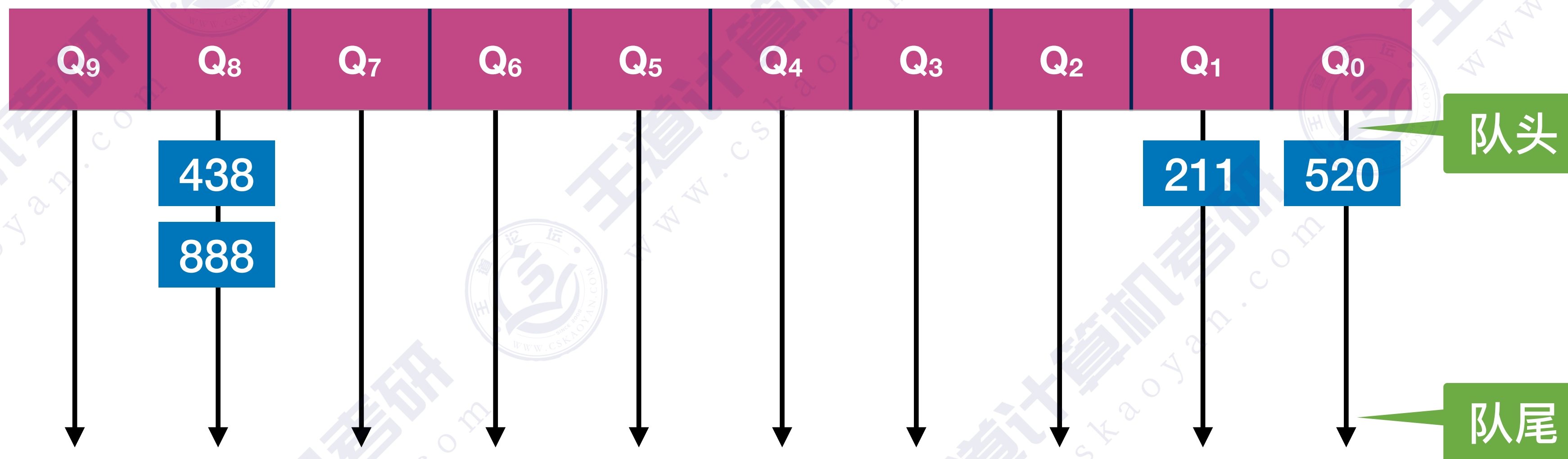
第一趟：以“个位”进行“分配”



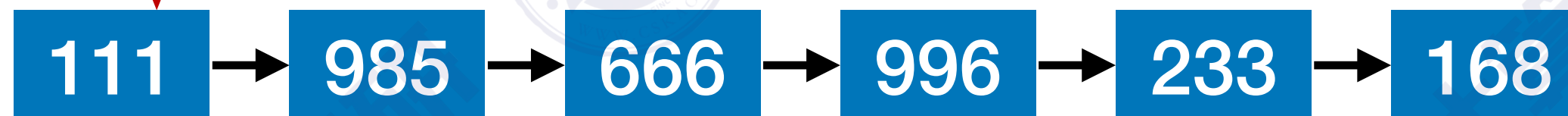
基数排序



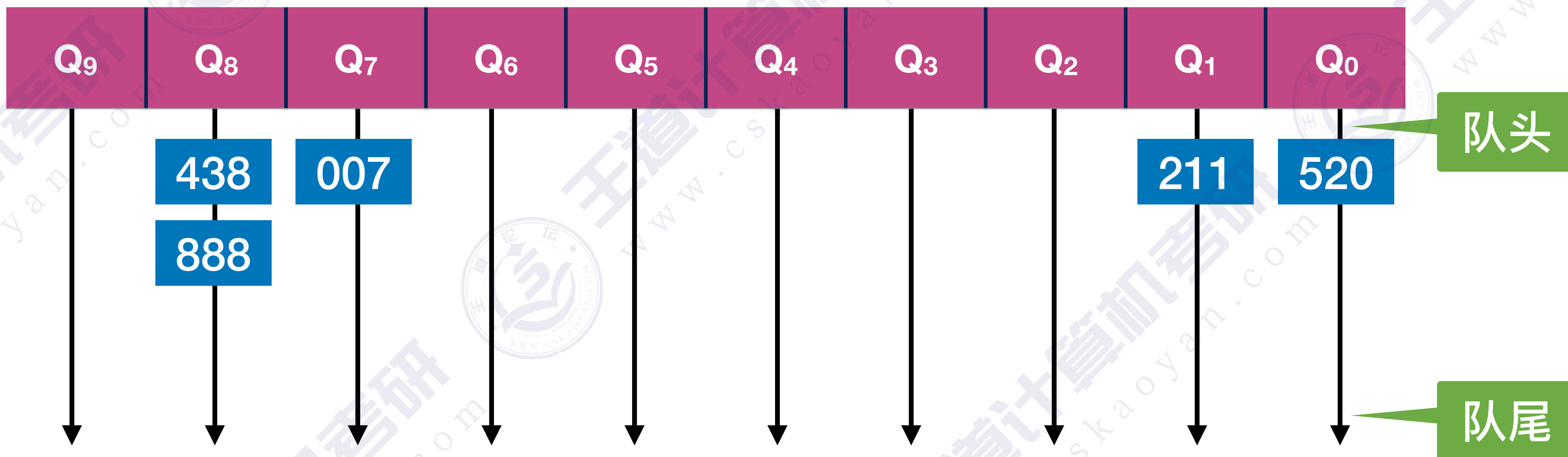
第一趟：以“个位”进行“分配”



基数排序

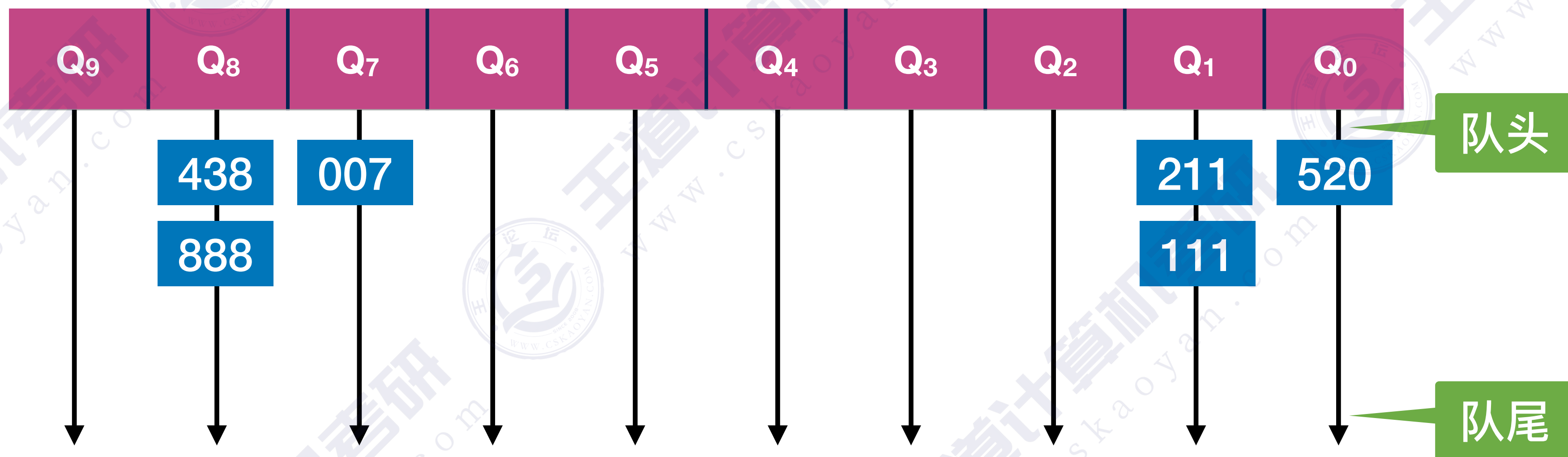


第一趟：以“个位”进行“分配”

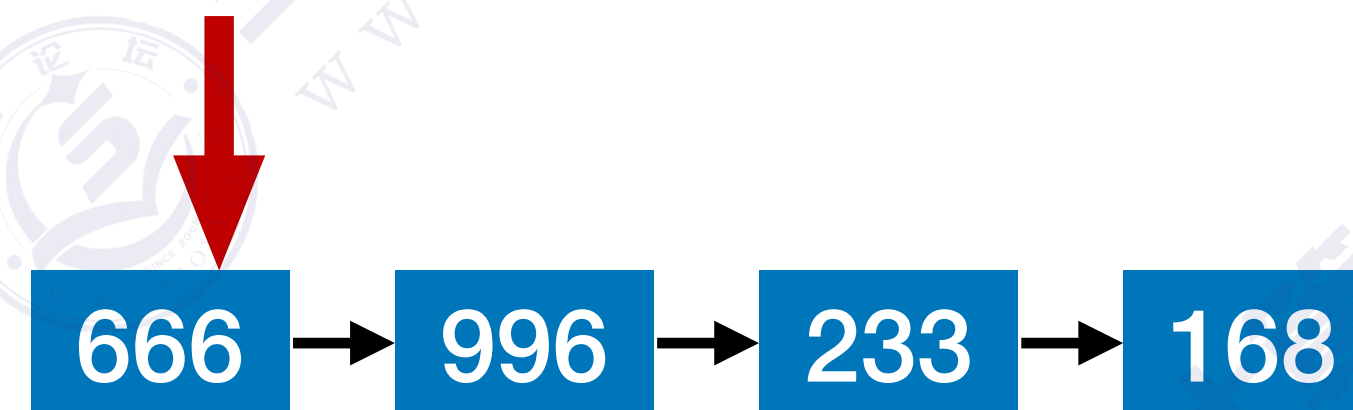


基数排序

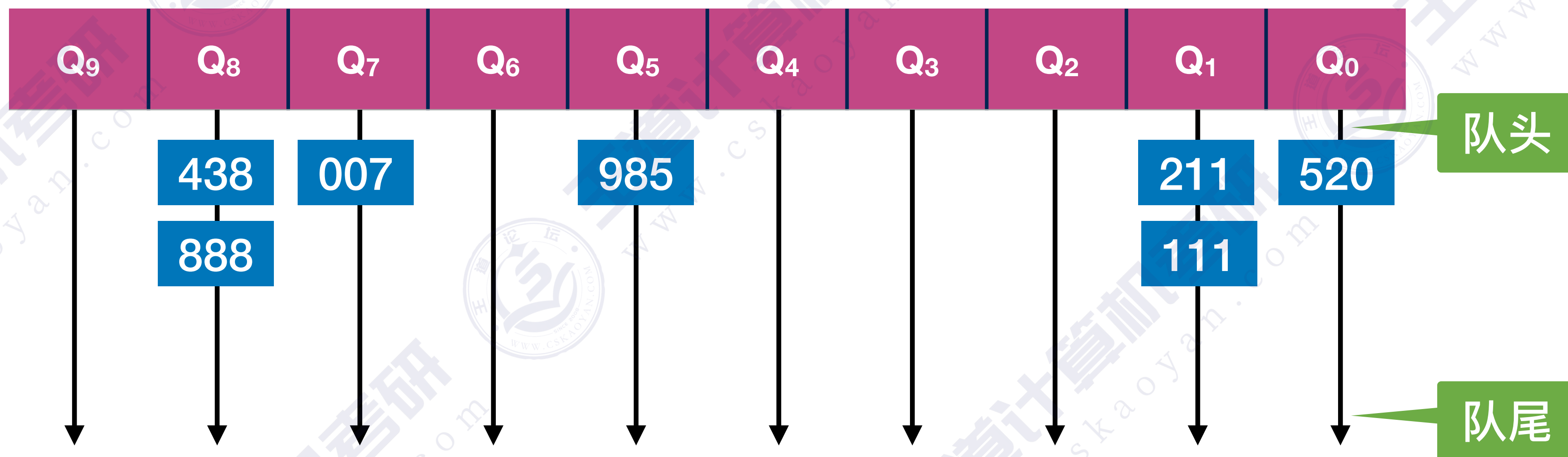
第一趟：以“个位”进行“分配”



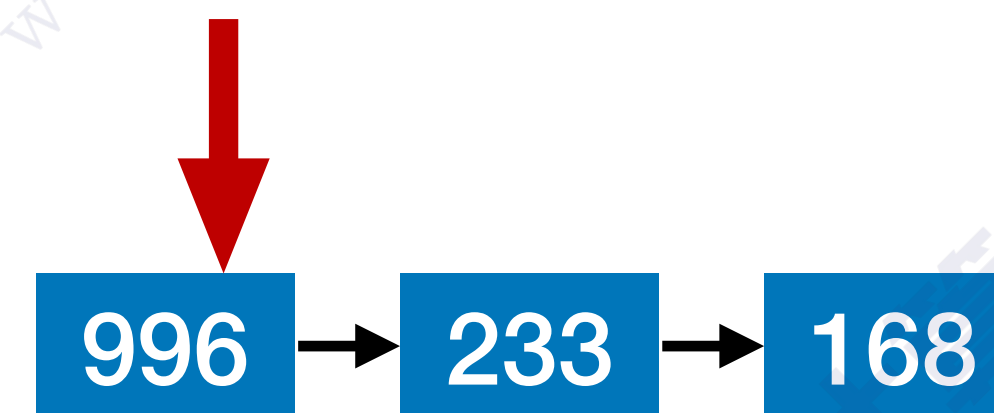
基数排序



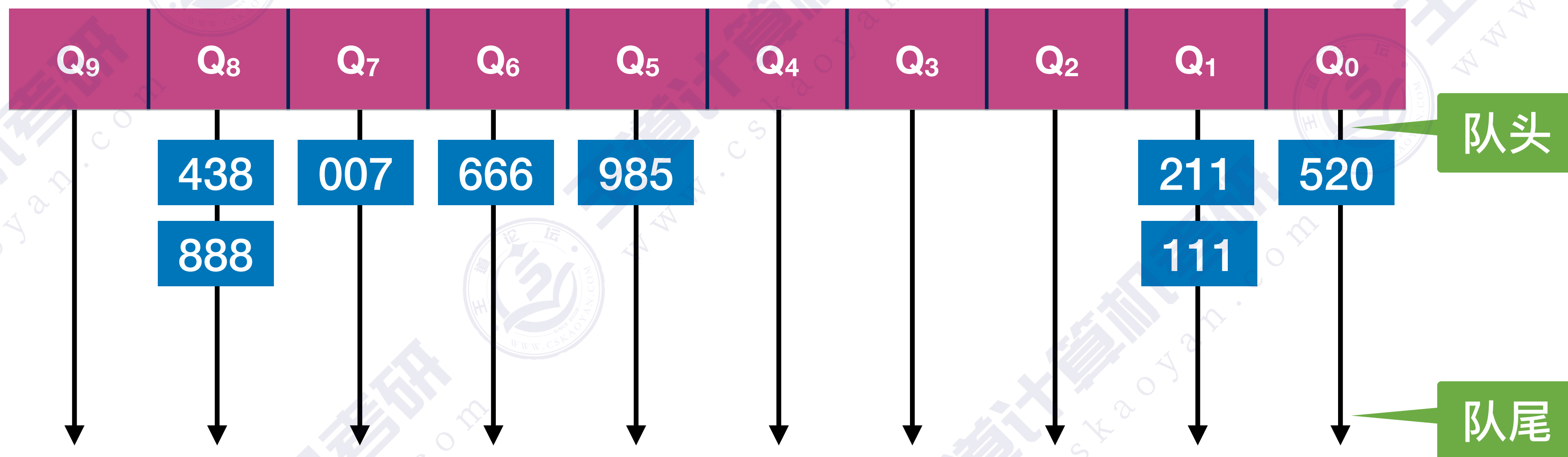
第一趟：以“个位”进行“分配”



基数排序

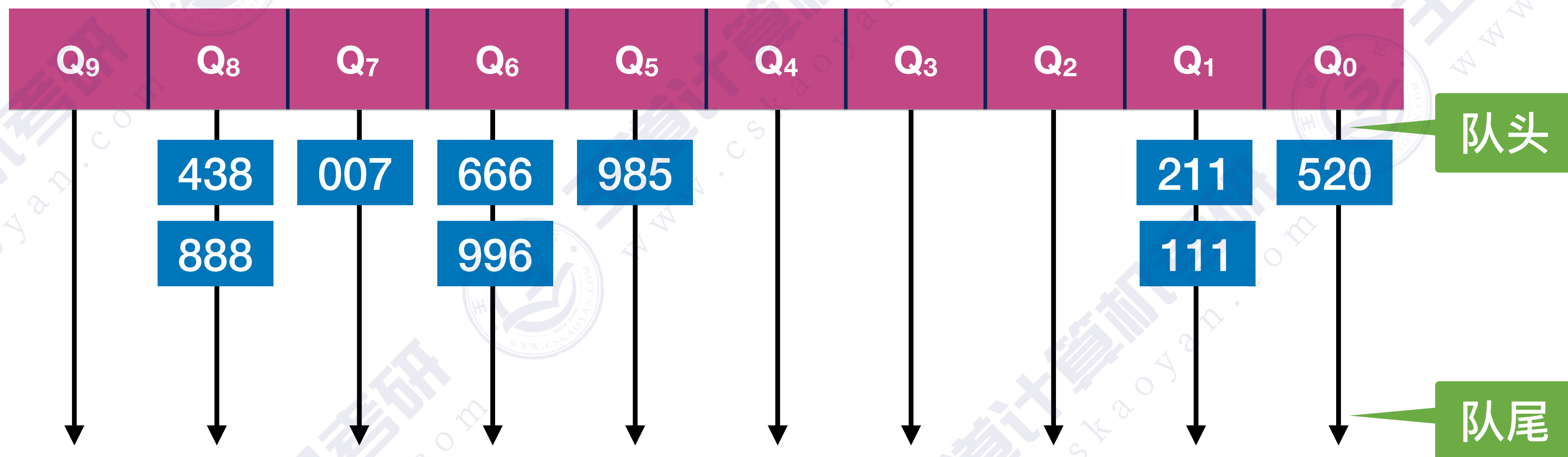


第一趟：以“个位”进行“分配”



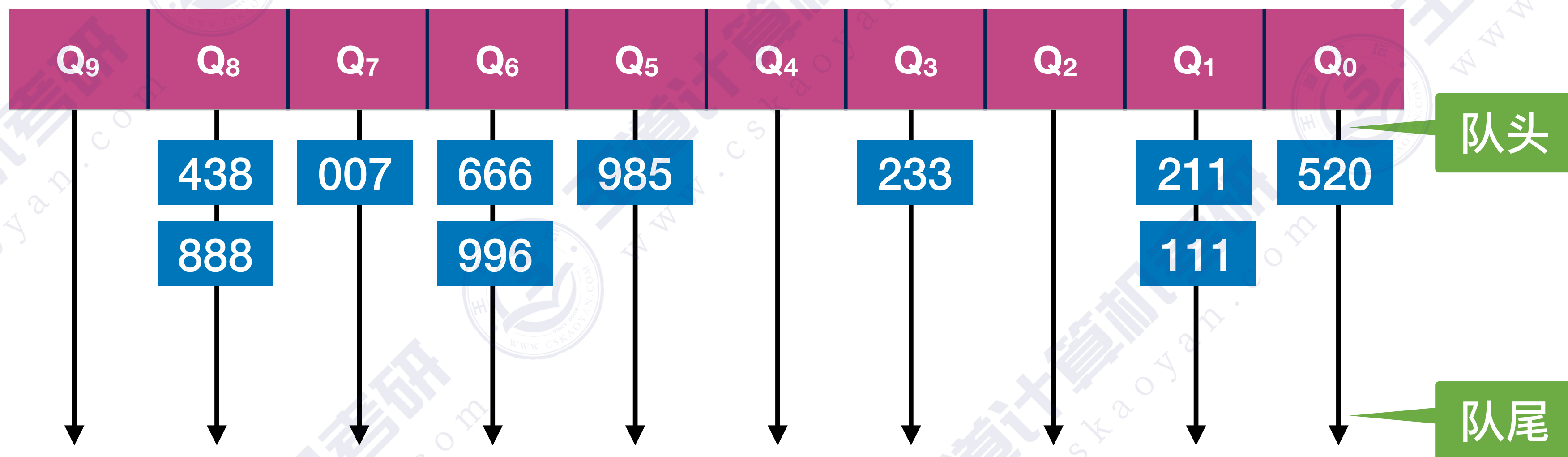
基数排序

第一趟：以“个位”进行“分配”



基数排序

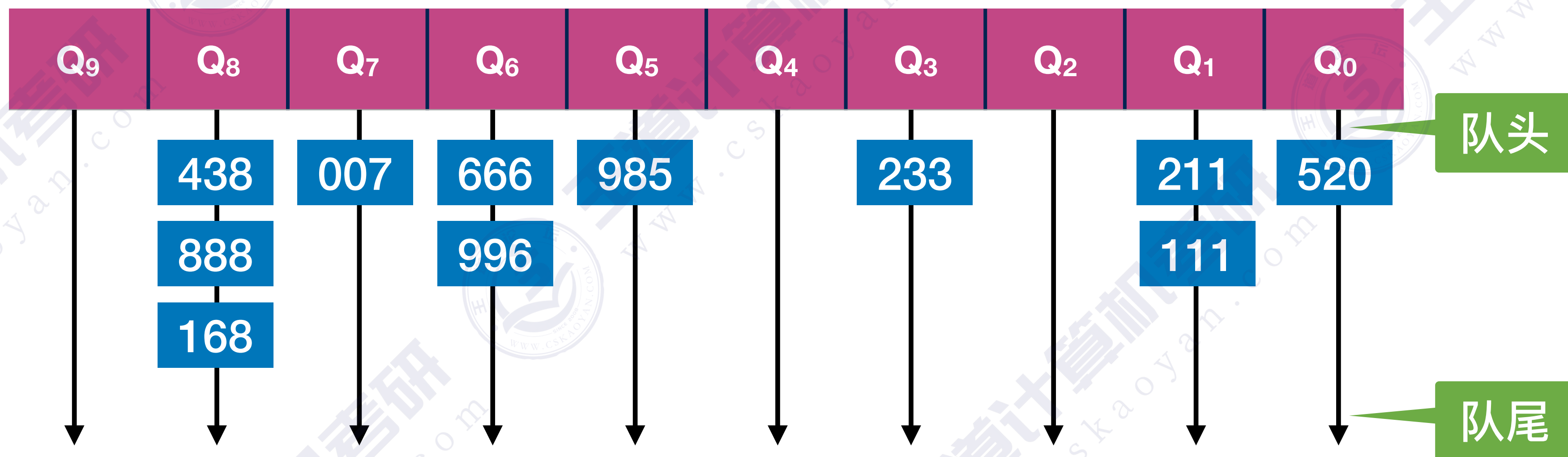
第一趟：以“个位”进行“分配”



168

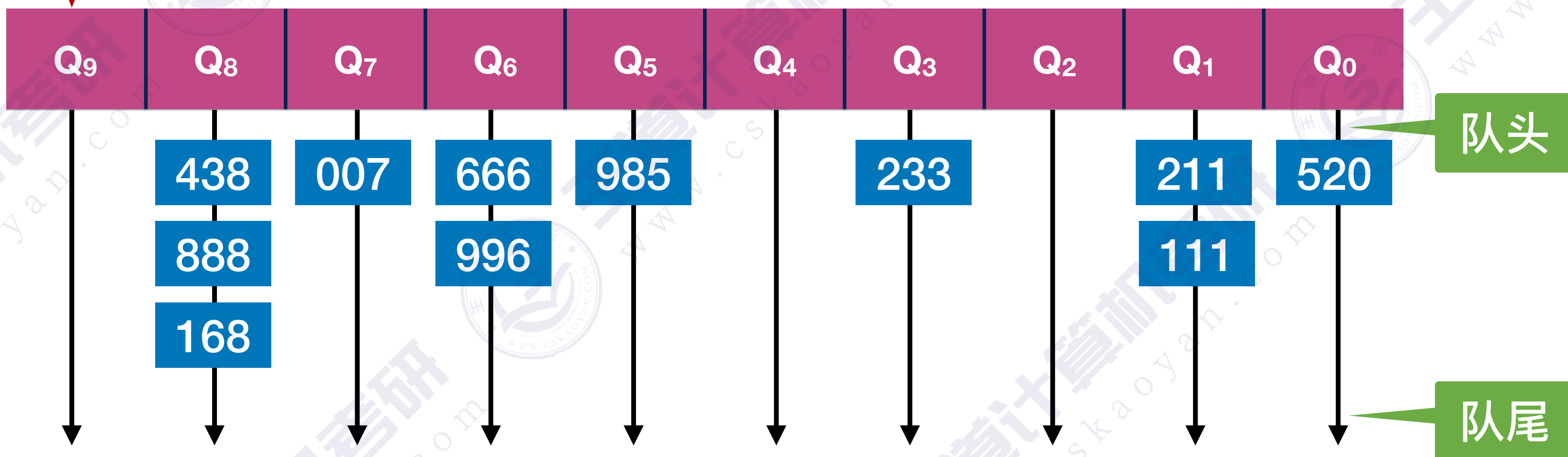
基数排序

第一趟“分配”结束



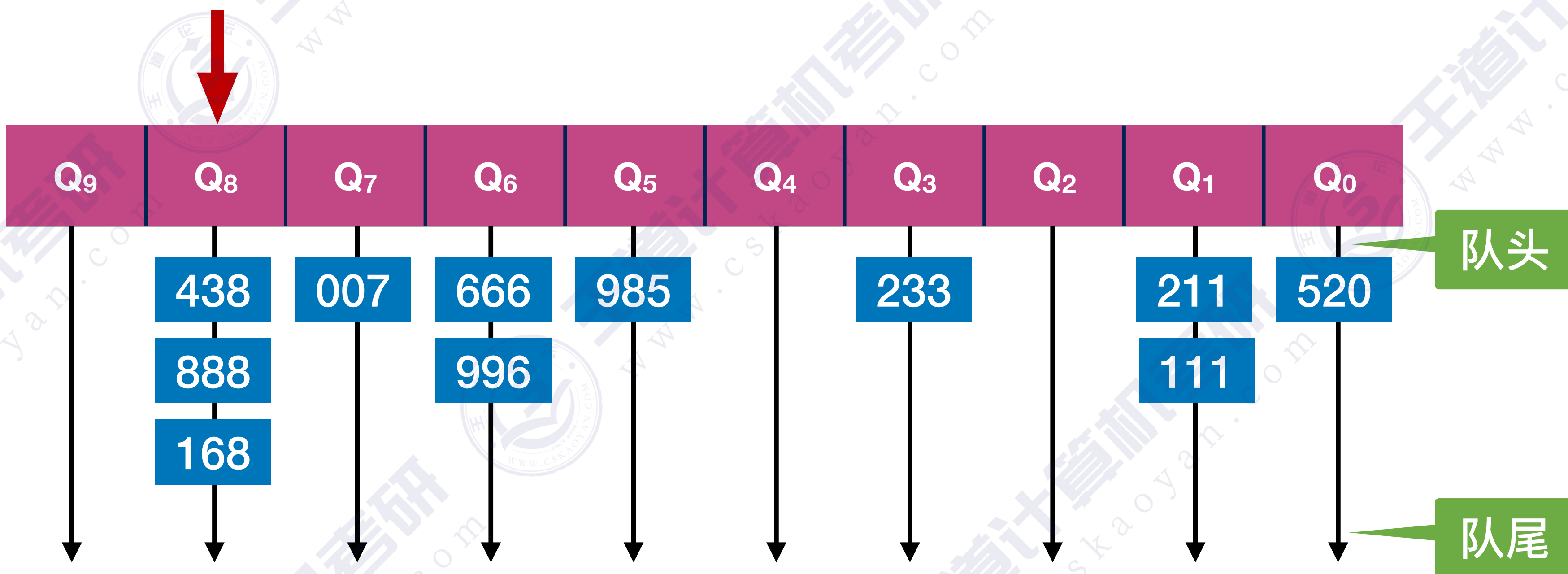
基数排序

第一趟“收集”



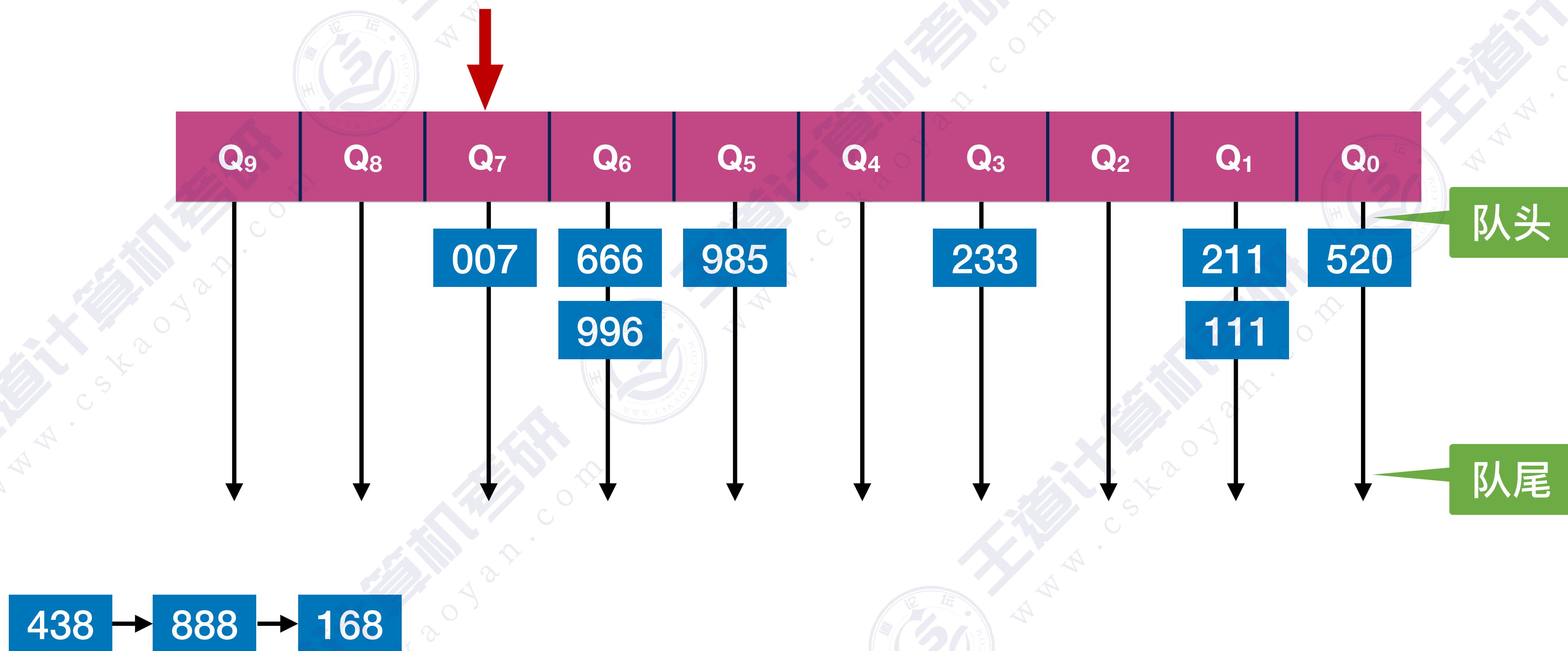
基数排序

第一趟“收集”



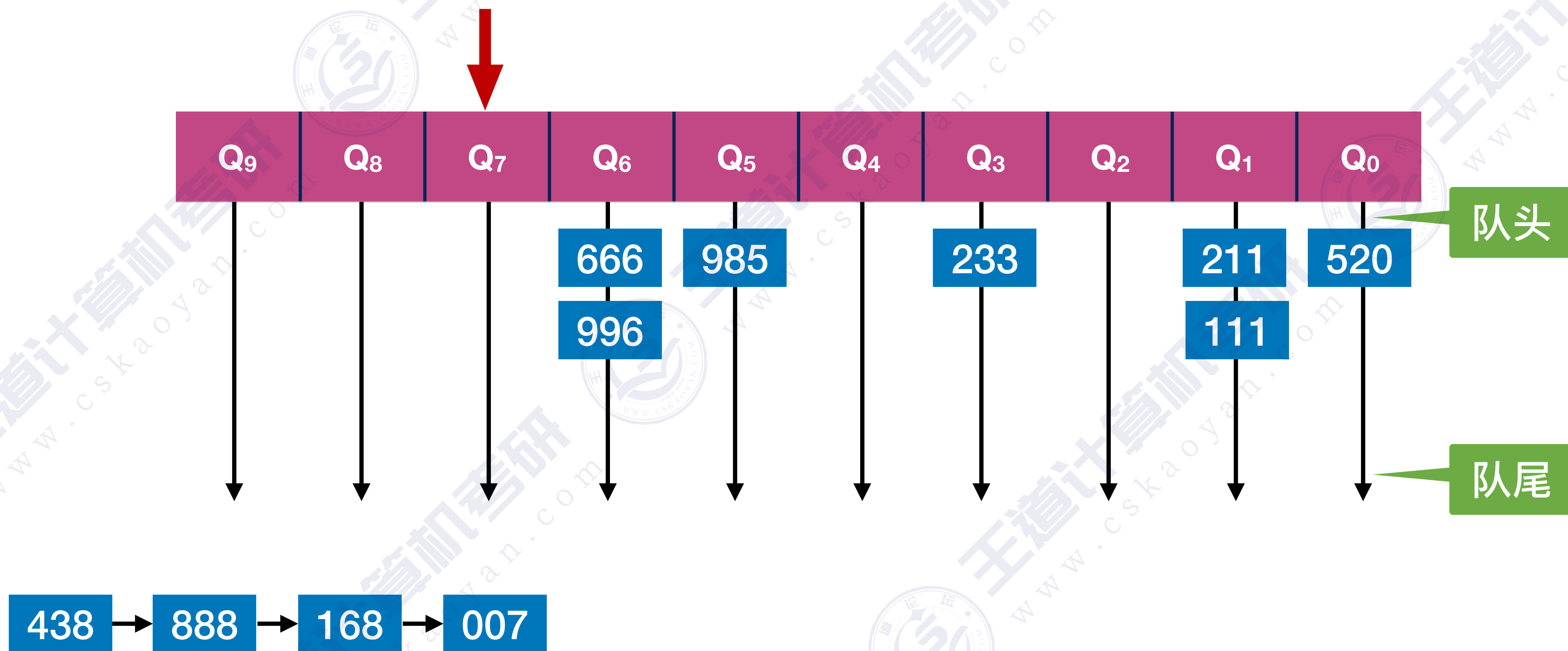
基数排序

第一趟“收集”



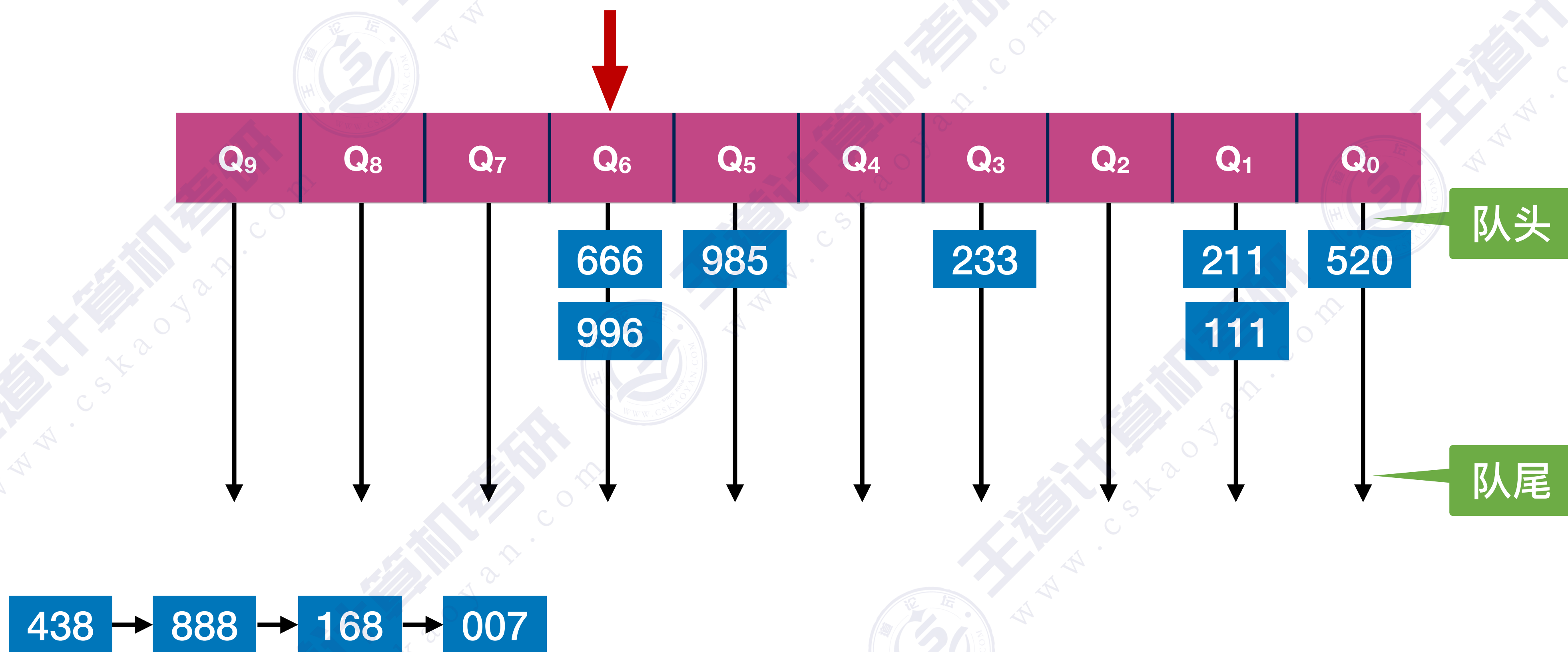
基数排序

第一趟“收集”



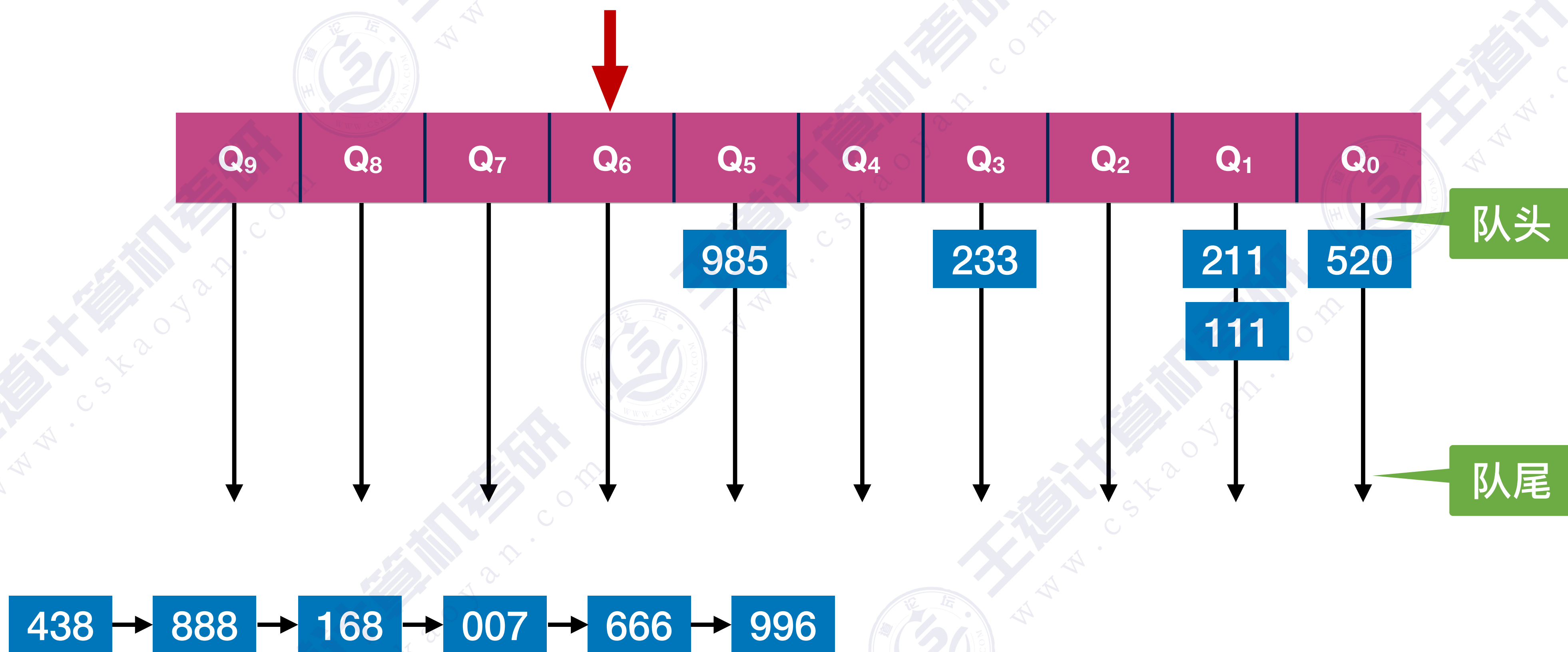
基数排序

第一趟“收集”



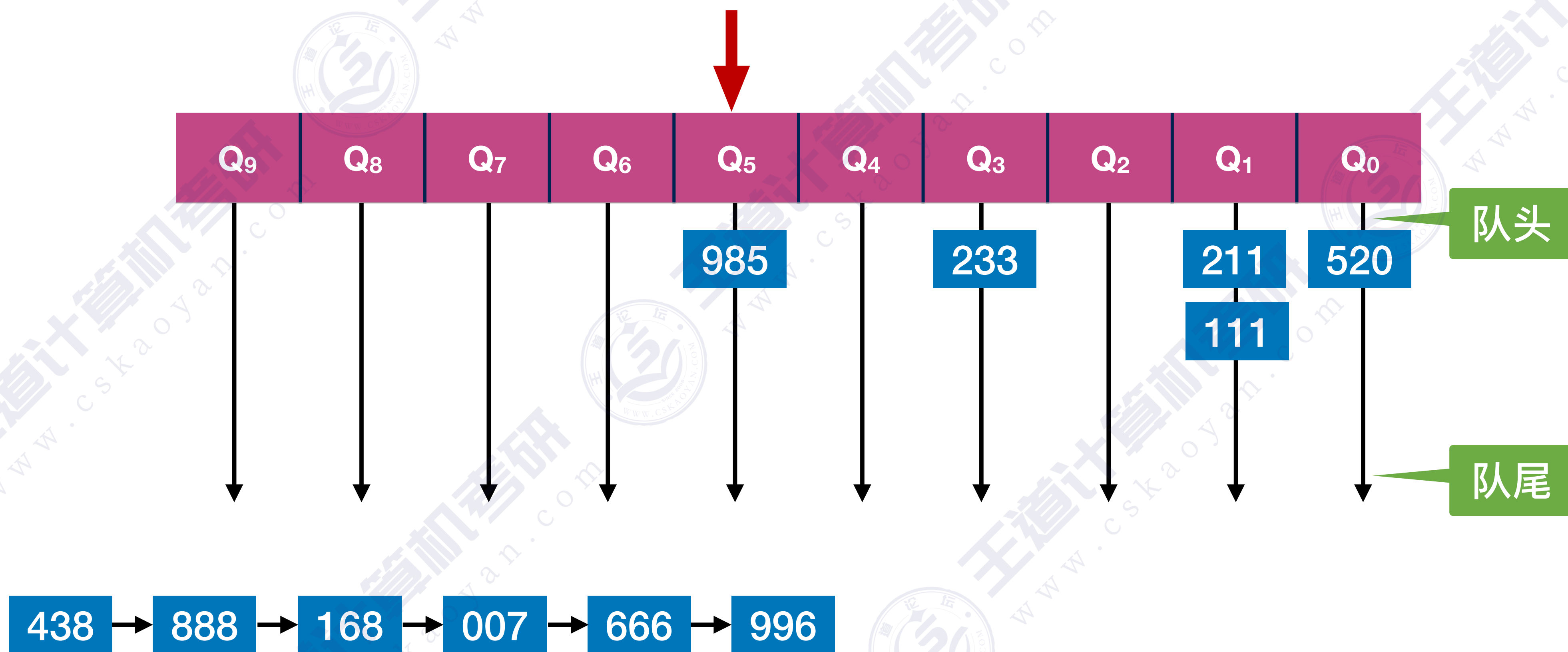
基数排序

第一趟“收集”



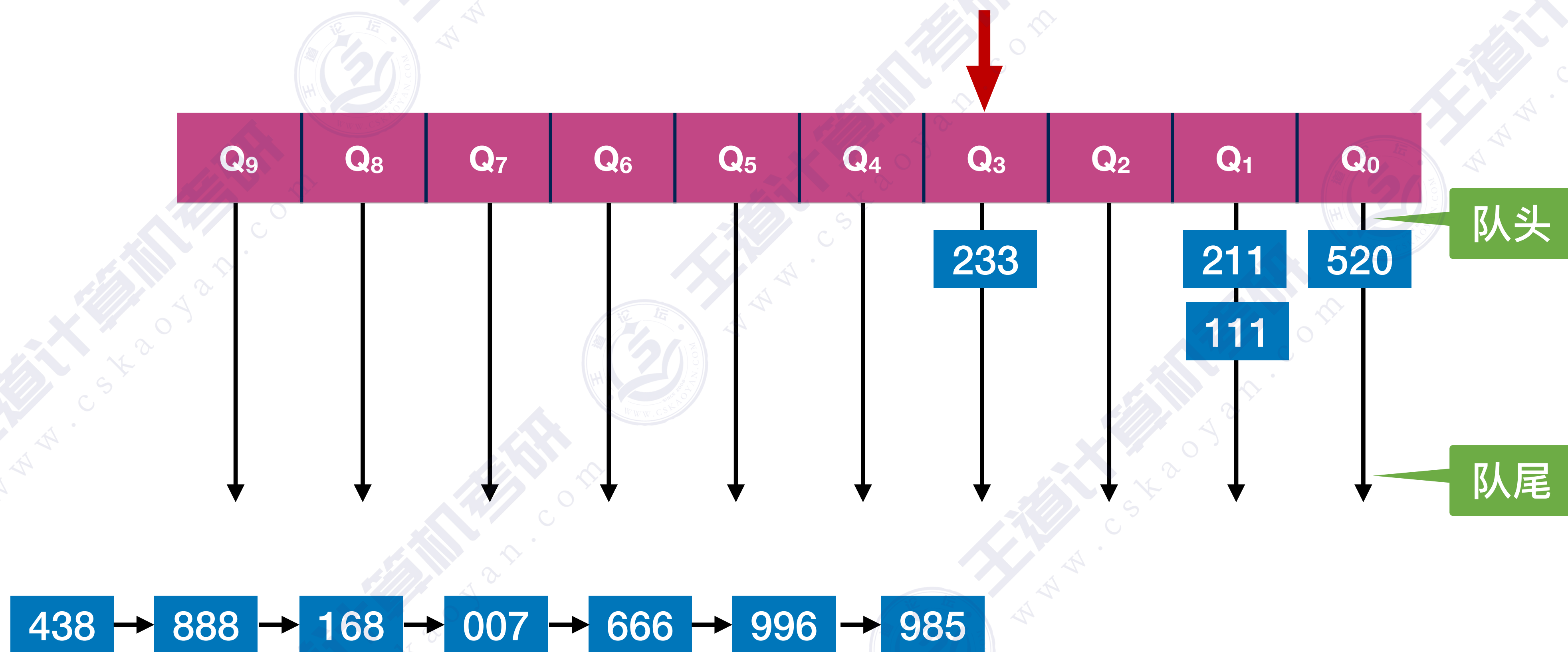
基数排序

第一趟“收集”



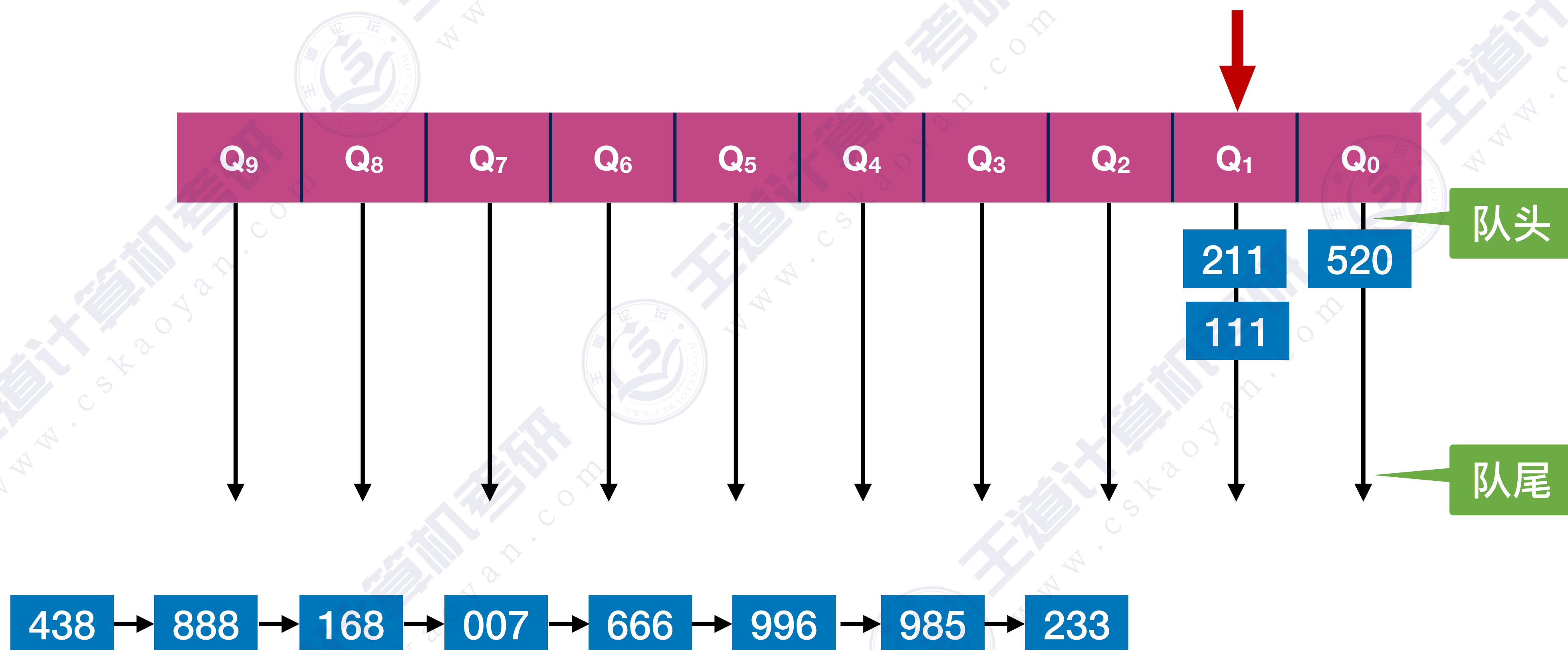
基数排序

第一趟“收集”



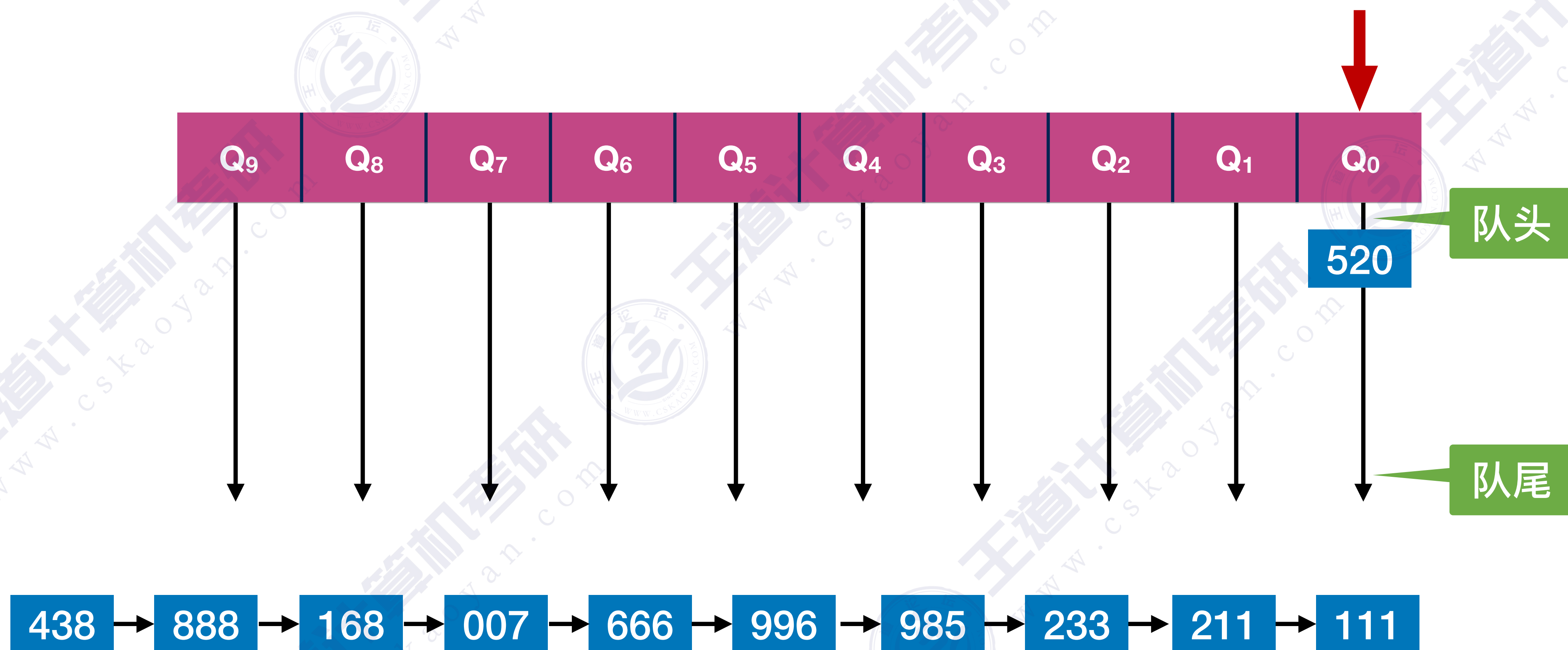
基数排序

第一趟“收集”

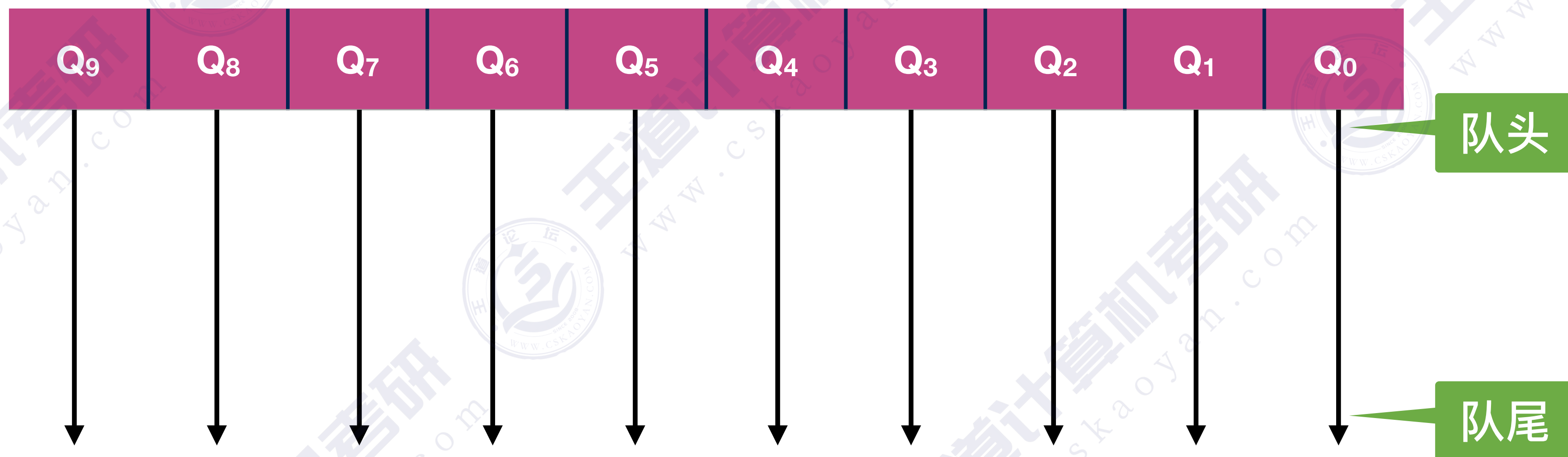


基数排序

第一趟“收集”



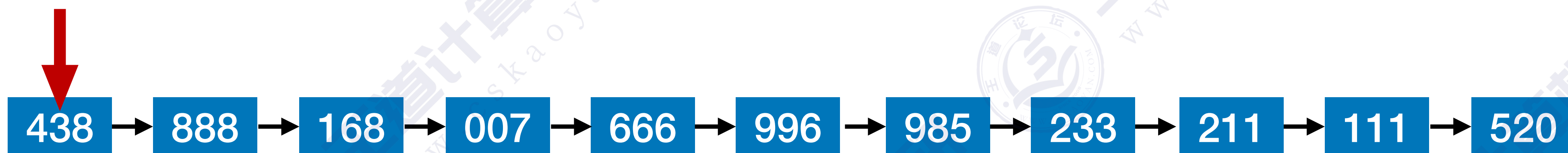
基数排序



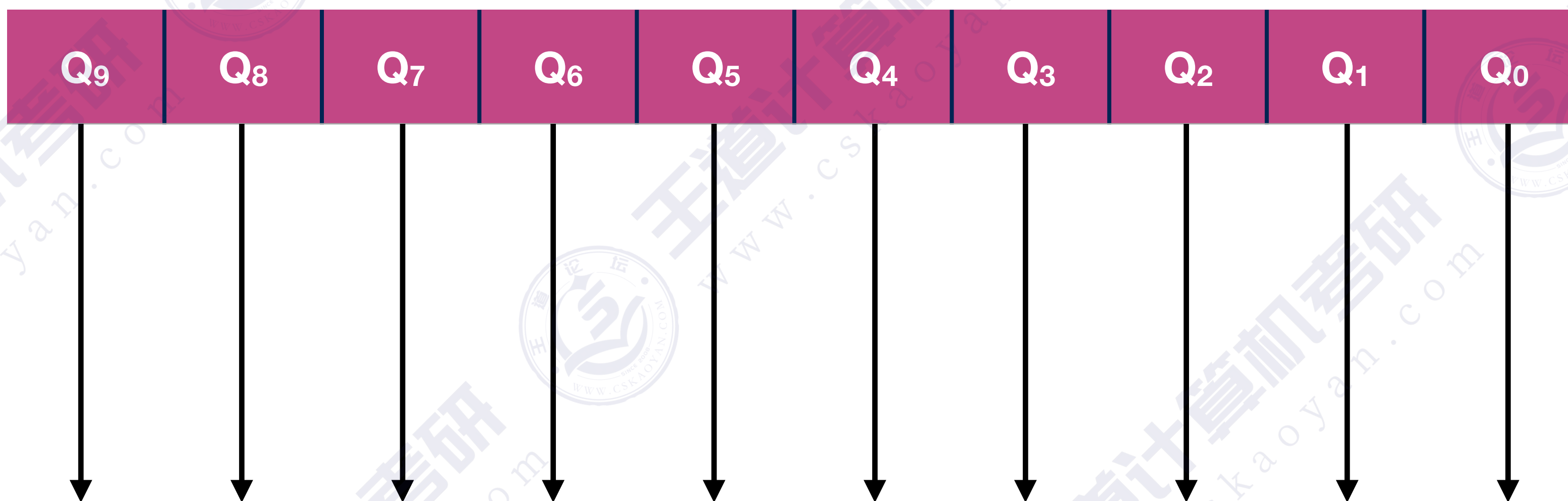
第一趟“收集”结束：得到按“个位”递减排序的序列

438 → 888 → 168 → 007 → 666 → 996 → 985 → 233 → 211 → 111 → 520

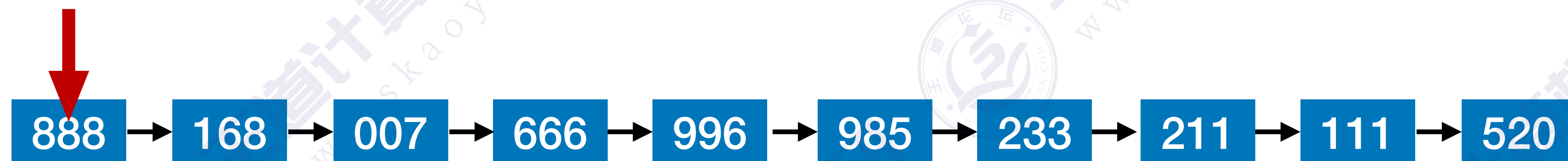
基数排序



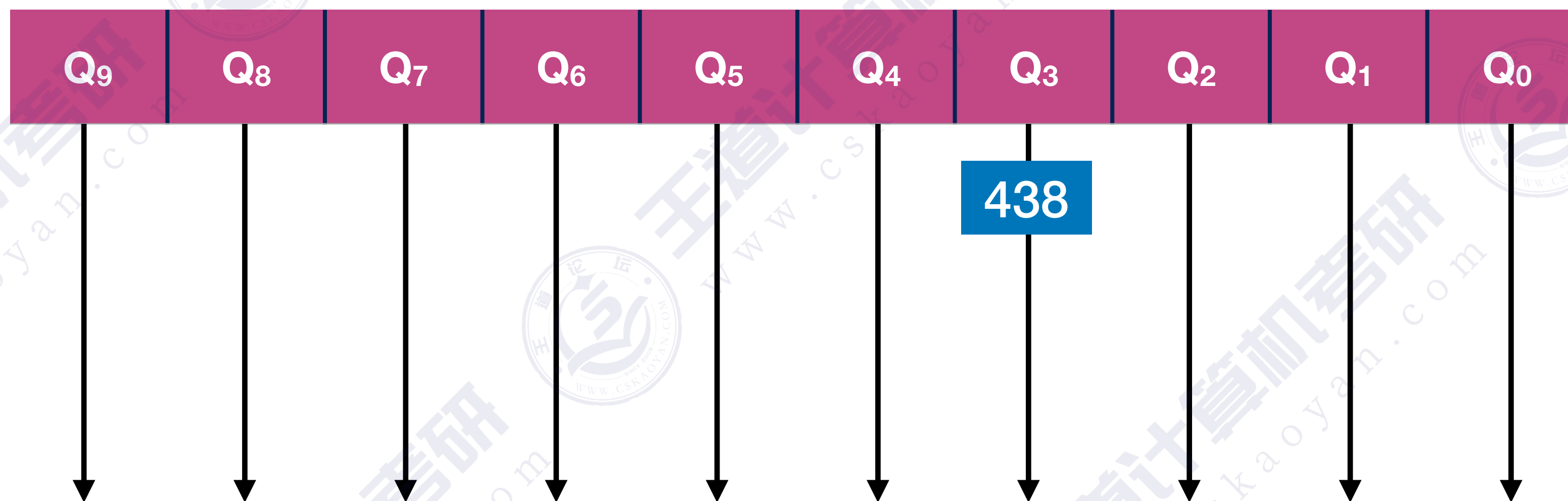
第二趟：以“十位”进行“分配”



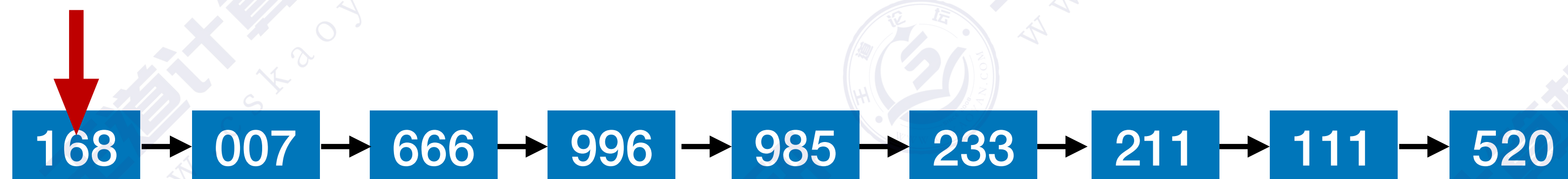
基数排序



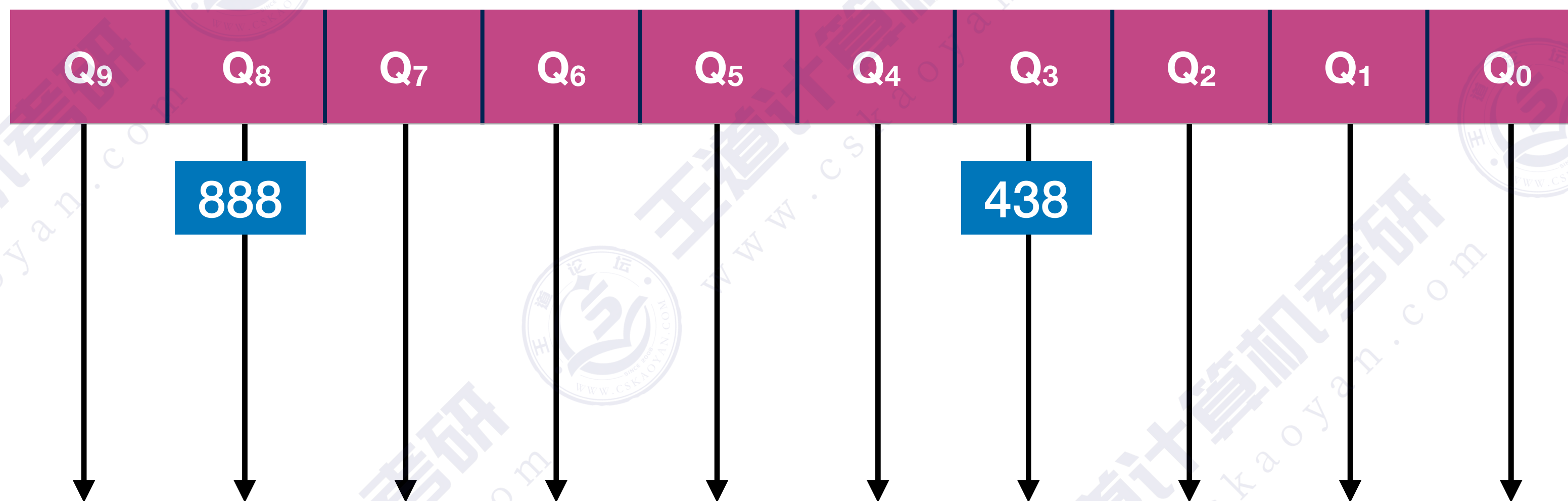
第二趟：以“十位”进行“分配”



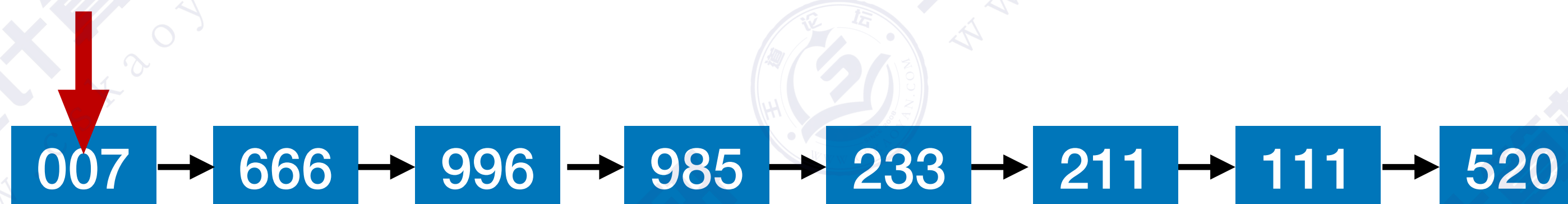
基数排序



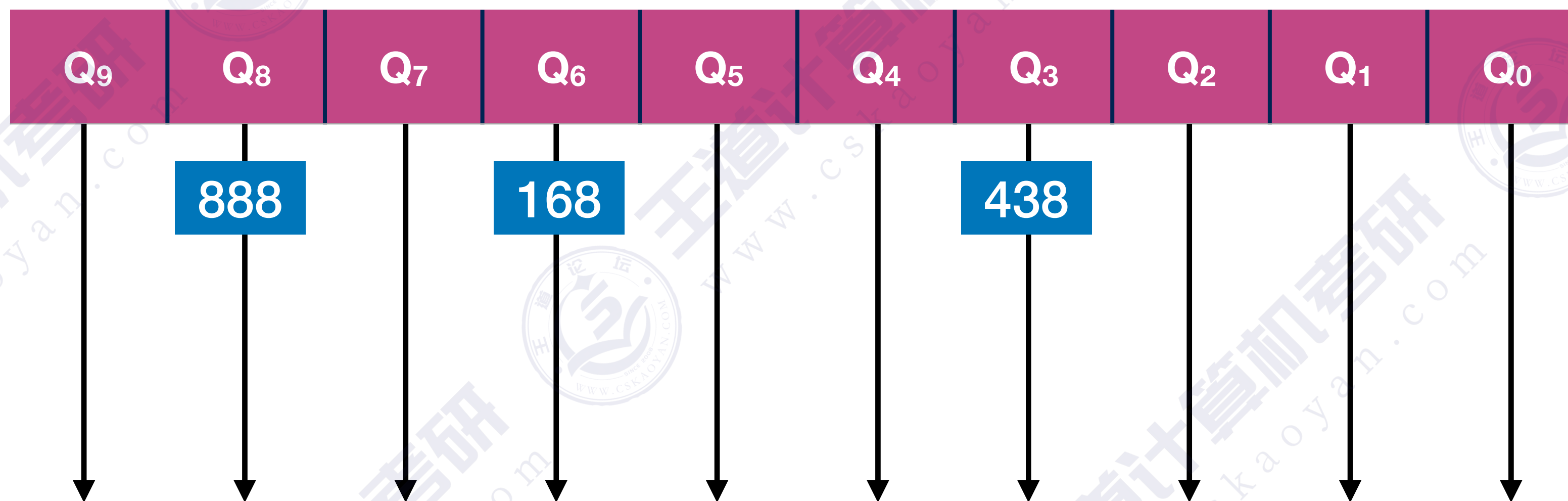
第二趟：以“十位”进行“分配”



基数排序



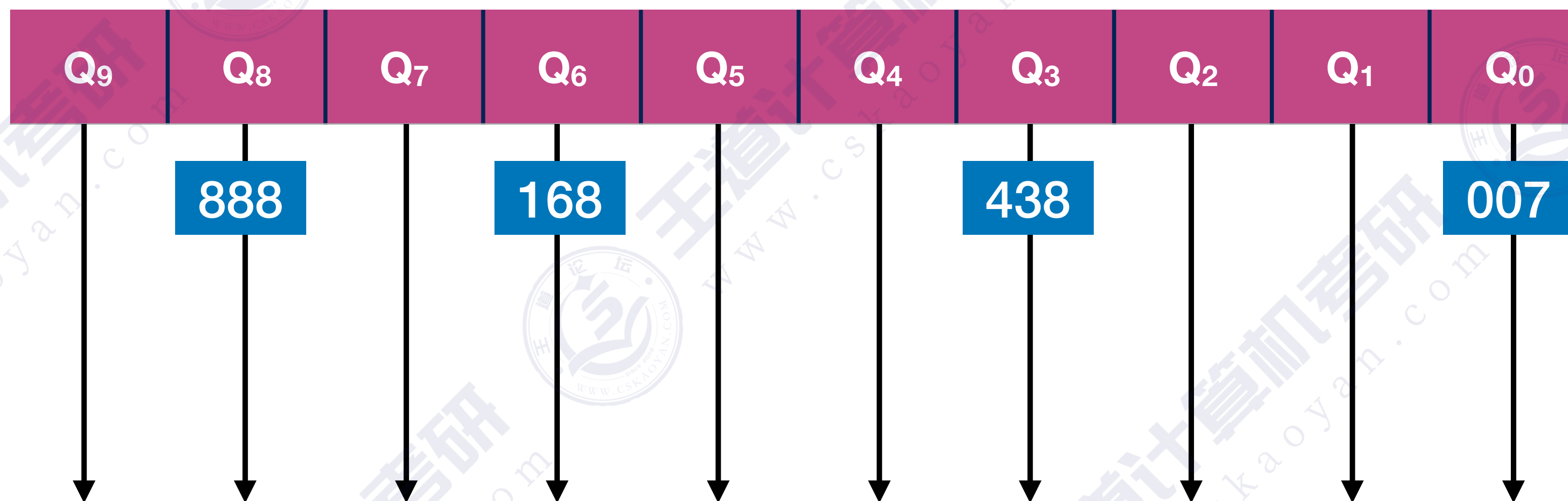
第二趟：以“十位”进行“分配”



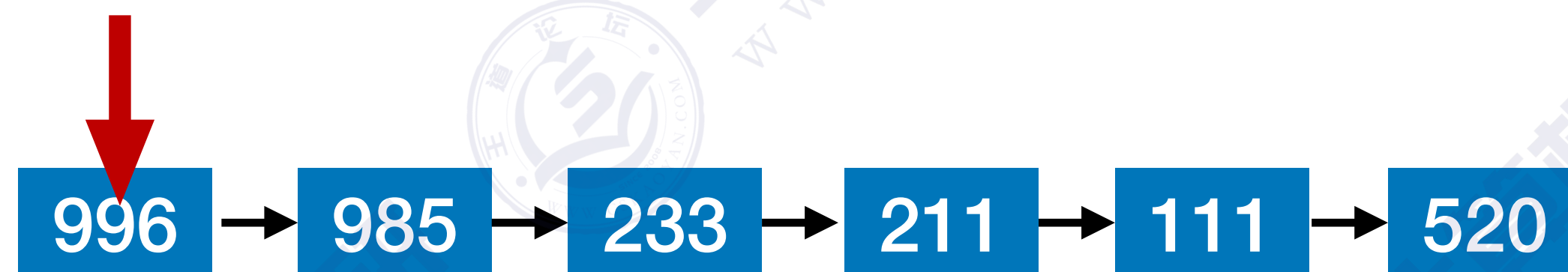
基数排序



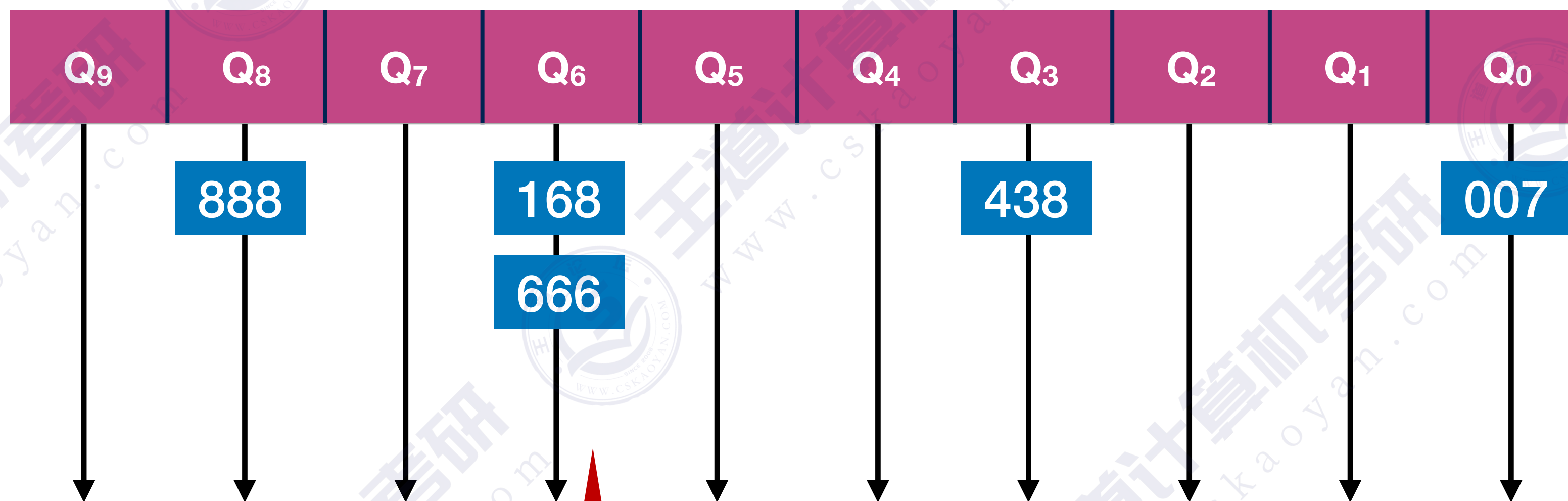
第二趟：以“十位”进行“分配”



基数排序



第二趟：以“十位”进行“分配”

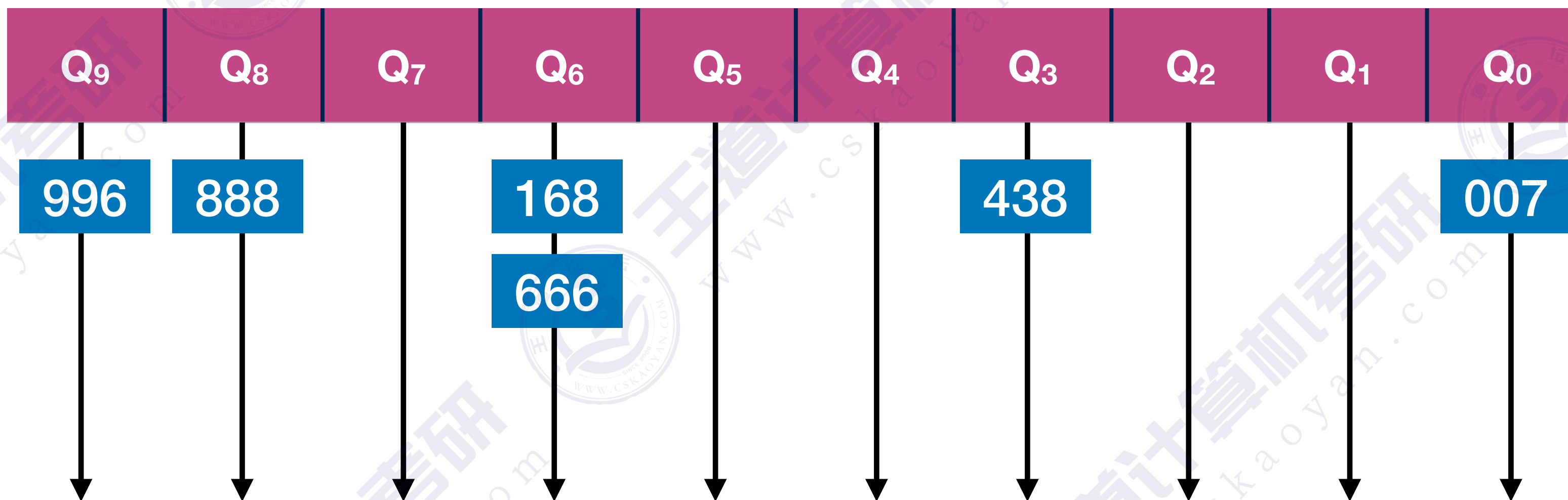


“个位”越大的越先入队

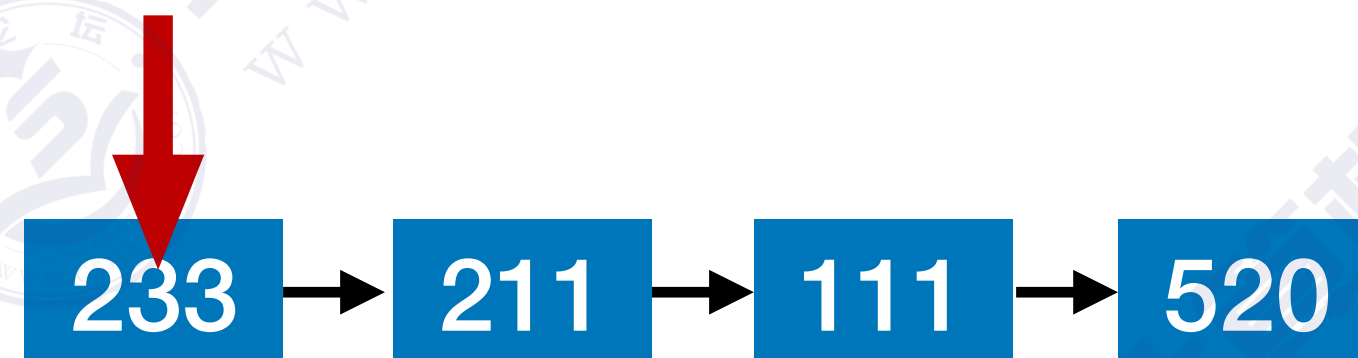
基数排序



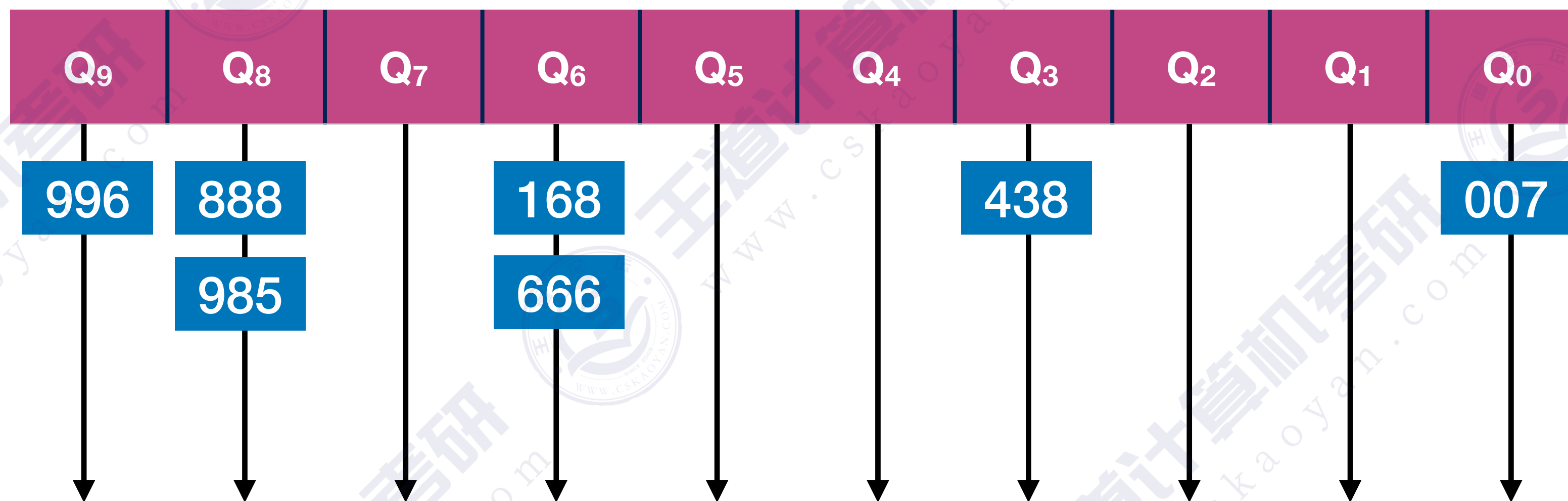
第二趟：以“十位”进行“分配”



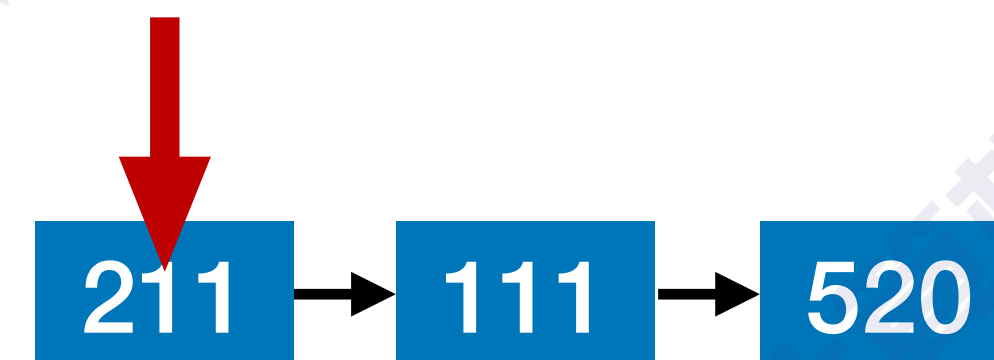
基数排序



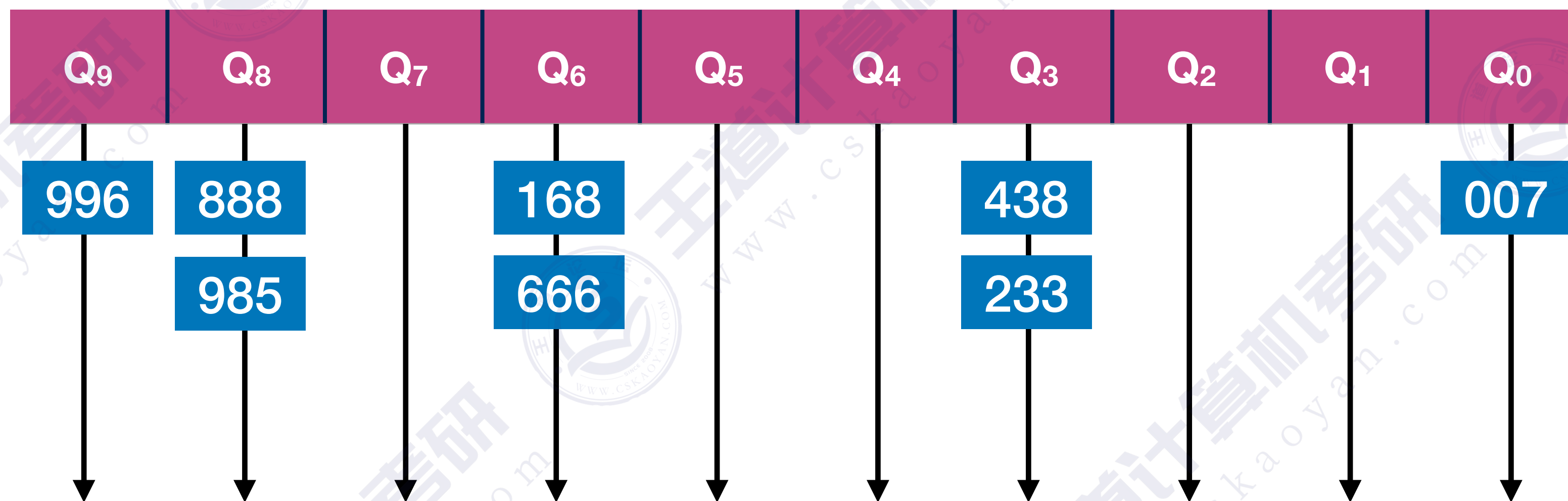
第二趟：以“十位”进行“分配”



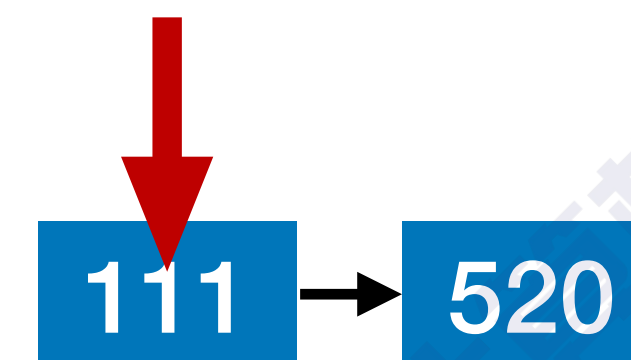
基数排序



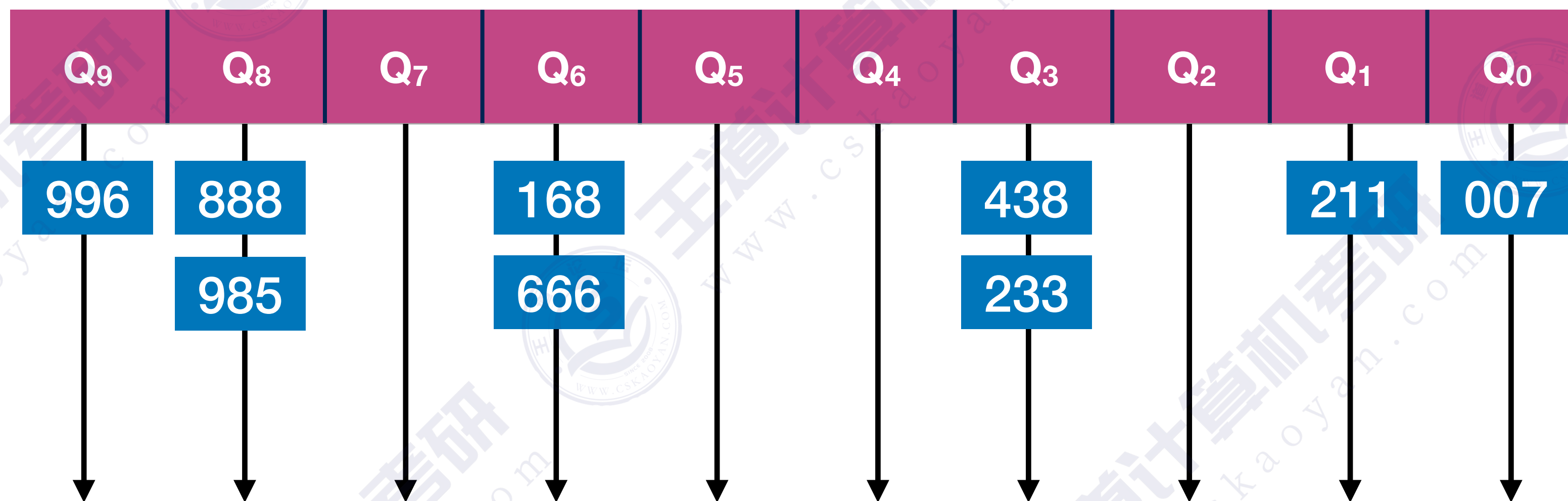
第二趟：以“十位”进行“分配”



基数排序



第二趟：以“十位”进行“分配”

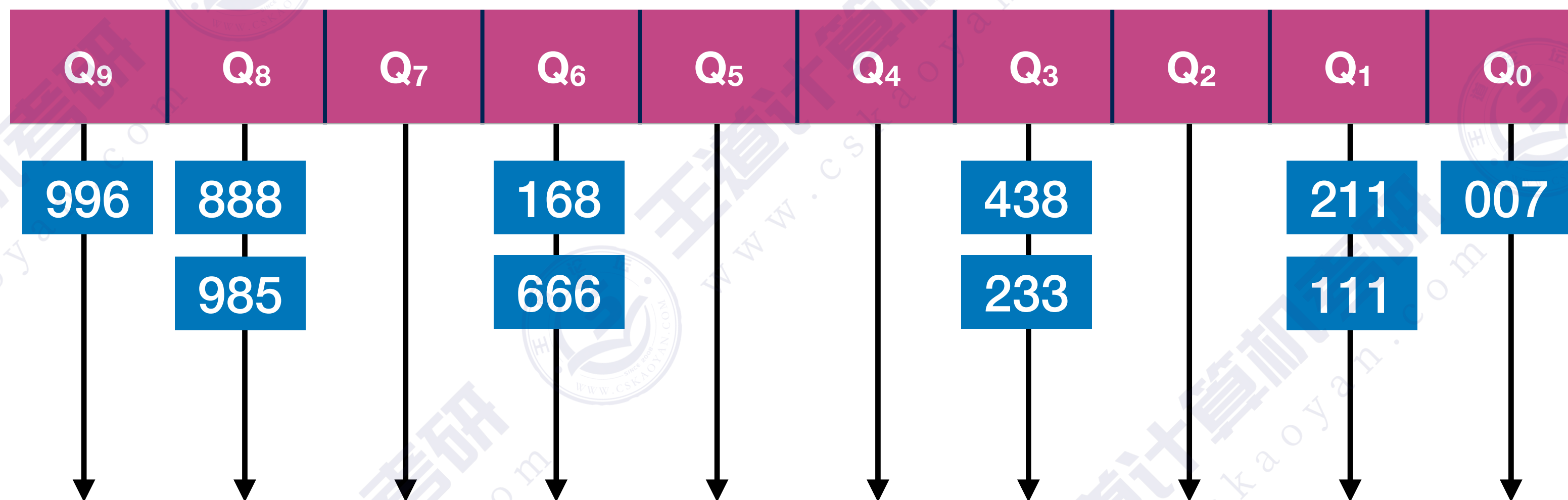


基数排序



520

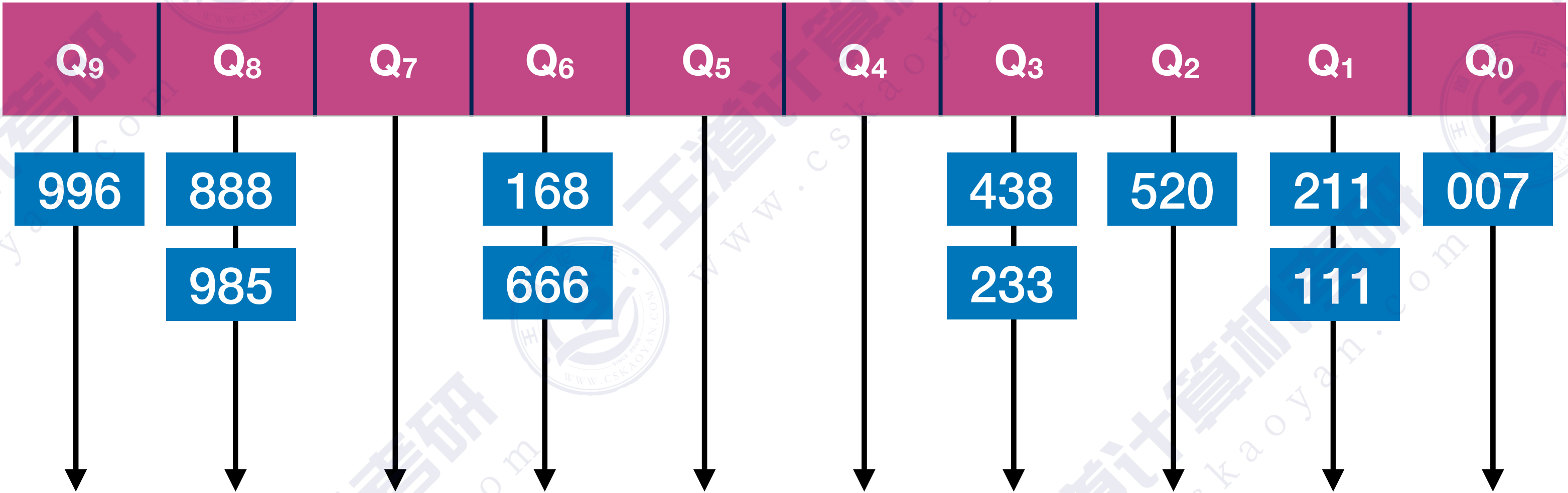
第二趟：以“十位”进行“分配”



基数排序

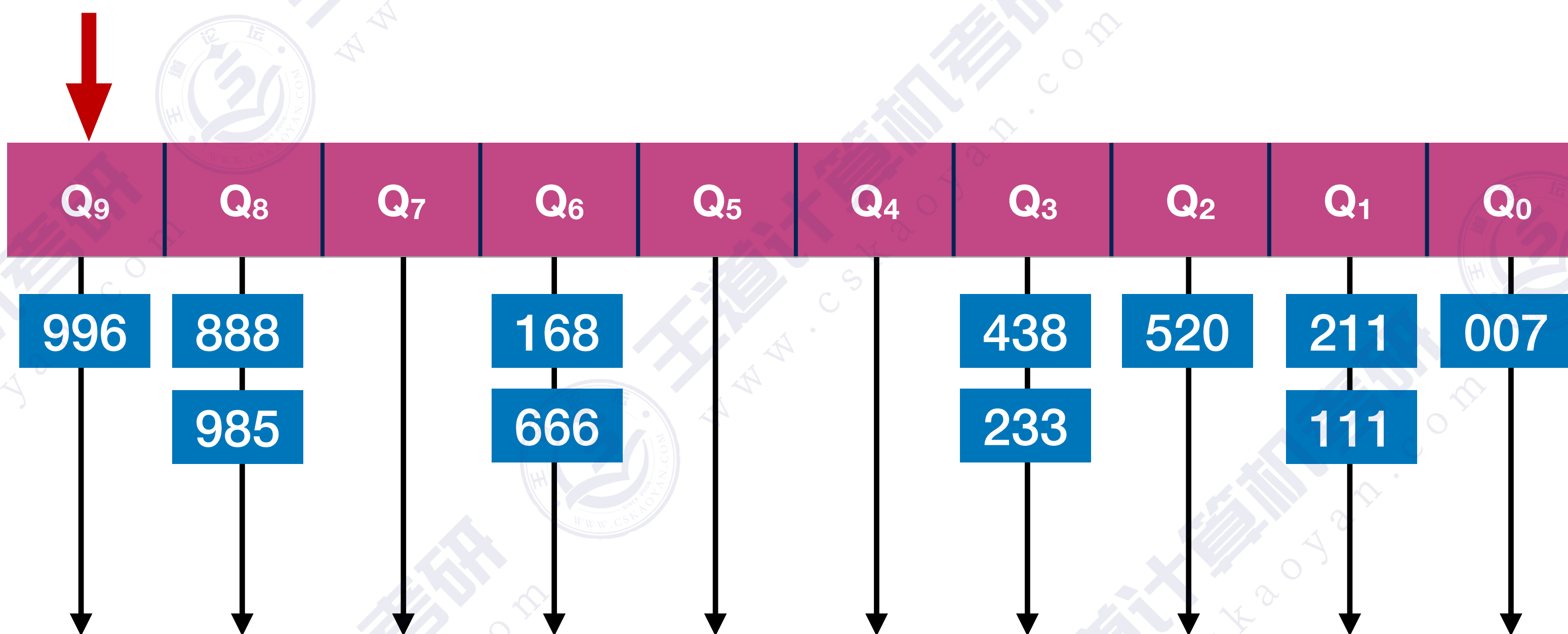


第二趟“分配”结束



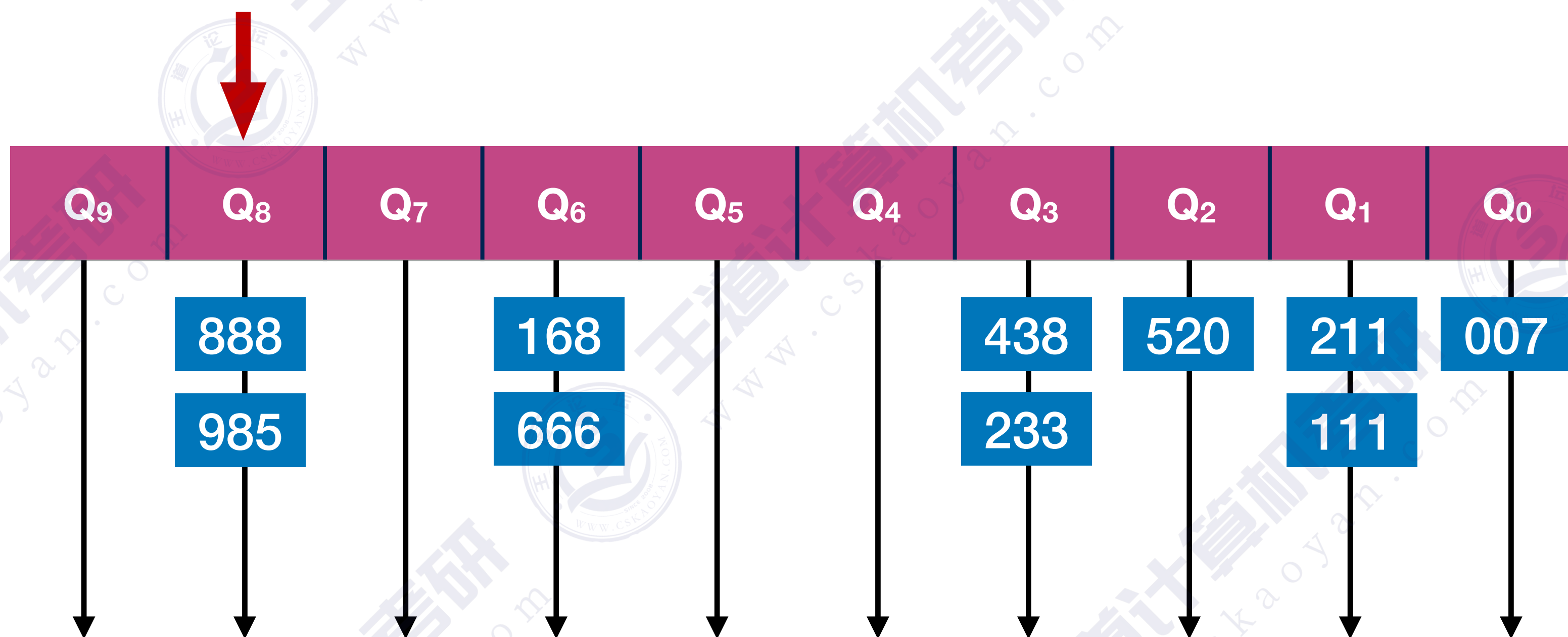
基数排序

第二趟“收集”：



基数排序

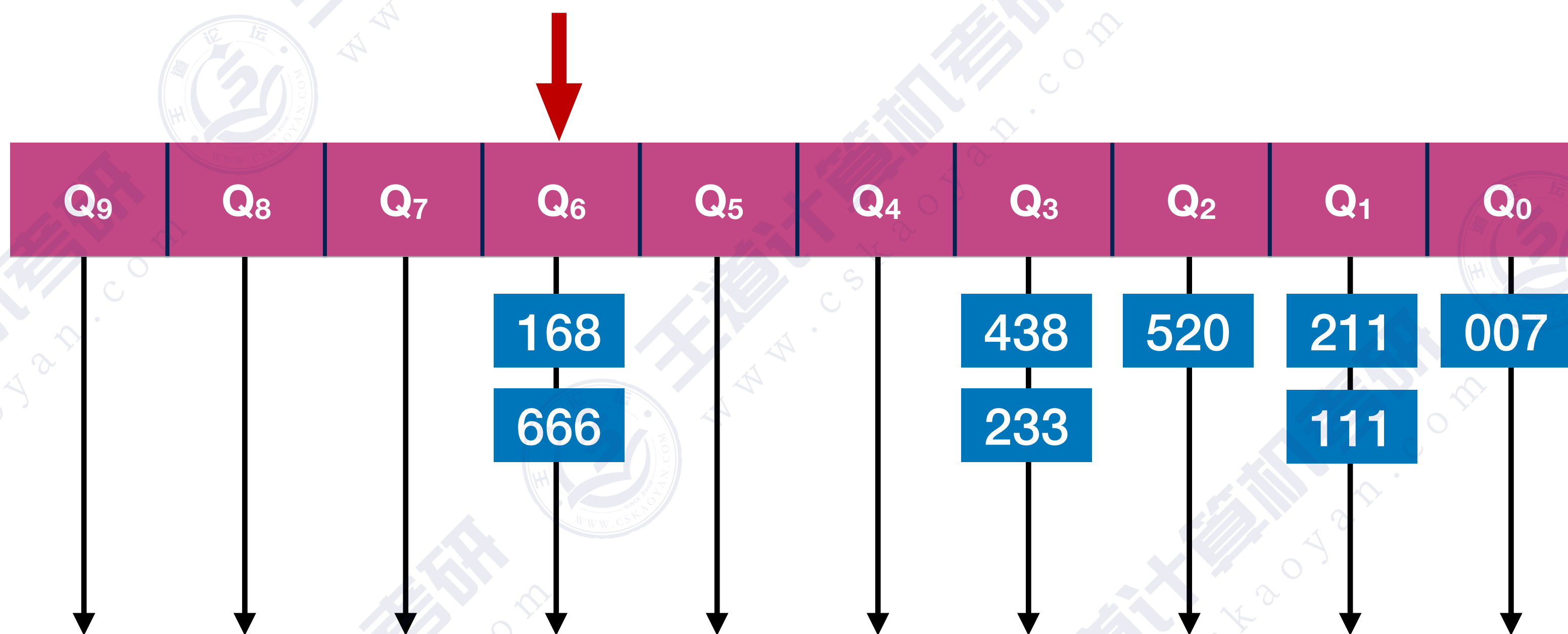
第二趟“收集”：



996

基数排序

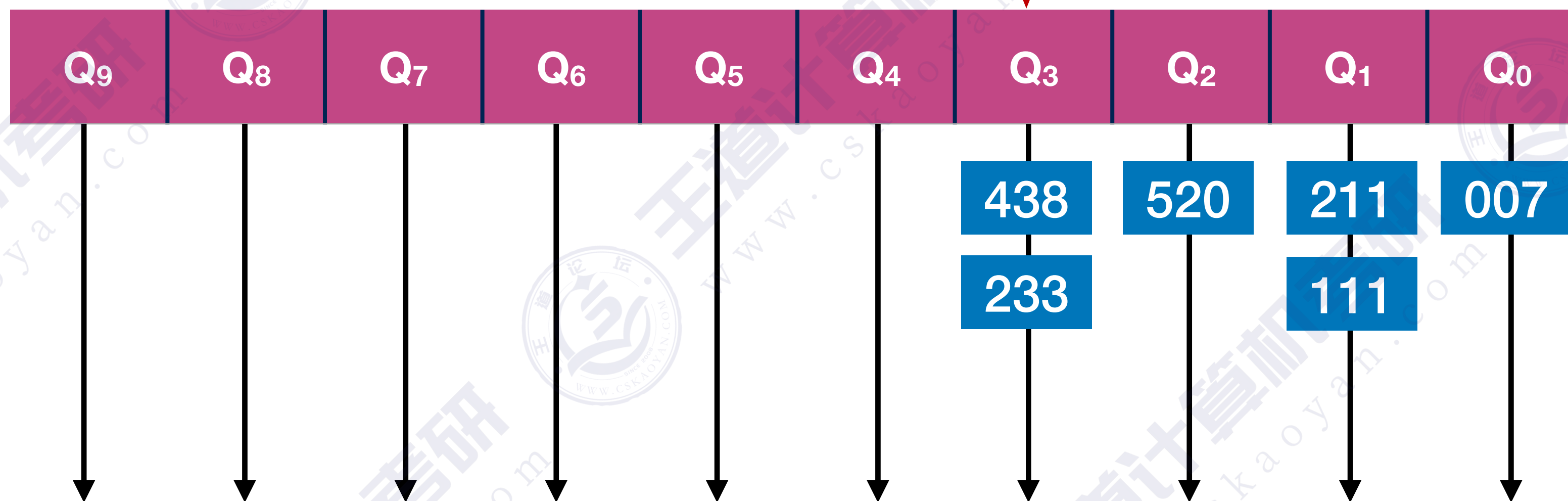
第二趟“收集”:



996 888 985

基数排序

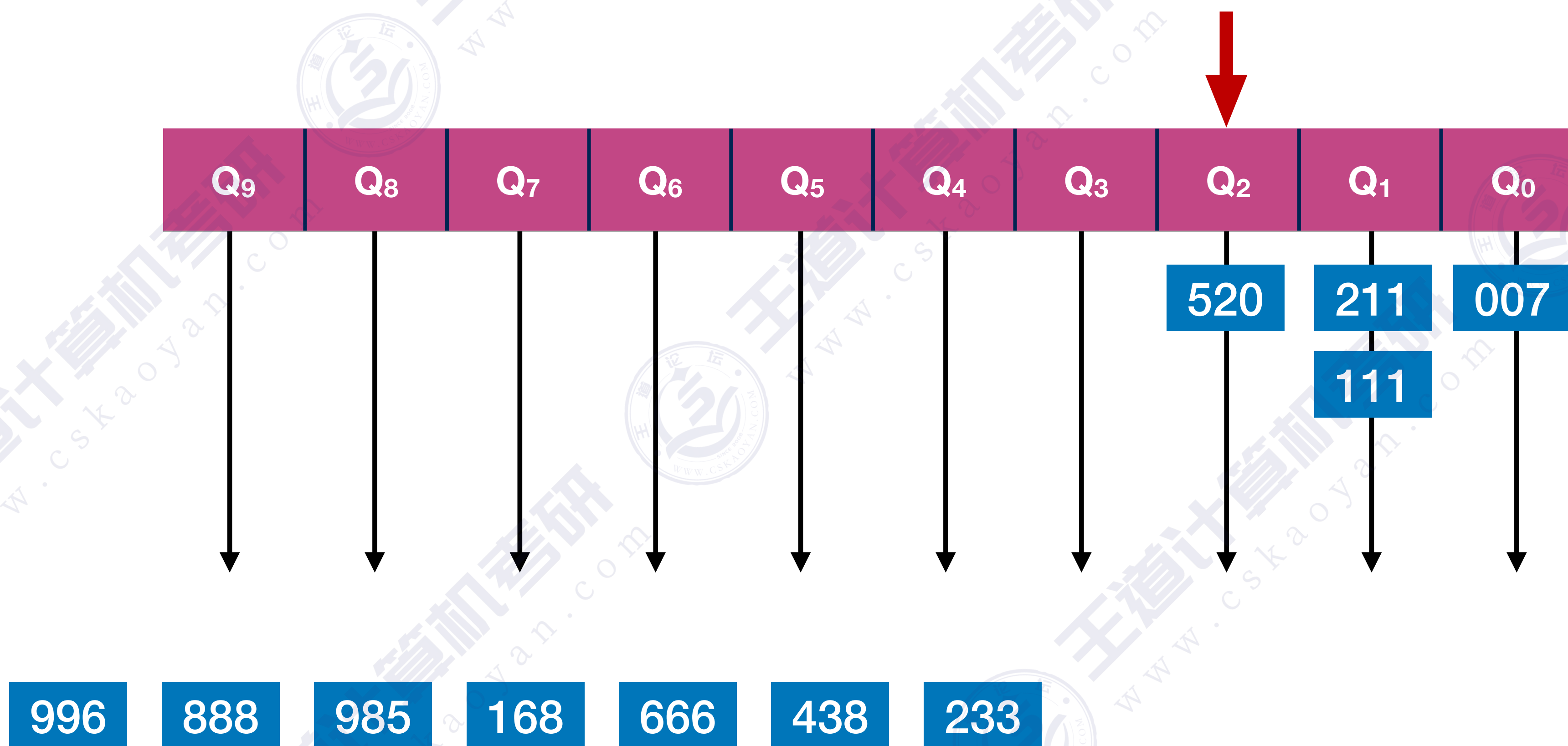
第二趟“收集”:



996 888 985 168 666

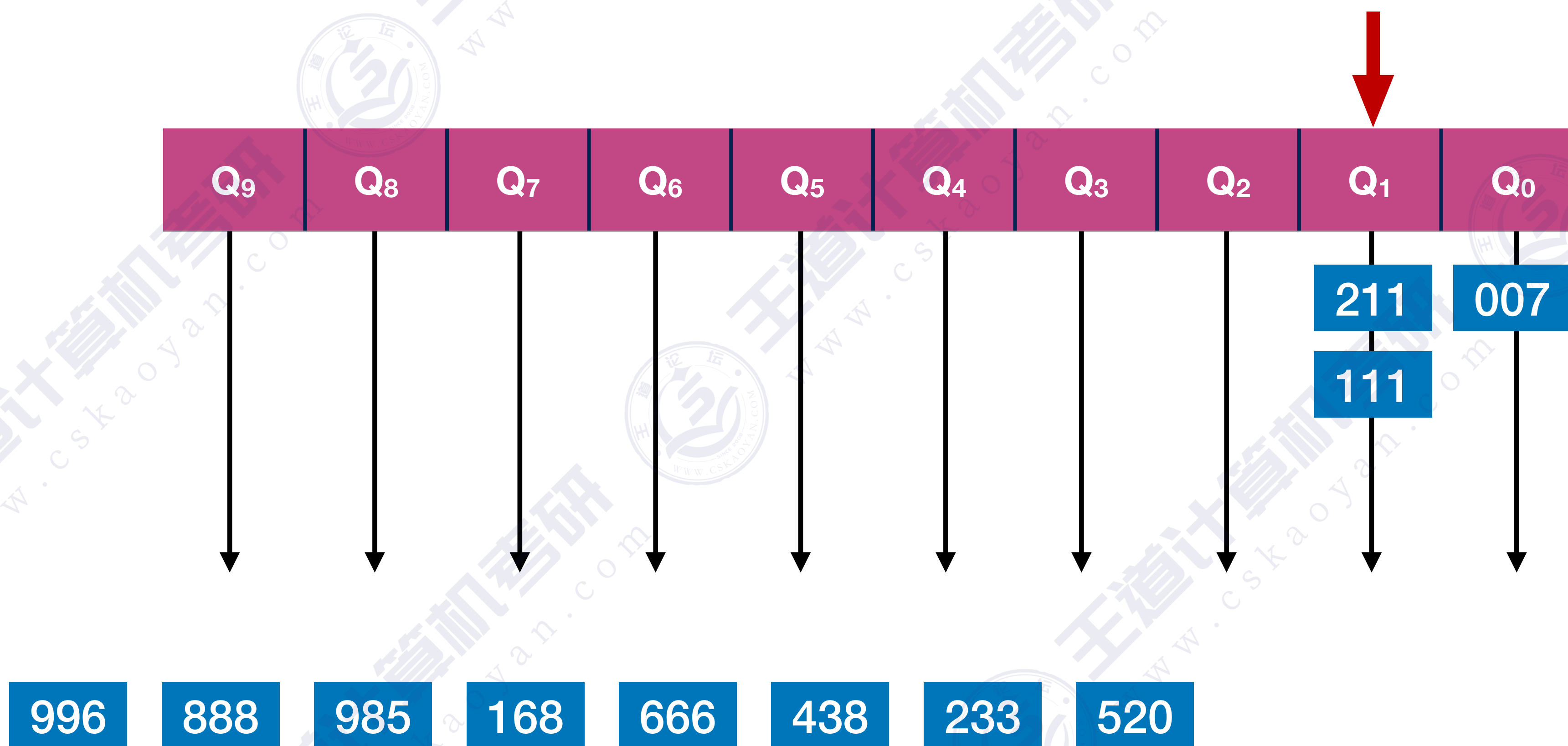
基数排序

第二趟“收集”：



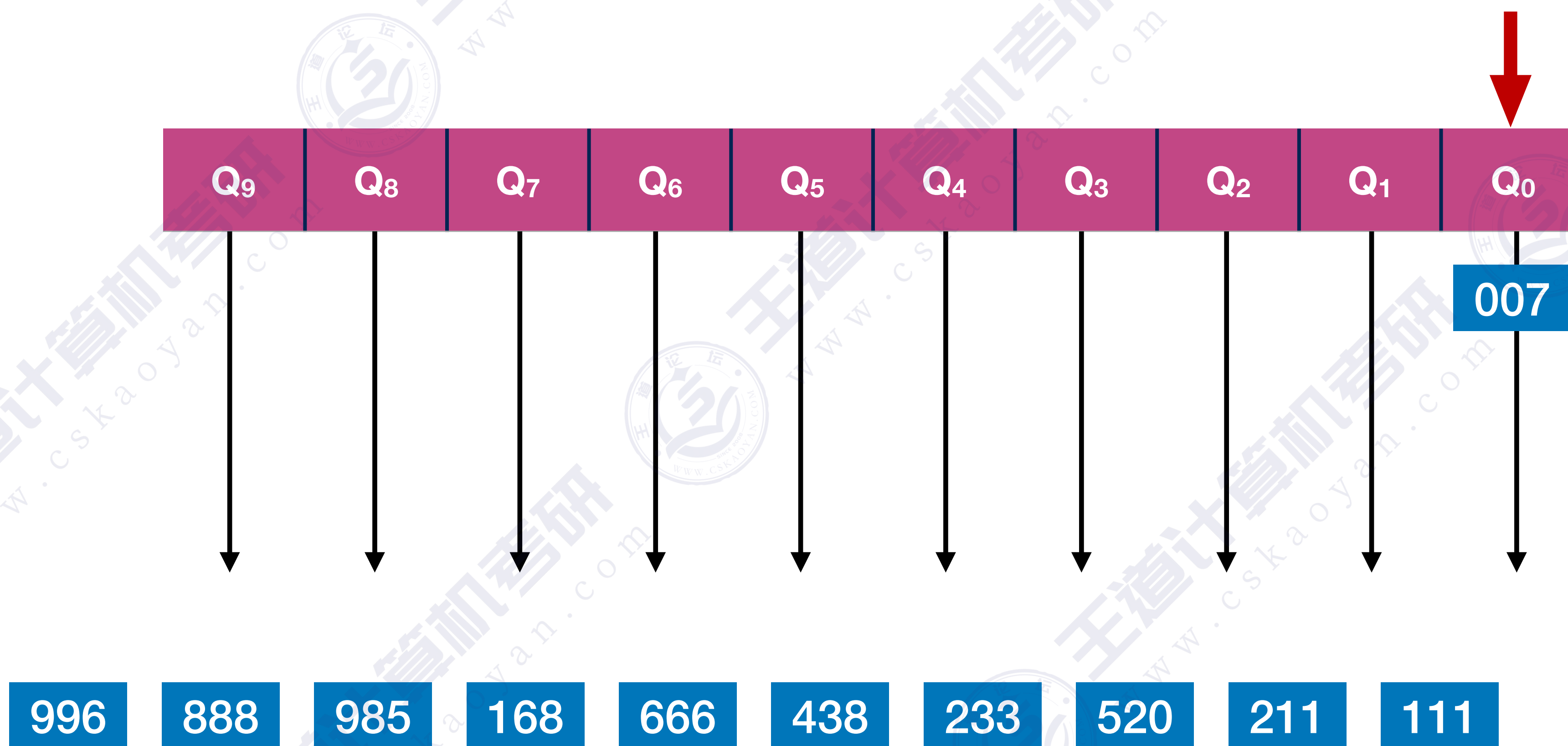
基数排序

第二趟“收集”:



基数排序

第二趟“收集”:



基数排序

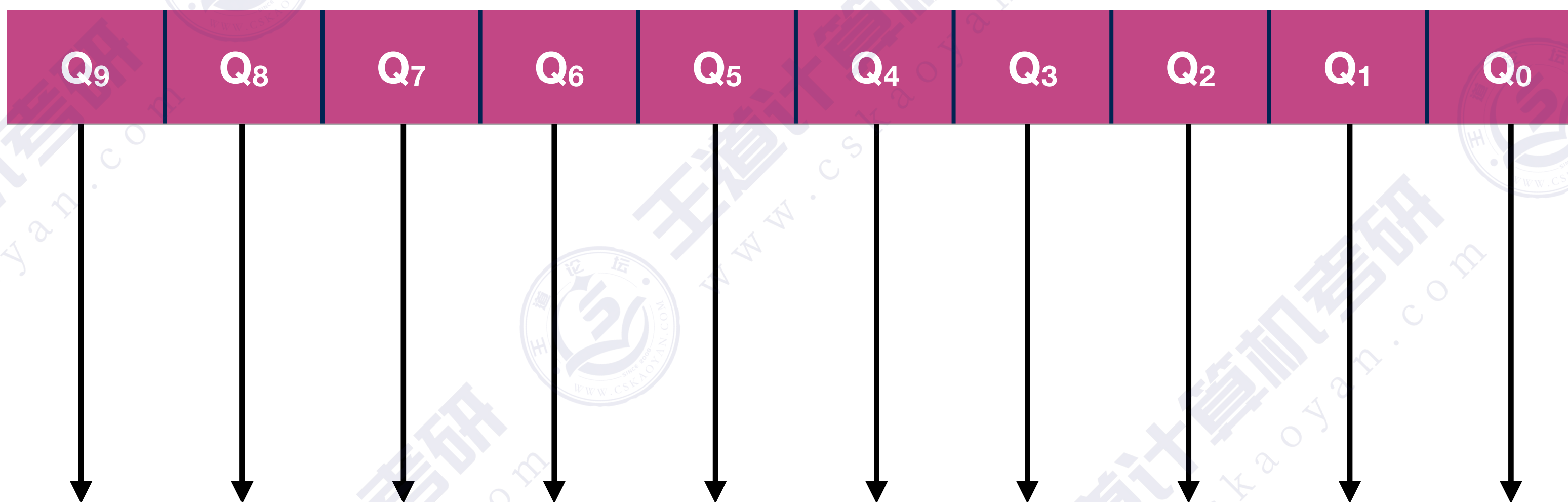


第二趟“收集”：



996 888 985 168 666 438 233 520 211 111 007

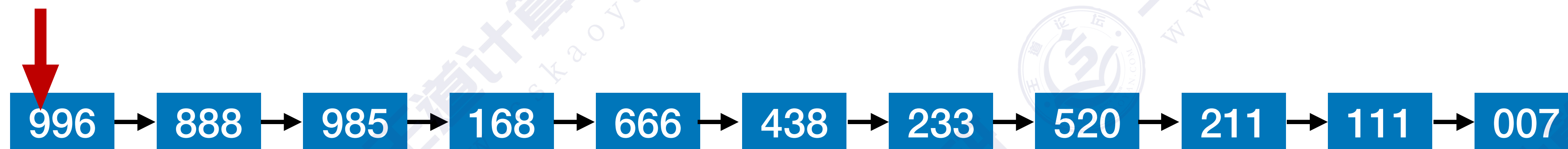
基数排序



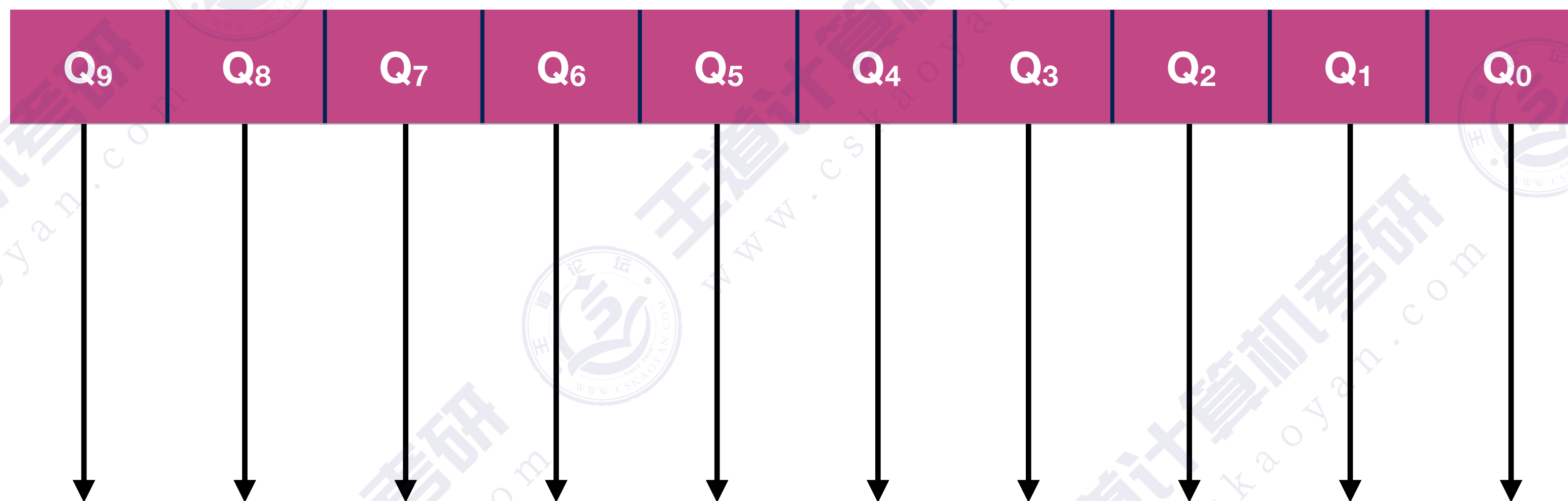
第二趟“收集”结束：得到按“十位”递减排序的序列，“十位”相同的按“个位”递减排序

996 → 888 → 985 → 168 → 666 → 438 → 233 → 520 → 211 → 111 → 007

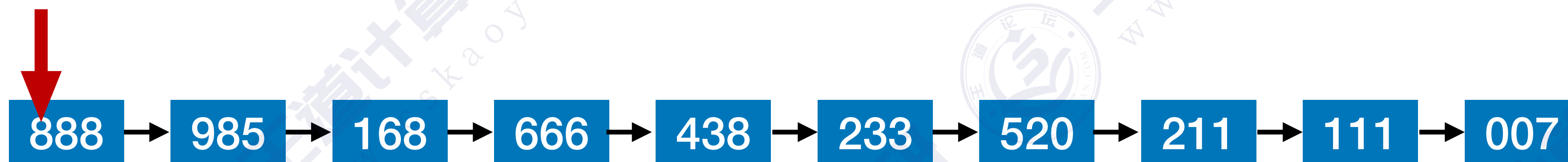
基数排序



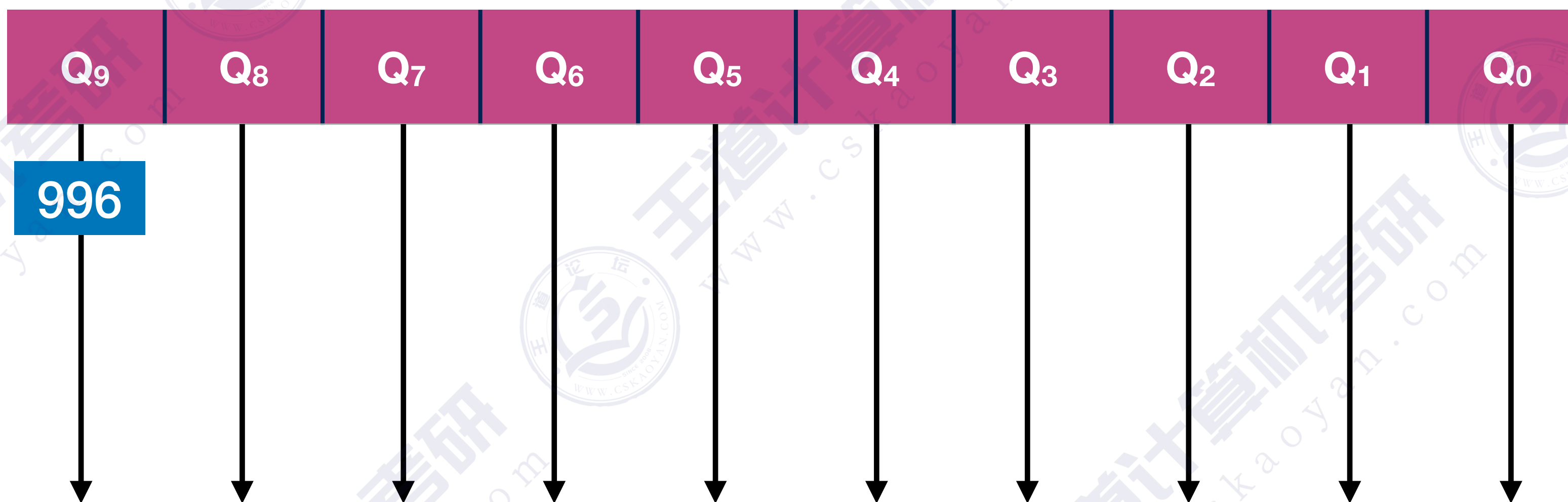
第三趟：以“百位”进行“分配”



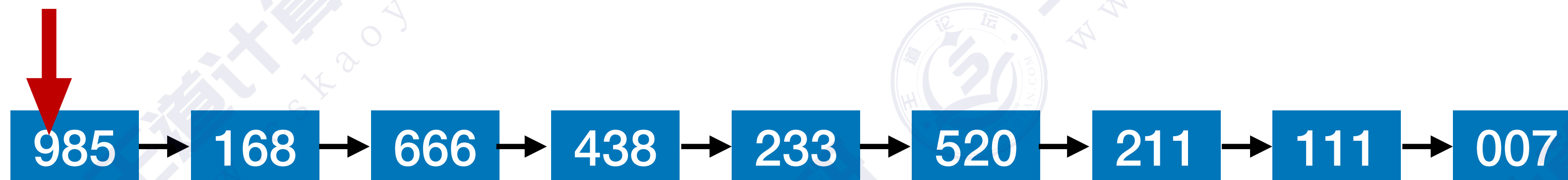
基数排序



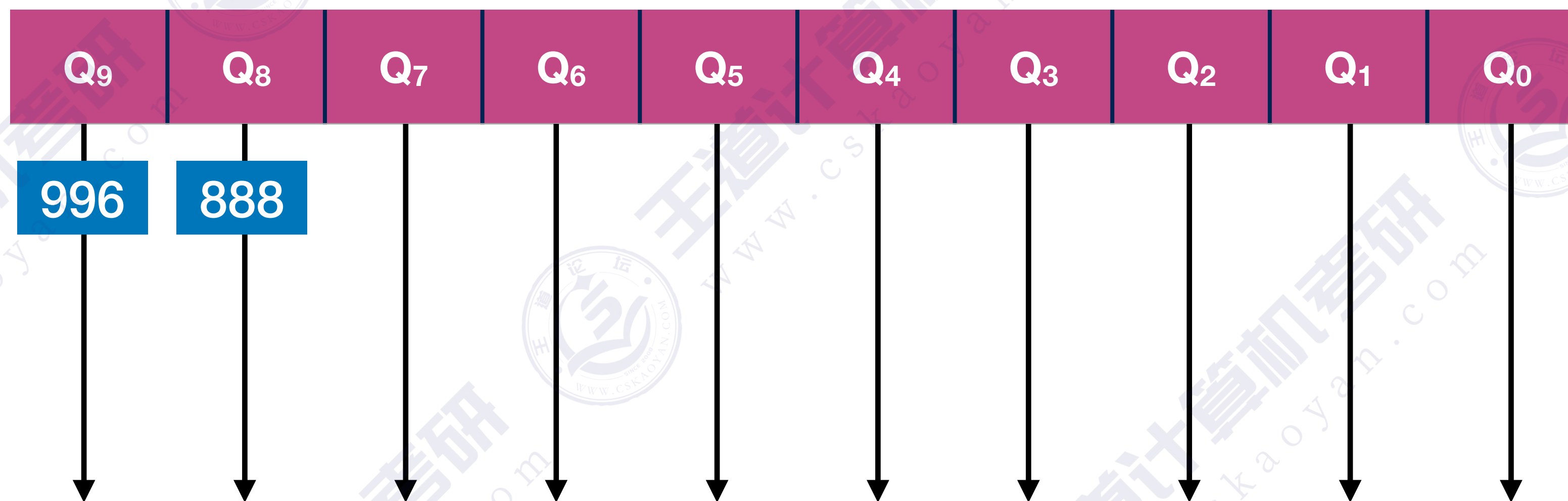
第三趟：以“百位”进行“分配”



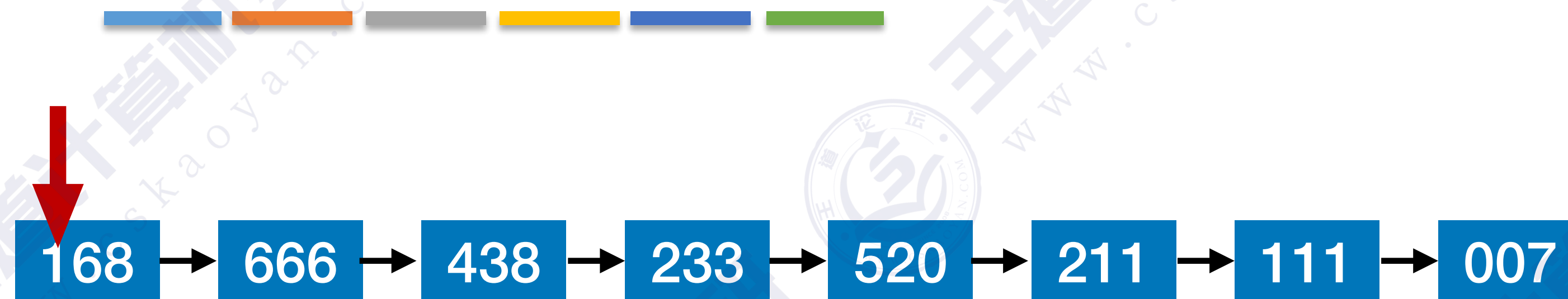
基数排序



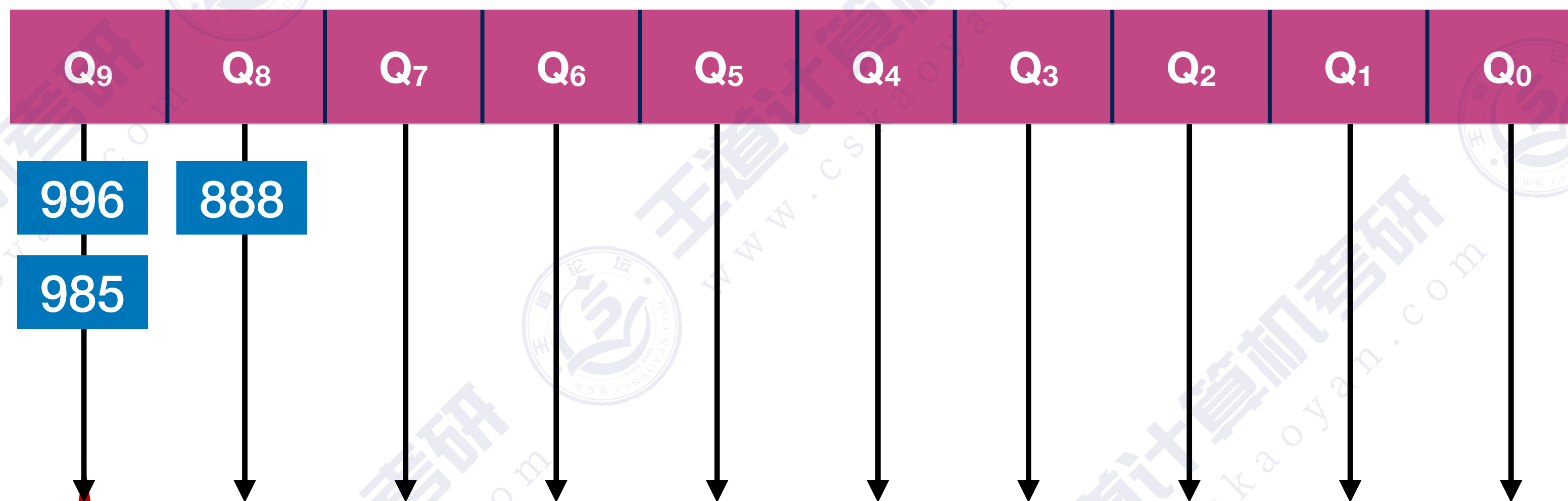
第三趟：以“百位”进行“分配”



基数排序



第三趟：以“百位”进行“分配”

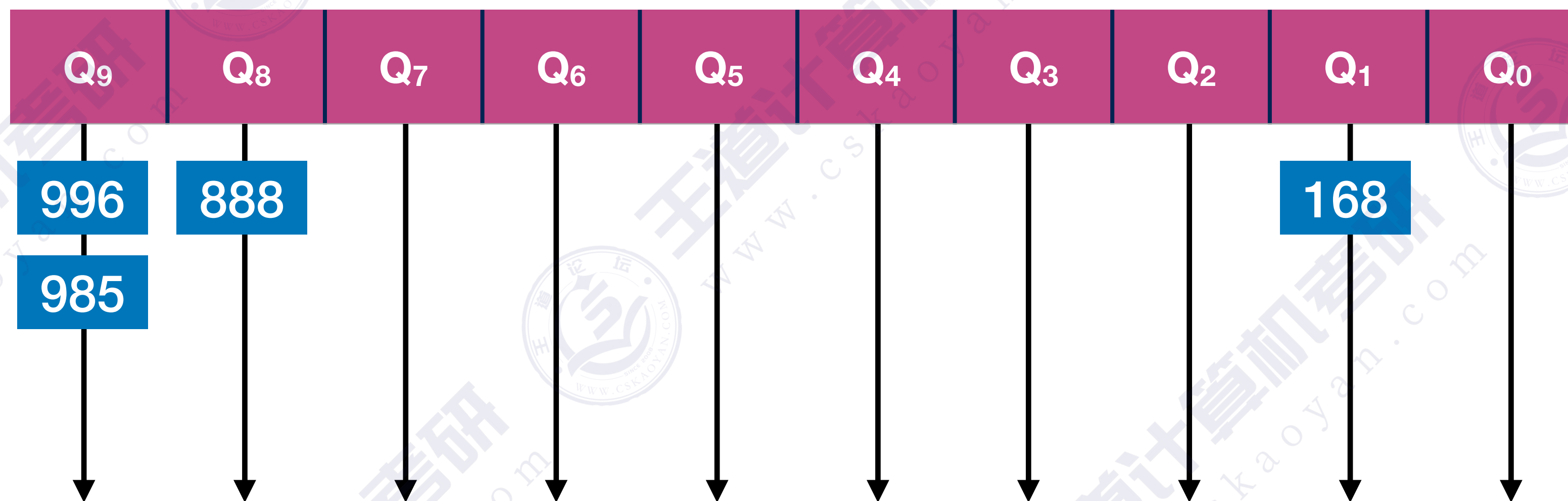


“十位”越大的越先入队

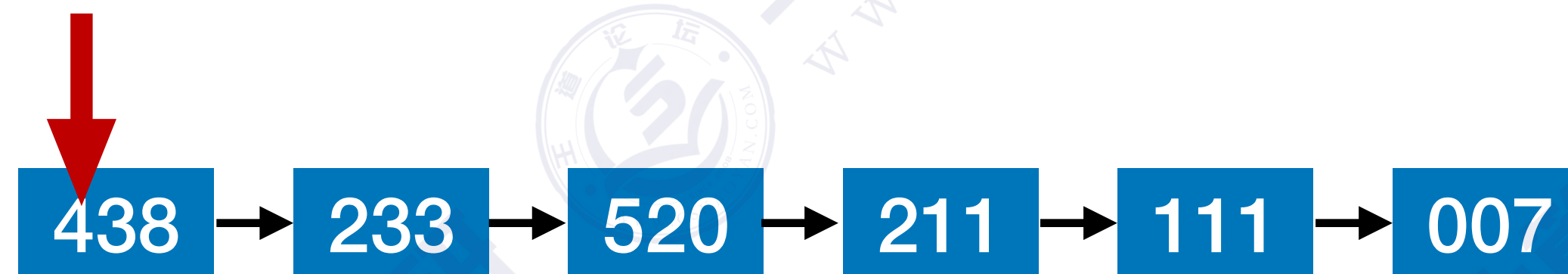
基数排序



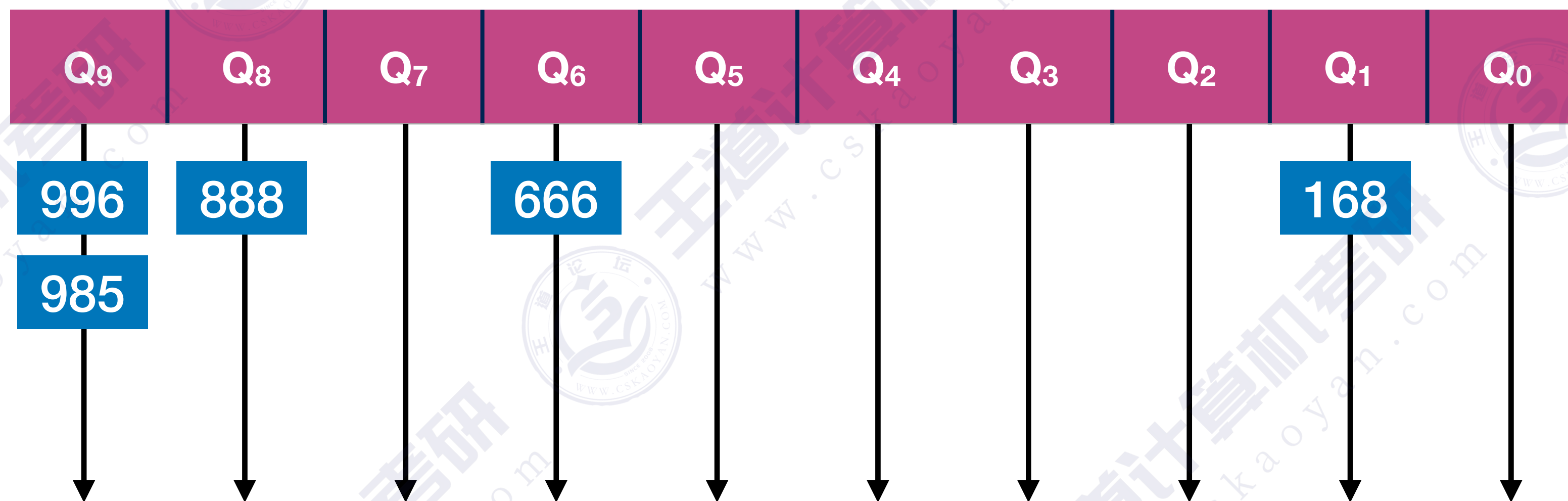
第三趟：以“百位”进行“分配”



基数排序



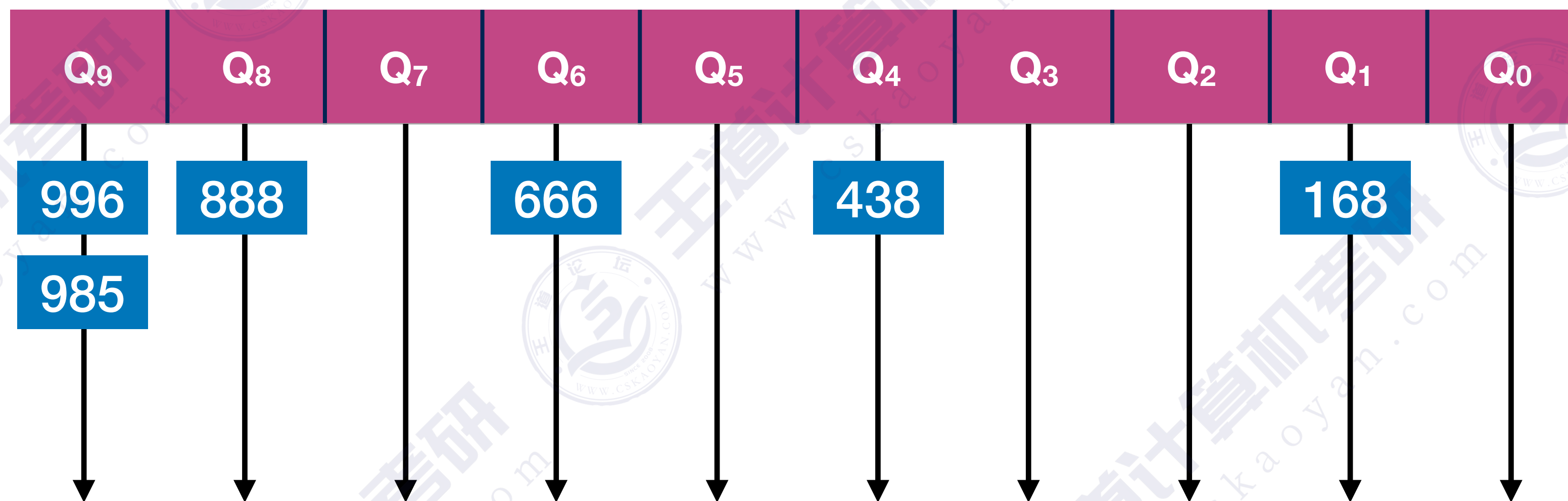
第三趟：以“百位”进行“分配”



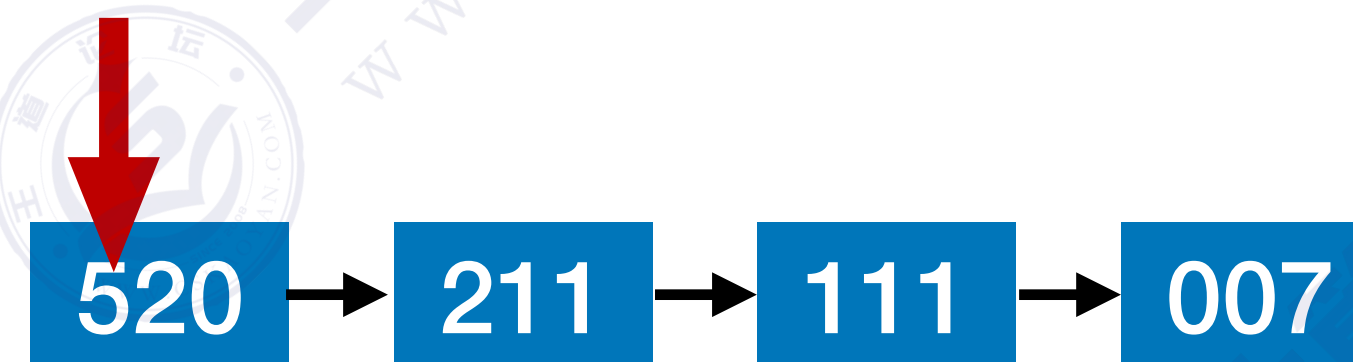
基数排序

233 → 520 → 211 → 111 → 007

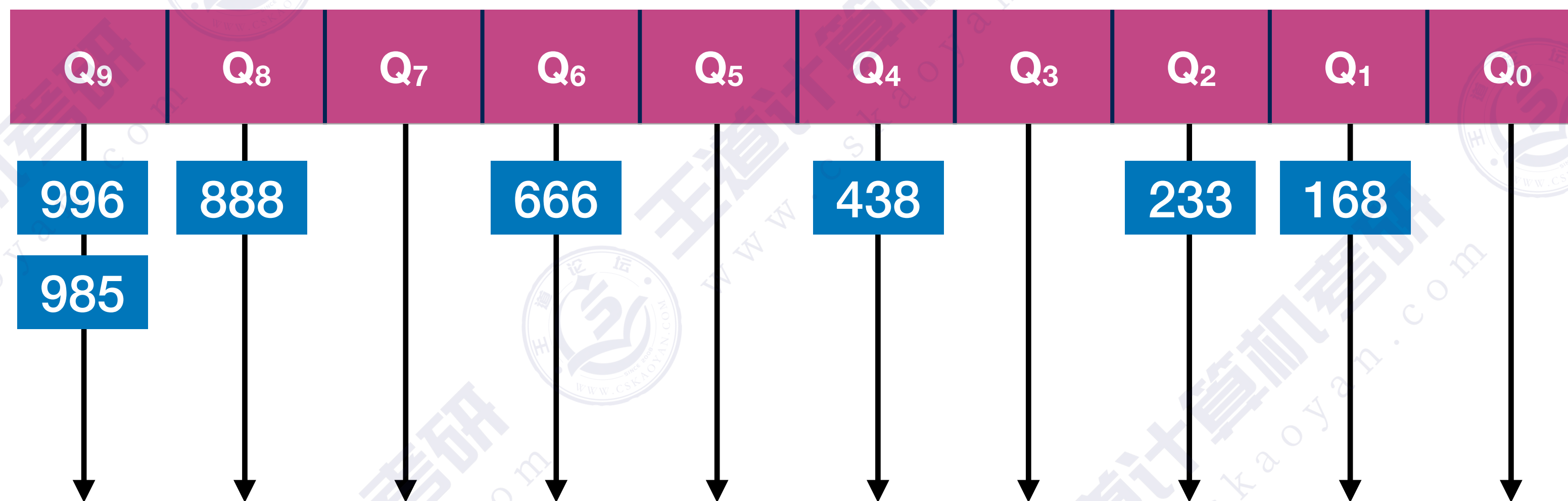
第三趟：以“百位”进行“分配”



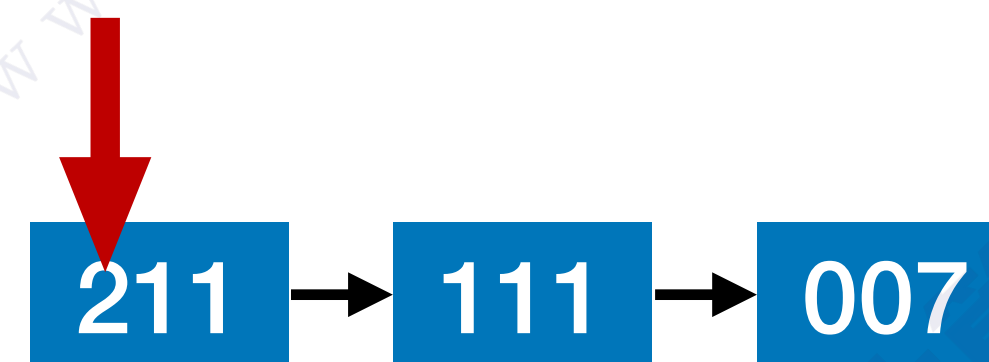
基数排序



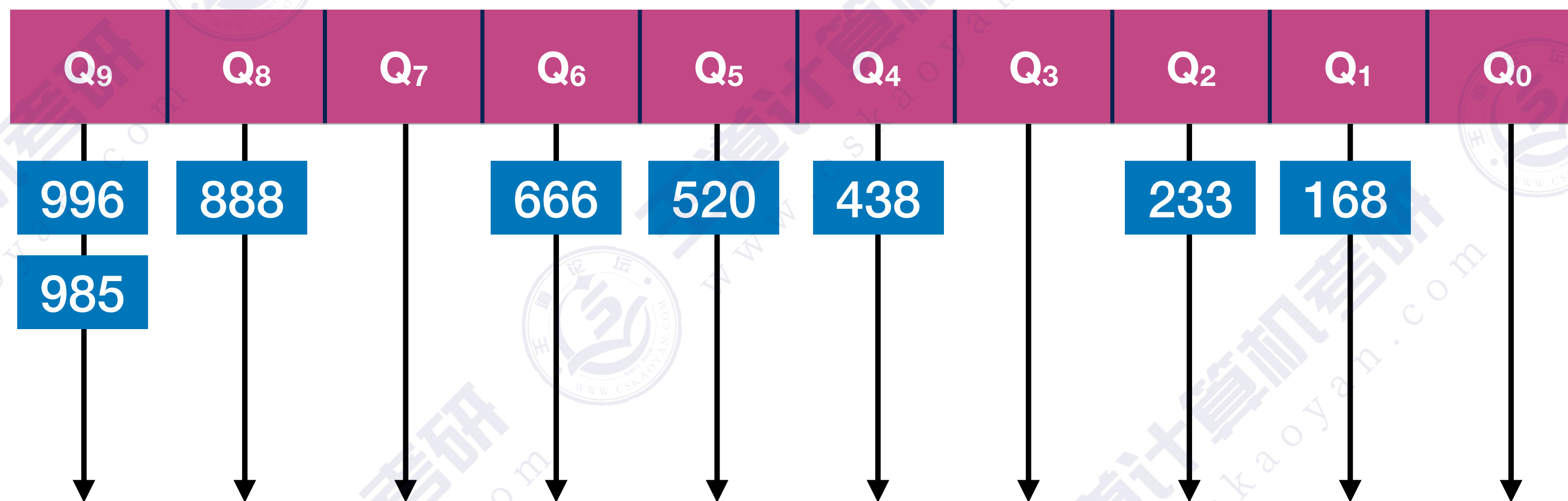
第三趟：以“百位”进行“分配”



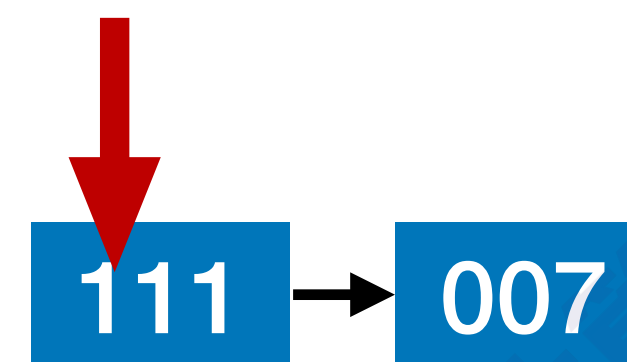
基数排序



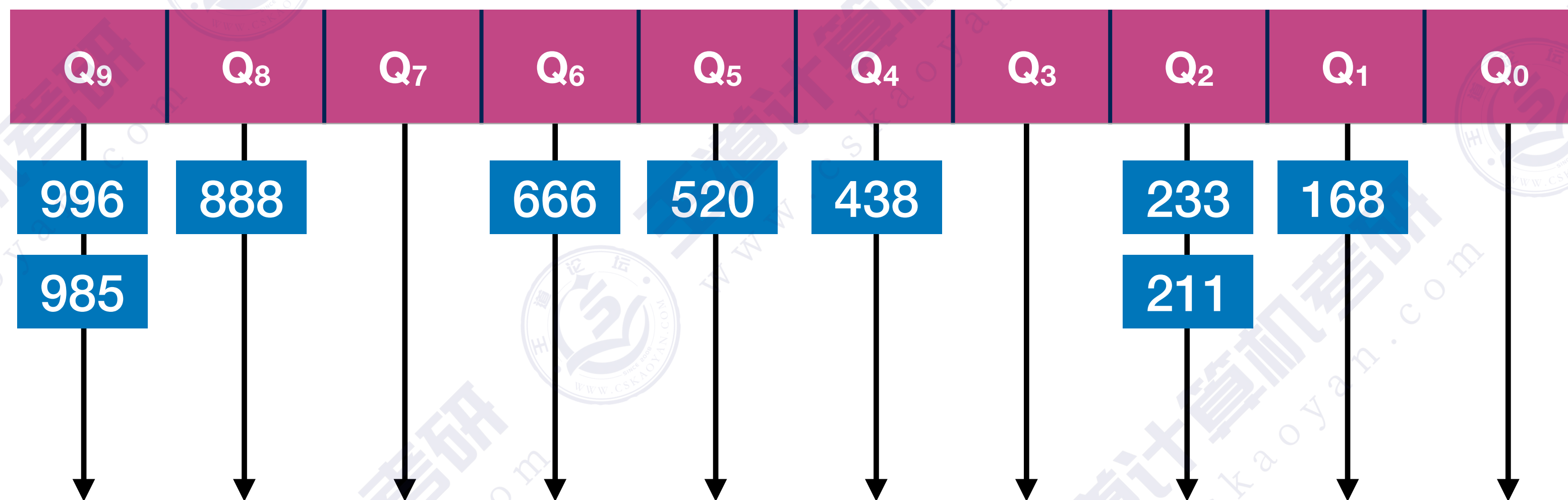
第三趟：以“百位”进行“分配”



基数排序



第三趟：以“百位”进行“分配”

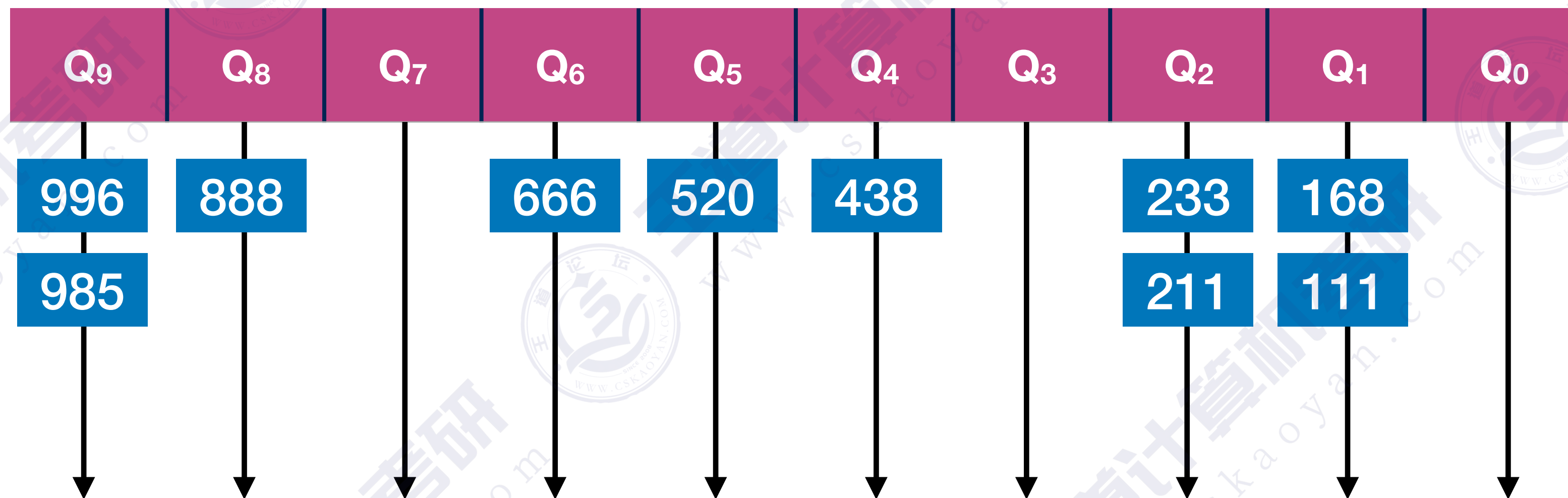


基数排序



007

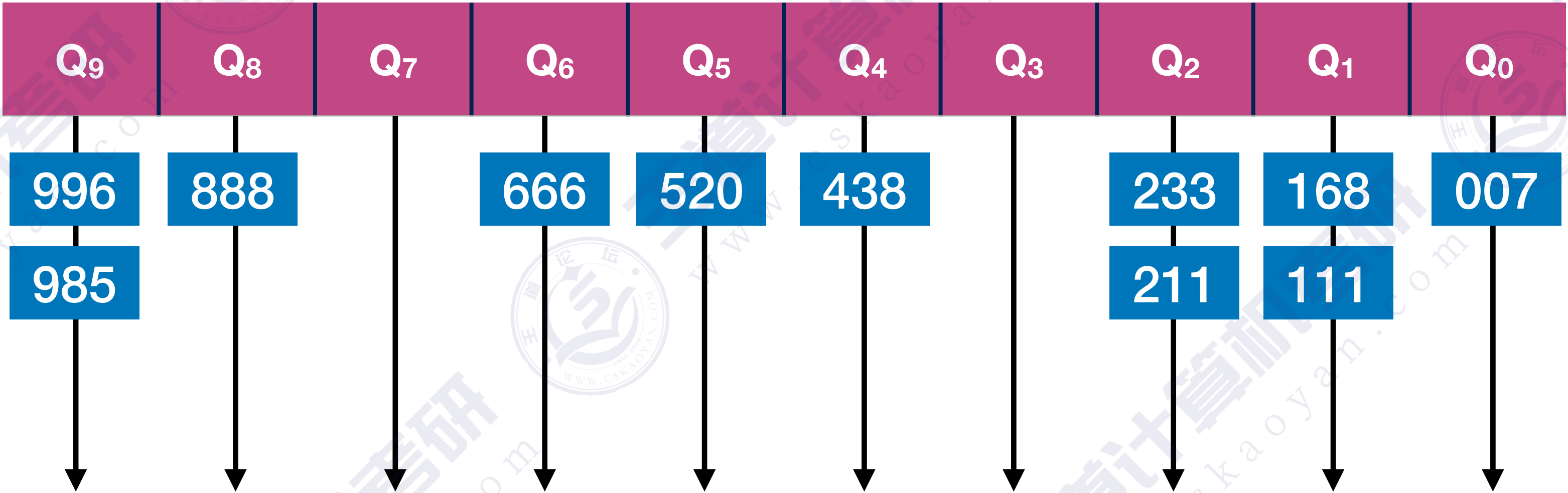
第三趟：以“百位”进行“分配”



基数排序

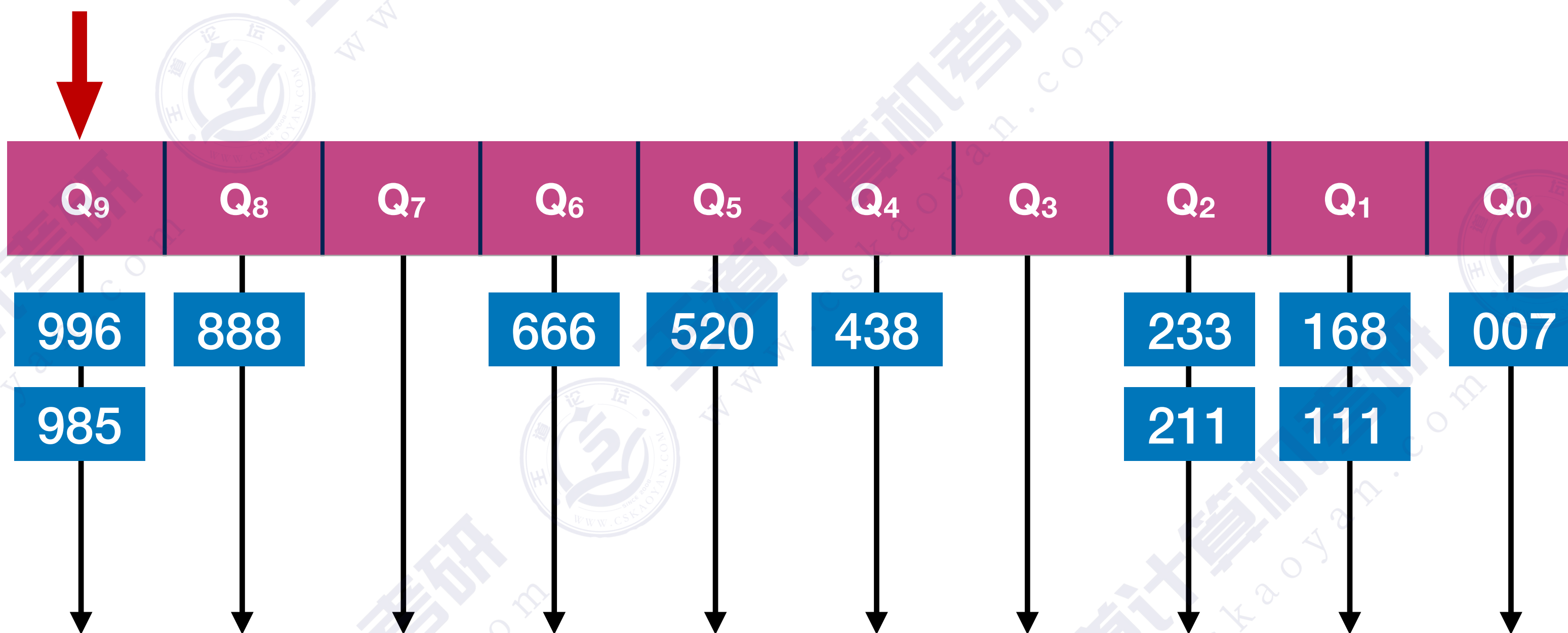


第三趟：以“百位”进行“分配”



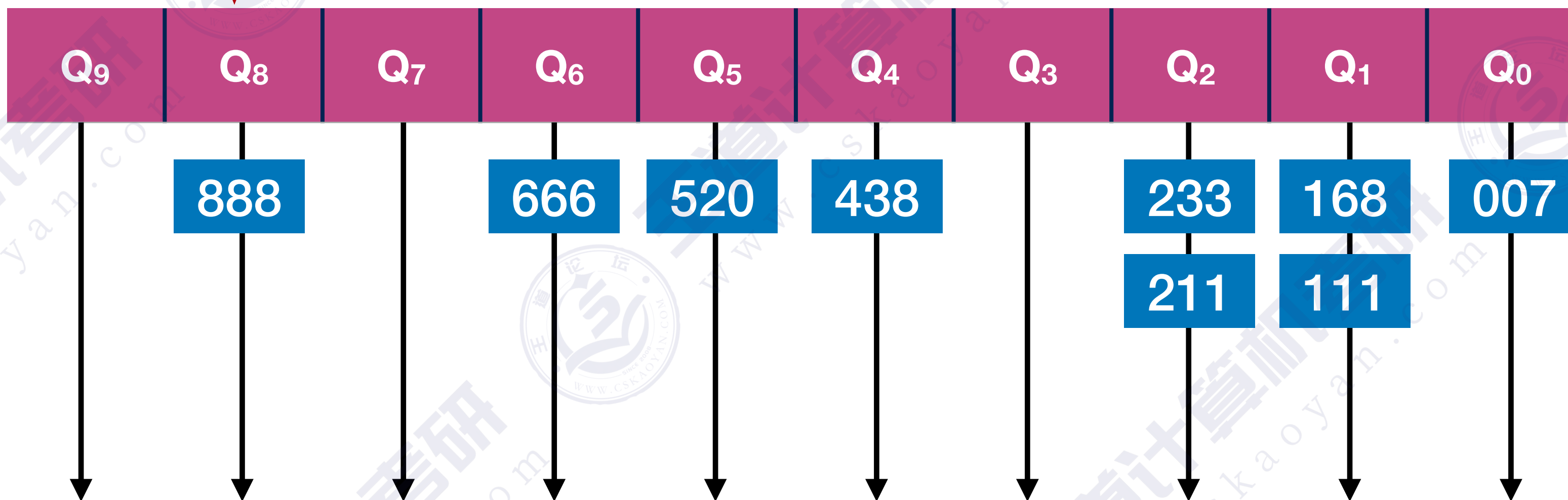
基数排序

第三趟“收集”



基数排序

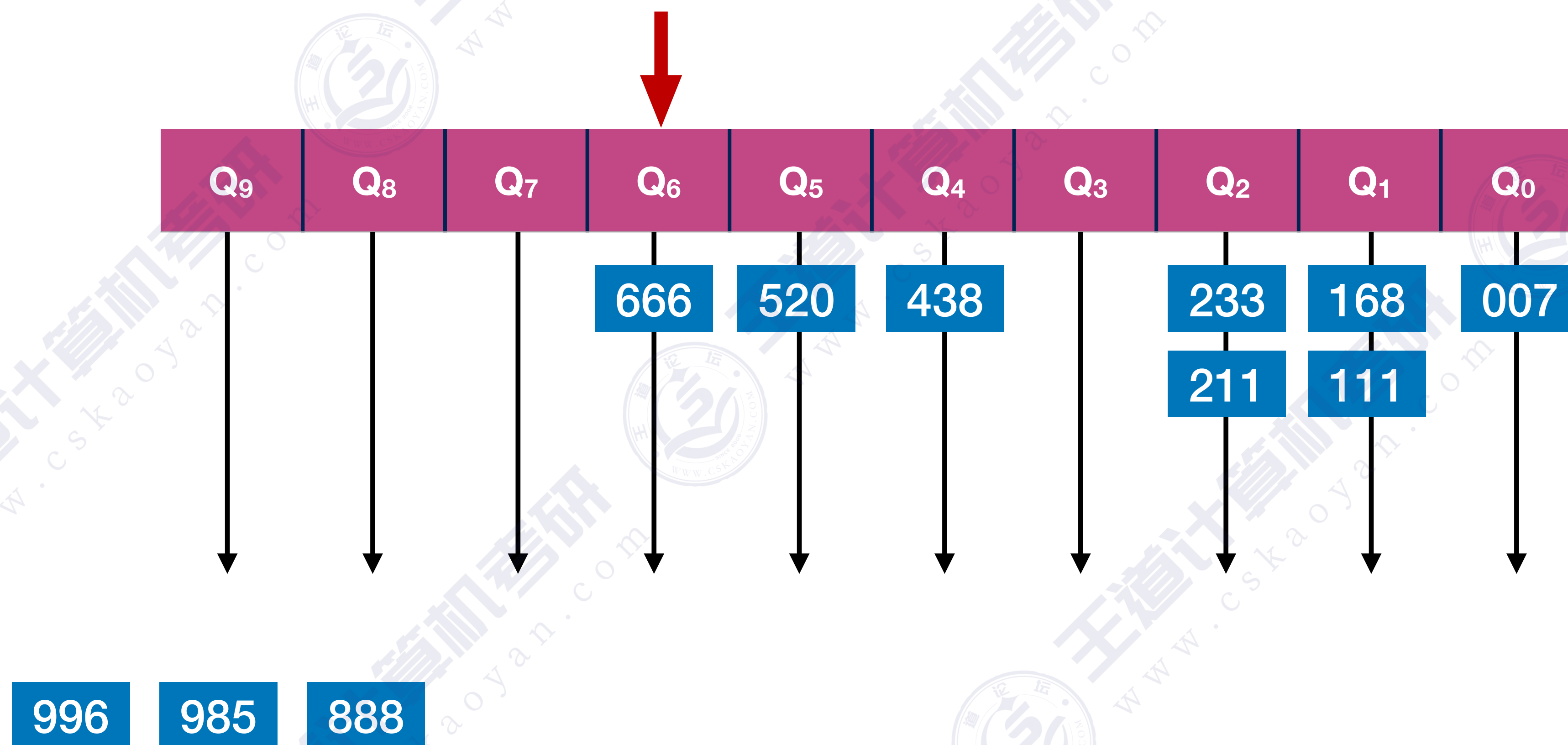
第三趟“收集”



996 985

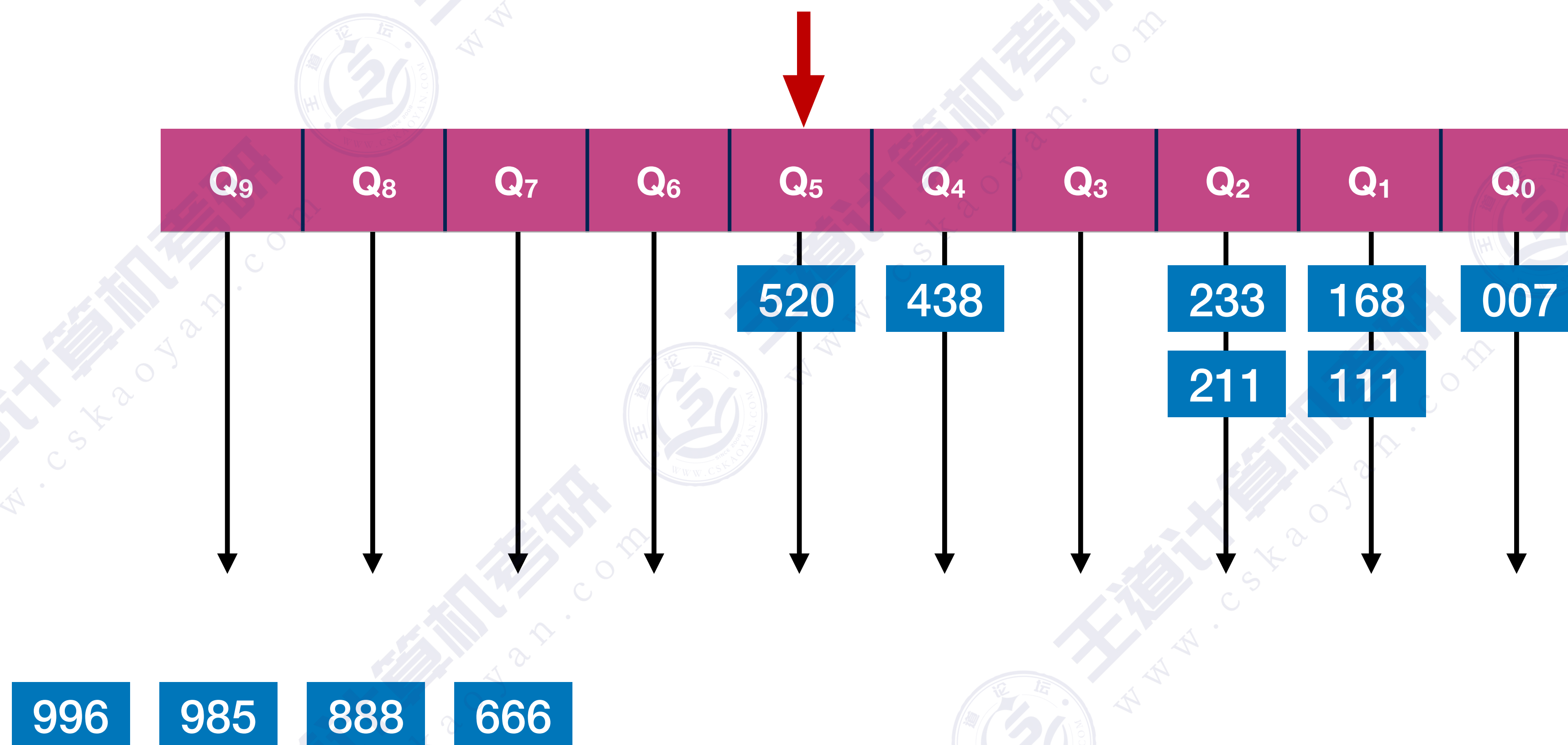
基数排序

第三趟“收集”



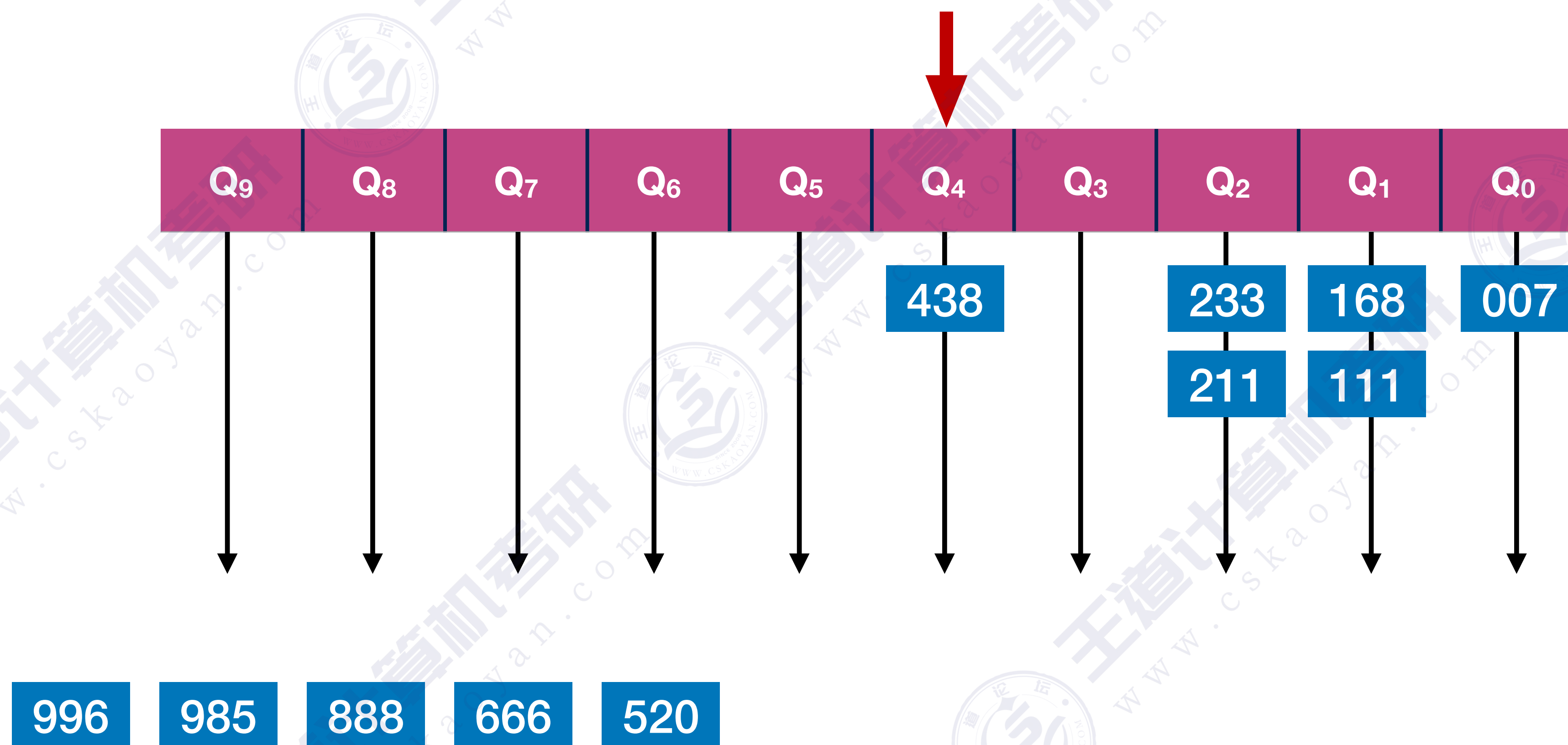
基数排序

第三趟“收集”



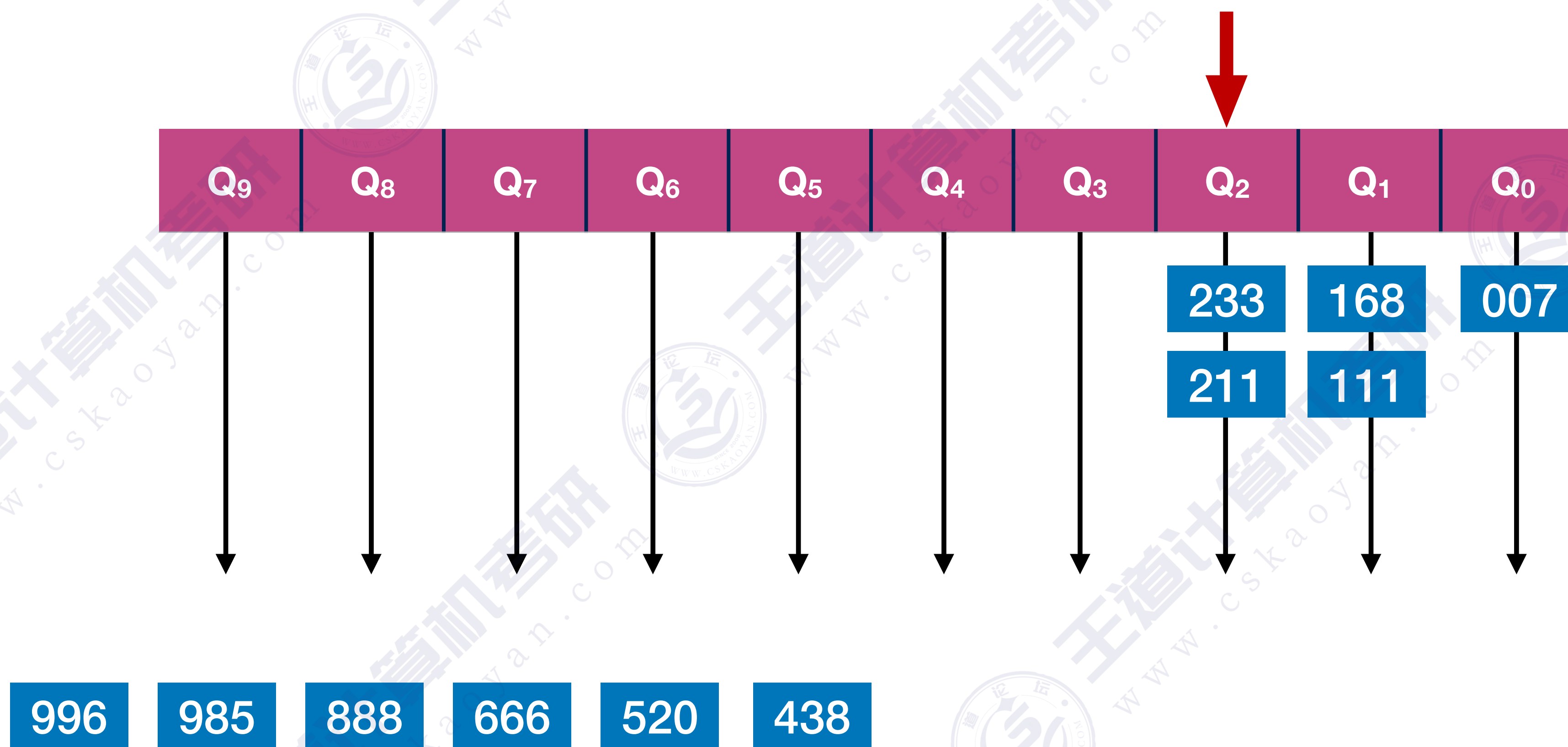
基数排序

第三趟“收集”



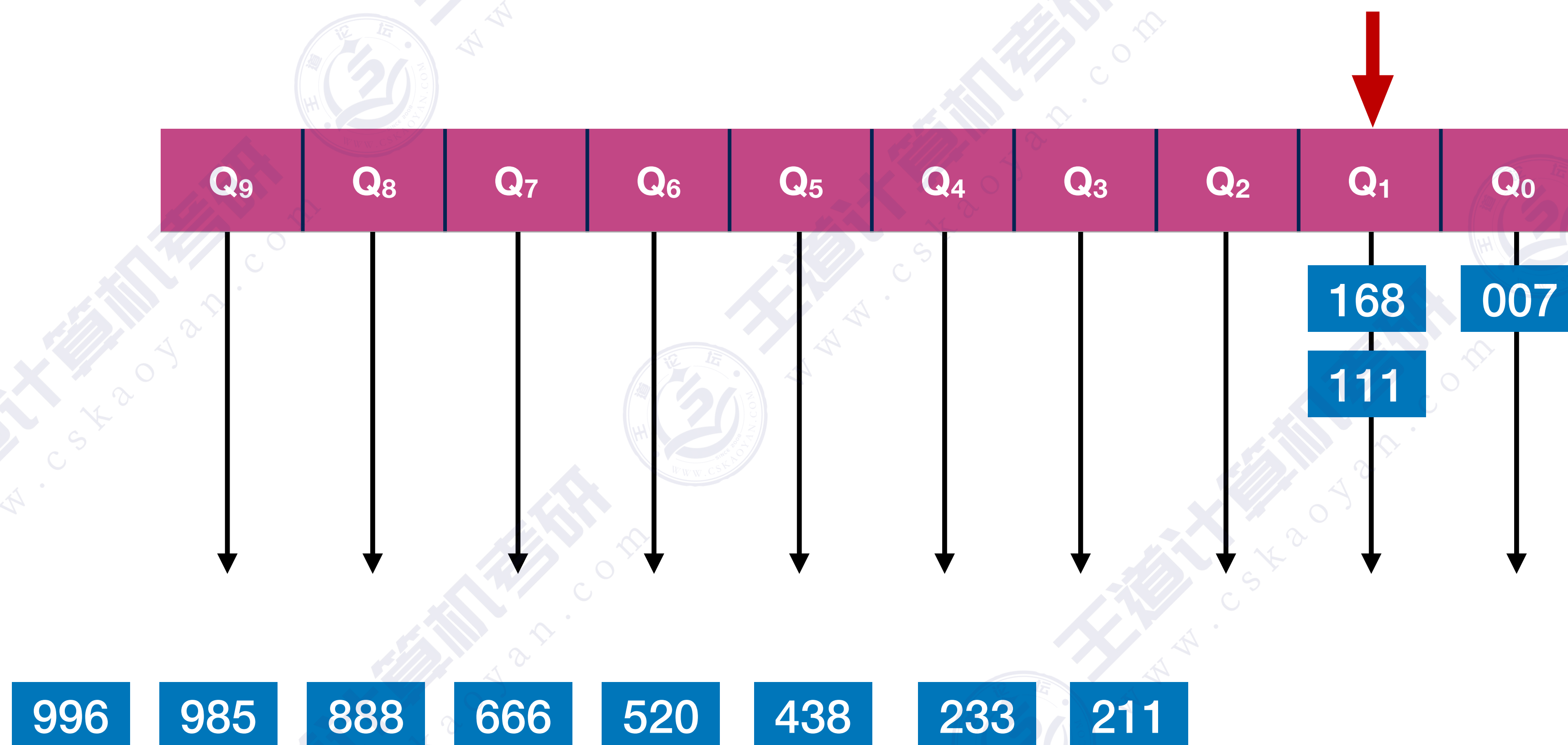
基数排序

第三趟“收集”



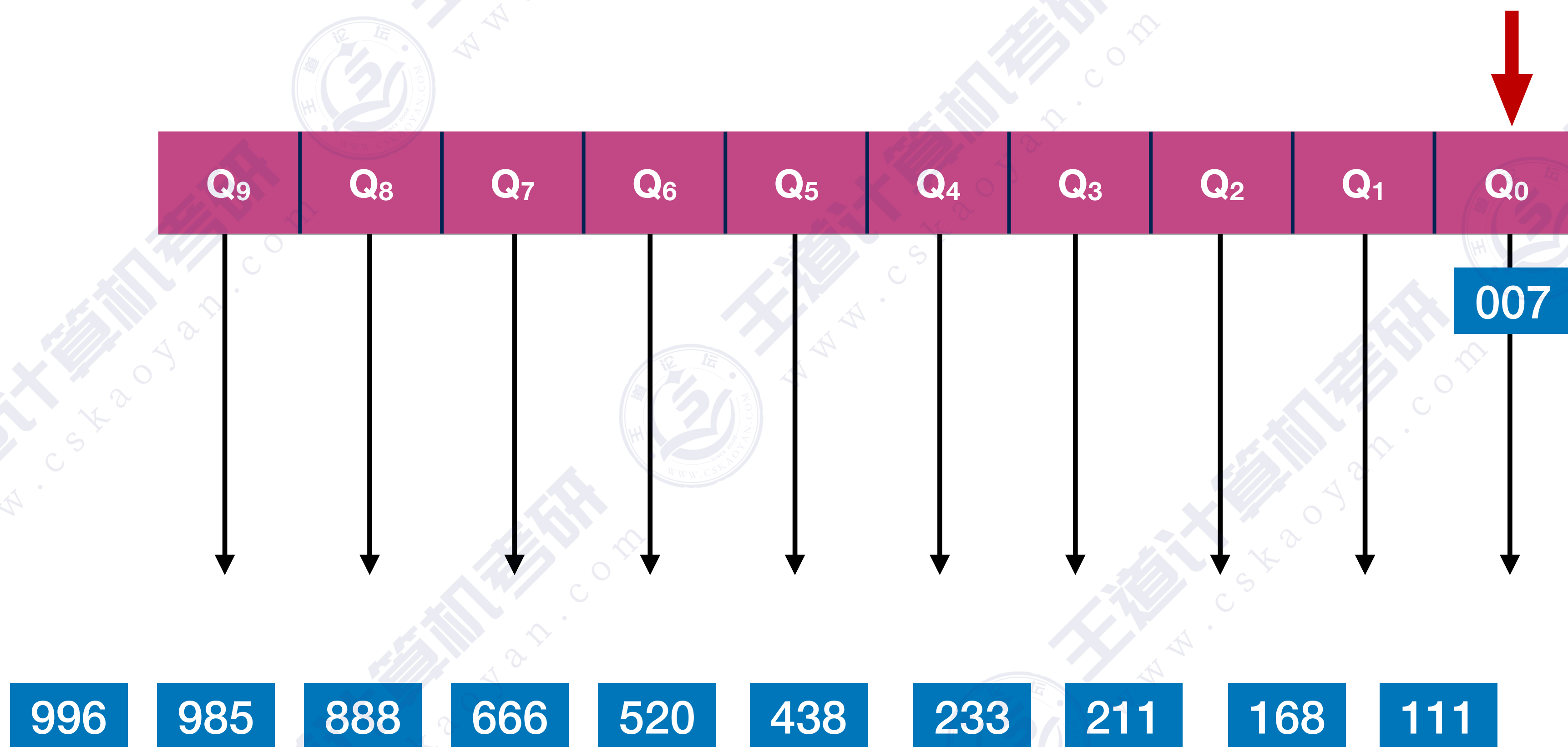
基数排序

第三趟“收集”



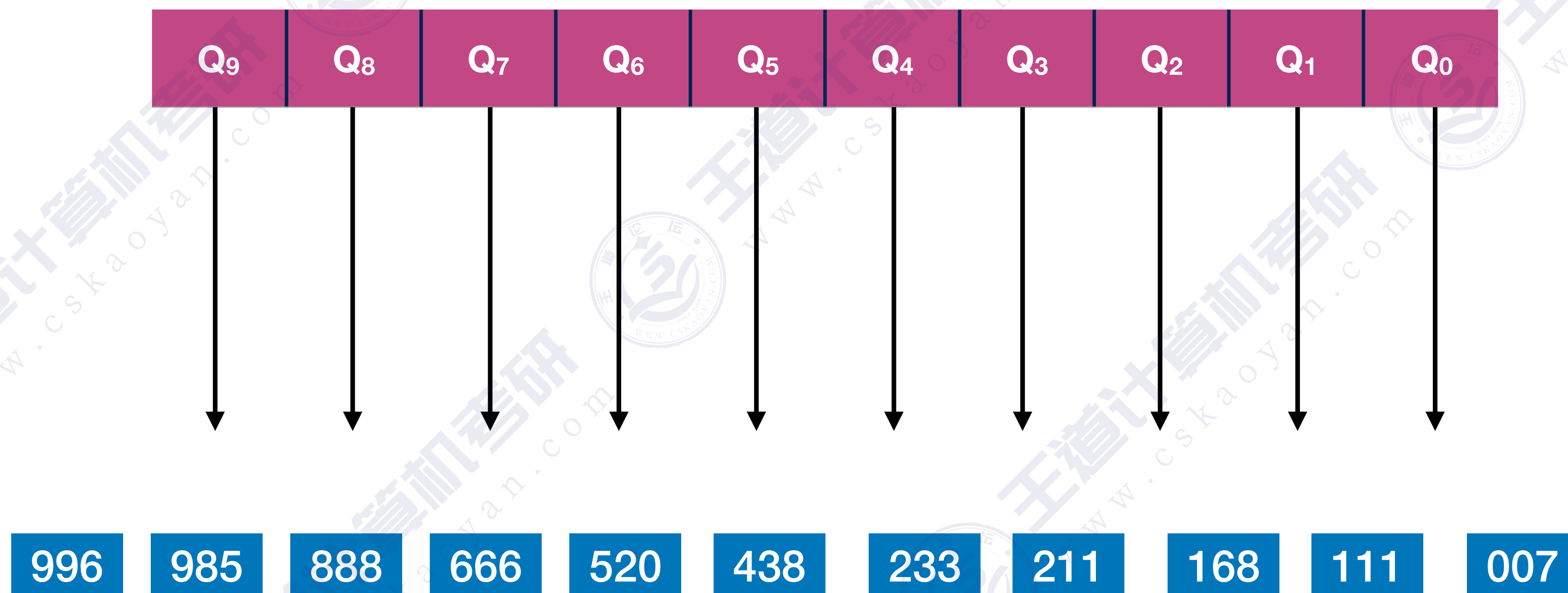
基数排序

第三趟“收集”



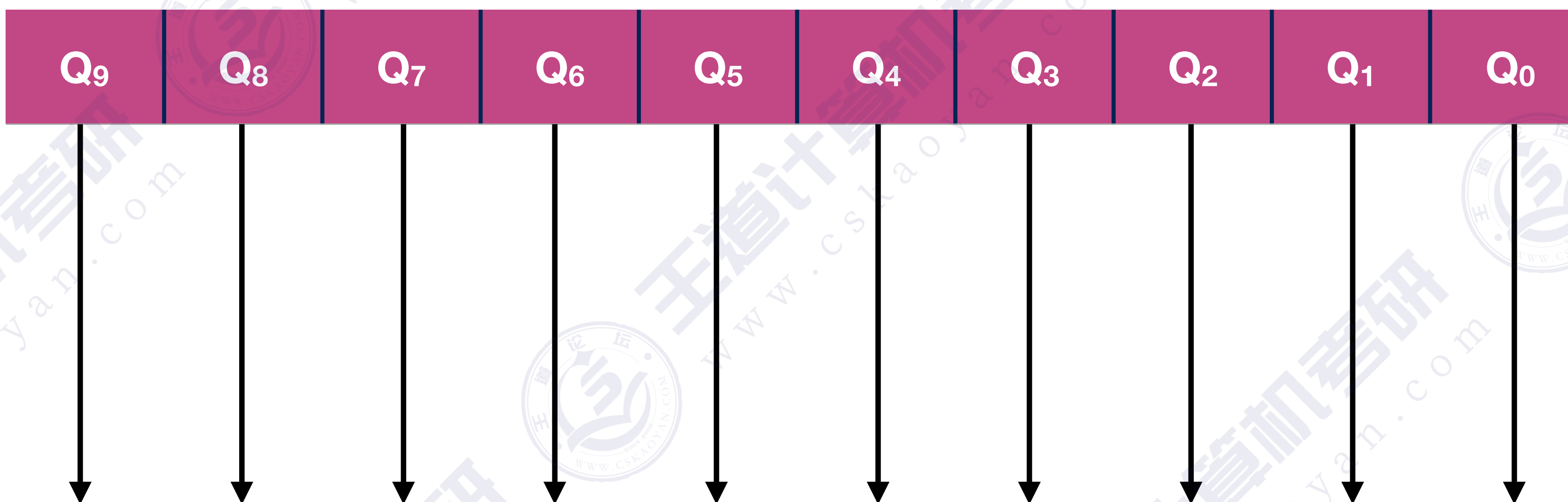
基数排序

第三趟“收集”



基数排序

第三趟“收集”



第三趟按“百位”分配、收集：得到一个按“百位”递减排列的序列，若“百位”相同则按“十位”递减排列，若“十位”还相同则按“个位”递减排列

996 → 985 → 888 → 666 → 520 → 438 → 233 → 211 → 168 → 111 → 007

基数排序

初始序列:

520 → 211 → 438 → 888 → 007 → 111 → 985 → 666 → 996 → 233 → 168

第一趟按“个位”分配、收集: 得到按“个位”递减排序的序列

438 → 888 → 168 → 007 → 666 → 996 → 985 → 233 → 211 → 111 → 520

第二趟按“十位”分配、收集: 得到按“十位”递减排序的序列, “十位”相同的按“个位”递减排序

996 → 888 → 985 → 168 → 666 → 438 → 233 → 520 → 211 → 111 → 007

第三趟按“百位”分配、收集: 得到一个按“百位”递减排列的序列, 若“百位”相同则按“十位”递减排列, 若“十位”还相同则按“个位”递减排列

996 → 985 → 888 → 666 → 520 → 438 → 233 → 211 → 168 → 111 → 007

基数排序

初始序列:



最高位关键字
(最主位关键字)

最低位关键字
(最次位关键字)

假设长度为 n 的线性表中每个结点 a_j 的关键字由 d 元组 $(k_j^{d-1}, k_j^{d-2}, k_j^{d-3}, \dots, k_j^1, k_j^0)$ 组成

其中, $0 \leq k_j^i \leq r - 1$ ($0 \leq j < n, 0 \leq i \leq d - 1$), r 称为“**基数**”

基数排序得到**递减**序列的过程如下,

初始化: 设置 r 个空队列, $Q_{r-1}, Q_{r-2}, \dots, Q_0$

按照各个 **关键字位 权重递增的次序** (个、十、百), 对 d 个关键字位分别做“分配”和“收集”

分配: 顺序扫描各个元素, 若当前处理的关键字位= x , 则将元素插入 Q_x 队尾

收集: 把 $Q_{r-1}, Q_{r-2}, \dots, Q_0$ 各个队列中的结点依次出队并链接

基数排序不是基于
“比较”的排序算法

基数排序

初始序列:



最高位关键字
(最主位关键字)

最低位关键字
(最次位关键字)

假设长度为 n 的线性表中每个结点 a_j 的关键字由 d 元组 $(k_j^{d-1}, k_j^{d-2}, k_j^{d-3}, \dots, k_j^1, k_j^0)$ 组成

其中, $0 \leq k_j^i \leq r - 1$ ($0 \leq j < n, 0 \leq i \leq d - 1$), r 称为“**基数**”

基数排序得到**递增**序列的过程如下,

初始化: 设置 r 个空队列, $Q_0, Q_1, \dots, Q_{r-1}$

按照各个 关键字位 权重递增的次序 (个、十、百), 对 d 个关键字位分别做“分配”和“收集”

分配: 顺序扫描各个元素, 若当前处理的关键字位= x , 则将元素插入 Q_x 队尾

收集: 把 $Q_0, Q_1, \dots, Q_{r-1}$ 各个队列中的结点依次出队并链接

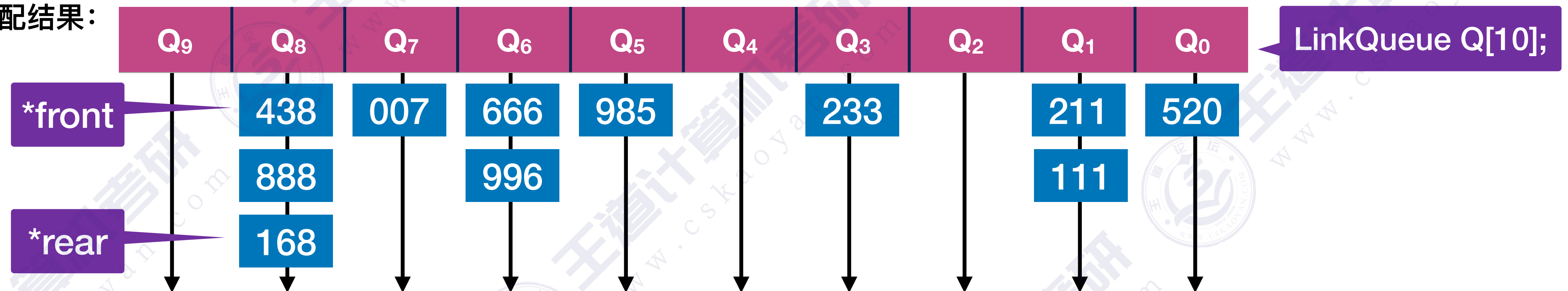
基数排序不是基于
“比较”的排序算法

算法效率分析

初始序列:



第一趟分配结果:



基数排序通常
基于链式存储实
现

```
typedef struct LinkNode{  
    ElemType data;  
    struct LinkNode *next;  
}LinkNode, *LinkList;
```

```
typedef struct{  
    LinkNode *front,*rear;  
}LinkQueue; //链式队列  
//队列的队头和队尾指针
```

需要 r 个辅助队列, 空间复杂度 = $O(r)$

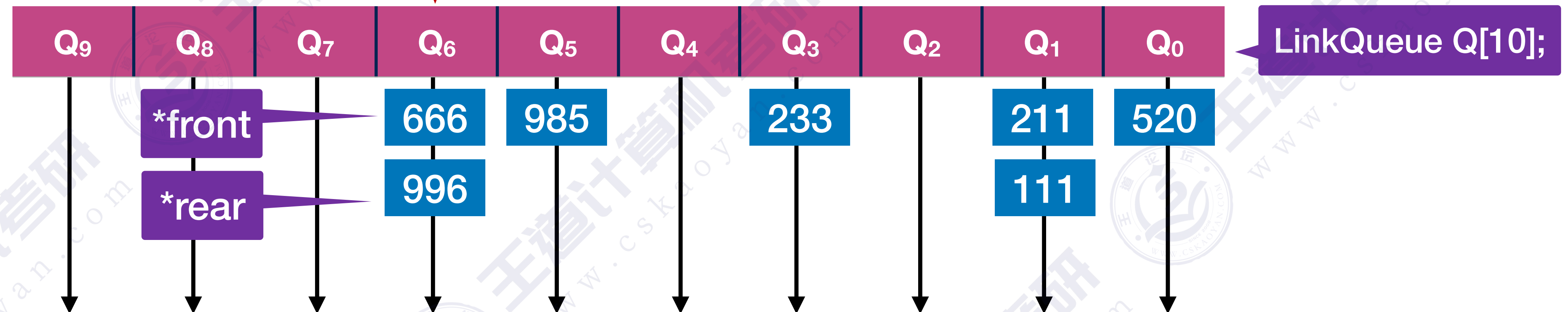
把关键字拆
为 d 个部分

每个部分可能
取得 r 个值

一趟分配 $O(n)$, 一趟收集 $O(r)$, 总共 d 趟分配、收集, 总的时间复杂度 = $O(d(n+r))$

算法效率分析

“收集”各个队列内元素



```
typedef struct LinkNode{  
    ElemType data;  
    struct LinkNode *next;  
}LinkNode, *LinkList;
```

*LinkList



```
typedef struct{  
    LinkNode *front,*rear;  
}LinkQueue;
```

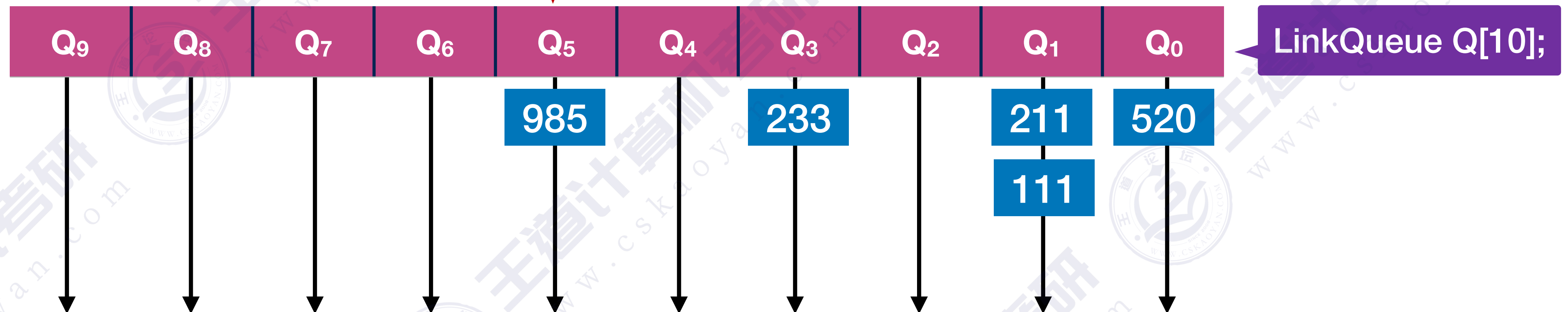
//链式队列
//队列的队头和队尾指针

```
p->next = Q[6].front;  
Q[6].front=NULL;  
Q[6].rear=NULL;
```

收集一个队列只
需O(1)时间

算法效率分析

“收集”各个队列内元素



```
typedef struct LinkNode{  
    ElemType data;  
    struct LinkNode *next;  
}LinkNode, *LinkList;
```

```
typedef struct{  
    LinkNode *front,*rear;  
}LinkQueue; //链式队列  
//队列的队头和队尾指针
```

*LinkList



```
p->next = Q[5].front;  
Q[5].front=NULL;  
Q[5].rear=NULL;
```

收集一个队列只
需O(1)时间

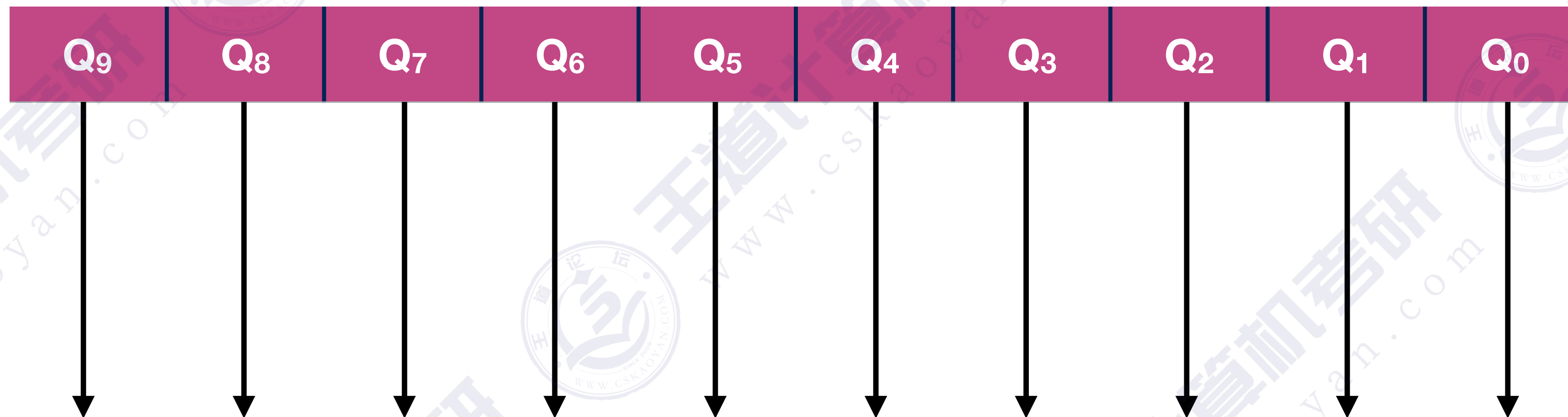
稳定性



初始序列:



第一趟分配:

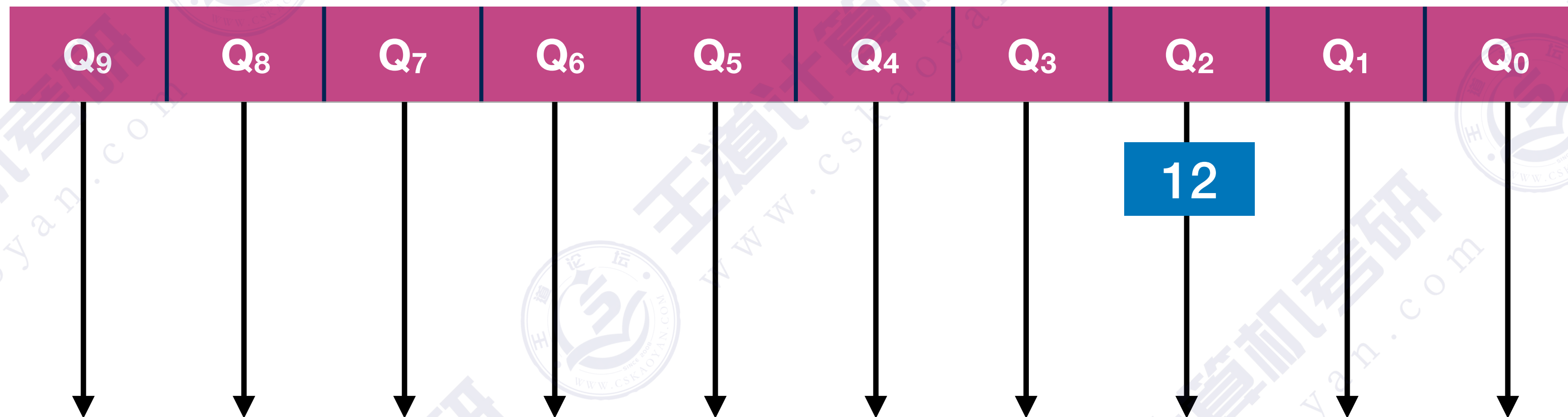


稳定性

初始序列:



第一趟分配:

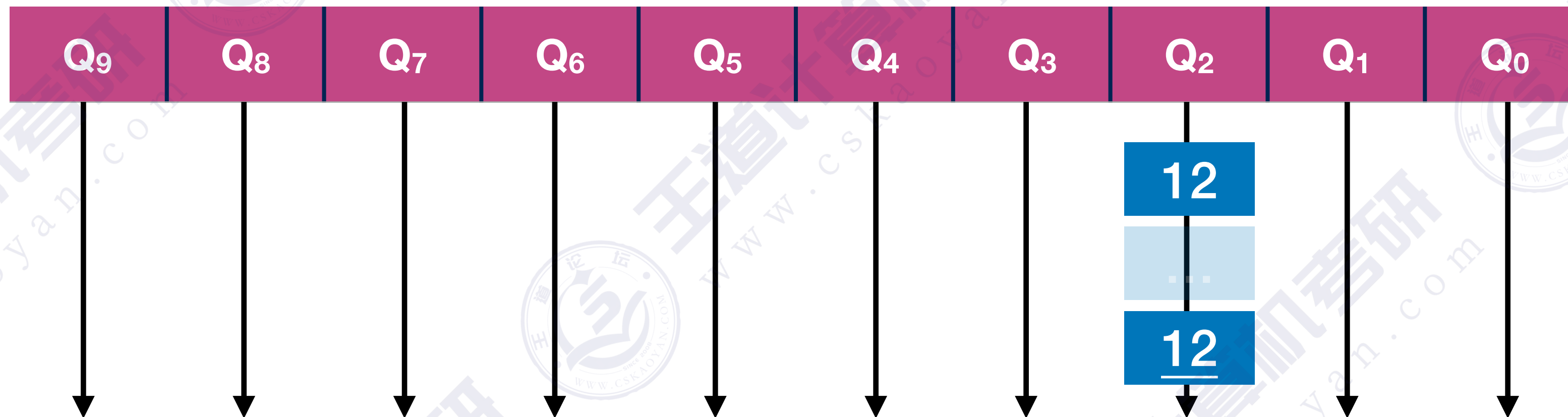


稳定性

初始序列:



第一趟分配:

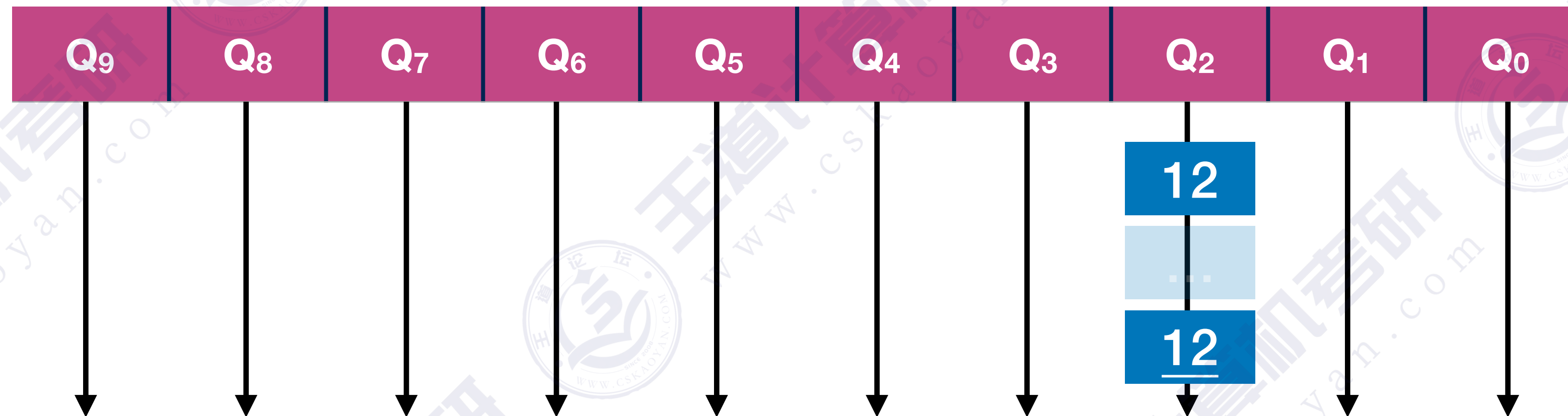


稳定性

初始序列:



第一趟分配:



第一趟收集:



基数排序是稳定的



基你太稳

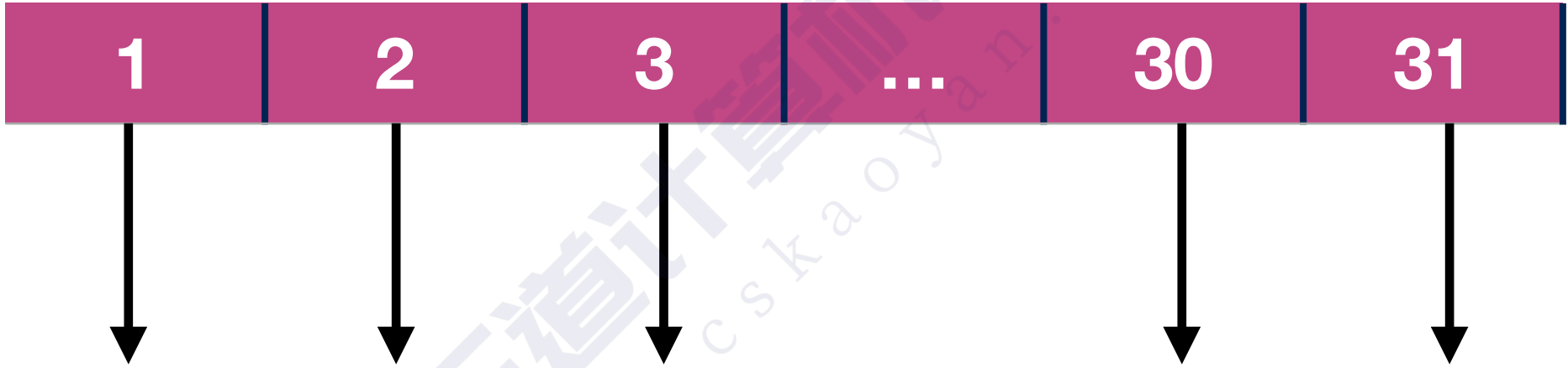
基数排序的应用

某学校有 10000 学生，将学生信息按**年龄递减**排序

生日可拆分为三组关键字：年(1991~2005)、月(1~12)、日(1~31)

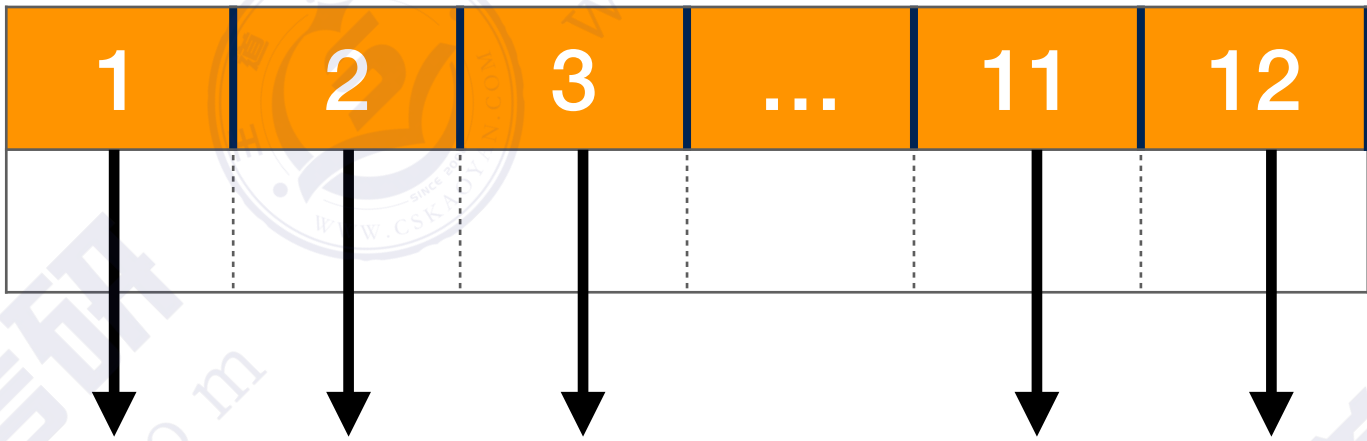
权重：年>月>日

第一趟分配、收集（按“日”递增）：



年、月、日越大，年龄越小

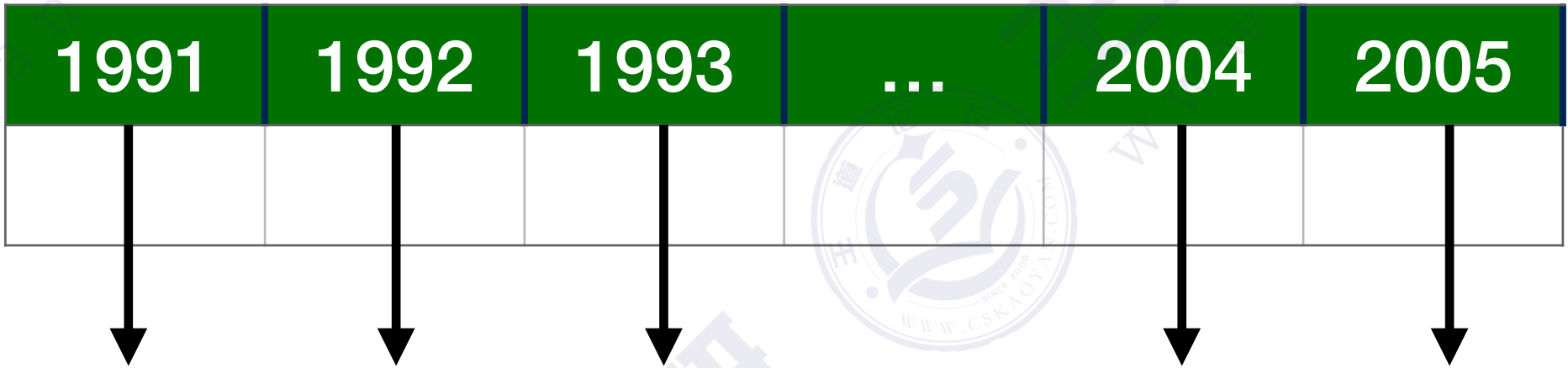
第二趟分配、收集（按“月”递增）：



基数排序，时间复杂度 = $O(d(n+r))$
 $\approx O(30000)$

若采用 $O(n^2)$ 的排序， $\approx O(10^8)$
若采用 $O(n\log_2 n)$ 的排序， $\approx O(140000)$

第三趟分配、收集（按“年”递增）：



基数排序的应用

某学校有 10000 学生，将学生信息按**年龄递减**排序

生日可拆分为三组关键字：年(1991~2005)、月(1~12)、日(1~31)

基数排序，时间复杂度 = $O(d(n+r))$
 $\approx O(30000)$

若采用 $O(n^2)$ 的排序， $\approx O(10^8)$

若采用 $O(n\log_2 n)$ 的排序， $\approx O(140000)$

基数排序擅长解决的问题：

- ①数据元素的关键字可以方便地拆分为 d 组，且 d 较小
- ②每组关键字的取值范围不大，即 r 较小
- ③数据元素个数 n 较大

基数排序的应用

基数排序，时间复杂度 = $O(d(n+r))$

基数排序擅长解决的问题：

- ①数据元素的关键字可以方便地拆分为 d 组，且 d 较小 反例：给5个人的身份证号排序
- ②每组关键字的取值范围不大，即 r 较小 反例：给中文人名排序
- ③数据元素个数 n 较大 擅长：给十亿人的身份证号排序

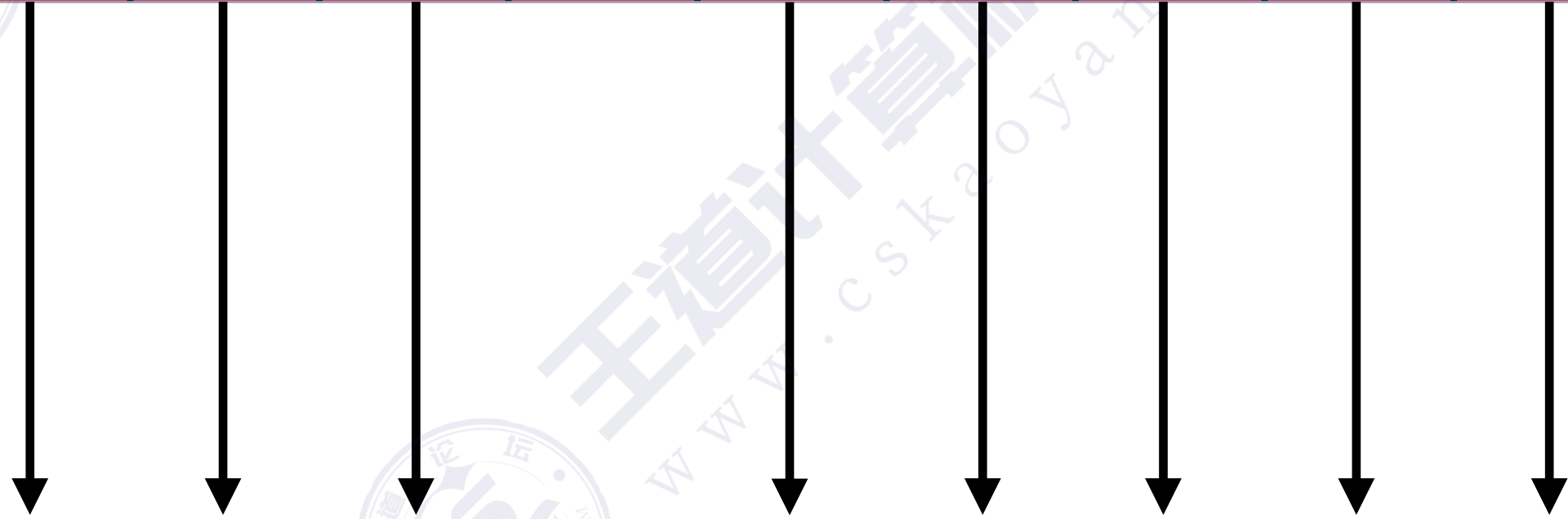
每个字可能有上万种取值

身份证号

XXXXXXXXXXXXXXXXXXXX

18位身份证号需要
分配、回收18趟

赵 钱 孙 ... 杰 伦 龔 谗 彧 ...



知识回顾与重要考点

基数排序

算法思想

将整个关键字拆分为 d 位（或“组”）

按照各个 关键字位 权重递增的次序（如：个、十、百），做 d 趟“分配”和“收集”，若当前处理的 关键字位 可能取得 r 个值，则需要建立 r 个队列

分配：顺序扫描各个元素，根据当前处理的关键字位，将元素插入相应队列。一趟分配耗时 $O(n)$

收集：把各个队列中的结点依次出队并链接。一趟收集耗时 $O(r)$

性能

空间复杂度 $O(r)$

时间复杂度 $O(d(n+r))$

稳定性 稳

擅长处理

① 数据元素的关键字可以方便地拆分为 d 组，且 d 较小

② 每组关键字的取值范围不大，即 r 较小

③ 数据元素个数 n 较大

本节内容

计数排序

Counting Sort

考研大纲对计数排序的要求 be like

大纲明面上
要求的...



考研大纲：没事哒，计数排序不用学，不要求掌握~

实际上...



考试加油

考场上：计数排序这么简单的东西，想必同学们都会吧？

知识总览

计数排序

算法思想

算法的实现

算法的性能

时间/空间复杂度

稳定性

计数排序在考试中的应用

Intro: 计数排序的算法思想

小学某班英语成绩:

旺财	90分
铁柱	97分
阿呆	76分
正南	83分
风间	100分
♥卧龙	3分
狗剩	95分
咸鱼	16分
大黄	84分
♥凤雏	12分

如何知道咸鱼的16分成绩排第几?



统计有多少个元素值 ≤ 16

共计3个元素（包含咸鱼自身）的值 ≤ 16 。
咸鱼应该排第3

成绩排名（倒数）:

卧龙	3分
凤雏	12分
咸鱼	16分
阿呆	76分
正南	83分
大黄	84分
旺财	90分
狗剩	95分
铁柱	97分
风间	100分

计数排序的实现

数组下标	0	1	2	3	4	5	6	7
A	2	4	3	0	2	3	0	3

使用计数排序的前提:

- 待排序元素的关键字是整数型
- 元素的取值范围是 $[0, k)$

Step 1: 创建一个长度为 k 的辅助数组, 统计每个元素的出现次数 (辅助数组的下标对应元素值)

此例中, 待排序元素取值范围是 $[0, 5)$

数组下标	0	1	2	3	4
C	2	0	2	3	1

0 出现过2次

1 出现过0次

3 出现过3次

Step 2: 再次处理辅助数组, 用 $C[i]$ 统计 $\leq i$ 的元素个数

数组下标	0	1	2	3	4
C	2	2	4	7	8

≤ 0 的元素累计出现了2个

≤ 3 的元素累计出现了7个

≤ 1 的元素累计出现了2个

Step 3: 利用辅助数组C实现排序

计数排序的实现



数组下标	0	1	2	3	4	5	6	7
A	2	4	3	0	2	3	0	3

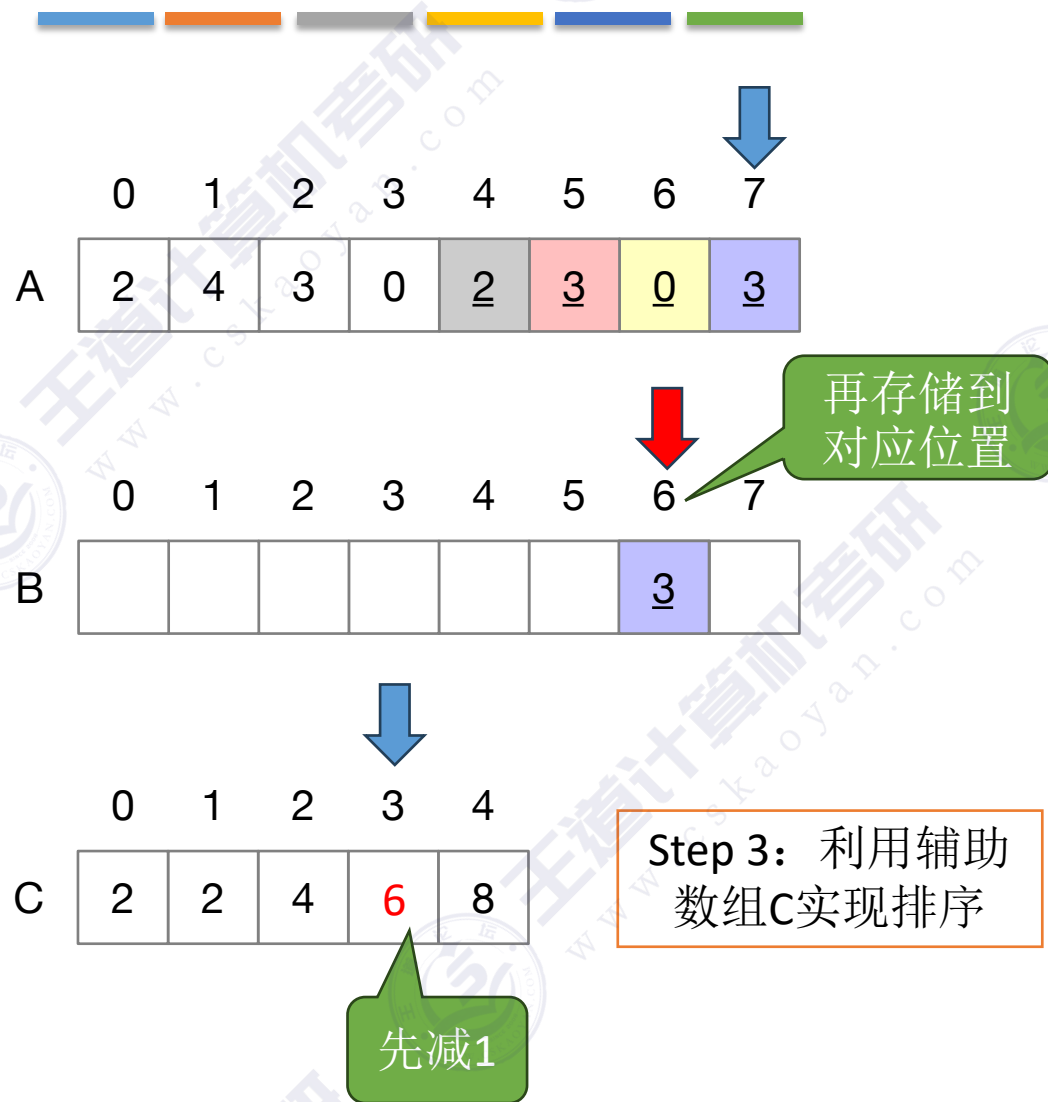
输出数组B的长度
与数组A相同，用
于存储排序结果

	0	1	2	3	4	5	6	7
B								

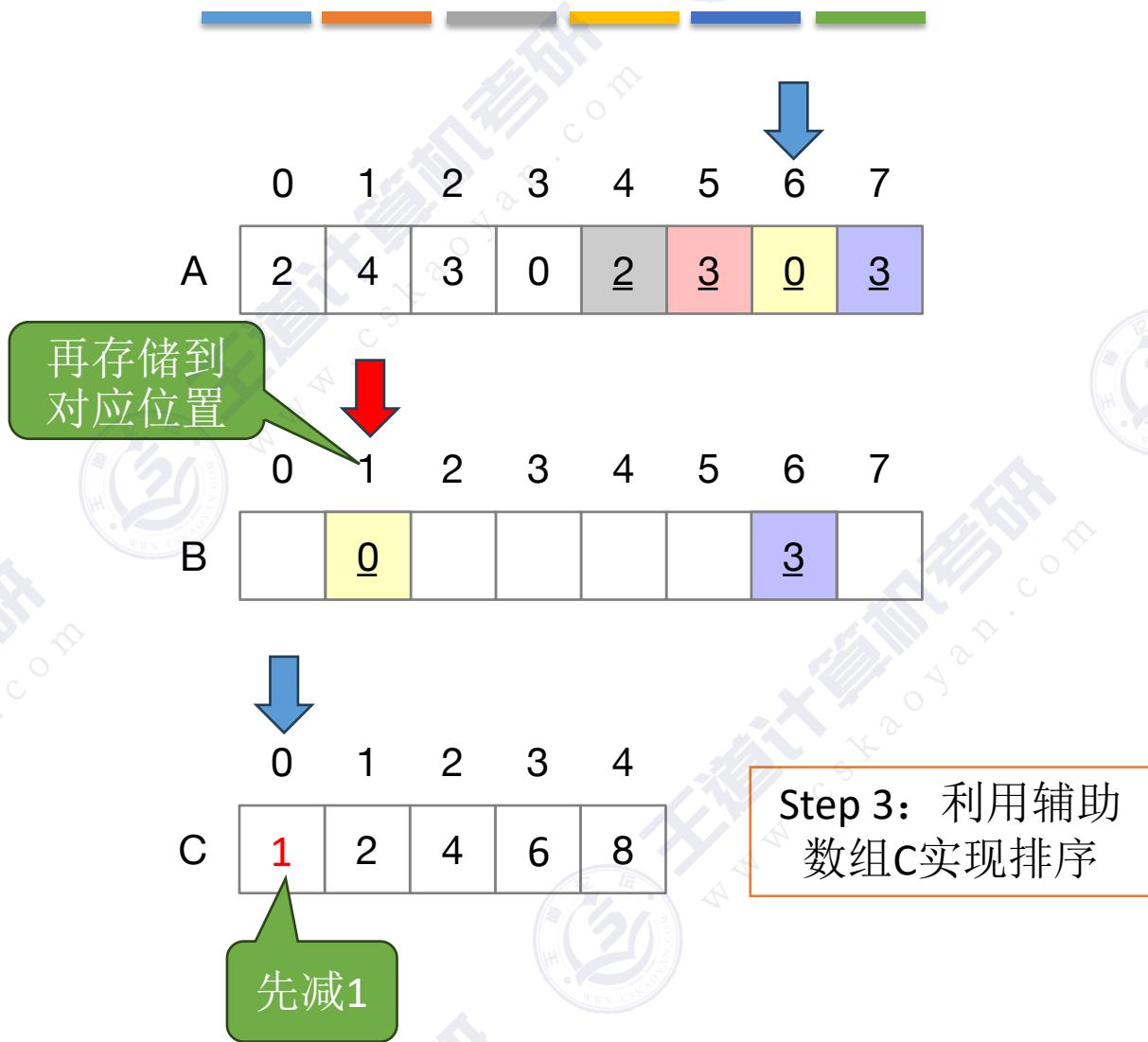
	0	1	2	3	4
C	2	2	4	7	8

Step 3: 利用辅助
数组C实现排序

计数排序的实现



计数排序的实现



计数排序的实现



	0	1	2	3	4	5	6	7
A	2	4	3	0	2	3	0	3



	0	1	2	3	4	5	6	7
B		0				3	3	



	0	1	2	3	4
C	1	2	4	5	8

Step 3: 利用辅助
数组C实现排序

计数排序的实现



	0	1	2	3	4	5	6	7
A	2	4	3	0	<u>2</u>	<u>3</u>	<u>0</u>	<u>3</u>



	0	1	2	3	4	5	6	7
B		<u>0</u>		<u>2</u>		<u>3</u>	<u>3</u>	



	0	1	2	3	4
C	1	2	3	5	8

Step 3: 利用辅助
数组C实现排序

计数排序的实现



	0	1	2	3	4	5	6	7
A	2	4	3	0	2	3	0	3



	0	1	2	3	4	5	6	7
B	0	0		2		3	3	



	0	1	2	3	4
C	0	2	3	5	8

Step 3: 利用辅助
数组C实现排序

计数排序的实现



	0	1	2	3	4	5	6	7
A	2	4	3	0	2	3	0	3



	0	1	2	3	4	5	6	7
B	0	0		2	3	3	3	



	0	1	2	3	4
C	0	2	3	4	8

Step 3: 利用辅助
数组C实现排序

计数排序的实现



	0	1	2	3	4	5	6	7
A	2	4	3	0	2	3	0	3



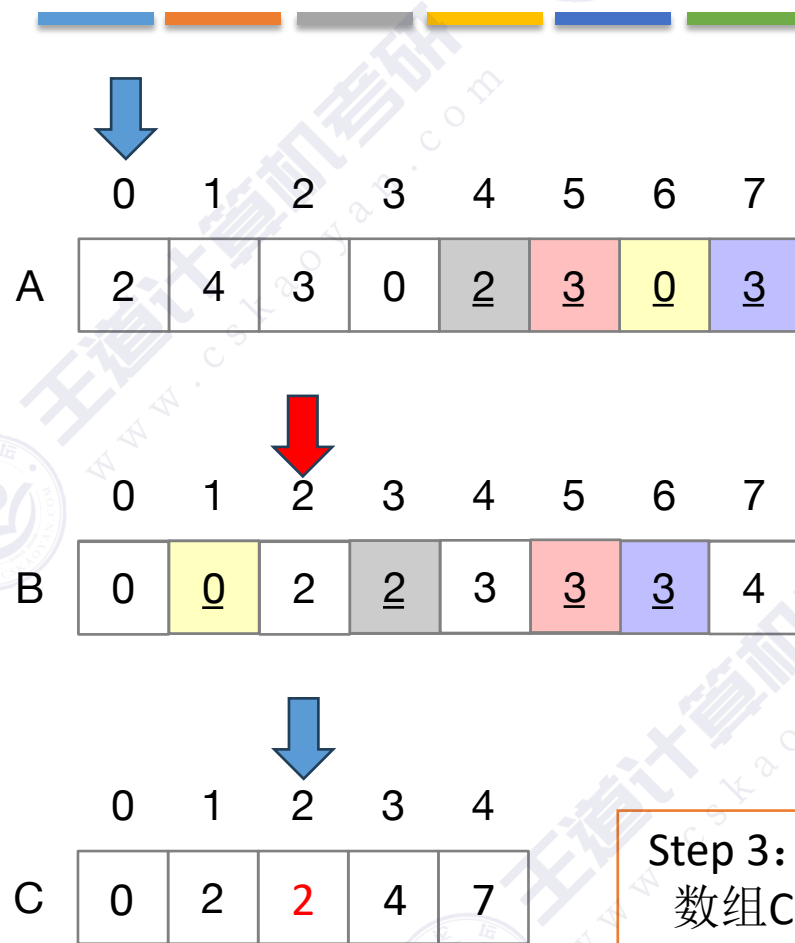
	0	1	2	3	4	5	6	7
B	0	0		2	3	3	3	4



	0	1	2	3	4
C	0	2	3	4	7

Step 3: 利用辅助
数组C实现排序

计数排序的实现



Step 3: 利用辅助
数组C实现排序

计数排序的实现

	0	1	2	3	4	5	6	7
A	2	4	3	0	<u>2</u>	<u>3</u>	<u>0</u>	<u>3</u>

	0	1	2	3	4	5	6	7
B								

	0	1	2	3	4
C	2	2	4	7	8



Step 3: 利用辅助
数组C实现排序

	0	1	2	3	4	5	6	7
B	0	<u>0</u>	2	<u>2</u>	3	<u>3</u>	<u>3</u>	4

	0	1	2	3	4
C	0	2	2	4	7

计数排序的实现（C语言）

A是输入数组
(待排序数组)

B是输出数组

n是A[]
的长度

k反映A[]的
取值范围

// 计数排序

```
void CountSort(int A[], int B[], int n, int k){  
    int i, C[k]; //辅助数组c的长度取决于待排序元素取值范围[0, k)  
    for(i=0; i<k; i++) //初始化计数数组  
        C[i]=0;  
    for(i=0; i<n; i++) //Step1: 遍历待排序数组, 统计每个关键字的出现次数  
        C[A[i]]++;  
    for(i=1; i<k; i++) //Step2: 再次处理辅助数组, 统计不大于 i 的元素个数  
        C[i]=C[i]+C[i-1]; //C[i]保存的是小于或等于 i 的元素个数  
    for(i=n-1; i>=0; i--) { //Step3: 利用辅助数组c实现计数排序 (从后往前处理)  
        C[A[i]]=C[A[i]]-1;  
        B[C[A[i]]]=A[i]; //将元素 A[i] 放在输出数组 B[] 的正确位置上  
    }  
}
```

注：这段C语言代码改写自《算法导论》，适合用于教学、考试。实践中可拓展这段代码，让计数排序更具备通用性（见计数排序代码附件）

计数排序的复杂度分析

来源于输出数组B

来源于辅助数组C

空间复杂度 = $O(n + k)$
时间复杂度 = $O(n + k)$

若 $k = O(n)$ ，计数排序的时间复杂度 = $O(n)$ ，此时计数排序的时间效率优于快速排序、堆排序等排序算法的 $O(n \log_2 n)$

若 $k > O(n \log_2 n)$ ，计数排序的时间复杂度 $> O(n \log_2 n)$ ，此时计数排序的时间效率不如快速排序、堆排序等排序算法的 $O(n \log_2 n)$

- 计数排序是典型的“空间”换“时间”思想
- 适用场景：当待排序数组的取值范围 k 较小时，可以考虑使用计数排序的思想

A是输入数组
(待排序数组)

B是输出数组

n是A[]
的长度

k反映A[]的
取值范围

// 计数排序

```
void CountSort(int A[], int B[], int n, int k){  
    int i, C[k];  
    // 辅助数组C的长度取决于待排序  
    // 初始化计数数组  
    for(i=0; i<k; i++){  
        C[i]=0;  
    }  
    // Step1: 遍历待排序数组，统计  
    for(i=0; i<n; i++){  
        C[A[i]]++;  
    }  
    // Step2: 再次处理辅助数组，  
    // C[i]保存的是小于或等于 i  
    for(i=1; i<k; i++){  
        C[i]=C[i]+C[i-1];  
    }  
    // Step3: 利用辅助数组C实现排序  
    for(i=n-1; i>=0; i--){  
        C[A[i]]=C[A[i]]-1;  
        B[C[A[i]]]=A[i];  
    }  
}
```

$O(k)$ →

$O(n)$ →

$O(k)$ →

$O(n)$ →

// 辅助数组C的长度取决于待排序
// 初始化计数数组

// Step1: 遍历待排序数组，统计

// Step2: 再次处理辅助数组，
// C[i]保存的是小于或等于 i

// Step3: 利用辅助数组C实现排序

// 将元素 A[i] 放在输出数组 B[]

计数排序在考试中的应用：2013年41题

41. (13 分) 已知一个整数序列 $A = (a_0, a_1, \dots, a_{n-1})$, 其中 $0 \leq a_i < n$ ($0 \leq i < n$). 若存在 $a_{p_1} = a_{p_2} = \dots = a_{p_m} = x$ 且 $m > n/2$ ($0 \leq p_k < n, 1 \leq k \leq m$), 则称 x 为 A 的主元素。例如 $A = (0, 5, 5, 3, 5, 7, 5, 5)$, 则 5 为主元素; 又如 $A = (0, 5, 5, 3, 5, 1, 5, 7)$, 则 A 中没有主元素。假设 A 中的 n 个元素保存在一个一维数组中, 请设计一个尽可能高效的算法, 找出 A 的主元素。若存在主元素, 则输出该元素; 否则输出-1。要求:

- 1) 给出算法的基本设计思想。
- 2) 根据设计思想, 采用 C、C++ 或 Java 语言描述算法, 关键之处给出注释。
- 3) 说明你所设计算法的时间复杂度和空间复杂度。

计数排序在考试中的应用：2018年41题



41. (13 分) 给定一个含 n ($n \geq 1$) 个整数的数组，请设计一个在时间上尽可能高效的算法，找出数组中未出现的最小正整数。例如，数组 $\{-5, 3, 2, 3\}$ 中未出现的最小正整数是 1；数组 $\{1, 2, 3\}$ 中未出现的最小正整数是 4。要求：
- 1) 给出算法的基本设计思想。
 - 2) 根据设计思想，采用 C 或 C++ 语言描述算法，关键之处给出注释。
 - 3) 说明你所设计算法的时间复杂度和空间复杂度。

知识回顾与重要考点

若 $O(k) > O(n \log_2 n)$ ，就不如用快速排序、堆排序

计数排序

适用条件

待排序元素的**关键字是整数型**

待排序**元素的取值范围是 $[0, k)$** ，且 **k 值较小**， k 与 $O(n)$ 同数量级

算法实现

创建一个长度为 k 的**辅助数组 $C[k]$**

Step 1: 遍历待排序数组 $A[n]$ ，用辅助数组 $C[i]$ 统计 $A[]$ 中每个元素值 i 出现的次数（元素值只可能是 $0 \sim k-1$ ，与辅助数组下标一一对应）

Step 2: 再次处理辅助数组 C ，最终用 $C[i]$ 统计 $\leq i$ 的元素个数

Step 3: **从后往前**依次处理各个待排序元素，结合辅助数组 $C[]$ 实现排序（注意：**从后往前处理各个元素可确保排序的稳定性**，详细过程建议自己手绘复习）

算法性能

时间复杂度

$O(n+k)$

当 k 与 $O(n)$ 同数量级时，时间复杂度 = $O(n)$

空间复杂度

$O(n+k)$

$O(n)$ 由输出数组 $B[n]$ 导致

$O(k)$ 由辅助数组 $C[k]$ 导致

当 k 与 $O(n)$ 同数量级时，空间复杂度 = $O(n)$

稳定性 —— **稳!**

计数排序（实战版）

注：计数排序代码（实战版）见代码pdf附件



我的心里只有一件事
就是 **学习**

计数排序的代码实现

1. 实战版本（更通用，以下代码可直接运行）

```
#include <stdio.h>
#include <stdlib.h>

void counting_sort(int A[], int n) {
    // 1. 找到数组 A 中的最大值和最小值
    int max = A[0];
    int min = A[0];
    for (int i = 1; i < n; i++) {
        if (A[i] > max) {
            max = A[i];
        }
        if (A[i] < min) {
            min = A[i];
        }
    }

    // 2. 创建计数数组
    int k = max - min + 1;
    int* C = (int*)calloc(k, sizeof(int)); // 动态分配内存

    // 3. 统计每个元素出现的次数
    for (int i = 0; i < n; i++) {
        C[A[i] - min]++;
    }

    // 4. 将计数数组转化为累加数组
    for (int i = 1; i < k; i++) {
        C[i] += C[i - 1];
    }

    // 5. 构建输出数组
    int* B = (int*)malloc(n * sizeof(int)); // 动态分配输出数组
    for (int i = n - 1; i >= 0; i--) { // 从后往前处理，确保稳定性
        B[C[A[i] - min] - 1] = A[i];
        C[A[i] - min]--;
    }

    // 6. 将输出数组B拷贝回原数组A
    for (int i = 0; i < n; i++) {
        A[i] = B[i];
    }

    // 7. 释放内存
    free(C);
    free(B);
}
```

```

int main() {
    int A[] = {4, 2, 2, -3, 5, 0, 3, 3, 1};
    int n = sizeof(A) / sizeof(A[0]);

    counting_sort(A, n);

    printf("排序后的数组: ");
    for (int i = 0; i < n; i++) {
        printf("%d ", A[i]);
    }
    printf("\n");

    return 0;
}

```

2. 教学用代码（更简洁易懂，改写自《算法导论》）

```

// 计数排序
void CountSort(int A[], int B[], int n, int k){
    int i, C[k];           //辅助数组c的长度取决于待排序元素取值范围[0, k)
    for(i=0; i<k; i++)      //初始化计数数组
        C[i]=0;
    for(i=0; i<n; i++)      //Step1: 遍历待排序数组，统计每个关键字的出现次数
        C[A[i]]++;
    for(i=1; i<k; i++)      //Step2: 再次处理辅助数组，统计不大于 i 的元素个数
        C[i]=C[i]+C[i-1];  //C[i]保存的是小于或等于 i 的元素个数
    for(i=n-1; i>=0; i--)   //Step3: 利用辅助数组C实现计数排序（从后往前处理）
        C[A[i]]=C[A[i]]-1;
    for(i=0; i<n; i++)      //将元素 A[i]放在输出数组 B[]的正确位置上
        B[C[A[i]]]=A[i];
}

```